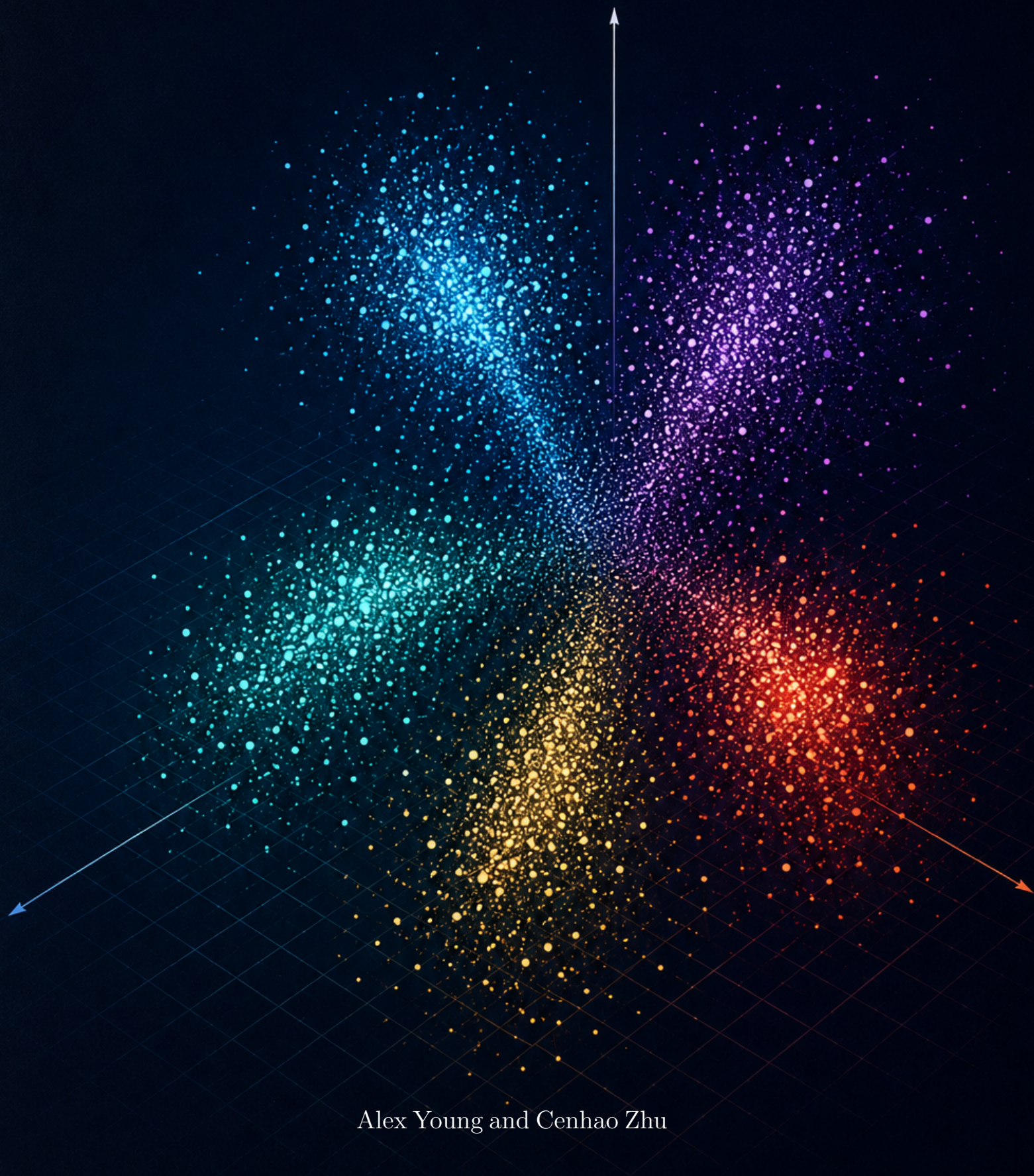


An Introduction to Unsupervised Learning



Alex Young and Cenhao Zhu

Contents

Preface	5
1 Introduction	7
1.1 Unsupervised learning among the branches of machine learning .	7
1.2 What can we learn without labels?	9
1.3 What this book covers	10
1.4 Prerequisites	11
1.5 How to use this book	12
2 Mathematical Background and Notation	13
2.1 Important notation	13
2.2 Random vectors in $\mathbb{R}^d$	14
2.3 Expectation, Mean, and Covariance	17
2.4 Linear Algebra	21
2.5 Summary	29
2.6 Exercises	30
3 Central goals and assumptions	35
3.1 Dimension reduction and manifold learning	35
3.2 Clustering	38
3.3 Generating synthetic data	39
3.4 Summary	45
3.5 Exercises	46
4 Linear Methods	47
4.1 Principal Component Analysis	47
4.2 Singular Value Decomposition	64
4.3 Nonnegative Matrix Factorization	70
4.4 Multidimensional Scaling	83
4.5 Summary	96
4.6 Exercises	97
5 Kernel Methods and Manifold Learning	101
5.1 Kernels and the Kernel Trick	101

5.2	Kernel PCA	103
5.3	Brief primer on manifolds and differential geometry	106
5.4	Isometric Feature Map (ISOMAP)	112
5.5	Locally Linear Embeddings (LLEs)	119
5.6	Laplacian Eigenmap (LEs)	127
5.7	Hessian Eigenmaps (HLEs)	131
5.8	t-Distributed Stochastic Neighbor Embedding (t-SNE)	137
5.9	Uniform Manifold Approximation and Projection (UMAP)	141
5.10	Autoencoders (AEs)	144
5.11	Comparison of Manifold Learning Methods	150
5.12	A Unifying View: Manifold Learning as Kernel PCA	152
5.13	Summary	154
5.14	Exercises	155
6	Clustering	161
6.1	Center-Based Clustering	161
6.2	Kernel k -means	169
6.3	Hierarchical Clustering	171
6.4	Model-Based Clustering	175
6.5	Spectral Clustering	186
6.6	Density-Based Spatial Clustering (DBSCAN)	192
6.7	Cluster Validation	196
6.8	Summary	201
6.9	Exercises	202
	Glossary	207
	Table of Notation	213
	Index	215

Preface

Copyright © 2023-2026 Alex Young and Cenhao Zhu. All rights reserved.

Chapter 1

Introduction

Unsupervised machine learning (UL) is an overarching term for methods designed to understand the patterns and relationships within a set of **unlabeled data**. As a concrete example, consider the MNIST dataset which contains a compendium of labeled digitized grayscale images of handwritten digits [Deng, 2012]. The images themselves are the features, $(\vec{x})$, and the identity of the digit (0 to 9) gives the label, y . A supervised learning algorithm trained on the MNIST dataset would be able to classify a handwritten digit in a new grayscale image. An unsupervised algorithm applied to the very same images – ignoring the digit labels entirely – would instead try to answer a different kind of question: do the images naturally organize into groups, and can we describe each image with a small number of meaningful numbers rather than 784 raw pixel intensities? Both questions are useful, but they are genuinely different questions, and answering them requires different tools.

1.1 Unsupervised learning among the branches of machine learning

Machine learning is often introduced through a small number of broad paradigms, distinguished by what kind of information is available during training and what kind of question we’re trying to answer. It’s worth briefly placing unsupervised learning among its neighbors before diving in, since the contrasts are genuinely clarifying.

Supervised learning is the setting most readers will have already encountered. One is primarily concerned with prediction and classification, and the machine learning algorithms therein focus on learning the relationship between a (often high dimensional) feature vector $\vec{x}$ and an observable outcome y by training on a labeled dataset $\{(\vec{x}_i, y_i)\}_{i=1}^N$. Prediction and classification are clearly defined goals which naturally translate to many settings. As such, supervised ML has

found numerous applications across diverse fields of research from healthcare and medicine to astronomy and chemistry. Given the clearly translatable goals of supervised learning, most texts on Machine Learning tend to emphasize this setting with much smaller discussion on the un- or semi-supervised setting. For example, both *The Elements of Statistical Learning* by Hastie et al [Hastie et al., 2001] and *Modern Multivariate Statistical Techniques* by Izenman [Izenman, 2008] are wonderful texts – which were central to the early development of this book – but lean towards supervised problems.

Unsupervised learning, the subject of this book, drops the labels $y_1, \dots, y_N$ entirely and works only with the features $\vec{x}_1, \dots, \vec{x}_N$. Rather than learning a mapping from $\vec{x}$ to y , UL algorithms look for structure that is already latent in the features themselves: groups of similar observations, simpler representations, or informative geometry we would otherwise never see by looking at raw, high-dimensional data.

Between these two extremes sit a few related paradigms worth knowing by name, if only because you're likely to encounter them elsewhere.

- **Semi-supervised learning** trains a supervised model using a small labeled dataset $\{(\vec{x}_i, y_i)\}_{i=1}^n$ together with a much larger pool of unlabeled features $\{\vec{x}_i\}_{i=n+1}^N$ (labels are often expensive or slow to collect, while unlabeled data is comparatively cheap). Many semi-supervised methods work by first uncovering structure in the unlabeled data – for instance, by clustering it or reducing its dimension using exactly the tools developed in this book – and then using that structure to make the most of the few labels that are available.
- **Self-supervised learning** also works with unlabeled data, but manufactures its own labels directly from the data rather than doing without them. A model might be trained to reconstruct an input from a compressed version of itself, or to predict a masked-out word from the words around it. The autoencoders we cover in Chapter 5 are a classic example: they have no labels of any kind, yet they are trained by the same kind of supervised loss-minimization machinery used elsewhere in ML, just with the input standing in as its own target.
- **Reinforcement learning** departs from the other paradigms most sharply. There is no fixed dataset at all; instead, an agent interacts sequentially with an environment, taking actions and receiving rewards, and the goal is to learn a policy that maximizes cumulative reward over time. Where the other paradigms all start from a static collection of observations, reinforcement learning is fundamentally about learning through interaction. (Harvard has recently introduced a dedicated undergraduate course on this topic, a sign of how distinct – and how quickly growing – this branch of ML has become.)

We will not return to semi-supervised, self-supervised, or reinforcement learning in any real depth in this text, but it's worth keeping the contrast in mind:

unsupervised learning is defined by the *absence* of labels, full stop, and by a focus on discovering structure for its own sake rather than as a means to a prediction.

1.2 What can we learn without labels?

Without labels, it may be difficult (particularly to students first seeing this branch of machine learning) to grasp the usefulness of UL including its applicability and what one is *learning* in practice. In this book, we hope to address this difficulty and provide readers with a clear understanding of UL by covering motivating ideas, fundamental techniques, and clear and compelling applications. For now, we'll discuss the high-altitude view of unsupervised learning.

We'll focus on those cases where we have a collection of independent observations of (preprocessed) features stored in vectors $\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}^d$. This setting will be formalized in Chapter 2. Broadly, UL learns patterns and similarities between the vectors which could allow us to

- 1) find subsets of the data which are more similar to each other (**clustering**), or
- 2) find simpler representations of the data which preserve important relationships (**dimension reduction**).

Each of these two cases provides a simplified lens through which we can view our data, and in doing so, can open up a number of interesting possibilities. A small sample of settings where these ideas have been put to genuine use:

- **Genomics and medicine.** Clustering gene expression data from tumor samples can reveal that a single clinically-defined cancer type is actually a mixture of several molecularly distinct subtypes, each of which may respond differently to treatment. We'll work through a real example of exactly this kind of analysis, using microarray data from 64 human tumor samples, in Chapter 6.
- **Computer vision.** Reducing a dataset of images to a handful of coordinates – as we will do repeatedly with the MNIST digits throughout this book – lets us visualize how thousands of images relate to one another, spot mislabeled or unusual images, and dramatically compress data for downstream storage or modeling.
- **Natural language and document collections.** A large corpus of emails, articles, or customer reviews can be clustered into topics or grouped by similarity without anyone having read and labeled a single document in advance, which is often the only realistic option when the corpus is large or labels don't yet exist (e.g. is this email spam, before anyone has flagged it as such?).
- **Anomaly and fraud detection.** A dimension reduction technique fit on “typical” transactions, network traffic, or sensor readings gives us a

notion of what normal data looks like in a low-dimensional representation; observations that reconstruct poorly or land far from the bulk of the data are natural candidates for further scrutiny.

- **Recommendation and personalization.** Clustering users or items by behavior, or reducing a large, sparse matrix of user-item ratings to a small number of latent factors, underlies many recommendation systems – we’ll see a version of this idea when we discuss the singular value decomposition (Chapter 4).
- **Scientific exploratory data analysis more broadly.** Long before any formal hypothesis is tested, dimension reduction and clustering are often the first tools scientists reach for simply to see what a new, high-dimensional dataset looks like.

The examples above are by no means exhaustive, but they do raise a few critical points.

- 1) **Where** Each application above is one step in a larger data science problem. Unsupervised learning is rarely detached from a broader data science pipeline. This is a stark difference from classification and prediction which are often viewed as isolated statistical problems (though careful practitioners recognize that data collection and cleaning and the communication of results are often of equal or greater importance than analysis). Examples are provided throughout the text demonstrating where UL can be useful.
- 2) **What** One data set could be approached from one or more different perspectives. For example, one could apply dimension reduction to the cancer data to *visualize* the potentially complex data in a manner that preserves important relationships. Combining many approaches together makes unsupervised learning a powerful tool to exploratory data analysis and featurization, particularly when combined with expert level content knowledge. Choosing a UL method is linked with what we hope to learn about our data.
- 3) **How** If we want an algorithm that clusters similar vectors or provides a visualization that keeps close points together, then we should be mindful of the meaning of similarity or proximity. UL algorithms, sometimes implicitly, prioritize different relationships. We explore how these algorithms work from a geometric perspective which is a helpful intellectual scaffolding.

1.3 What this book covers

To be precise about scope: this book is organized around the two pillars introduced above, dimension reduction and clustering, and says relatively little about other corners of unsupervised learning (association rule mining and generative modeling being two notable examples we do not cover in depth). Concretely, after a review chapter, the book proceeds as follows.

- Chapter 2 briefly reviews the probability, linear algebra, and calculus this book leans on most heavily, and fixes the notation used throughout (see the Table of Notation at the end of the book for a quick-reference version).
- Chapter 4 covers **linear** dimension reduction methods: principal component analysis, the singular value decomposition, nonnegative matrix factorization, and multidimensional scaling.
- Chapter 5 extends these ideas to **nonlinear** structure: the kernel trick and kernel PCA, the manifold hypothesis, and a tour of manifold learning methods (ISOMAP, locally linear embeddings, Laplacian and Hessian eigenmaps, UMAP) and autoencoders.
- Chapter 6 covers **clustering**: center-based methods (k -means, k -medoids, k -center, and their kernelized versions), hierarchical clustering, model-based clustering via Gaussian mixture models, spectral clustering, and how to validate a clustering once you have one.

Every method is paired with worked examples (many using real data), a discussion of its strengths and weaknesses, and exercises at the end of each chapter.

1.4 Prerequisites

This text is targeted at upper level undergraduates with a well rounded background consistent with an undergraduate course in each of the following:

- **Probability**
 - Major concepts: mean, covariance, correlation, (in)dependence
 - Minor concepts: likelihood functions, conditional expectation, parametric distributions (e.g. Gaussian)
- **Linear algebra**
 - Major concepts: interpretations of matrix-vector and matrix-matrix multiplication, spans, bases, matrix decompositions
 - Minor concepts: orthonormal transformations, vector and matrix norms, Hadamard operations
- **Multivariable calculus**
 - Major concepts: gradients (computation and interpretation), optimization with and without constraints
 - Minor concepts: multivariate Taylor expansions, determining properties of local optima via the Hessian matrix

A brief review of the most important ideas above, along with the notation used throughout the book, is covered in Chapter 2. Additional tools and techniques needed for specific algorithms are covered at a cursory level as needed. References to more thorough discussions are provided throughout for the interested reader.

1.5 How to use this book

This book grew out of *Stat 185: Unsupervised Learning*, taught at Harvard University since Fall 2019. The book emphasizes high-dimensional geometry and the insight linear algebra provides into *why* these methods work, rather than treating each algorithm as a black box.

A few suggestions depending on how you’re approaching the text:

- **If you’re using this book for a semester-long course** (as in its original setting), the chapters are designed to be read in order. Chapter 2 is intended as a review rather than a first introduction to its material – if any of it is genuinely new to you, consider supplementing it with a dedicated probability, linear algebra, or multivariable calculus reference before proceeding. Chapters 4 through 6 build on each other: the kernel methods in Chapter 5 rely on the linear methods of Chapter 4, and the “Unifying View” closing that chapter connects manifold learning directly to spectral clustering in Chapter 6.
- **If you already know some of this material**, each chapter is reasonably self-contained past the review chapter. A reader already comfortable with PCA and the SVD, for instance, could reasonably start at Chapter 5.
- **If you’re using this book for self-study**, work the exercises. Nearly every section closes with a mix of derivations, hand-computable numerical examples, and applied problems using real or synthetic data in R; the exercises are where the geometric intuition described above actually gets tested.
- Throughout the text, definitions and theorems are set off in numbered boxes, worked examples are marked **Example**, and R code is shown whenever it clarifies *how* a method is actually computed rather than just *what* it computes.
- A Table of Notation, Glossary, and Index are collected at the end of the book for quick reference.

Chapter 2

Mathematical Background and Notation

The required mathematical background is summarized in the Prerequisites section of Chapter 1. In this chapter, we identify common notation and review the most important material from each.

2.1 Important notation

Throughout this text, we will be working with vectors and matrices quite often so we begin with a bit of notation and a few important conventions we will adopt hereafter. We'll use notation $\vec{x} \in \mathbb{R}^d$ to denote a d -dimensional vector. Importantly, we adopt the convention that vectors are column vectors by default so that

$$\vec{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$$

where $x_1, \dots, x_d$ are the entries or coordinates of vector $\vec{x}$. Row vectors are then the transpose of column vectors so that $\vec{x}^T = (x_1, \dots, x_d)$. When needed we'll let $\vec{0}$ denote a vector of all zeros and $\vec{1}$ a vector of all ones with the dimensionality defined implicitly, e.g. if $\vec{x} \in \mathbb{R}^d$ then in the expression $\vec{x} + \vec{1}$, you may interpret $\vec{1} \in \mathbb{R}^d$ so the summation is well defined.

Matrices will be denoted in **bold** so that $\mathbf{A} \in \mathbb{R}^{m \times n}$ denotes an $m \times n$ matrix with real entries. Subscripts are read as *row, column* so that $\mathbf{A}_{ij}$ is the entry of $\mathbf{A}$ in the i th row and j th column. A superscript T denotes the transpose of a matrix. For a square matrix $\mathbf{B} \in \mathbb{R}^{n \times n}$, we use notation $Tr(\mathbf{B})$ to denote the trace of $\mathbf{B}$ and $det(\mathbf{B}) = |\mathbf{B}|$ to denote its determinant.

Using this above notation, we may also define the inner product and outer product of two vectors. For vectors $\vec{x}$ and $\vec{y}$, the inner product or dot product of $\vec{x}$ and $\vec{y}$ is the scalar $\vec{x}^T \vec{y} = \sum_{i=1}^d x_i y_i$. Alternatively, we may also consider the outer product $\vec{x} \vec{y}^T$ which is a matrix such that $(\vec{x} \vec{y}^T)_{ij} = x_i y_j$. For the inner product to be well defined $\vec{x}$ and $\vec{y}$ must have the same dimension. This is not the case for the outer product. If $\vec{x} \in \mathbb{R}^m$ and $\vec{y} \in \mathbb{R}^n$ then $\vec{x} \vec{y}^T \in \mathbb{R}^{m \times n}$. If we view a d -dimensional vector as a $d \times 1$ matrix, then both of these algebraic computations are completely consistent with standard matrix multiplication which we will revisit near the end of this chapter.

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a function of d -dimensional vector $\vec{x}$. Then we define the gradient of f with respect to $\vec{x}$, denoted ∇f , to be the d -dimensional vector of partial derivatives of f with respect to the coordinates of $\vec{x}$ so that

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix}.$$

The Hessian matrix of f with respect to $\vec{x}$, denoted as $\nabla^2 f$, is the $d \times d$ matrix of second order partial derivatives of f with respect to the coordinates of $\vec{x}$ so that $(\nabla^2 f)_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$. If we are considering a function of multiple vector valued variables, e.g. $f(\vec{x}, \vec{y}, \vec{z})$ then we use $\nabla_{\vec{x}}$ to denote the gradient of f w.r.t. the vector $\vec{x}$ only.

2.2 Random vectors in $\mathbb{R}^d$

Throughout this text, we will consider independent, identically distributed (**iid**) samples of a d -dimensional random vector $\vec{x} = (x_1, \dots, x_d)^T$. Each coordinate x_i is a random variable and we may view the distribution of the random vector $\vec{x}$ as the joint distribution of all of its coordinates. The cumulative distribution function of $\vec{x}$ is then

$$F(\vec{x} \leq \vec{x}^o) = P(\vec{x} \leq \vec{x}^o) = P(x_1 \leq x_1^o, \dots, x_d \leq x_d^o).$$

We'll largely consider continuous entries so we can rewrite the above in terms of the joint density, $f : \mathbb{R}^d \rightarrow [0, \infty)$, of $\vec{x}$ such that

$$F(\vec{x} \leq \vec{x}^o) = \int_{-\infty}^{x_1^o} \cdots \int_{-\infty}^{x_d^o} f(x_1, \dots, x_d) dx_d \cdots dx_1.$$

To simplify notation, we'll often write the above as

$$\int_{-\infty}^{x_1^o} \cdots \int_{-\infty}^{x_d^o} f(x_1, \dots, x_d) dx_d \cdots dx_1 = \int_{-\infty}^{\vec{x}^o} f(\vec{x}) d\vec{x}.$$

In the case that $\vec{x}$ may only take one of a countable set of outcomes, one can replace the integral above with a corresponding summation.

Generally speaking we will be considering data drawn from an unknown distribution. However, considering known cases which we can analyze and sample from is often helpful to study how different algorithms perform. With this idea in mind, let's define a few different distributions which we will revisit throughout this chapter as examples. In each case, we will also provide scatterplots of independent samples from these distributions so that you can visualize the distributions more directly.

Definition 2.1 (Multivariate Gaussian Distribution). The multivariate Gaussian distribution in $\mathbb{R}^d$ with mean $\vec{\mu} \in \mathbb{R}^d$ and symmetric positive definite covariance matrix $\Sigma \in \mathbb{R}^{d \times d}$ is the random d -dimensional vector $\vec{x}$ with probability density function

$$f(\vec{x}) = \frac{1}{(2\pi)^{d/2} \det(\Sigma)^{1/2}} \exp\left(-\frac{1}{2}(\vec{x} - \vec{\mu})^T \Sigma^{-1}(\vec{x} - \vec{\mu})\right).$$

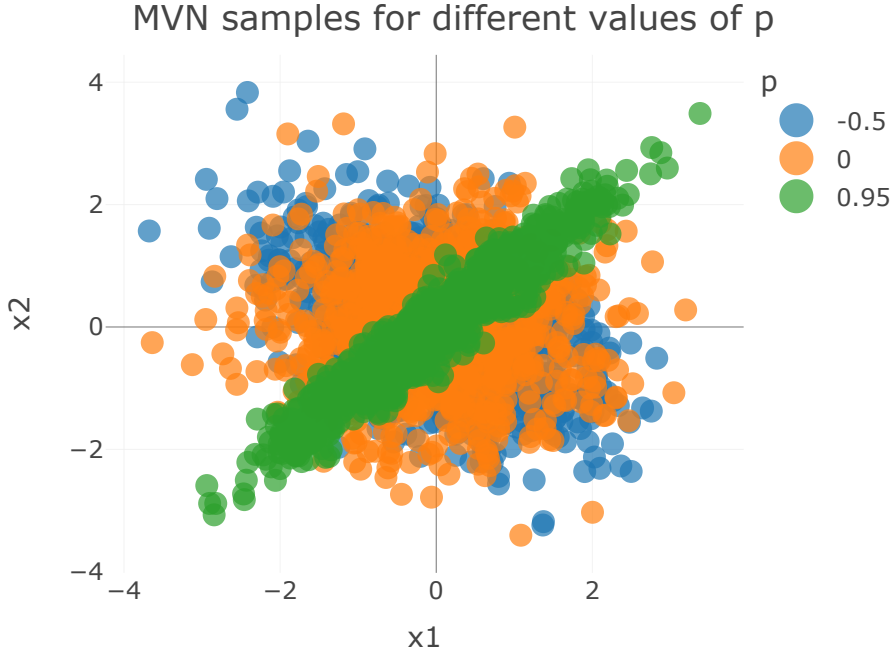
We use shorthand $\vec{x} \sim \mathcal{N}(\vec{\mu}, \Sigma)$ to indicate $\vec{x}$ follows this distribution.

The Multivariate Gaussian distribution is also often called the Multivariate Normal (MVN) distribution.

For example of the MVN, first consider the two-dimensional case with $\vec{\mu} = \vec{0}$ and

$$\Sigma = \begin{bmatrix} 1 & p \\ p & 1 \end{bmatrix}.$$

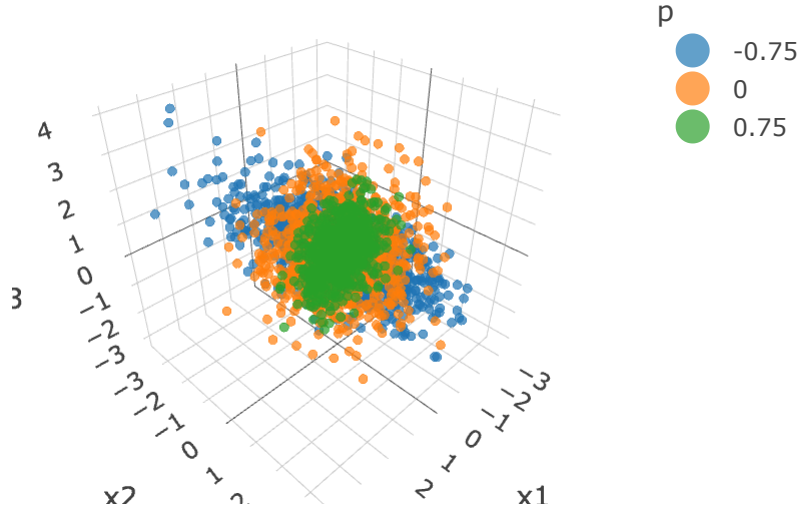
Below, we show scatterplots of 1000 independent samples from this distribution for three different values of p . We will also refer to a collection of points in $\mathbb{R}^d$ as a point cloud.



For an examples in $\mathbb{R}^3$ we again consider case where $\vec{\mu} = 0$ and let

$$\Sigma = \begin{bmatrix} 1 & p & p^2 \\ p & 1 & p \\ p^2 & p & 1 \end{bmatrix}.$$

MVN samples for different values of p



In the preceding examples, different choices of p , hence different covariance matrices, resulted in point clouds with different orientations and shapes. Later, we'll discuss how we can determine the shape and orientation from the covariance matrix with the aid of linear algebra. What about changes to $\vec{\mu}$? Changing $\vec{\mu}$ translates the point cloud. If in the preceding examples, we had taken $\vec{\mu} = \vec{1}$ the scatterplots would have had the same shape and orientation, but they would have been translated by a shift of $\vec{1}$.

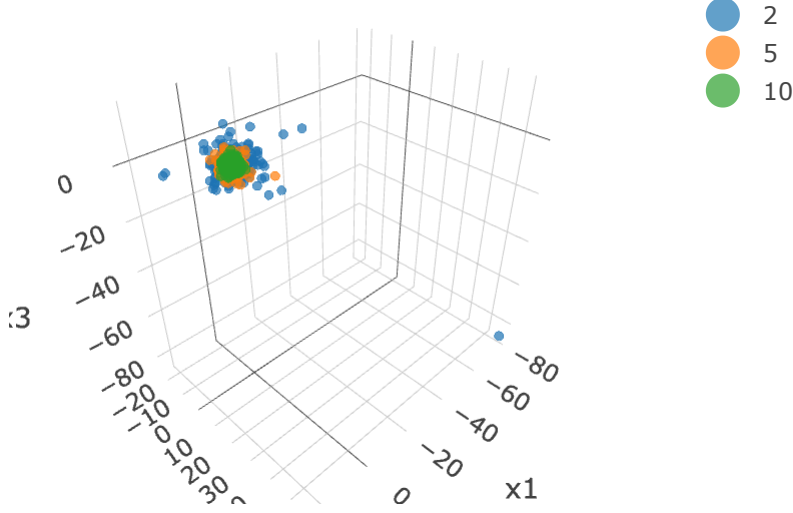
Definition 2.2 (Multivariate t Distribution). The multivariate t-distribution on $\mathbb{R}^d$ with location vector $\vec{\mu} \in \mathbb{R}^d$, positive definite scale matrix $\Sigma \in \mathbb{R}^{d \times d}$ and degrees of freedom ν has density

$$f(\vec{x}) = \frac{\Gamma\left(\frac{\nu+d}{2}\right)}{\Gamma(\nu/2)\nu^{d/2}\pi^{d/2}|\Sigma|^{1/2}} \left[1 + \frac{1}{\nu}(\vec{x} - \vec{\mu})^T \Sigma^{-1}(\vec{x} - \vec{\mu})\right]^{-(\nu+d)/2}.$$

We use shorthand $\vec{x} \sim t_\nu(\vec{\mu}, \Sigma)$ to indicate $\vec{x}$ follows this distribution.

We'll only consider a three dimensional case where the location, which determines the mode of the distribution, is $\vec{0}$ and the scale is the identity matrix. As in the Gaussian case, changing $\vec{\mu}$ translates the point cloud and different values of Σ give point clouds with different shapes. The remaining parameter to consider here is the degrees of freedom, ν , which controls how spread out the samples can

be. We show results for three different choices of the degrees of freedom. For smaller degrees of freedom, there are more points which are far from the mode at $\vec{0}$.



2.3 Expectation, Mean, and Covariance

As in the one-dimensional case, the cumulative distribution function determines the distribution of the random vector, and using the density we may establish a few important quantities which will appear often throughout this text.

The first is the mean or expected value of the random vector which is the vector of expected values of each entry so that

$$E[\vec{x}] = \int_{\mathbb{R}^d} \vec{x} f(\vec{x}) d\vec{x} = \begin{bmatrix} E[x_1] \\ \vdots \\ E[x_d] \end{bmatrix} = \begin{bmatrix} \int_{\mathbb{R}^d} x_1 f(\vec{x}) d\vec{x} \\ \vdots \\ \int_{\mathbb{R}^d} x_d f(\vec{x}) d\vec{x} \end{bmatrix} \quad (2.1)$$

where

$$\int_{\mathbb{R}^d} x_i f(\vec{x}) d\vec{x} = \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} x_i f(x_1, \dots, x_d) dx_1 \dots dx_d.$$

Note, we are assuming each of the integrals in (2.1) is well defined, which is a convention we adhere to throughout this text. Often, we'll often use $\vec{\mu}$ to denote the mean vector. When we are considering more than multiple random vectors $\vec{x}$ and $\vec{y}$ we will add a corresponding subscript $\vec{\mu}_{\vec{x}}$ to denote the corresponding mean of $\vec{x}$.

The linearity of expectation for univariate random vectors holds here as well. If $\vec{x} \in \mathbb{R}^d$ is a random vector, $\mathbf{A} \in \mathbb{R}^{k \times d}$ is a matrix of constant entries, and $\vec{b} \in \mathbb{R}^k$

is a vector of constant entries then

$$E[\mathbf{A}\vec{x} + \vec{b}] = \mathbf{A}\vec{\mu} + \vec{b}.$$

Importantly, for non-squared matrices $\mathbf{A}$ then mean of $\mathbf{A}\vec{x}$ will be of a different dimension than $\vec{x}$.

In general, the coordinates of a random vector will not be independent. To quantify the pairwise dependence, we could consider the covariance

$$\text{Cov}(x_i, x_j) = E[(x_i - \mu_i)(x_j - \mu_j)] = \int_{\mathbb{R}^d} (x_i - \mu_i)(x_j - \mu_j) f(\vec{x}) d\vec{x}$$

for $1 \leq i, j \leq d$. In the case $i = j$, this simplifies to $\text{Cov}(x_i, x_i) = \text{Var}(x_i)$.

Importantly, we do not want to consider each all of the pairwise covariance separately. Instead, we can organize them as a $d \times d$ matrix Σ with entries $\Sigma_{ij} = \text{Cov}(x_i, x_j)$. Hereafter, we will refer to Σ as the covariance matrix of $\vec{x}$. When we are considering multiple random vectors we will use subscripts so that $\Sigma_{\vec{x}}$ and $\Sigma_{\vec{y}}$ denote the covariance matrices of random vectors $\vec{x}$ and $\vec{y}$ respectively. Following the notational conventions, it follows that $\vec{x} - E[\vec{x}] = \vec{x} - \vec{\mu} \in \mathbb{R}^d$ so that the outer product of $\vec{x} - \vec{\mu}$ with itself is the $d \times d$ matrix with entries

$$[(\vec{x} - \vec{\mu})(\vec{x} - \vec{\mu})^T]_{ij} = (x_i - \mu_i)(x_j - \mu_j)$$

so that we may more compactly write

$$\text{Var}(\vec{x}) = E[(\vec{x} - \vec{\mu})(\vec{x} - \vec{\mu})^T] \quad (2.2)$$

where we interpret the expectation operation as applying to each entry of the matrix $(\vec{x} - \vec{\mu})(\vec{x} - \vec{\mu})^T$. This looks very similar to the univariate case save that we must be mindful of the multidimensional nature of our random vector. In fact with some algebra, we have the following alternative formula for the covariance matrix

$$\Sigma = E[\vec{x}\vec{x}^T] - \vec{\mu}\vec{\mu}^T$$

which is again reminiscent of the univariate case. Showing this result is left as a short exercise. One brief note to avoid confusion. Other texts refer to $\text{Var}(\vec{x})$ as the variance matrix or variance-covariance matrix. Herein, we use the term covariance matrix for $\text{Var}(\vec{x})$.

Recall the univariate case,

$$\text{Var}(aX + b) = a^2 \text{Var}(X)$$

for constants a and b and (one-dimensional) random variable X . Similar to the univariate case, there is a formula for the covariance of an affine mapping of a random vector, but the specific form requires us to be mindful of the matrix structure of the covariance matrix. For random vector $\vec{x} \in \mathbb{R}^d$, constant matrix $\mathbf{A} \in \mathbb{R}^{k \times d}$ and constant vector $\vec{b} \in \mathbb{R}^k$, it follows (see exercises) that

$$\Sigma_{\mathbf{A}\vec{x} + \vec{b}} = \mathbf{A}\Sigma\mathbf{A}^T.$$

Importantly, note that $\mathbf{A}\Sigma\mathbf{A}^T$ is a $k \times k$ matrix which is consistent with the fact that $\mathbf{A}\vec{y} + \vec{b}$ is a k -dimensional vector.

Example 2.1 (Mean and Covariance of MVN). If $\vec{x} \sim \mathcal{N}(\vec{\mu}, \Sigma)$ then $E[\vec{x}] = \vec{\mu}$ and $\text{Var}(\vec{x}) = \Sigma$

Example 2.2 (Mean and Covariance of Multivariate t-distribution). Let $\vec{x} \sim t_\nu(\vec{\mu}, \Sigma)$. If $\nu > 1$ then $E[\vec{x}] = \vec{\mu}$; otherwise the mean does not exist. If $\nu > 2$, then $\text{Var}(\vec{x}) = \frac{\nu}{\nu-2}\Sigma$; otherwise, the covariance matrix does not exist.

Verifying these examples is left to the exercises and rely on multivariate change of variables which are not covered here.

2.3.1 Sample Mean and Sample Covariance

In many cases, we'll consider a collection of N *iid* vectors $\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}$. Again, subscripts are used here, but importantly when accompanied by the $\vec{\cdot}$ sign a subscript does not refer to a specific coordinate of a vector but rather one vector in a set. Given *iid* observations $\vec{x}_1, \dots, \vec{x}_N$, we will use sample averages to estimate the expectation and covariance of the data generating distribution. We'll use bars to denote sample averages so that $\bar{x}$ denotes the sample mean and $\bar{\Sigma}$ the sample covariance. In this case, we have

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N \vec{x}_i. \quad (2.3)$$

Similarly, we define the sample covariance matrix to be

$$\bar{\Sigma} = \frac{1}{N} \sum_{i=1}^N (\vec{x}_i - \bar{x})(\vec{x}_i - \bar{x})^T = \left(\frac{1}{N} \sum_{i=1}^N \vec{x}_i \vec{x}_i^T \right) - \bar{x} \bar{x}^T \quad (2.4)$$

In (2.4), dividing by N rather than $N - 1$ yields biased estimates of the terms of the sample covariance matrix. However, the final formula in (2.4) more directly matches the corresponding term in the definition of the covariance matrix. Had we used a factor of $1/(N - 1)$ instead, we would have

$$\frac{1}{N-1} \sum_{i=1}^N (\vec{x}_i - \bar{X})(\vec{x}_i - \bar{X})^T = \left(\frac{1}{N} \sum_{i=1}^N \vec{x}_i \vec{x}_i^T \right) - \frac{N}{N-1} \bar{x} \bar{x}^T$$

which is slightly more cumbersome. In the examples we will consider, N will typically be large enough so that the numerical difference is small. As such, we will opt for algebraically convenient definition form of (2.4) as our definition of the sample covariance matrix.

Alternatively, we can view the sample mean and sample covariance as the mean and covariance (using expectation rather than averages) of the empirical distribution from a collection of samples $\vec{x}_1, \dots, \vec{x}_N$ defined below.

Definition 2.3 (Empirical Distribution). Given a finite set of points $\mathcal{X} = \{\vec{x}_1, \dots, \vec{x}_N\} \subset \mathbb{R}^d$, we say that random vector $\vec{z}$ follows the empirical distribution from data $\mathcal{X}$ if

$$P(\vec{z} = \vec{x}_i) = \frac{1}{N}, \quad i = 1, \dots, N$$

and is zero otherwise.

If $\vec{z}$ follows the empirical distribution on a set of N points $\mathcal{X} = \{\vec{x}_1, \dots, \vec{x}_N\}$, then the expectation and covariance matrix of $\vec{z}$ are equivalent to the sample mean and sample covariance for data $\vec{x}_1, \dots, \vec{x}_N$.

2.3.2 The Data Matrix

Both (2.3) and (2.4) involve summations. Working with sums will prove cumbersome, so briefly let us introduce a more compact method for representing these expressions. Hereafter, we will organize the vectors $\vec{x}_1, \dots, \vec{x}_N$ into a **data matrix**

$$\mathbf{X} = \begin{bmatrix} \vec{x}_1^T \\ \vdots \\ \vec{x}_N^T \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

In this setup, $\mathbf{X}_{ij}$ is the j th coordinate of $\vec{x}_i$, or equivalently, the j th measurement taken from the i th subject. Thus, rows of $\mathbf{X}$ index subjects (realizations of the random vector) whereas columns index common measurements across all subjects. Using the data matrix, we can forgo the summation notation giving the following formulas for the sample mean

$$\bar{x} = \frac{1}{N} \mathbf{X}^T \vec{1} \quad (2.5)$$

and the sample covariance matrix

$$\bar{\Sigma} = \frac{1}{N} (\mathbf{H}\mathbf{X})^T \mathbf{H}\mathbf{X} = \frac{1}{N} \mathbf{X}^T \mathbf{H}\mathbf{X} \quad (2.6)$$

where $\mathbf{H} = \mathbf{I} - \frac{1}{N} \vec{1} \vec{1}^T \in \mathbb{R}^{N \times N}$ is known as the centering matrix. We have used the fact that $\mathbf{H}$ is symmetric and idempotent, e.g. $\mathbf{H}^2 = \mathbf{H}$ which is left as an exercise. The vector $\vec{1}$ is the N -dimensional vector with 1 in each entry and $\mathbf{I}$ is the $N \times N$ identity matrix. One can show (see exercises) that the matrix-vector and matrix-matrix multiplication implicitly handles the summations in (2.3) and (2.4).

To conclude this section, we compare the sample mean and covariance matrix computed from random draws from the *MVN* distribution.

Example 2.3 (Draws from the MVN). We draw $N = 100$ samples from the $\mathcal{N}(\vec{0}, \Sigma)$ distribution where

$$\Sigma = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 9 \end{bmatrix}$$

```

.
N <- 100
X <- mvrnorm(n=N, mu = rep(0,3), Sigma = c(1,4,9)*diag(3))

```

To compute the sample mean and sample covariance, we implement (2.3) and (2.4).

```

xbar <- (1/N) * t(X) %*% rep(1,N)
H <- diag(N) - (1/N)*matrix(1,nrow = N, ncol = N)
S <- (1/N) * t(H %*% X) %*% (H %*% X)

```

The results are shown below (rounded to three decimal places)

$$\bar{x} = \begin{bmatrix} 0.008 \\ 0.078 \\ -0.239 \end{bmatrix} \quad \text{and} \quad \bar{\Sigma} = \begin{bmatrix} 1.148 & 0.216 & 0.121 \\ 0.216 & 4.181 & 0.087 \\ 0.121 & 0.087 & 9.366 \end{bmatrix}$$

which are close to the true values. If we increase the sample size to $N = 10^4$ samples, we get estimates which are closer to the true values (shown below).

$$\bar{x} = \begin{bmatrix} 0.001 \\ 0.02 \\ 0.051 \end{bmatrix} \quad \text{and} \quad \bar{\Sigma} = \begin{bmatrix} 0.991 & -0.02 & -0.04 \\ -0.02 & 3.997 & 0.095 \\ -0.04 & 0.095 & 8.992 \end{bmatrix}$$

2.4 Linear Algebra

2.4.1 Assumed Background

This text assumes familiarity with definitions from a standard undergraduate course in linear algebra including but not limited to linear spaces, subspaces, spans and bases, and matrix multiplication. However, we have elected to provide review of some of the most commonly used ideas in the methods we'll cover in the following subsections. For a more thorough treatment of linear algebra, please see [Trefethen and Bau, 1997, Strang [2006]]

2.4.2 Interpretations of Matrix Multiplication

Throughout this text, comfort with common calculations in linear algebra will be very important. Herein, we assume the reader has some exposure to these materials at an undergraduate level including the summation of vectors or matrices. The familiarity with matrix-vector and matrix-matrix multiplication will play a central role as we have already seen in the case of the data matrix formulation of the sample mean and sample covariance. However, rote familiarity with computation will not be sufficient to build intuition for the methods we'll discuss. As such, we'll begin with a review of a few important ways one can view matrix-vector (and matrix-matrix) multiplication which will be helpful

later. Those who feel comfortable with the myriad interpretations of matrix multiplication in terms of linear combinations of the rows and columns may skip to the next section.

Suppose we have matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and vector $\vec{x} \in \mathbb{R}^n$. If we let $\vec{a}_1^T, \dots, \vec{a}_m^T \in \mathbb{R}^n$ denote the rows of $\mathbf{A}$, then the most commonly cited formula for computing $\mathbf{A}\vec{x}$ is

$$\mathbf{A}\vec{x} = \begin{bmatrix} \vec{a}_1^T \\ \vdots \\ \vec{a}_m^T \end{bmatrix} \vec{x} = \begin{bmatrix} \vec{a}_1^T \vec{x} \\ \vdots \\ \vec{a}_m^T \vec{x} \end{bmatrix}$$

wherein we take the inner product of the rows of $\mathbf{A}$ with vector $\vec{x}$. We can expand this definition to matrix-matrix multiplication. If $\mathbf{B} \in \mathbb{R}^{n \times k}$ has columns $\vec{b}_1, \dots, \vec{b}_k$ then

$$(\mathbf{AB})_{ij} = \vec{a}_i^T \vec{b}_j$$

where we take the inner product of the i th row of $\mathbf{A}$ with the j th column of $\mathbf{B}$ to get the ij th entry of $\mathbf{AB}$. This is perfectly reasonable method of computation, but alternative perspectives are helpful, particularly when we consider different factorization of the data matrix in later chapters..

Returning to $\mathbf{A}\vec{x}$, suppose now that $\mathbf{A}$ has columns $\vec{\alpha}_1, \dots, \vec{\alpha}_n$ and $\vec{x} = (x_1, \dots, x_n)^T$, then we may view $\mathbf{A}\vec{x}$ as a linear combination of the columns of $\mathbf{A}$ so that

$$\mathbf{A}\vec{x} = [\vec{\alpha}_1 \mid \cdots \mid \vec{\alpha}_n] \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = x_1 \vec{\alpha}_1 + \cdots + x_n \vec{\alpha}_n = \sum_{j=1}^n x_j \vec{\alpha}_j.$$

Here, we added vertical columns between the vectors $\vec{\alpha}_i$ to make clear that $[\vec{\alpha}_1 \mid \cdots \mid \vec{\alpha}_n]$ is a matrix. We can extend this perspective to see that the columns of $\mathbf{AB}$ are comprised of different linear combinations of the columns of $\mathbf{A}$. Specifically, the j column of $\mathbf{AB}$ is a linear combination of the columns of $\mathbf{A}$ using the entries in the j th column of $\mathbf{B}$. More specifically, the j th column of $\mathbf{AB}$ is the linear combination

$$\sum_{i=1}^n \mathbf{B}_{ij} \vec{\alpha}_i.$$

Our final observations follows by taking these insights on linear combinations of columns and transposing the entire operation. What can we say about the rows of $\mathbf{AB}$? We can rewrite $\mathbf{AB} = (\mathbf{B}^T \mathbf{A}^T)^T$. The columns of $\mathbf{B}^T \mathbf{A}^T$ are linear combinations of the columns of $\mathbf{B}^T$. Since the columns of $\mathbf{B}^T$ are the rows of $\mathbf{B}$, it follows that the rows of $\mathbf{AB} = (\mathbf{B}^T \mathbf{A}^T)^T$ are linear combinations of the rows of $\mathbf{B}$ with weights given by the entries in each row of $\mathbf{A}$ respectively. In mathematical notation, the i th row of $\mathbf{AB}$ is

$$\sum_{j=1}^n \mathbf{A}_{ij} \vec{\beta}_j^T$$

where $\vec{\beta}_1^T, \dots, \vec{\beta}_n^T$ are the rows of $\mathbf{B}$.

2.4.3 Norms and Distances

Throughout this text, we will use $\|\cdot\|$ to denote the usual Euclidean (or ℓ_2) norm, which for a vector, $\vec{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$, is

$$\|\vec{x}\| = \left(\sum_{j=1}^d x_j^2 \right)^{1/2}.$$

We may then define the Euclidean distance between two d -dimension vectors $\vec{x}$ and $\vec{y}$ to be

$$\|\vec{x} - \vec{y}\| = \left(\sum_{j=1}^d (x_j - y_j)^2 \right)^{1/2}.$$

Euclidean distance is the most commonly used notion of distance (or norm or metric) between two vectors, but it is far from the only option. We can consider the general ℓ_p norm

$$\|\vec{x}\|_p = \left(\sum_{j=1}^d x_j^p \right)^{1/p}$$

which coincides with the Euclidean norm for $p = 2$. Two other special cases include $p = 1$ also known as the Manhattan distance and $p = \infty$ also known as the sup-norm

$$\|\vec{x}\|_\infty = \max_{j=1, \dots, d} |x_j|.$$

We can also extend this notions of vector norms to a measure of the norm of a matrix. Two important cases are the ℓ_2 norm of a matrix and the Frobenius norm. For matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, the ℓ_2 norm is

$$\|\mathbf{A}\| = \sup_{\vec{x} \in \mathbb{R}^n \text{ s.t. } \vec{x} \neq \vec{0}} \frac{\|\mathbf{A}\vec{x}\|}{\|\vec{x}\|} = \sup_{\vec{x} \in \mathbb{R}^n \text{ s.t. } \|\vec{x}\|=1} \|\mathbf{A}\vec{x}\|. \quad (2.7)$$

You can interpret $\|\mathbf{A}\|$ as the largest relative change in the Euclidean length of a vector after it is multiplied by $\mathbf{A}$. The Frobenius norm extends the algebraic definition notion of Euclidean length to a matrix. For $\mathbf{A} \in \mathbb{R}^{m \times n}$, its Frobenius norm is

$$\|\mathbf{A}\|_F = \left(\sum_{i=1}^m \sum_{j=1}^n \mathbf{A}_{ij}^2 \right)^{1/2}. \quad (2.8)$$

The ℓ_2 distance between two matrices is then $\|\mathbf{A} - \mathbf{B}\|$ and the Frobenius distance between two matrices is $\|\mathbf{A} - \mathbf{B}\|_F$ (where both $\mathbf{A}$ and $\mathbf{B}$ have the same number of rows and columns).

2.4.4 Important properties

A few additional definitions that we will use throughout the text are provided below without examples.

Definition 2.4 (Symmetric Matrix). Matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$ is symmetric if $\mathbf{A} = \mathbf{A}^T$.

Definition 2.5 (Eigenvectors and Eigenvalues). Let $\mathbf{A} \in \mathbb{R}^{d \times d}$. If there is a scalar λ and vector $\vec{x} \neq \vec{0}$ such that $\mathbf{A}\vec{x} = \lambda\vec{x}$ then we say λ is an eigenvalue of $\mathbf{A}$ with associated eigenvector $\vec{x}$.

2.4.5 Matrix Factorizations

Two different matrix factorization will arise many times throughout the text. The first, which is commonly presented in linear algebra courses, is the spectral decomposition of a square matrix which is also known as diagonalization or eigenvalue decomposition. Herein, we assume familiarity with eigenvalues and eigenvectors. The second factorization is the singular value decomposition. In the subsequent subsections, we briefly discuss these two factorizations, their geometric interpretation, and some notation that will typically be used in each case.

2.4.5.1 Eigenvalue Decomposition

We begin with the eigenvalue decomposition of a square matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$. As you may recall, $\mathbf{A}$ will have a set of d eigenvalues $\lambda_1, \dots, \lambda_d$ (which may include repeated values) and associated eigenvectors. A number, λ , may be repeated in the list of eigenvalues, and the number of times is called the algebraic multiplicity of λ . Each eigenvalue has a least one eigenvector. In cases where the eigenvalue has algebraic multiplicity greater than one, we refer to its geometric multiplicity as the number of linearly independent eigenvectors associated with the eigenvalue.

The algebraic multiplicity is always greater than or equal to the geometric multiplicity. However, this is not always the case, and when this occurs, the matrix cannot be diagonalized. Fortunately, we will largely be dealing with symmetric matrices for which diagonalization is guaranteed by the following theorem.

Theorem 2.1 (Spectral Decomposition Theorem for Symmetric Matrices). *Any symmetric matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$ can be written as*

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$$

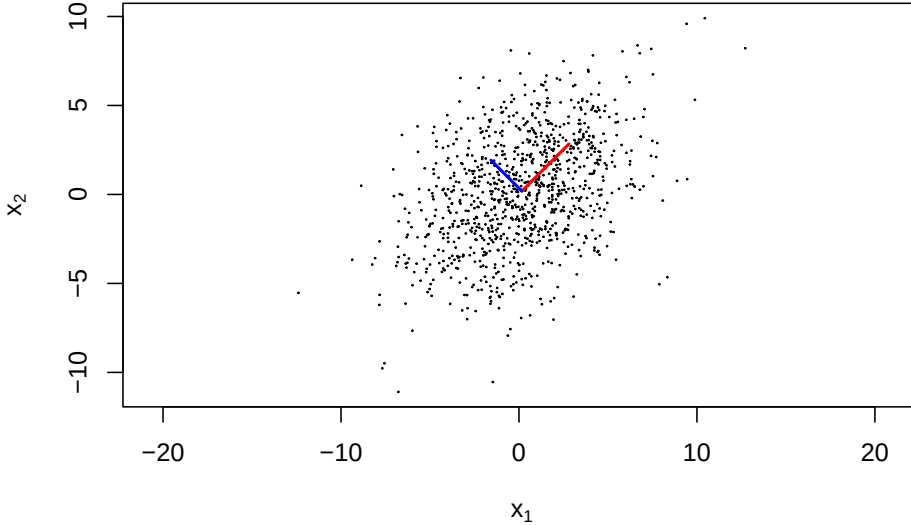
where $\mathbf{U} \in \mathbb{R}^{d \times d}$ is an orthonormal matrix and $\mathbf{\Lambda}$ is a diagonal matrix

$$\mathbf{\Lambda} = \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \lambda_d \end{bmatrix}$$

where the scalars $\lambda_1, \dots, \lambda_d \in \mathbb{R}$ are the eigenvalues of $\mathbf{A}$ and the corresponding columns of $\mathbf{U}$ are their associated eigenvectors.

By convention, we will always assume the eigenvalues are in decreasing order so that $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$. The most common types of symmetric matrices that we will encounter are covariance matrices. In those cases, the spectral decomposition of the covariance can provide some helpful insight about the shape generated by many *iid* samples from a distribution. We demonstrate this idea graphically in the following examples using the MVN.

Example 2.4 (Level curves of MVN in). A scatterplot of 1000 *iid* samples from two-dimensional Gaussian distribution with mean $\vec{0}$ and covariance $\begin{bmatrix} 10 & 2 \\ 2 & 10 \end{bmatrix}$ is shown below. The eigenvector associated with the largest eigenvalue of the sample covariance matrix is shown in red, whereas the eigenvector shown associated with the second eigenvalue is shown in blue. Note, both eigenvectors have been rescaled so their length is the square root of the associated eigenvalues.



Importantly, observe how these eigenvectors follow the spread of the data. As we will see in 4.1, this idea can be used to compress data in a geometrically interpretable manner through PCA.

2.4.5.2 Singular Value Decomposition

The spectral theorem is limited in that it requires a matrix to be both square and symmetric. When focusing on data matrices $\mathbf{X} \in \mathbb{R}^{N \times d}$ both assumptions are extremely unlikely to be satisfied, and we will need a more flexible class of methods. This idea is explored in much greater detail in Chapter 4. For now, we briefly introduce the Singular Value Decomposition and how this factorization provides some insight on the geometric structure of matrix-vector multiplication.

For a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ the mapping $T_{\mathbf{A}} : \mathbb{R}^n \mapsto \mathbb{R}^m$ defined as $T_{\mathbf{A}}(\vec{x}) = \mathbf{A}\vec{x}$ for $\vec{x} \in \mathbb{R}^n$. Since this mapping is linear, we can understand its geometric nature by investigating how the unit sphere in $\mathbb{R}^n$ is transformed by the mapping $T_{\mathbf{A}}$. More directly, let's suppose we took every vector on the unit sphere $\mathcal{S} = \{\vec{x} \in \mathbb{R}^n : \|\vec{x}\| = 1\}$ and computed the vector $\mathbf{A}\vec{x}$. The result is a hyperellipse in $\mathbb{R}^m$. The specifics on the orientation and shape of the hyperellipse depend on the matrix $\mathbf{A}$ which we demonstrate through a brief example.

Example 2.5. In Figure 2.1, we show the unit sphere in $\mathbb{R}^2$ and the resulting hyperellipse in $\mathbb{R}^3$ after multiplication by the matrix

$$\mathbf{A} = \begin{bmatrix} 3 & 2 \\ 2 & 1 \\ 1 & 0 \end{bmatrix}.$$

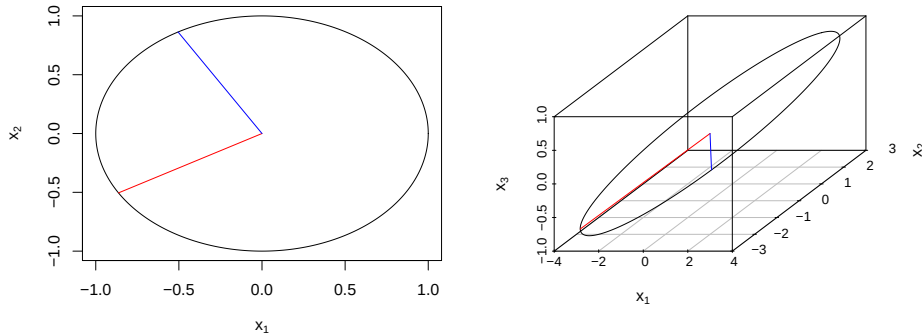
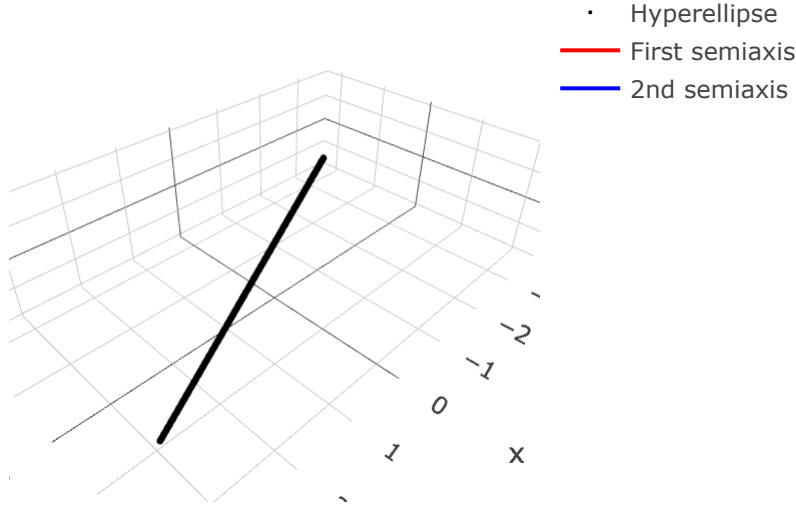


Figure 2.1: (Left) Unit sphere in 2D, (Right) Hyperellipse that results after multiplication by $\mathbf{A}$

We have also added two vectors. On the hyperellipse, the red vector is the longest semi-axis of the ellipse; the blue vector is the shortest semi-axis. The red and blue vectors on the left are the preimages of the semi-axes of the ellipse (those vectors which get mapped to the semi-axes after multiplication with $\mathbf{A}$). It may not be clear from the static images above, but the semi-major axes on the right are orthogonal. We provide an interactive plot of hyperellipse below so that you can better see this result. Interestingly, the preimages on the left are also perpendicular. Since they are on the unit sphere, they are also orthonormal.



In the preceding example, we have made a few important observations which generalize to any multiplication by a matrix, $\mathbf{A} \in \mathbb{R}^{m \times n}$. For now, assume that $m > n$ and $\mathbf{A}$ is full rank. In this case, the hyperellipse $\mathbf{A}\mathcal{S} = \{\mathbf{A}\vec{x} : \vec{x} \in \mathcal{S}\}$ has orthogonal semi-axes $\sigma_1\vec{u}_1, \dots, \sigma_n\vec{u}_n$ with corresponding orthonormal preimages $\vec{v}_1, \dots, \vec{v}_n$. We use the assumption that the lengths of the axes are listed in decreasing order so that $\sigma_1 \geq \dots \geq \sigma_n$. We may express these mappings in the following manner

$$\underbrace{\mathbf{A} \begin{bmatrix} \vec{v}_1 & \dots & \vec{v}_n \end{bmatrix}}_{\tilde{\mathbf{V}}} = \begin{bmatrix} \sigma_1\vec{u}_1 & \dots & \sigma_n\vec{u}_n \end{bmatrix} = \underbrace{\begin{bmatrix} \vec{u}_1 & \dots & \vec{u}_n \end{bmatrix}}_{\tilde{\mathbf{U}}} \underbrace{\begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \sigma_n \end{bmatrix}}_{\tilde{\mathbf{S}}} \quad (2.9)$$

Since $\tilde{\mathbf{V}} \in \mathbb{R}^{n \times n}$ has orthonormal columns so that $\tilde{\mathbf{V}}^{-1} = \tilde{\mathbf{V}}^T$, we arrive at the following decomposition for $\mathbf{A}$.

Definition 2.6 (The Singular Value Decomposition (SVD)). Let $\mathbf{A} \in \mathbb{R}^{m \times n}$. Then $\mathbf{A}$ may be factored as

$$\mathbf{A} = \tilde{\mathbf{U}}\tilde{\mathbf{S}}\tilde{\mathbf{V}}^T \quad (2.10)$$

where $\tilde{\mathbf{U}} \in \mathbb{R}^{m \times n}$ and $\tilde{\mathbf{V}} \in \mathbb{R}^{n \times n}$ have orthonormal columns and $\tilde{\mathbf{S}} \in \mathbb{R}^{n \times n}$ is a diagonal matrix with real entries $\sigma_1 \geq \dots \geq \sigma_n \geq 0$ along the diagonal. We refer to (2.10) as the reduced singular value decomposition of $\mathbf{A}$. The columns of $\tilde{\mathbf{U}}$ ($\tilde{\mathbf{V}}$) are called the left (right) singular vectors of $\mathbf{A}$ and the scalars $\sigma_1 \geq \dots \geq \sigma_n$ are referred to as the singular values of $\mathbf{A}$.

Let $\vec{u}_1, \dots, \vec{u}_n \in \mathbb{R}^m$ be the columns of $\tilde{\mathbf{U}}$. When $m > n$, we may find additional vectors $\vec{u}_{n+1}, \dots, \vec{u}_m$ such that $\{\vec{u}_1, \dots, \vec{u}_m\}$ are an orthonormal basis for $\mathbb{R}^m$.

This gives the (full) singular value decomposition of $\mathbf{A}$ is

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T \quad (2.11)$$

where

$$\mathbf{U} = [\vec{u}_1 \quad \dots \quad \vec{u}_m] \in \mathbb{R}^{m \times m} \quad \mathbf{V} = [\vec{v}_1 \quad \dots \quad \vec{v}_n] \in \mathbb{R}^{n \times n}$$

are orthonormal matrices. The matrix $\mathbf{S} \in \mathbb{R}^{m \times n}$ is formed by taking $\tilde{S}$ and padding it with zero along the bottom and right so that it is $m \times n$.

We may drop the preceding assumptions that $m > n$ or the $\mathbf{A}$ is full rank. In fact, **every** matrix has a singular value decomposition. A proof of this fact using induction may be found in [Trefethen and Bau, 1997]. Furthermore, a few important matrix properties are directly related to the SVD.

- 1) The rank of a matrix is equal to its number of positive singular values
- 2) The column space of a matrix is the span of its first $\text{rank}(\mathbf{A})$ left singular vectors
- 3) The kernel of a matrix is the span of its last $n - \text{rank}(\mathbf{A})$ right singular vectors.

When considering matrix multiplication $\mathbf{A}\vec{x}$, the full SVD of $\mathbf{A}$ is helpful for decomposition this matrix multiplication into three more interpretable steps: a rotation/reflection, a stretch/compression, and a final rotation/reflection.

$$\mathbf{A}\vec{x} = \mathbf{U} \underbrace{\underbrace{\mathbf{D}(\mathbf{V}^T \vec{x})}_{\text{rot/ref}}}_{\text{str/com}}_{\text{rot/ref}} \quad (2.12)$$

2.4.6 Positive Definiteness and Matrix Powers

Finally, we review a few brief properties for square matrices.

Definition 2.7 (Positive Definite Matrices). A matrix, $\mathbf{A} \in \mathbb{R}^{n \times n}$ is positive definite if $\vec{x}^T \mathbf{A} \vec{x} > 0$ for all $\vec{x} \neq 0$. Equivalently, $\mathbf{A}$ is positive definite if all of its eigenvalues are positive.

A matrix is positive semidefinite if $\vec{x}^T \mathbf{A} \vec{x} \geq 0$ for all $\vec{x} \neq 0$. Equivalently, $\mathbf{A}$ is positive semidefinite if all of its eigenvalues are non-negative.

One may interpret the powers of a matrix as repeated multiplication of a square matrix. More specifically, for any $m \in \{1, 2, \dots\}$, let $\mathbf{A}^m = \underbrace{\mathbf{A} \dots \mathbf{A}}_{m \text{ times}}$. For

diagonalizable positive definite matrices, we may extend this interpretation to negative and fractional power using the eigenvalue decomposition. If the eigenvalue decomposition of the $\mathbf{A}$ is $\mathbf{A} = \mathbf{W}\mathbf{\Lambda}\mathbf{W}^{-1}$ for some diagonal matrix

Λ with positive entries, $\lambda_1, \dots, \lambda_n$ along its diagonal. For any $s \in \mathbb{R}$, we define $\mathbf{A}^s = \mathbf{W}\Lambda^s\mathbf{W}^{-1}$ where

$$\Lambda^s = \begin{bmatrix} \lambda_1^s & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \lambda_n^s \end{bmatrix}.$$

2.4.7 Hadamard (elementwise) operations

Given two matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$ with the same number of rows and columns, we define Hadamard product and division as

$$\mathbf{A} \odot \mathbf{B} \in \mathbb{R}^{m \times n} \text{ such that } (\mathbf{A} \odot \mathbf{B})_{ij} = \mathbf{A}_{ij} \mathbf{B}_{ij}$$

$$\mathbf{A} \oslash \mathbf{B} \in \mathbb{R}^{m \times n} \text{ such that } (\mathbf{A} \oslash \mathbf{B})_{ij} = \frac{\mathbf{A}_{ij}}{\mathbf{B}_{ij}}$$

Unlike standard matrix multiplication, the Hadamard product $\odot$ is commutative such that $\mathbf{B} \odot \mathbf{A} = \mathbf{A} \odot \mathbf{B}$. Commutativity does not hold for Hadamard division however.

Furthermore, we define Hadamard powers as

$$\mathbf{A}^{\circ k} \in \mathbb{R}^{m \times n} \text{ such that } (\mathbf{A}^{\circ k})_{ij} = (\mathbf{A}_{ij})^k.$$

For the special case $k = -1$, we refer to $\mathbf{A}^{\circ(-1)}$ as the Hadamard inverse.

2.5 Summary

This chapter built the mathematical vocabulary the rest of the book relies on, but it is worth pausing to notice that almost every piece of it is secretly geometric. A random vector's covariance matrix is not just a table of variances and covariances – its eigendecomposition $\Sigma = \mathbf{Q}\Lambda\mathbf{Q}^T$ describes an ellipsoid, with the eigenvectors giving the ellipsoid's axes and the eigenvalues giving their lengths. This picture is what makes a multivariate Gaussian's density contours literal ellipses (or ellipsoids) in $\mathbb{R}^d$, and it is the same picture we will lean on repeatedly once we start asking which directions in a dataset carry the most information.

Quadratic forms $\vec{x}^T \mathbf{A} \vec{x}$ generalize this idea beyond a fixed distribution: any symmetric matrix $\mathbf{A}$ defines a quadratic surface, and positive semidefiniteness is exactly the statement that this surface curves upward in every direction, never downward. We will see this exact condition reappear as Mercer's condition for a valid kernel (5.1) and as the reason a graph Laplacian's eigenvalues are always nonnegative (5.6) – two settings that look unrelated to covariance matrices at first glance, but rest on the same underlying geometric fact. Gradients and Hessians extend this local, curvature-based way of thinking to arbitrary smooth functions, and will return in force when we discuss how a function behaves on a curved manifold rather than a flat Euclidean space (5.7).

The throughline is this: almost everything unsupervised learning does is a statement about the *shape* of data – its spread, its curvature, its local structure – and the algebra in this chapter is simply the language for making those shapes precise. Keep that in mind as a guide going forward, since the specific tool (an eigendecomposition, a norm, a trace) will change from method to method, but the goal of describing geometric structure will not.

2.6 Exercises

2.6.1 Probability

1. Given a random vector $\vec{x} = (\mathbf{x}_1, \mathbf{x}_2)^T \sim \mathcal{N}(\vec{\mu}, \Sigma)$ where:

$$\vec{\mu} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 1 & 0 \\ 0 & 10 \end{pmatrix}$$

- a. What are the means and variances of the individual components x_1 and x_2 ?
 - b. What is the covariance and correlation between x_1 and x_2 ?
 - c. Are $\mathbf{x}_1$ and $\mathbf{x}_2$ independent?
2. Given a random vector $\vec{x} = (x_1, x_2)^T \sim \mathcal{N}(\vec{\mu}, \Sigma)$ where:

$$\vec{\mu} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

- a. What are the means and variances of the individual components $\mathbf{x}_1$ and $\mathbf{x}_2$?
 - b. What is the covariance and correlation between $\mathbf{x}_1$ and $\mathbf{x}_2$?
 - c. Are $\mathbf{x}_1$ and $\mathbf{x}_2$ independent?
3. Given a random vector $\vec{x} = (x_1, x_2) \sim t_{10}(\vec{\mu}, \Sigma)$ where:

$$\vec{\mu} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 1 & 0 \\ 0 & 10 \end{pmatrix}$$

- a. What are the means and variances of the individual components $\mathbf{x}_1$ and $\mathbf{x}_2$?
 - b. What is the covariance and correlation between $\mathbf{x}_1$ and $\mathbf{x}_2$?
 - c. Are $\mathbf{x}_1$ and $\mathbf{x}_2$ independent?
4. Given a random vector $\vec{x} = (x_1, x_2) \sim t_{10}(\vec{\mu}, \Sigma)$ where:

$$\vec{\mu} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

- a. What are the means and variances of the individual components $\mathbf{x}_1$ and $\mathbf{x}_2$?
- b. What is the covariance and correlation between $\mathbf{x}_1$ and $\mathbf{x}_2$?
- c. Are $\mathbf{x}_1$ and $\mathbf{x}_2$ independent?

2.6.2 Calculus

1. Let $f(\vec{x}) = \vec{x}^T \mathbf{A} \vec{x}$ for a vector $\vec{x} \in \mathbb{R}^d$ and a matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$ which is constant. Give expressions for ∇f and $\nabla^2 f$ using only matrices and vectors (no summation notation is allowed).
2. Let $\vec{x} \in \mathbb{R}^m$ and $\vec{y} \in \mathbb{R}^n$. Compute the gradient of $\|\vec{x}\vec{y}^T\|_F^2$ with respect to $\vec{x}$.
3. Consider the function

$$f(x_1, x_2, x_3) = 2x_1^2 + 3x_2^2 + 4x_3^2 + 5x_1x_2 + 6x_1x_3 + 7x_2x_3 + 8x_1 + 9x_2 + 10x_3 + 11$$

- a. Find the critical point(s) of f . Hint: it may be helpful to express f in the form $f(\vec{x}) = \vec{x}^T \mathbf{A} \vec{x} + \vec{b}^T \vec{x} + c$ for some matrix $\mathbf{A}$, vectors $\vec{b}$, $\vec{x} = (x_1, x_2, x_3)^T \in \mathbb{R}^3$ and scalar c .
- b. Classify the critical point(s) found in part (a) as minima, maxima, or saddle points. Explain your reasoning.
4. Verify that the mean and variance of the $\mathcal{N}(\vec{\mu}, \Sigma)$ distribution are indeed $\vec{\mu}$, Σ respectively by computing the integrals directly.
5. Verify that the mean and variance of the $t_\nu(\vec{\mu}, \Sigma)$ distribution by computing the integrals directly. You may assume that $\nu > 2$.

2.6.3 Linear Algebra

1. Given a matrix $\mathbf{A} \in \mathbb{R}^{3 \times 3}$ and a vector $\mathbf{x} \in \mathbb{R}^3$, where:

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}$$

- a. Compute the product $\mathbf{A}\mathbf{x}$ using the column interpretation of matrix-vector multiplication.
- b. Compute the product $\mathbf{A}\mathbf{x}$ using the row interpretation of matrix-vector multiplication.
2. Given matrices $\mathbf{A} \in \mathbb{R}^{2 \times 3}$ and $\mathbf{B} \in \mathbb{R}^{3 \times 2}$, where:

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

- a. Compute the product $\mathbf{C} = \mathbf{AB}$ using the dot product interpretation of matrix-matrix multiplication.
 - b. Compute the product $\mathbf{C} = \mathbf{AB}$ using the column interpretation of matrix-matrix multiplication.
 - c. Compute the product $\mathbf{C} = \mathbf{AB}$ using the row interpretation of matrix-matrix multiplication.
3. Let $\mathbf{A} \in \mathbb{R}^{m \times n}$.
- a. Explain why $\mathbf{AA}^T$ and $\mathbf{A}^T\mathbf{A}$ are diagonalizable and positive semi-definite.
 - b. Give expressions for the singular vectors and singular values of $\mathbf{A}$ in terms of the eigenvectors and eigenvalues of $\mathbf{AA}^T$ and $\mathbf{A}^T\mathbf{A}$.
4. Let

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

- a. Show that $\mathbf{A}^2 \neq \mathbf{A}^{\circ 2}$.
- b. Under what conditions does $\mathbf{A}^k = \mathbf{A}^{\circ k}$ for a square matrix $\mathbf{A}$?

2.6.4 Hybrid Problems

1. Consider the data matrix

$$\mathbf{X} = \begin{bmatrix} 2 & 3 \\ 4 & 5 \\ 6 & 7 \\ 8 & 9 \end{bmatrix}.$$

- a. Compute the sample mean of the data.
 - b. Provide the centered data matrix.
 - c. Compute the sample covariance matrix of $\mathbf{X}$.
2. Consider a random vector $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ where:

$$\boldsymbol{\mu} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}.$$

Let $\mathbf{Y} = \mathbf{AX} + \mathbf{b}$ where:

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

- a. What is the mean vector $\boldsymbol{\mu}_{\mathbf{Y}}$ of $\mathbf{Y}$?
- b. What is the covariance matrix $\boldsymbol{\Sigma}_{\mathbf{Y}}$ of $\mathbf{Y}$?

3. Given a random vector $\vec{x}$ with mean $\vec{\mu}$ and covariance matrix $\Sigma = E[(\vec{x} - \vec{\mu})(\vec{x} - \vec{\mu})^T]$ verify the identity $\Sigma = E[\vec{x}\vec{x}^T] - \vec{\mu}\vec{\mu}^T$.
4. Suppose $\vec{x} \in \mathbb{R}^n$ is a random vector with covariance $\Sigma \in \mathbb{R}^{n \times n}$, and let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be a matrix with constant entries (non-random). Show that the covariance matrix of $\mathbf{A}\vec{x}$ is $\mathbf{A}\Sigma\mathbf{A}^T$.
5. Show that the $N \times N$ centering matrix $\mathbf{H} = \mathbf{I} - \frac{1}{N}\mathbf{1}\mathbf{1}^T$ is idempotent, i.e. $\mathbf{H}^2 = \mathbf{H}$.
6. Consider the data matrix

$$\mathbf{X} = \begin{bmatrix} \vec{x}_1^T \\ \vdots \\ \vec{x}_N^T \end{bmatrix}.$$

Show that

$$\mathbf{H}\mathbf{X} = \begin{bmatrix} \vec{x}_1^T - \bar{x}^T \\ \vdots \\ \vec{x}_N^T - \bar{x}^T \end{bmatrix}$$

where $\mathbf{H}$ is the centering matrix and $\bar{x}$ is the sample mean of vectors $\vec{x}_1, \dots, \vec{x}_N$.

7. For data $\vec{x}_1, \dots, \vec{x}_N$ with sample mean

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^N \vec{x}_i$$

and sample covariance

$$\hat{\Sigma} = \frac{1}{N} (\vec{x}_i - \hat{\mu})(\vec{x}_i - \hat{\mu})^T$$

verify the following identities

- a. $\hat{\Sigma} = \frac{1}{N} \left(\sum_{i=1}^N \vec{x}_i \vec{x}_i^T \right) - \hat{\mu} \hat{\mu}^T$.
- b. $\hat{\Sigma} = \mathbf{X}^T \mathbf{H} \mathbf{X}$ where $\mathbf{X}$ is the data matrix formed from vectors $\vec{x}_1, \dots, \vec{x}_N$.

Chapter 3

Central goals and assumptions

In the remainder of this text, we will largely focus on the case where we are given a dataset containing samples $\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}^d$. We will assume that the vectors were drawn independently from some unknown data generating process. As we discussed briefly in Chapter 1, in UL we want to learn important relationships within the dataset that can provide a simplified but meaningful summary of the data. The central assumption to UL is that such structure exists, though the specifics vary depending on the setting.

The two largest areas of focus herein are dimension reduction/manifold learning and clustering which can both be used for feature engineering, data compression, and exploratory data analysis. Dimension reduction is also commonly used for visualization.

3.1 Dimension reduction and manifold learning

Algorithms for dimension reduction and manifold learning have a number of different applications which are all based on the Manifold Hypothesis

Hypothesis 3.1 (Manifold Hypothesis). The points in a high-dimensional dataset live on a latent low-dimension surface (also called a manifold).

The manifold hypothesis implies that the dataset can be described by a much smaller number of dimensions. Determining the manifold structure and a simplified set of coordinates for each point on the manifold is the central goal of these algorithms.

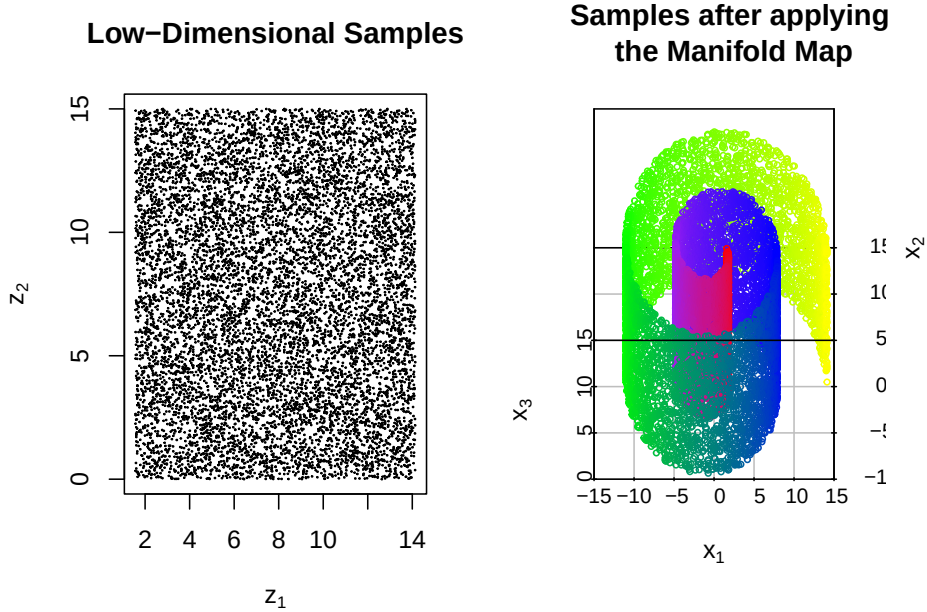
A simple mechanism for generating data on a manifold makes the hypothesis above concrete. Assume there are points $\vec{z}_1, \dots, \vec{z}_N \in A \subset \mathbb{R}^t$ which are *iid*

random samples from some probability distribution. These points are (nonlinearly) mapped into a higher dimensional space $\mathbb{R}^d$ by a smooth map Ψ giving data $\vec{x}_i = \Psi(\vec{z}_i)$ for $i = 1, \dots, N$. Hereafter, we refer to Ψ as the manifold map. In this setting, we are only given $\vec{x}_1, \dots, \vec{x}_N$, and we want to recover the lower-dimensional $\vec{z}_1, \dots, \vec{z}_N$. If possible, we would also like recover Ψ and Ψ^{-1} and in the most ideal case, the sampling distribution that generated the lower-dimensional coordinates $\vec{z}_1, \dots, \vec{z}_N$.

Example 3.1 (Mapping to the Swiss Roll). Let $A = (\pi/2, 9\pi/2) \times (0, 15)$. We define the map $\Psi : A \rightarrow \mathbb{R}^3$ as follows

$$\Psi(\vec{z}) = \Psi(z_1, z_2) = \begin{bmatrix} z_1 \sin(z_1) \\ z_1 \cos(z_1) \\ z_2 \end{bmatrix}$$

Below we show $N = 10^4$ *iid* samples which are drawn uniformly from A . We then show the resulting observations after applying map Ψ to each sample.



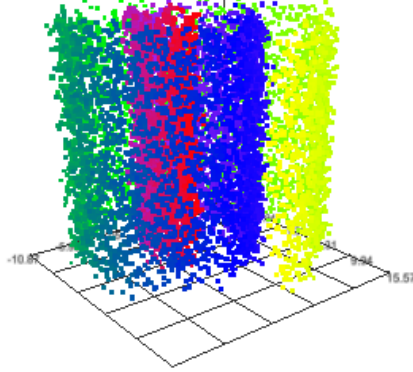
We may also consider the more complicated case where the observations are corrupted by additive noise. In this setting, the typical assumption is that the noise follows after the manifold map so that our data are

$$\vec{x}_i = \Psi(\vec{z}_i) + \vec{\epsilon}_i, \quad i = 1, \dots, N$$

for some *iid* noise vectors $\{\vec{\epsilon}_i\}_{i=1, \dots, N}$.

Example 3.2 (Swiss Roll with Additive Gaussian Noise). Here, we perturb the observations in the preceding example with additive $\mathcal{N}(\vec{0}, 0.1\mathbf{I})$ noise.

Swiss Roll Data Perturbed by Additive Noise



In addition to the goals in the noiseless case, we may also add the goal of learning the noiseless version of the data which reside on a manifold.

However, there are a number of practical issues to this setup. First, the dimension, t , of the original lower-dimensional points is typically unknown. Similar to previous methods, we could pick a value of t with the goal of visualization, base our choice off of prior knowledge, or run our algorithms different choices of t and compare the results. More advanced methods for estimating the true value of t are an open area of research [Campadelli et al., 2015].

There is also a issue with the uniqueness of this problem statement. Given only the high dimensional observations, there is no way we could identify the original lower-dimensional points without more information. In fact, one could find an unlimited number of equally suitable results.

Let $\Phi : \mathbb{R}^t \rightarrow \mathbb{R}^d$ be an invertible function. As an example, you could think of Φ as defining a translation, reflection, rotation, or some composition of these operations – we will see a similar identifiability issue when we discuss classical scaling and dimension reduction more generally later in this book. If our original observed data are $\vec{x}_i = \Psi(\vec{z}_i)$, our manifold learning algorithm could instead infer that the manifold map is $\Psi \circ \Phi^{-1}$ and the lower-dimensional points are $\Phi(\vec{z}_i)$. This is a perfectly reasonable result since $(\Psi \circ \Phi^{-1}) \circ \Phi(\vec{z}_i) = \Psi(\vec{z}_i) = \vec{x}_i$ for $i = 1, \dots, N$, which is the only result we require. Without additional information, there is little we could do to address this issue. For the purposes of visualization, however, we will typically be most interested in the relationship between the lower-dimensional points rather than their specific location or orientation. As such, we need not be concerned about a manifold learning algorithm that provides a translated or rotated representation of $\vec{z}_1, \dots, \vec{z}_N$. More complicated transformations of the lower-dimensional coordinates are of greater concern and may be addressed through additional assumptions about the manifold map Ψ .

We will review a small collection of different methods which address the manifold learning problem in Chapter 5. Many of these methods are motivated based on important concepts from differential geometry, the branch of mathematics focused on manifolds. Many of the details of differential geometry are beyond the scope of this book, so we will focus on a few key ideas when we return to this topic. For the more mathematically-minded reader, see [Spivak, 1979, Lee, 2019].

3.2 Clustering

Hypothesis 3.2 (Clustering Hypothesis). The points in a dataset can be grouped into well defined subsets. Points in each subset are similar and points in different subsets are not.

The cluster hypothesis suggests there are non-overlapping regions. Within each region there are (many) similar points, and there are no points whose are comparably similar to those in more than one subset.

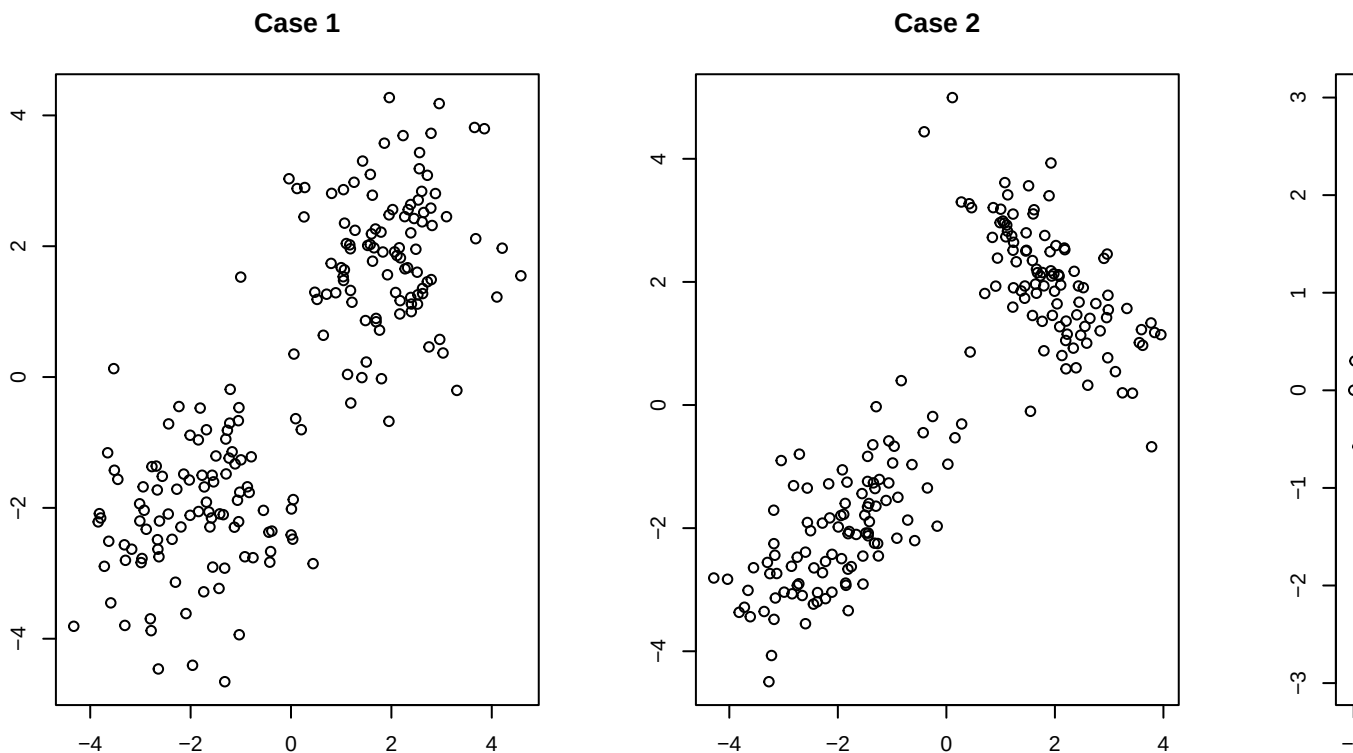
Turning this hypothesis into a working method for finding clusters ourselves raises a number of questions:

- i) how many clusters (if any) exist?
- ii) what do we mean by similar observations?

As we will see through a series of examples, these two points are linked. Given a notion of similarity (which is implicitly defined for some clustering algorithms and tuneable for others), a certain number of clusters and clustering of data may be optimal. Given a different notion of similarity, an entirely different organization of the data into a different number of clusters may arise.

For now, let us focus on the latter question regarding similarity. Consider the following three examples of data sets in $\mathbb{R}^2$. Visually, how would you cluster the observations? In particular, how many subgroups would you say exist and how would you partition the data into these subgroups?

Example 3.3 (Different notions of similar subgroups).



The questions posed above can be reasonably answered based on the figures but will not be so easy in high dimensions. We'll explore these questions and discuss data driven ways to estimate the appropriate number of clusters for a specific algorithm in Chapter 6.

3.3 Generating synthetic data

3.3.1 Data on manifolds

At the beginning of this chapter, we indicated that our data are drawn from some unknown distribution. This is a practical assumption, but in many cases, it is also helpful to consider examples where we generate the data ourselves. In doing so, we can create whatever complicated structure we would like such as different clustering arrangements or lower dimensional structure. We can test an Unsupervised Learning algorithm of interest on these synthetically generated data to see if important relationships or properties are accurately preserved. This is a helpful method for evaluating how well an algorithm works in a specific case, and importantly, can be used to build intuition on a number of natural complexities such as appropriately choosing tuning parameters, evaluating the effects of noise, and seeing how these algorithms may break when certain assumptions are not met.

First, let us consider the case of generating data with a known lower dimensional structure which will be valuable when testing a dimension reduction or manifold learning algorithm. We'll begin with data on a hyperplane. Later in Chapter 4, we consider data on a hyperplane with additional constraints which can be generated by small changes to the method discussed below.

Example 3.4 (Generating data on a hyperplane). Suppose we want to generate a set of d -dimensional data which is on a $k < d$ dimensional hyperplane. The span of k linearly independent vectors $\vec{z}_1, \dots, \vec{z}_k \in \mathbb{R}^d$ defines a k dimensional hyperplane. If we then generated random coefficient $c_1, \dots, c_k$, then the vector

$$\vec{x} = c_1 \vec{z}_1 + \dots + c_k \vec{z}_k$$

would be an element on this hyperplane. To generate a data set we could then

- 1) Specify $\vec{z}_1, \dots, \vec{z}_k$ or generate them randomly
- 2) Draw random coefficients $c_1, \dots, c_k$ and compute the random sample $\vec{x} = c_1 \vec{z}_1 + \dots + c_k \vec{z}_k$.
- 3) Repeat step 2, N times to generate the N samples.

In Figure 3.1, we show an example of data generated to reside on a random plane in $\mathbb{R}^3$. We first generate $\vec{z}_1, \dots, \vec{z}_k$ randomly by drawing each vector from a $\mathcal{N}(\vec{0}, \mathbf{I})$ distribution. These vectors will be independent with probability 1. When then take coefficients $c_1, \dots, c_k$ which are **iid** $N(0, 1)$.

Generating data on a curved surface is generally more complicated. In some cases, the curved surface is defined implicitly via a constraint such as the unit sphere in d -dimensions

$$S^{d-1} = \{\vec{x} \in \mathbb{R}^d : \|\vec{x}\| = 1\}.$$

Generating data on the unit sphere can then be accomplished by drawing a vector from any distribution on $\mathbb{R}^d$ then rescaling the vector to have unit length. Different choices of the original distribution will result in different distributions over the unit sphere.

Alternatively, we could consider a function which parameterizes a curve or surface. We show one such example below.

Example 3.5 (Generating data on a Ribbon). The function

$$R(s) = (\cos(2\pi s), \sin(2\pi s), 2s)^T$$

maps the interval $(0, 5)$ to a helix in $\mathbb{R}^3$. For a given choice of s , if we let the third coordinate vary from $2s$ to $2s + 1$ we would then trace out a ribbon in $\mathbb{R}^3$. To do this, let's add a second coordinate t which ranges from 0 to 1. We then have function $R(s, t) = (\cos(2\pi s), \sin(2\pi s), 2s + t)$ which maps the rectangle $(0, 5) \times (0, 2)$ to a curved ribbon shape in $\mathbb{R}^3$. To generate data on the ribbon, we draw **iid** points uniformly from $(0, 5) \times (0, 1)$ then apply the ribbon function. The results for $N = 1000$ samples are shown below.

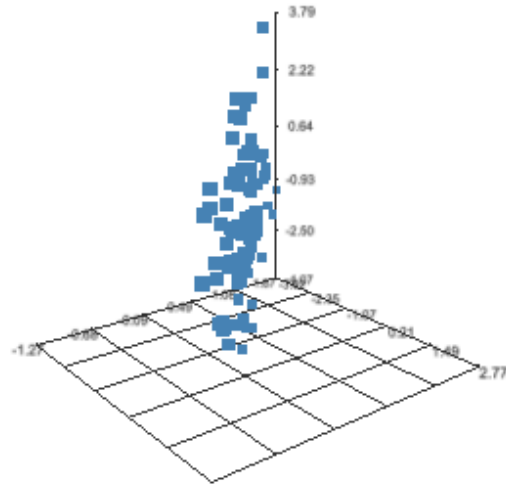


Figure 3.1: Randomly generated points concentrated on a two-dimensional hyperplane.

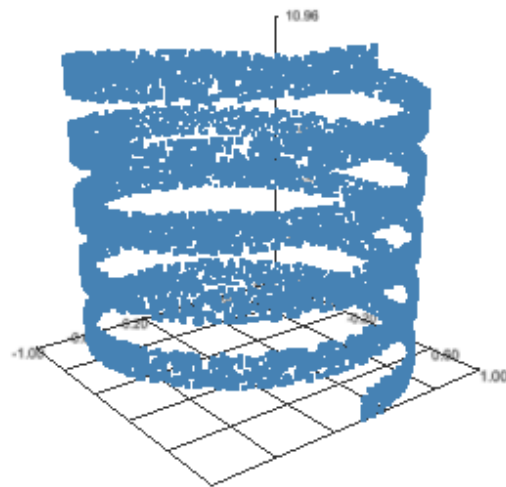


Figure 3.2: Realizations of points on a ribbon

In both the surface is two-dimensional (if you were to stand on the surface and look at nearby data points, they would appear to be planar). A dimension reduction would be able to recover this two-dimensional structure which in the case of the plane corresponds to finding the coefficients used to generate each point. For the ribbons, the coordinates (s, t) used in the ribbon mapping would provide the simplified representation.

The previous examples have shown data in $\mathbb{R}^3$. While this is helpful for visualization, it is not representative of many modern data sources where the dimensionality is extremely high! Suppose we want to generate data in $\mathbb{R}^d$ for some specific large d which has a simpler representation in $\mathbb{R}^t$ (with $t \ll d$) that we know.

In general we can do this through a mapping, $f: \mathbb{R}^t \rightarrow \mathbb{R}^d$. (More on this idea in 5 later). First generate synthetic data $\vec{x}'_1, \dots, \vec{x}'_N$ then apply the map to create high dimensional observations

$$\vec{x}_i = f(\vec{x}'_i), \quad i = 1, \dots, N.$$

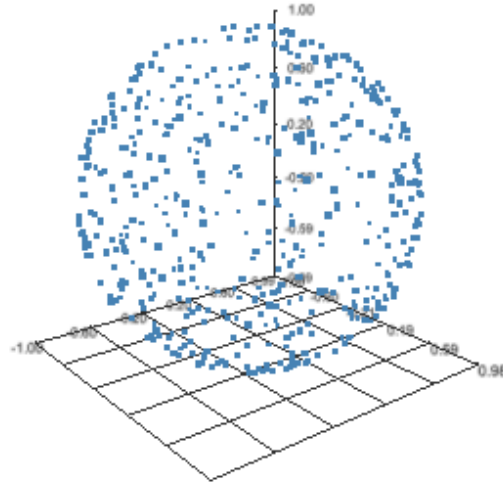
There are infinite possibilities for f , but for now, we will focus on affine mappings. For a matrix $\mathbf{A} \in \mathbb{R}^{d \times t}$ and vector $\vec{b} \in \mathbb{R}^d$, the affine function

$$f(\vec{x}') = \mathbf{A}\vec{x}' + \vec{b}$$

maps vectors in $\mathbb{R}^t$ to points in $\mathbb{R}^d$. Properties of this mapping will depend on the columns of $\mathbf{A}$. For example, if the columns are orthonormal, then Euclidean distances between corresponding pairs of vectors will be the same in the low and high dimensional spaces.

Example 3.6 (Mapping unit sphere in $\mathbb{R}^3$ to higher dimensions). We begin by drawing sample from $\mathcal{N}(\vec{0}, \mathbf{I}_3)$ then rescaling each vector to have length 1.

```
N <- 500
X <- mvrnorm(n=N, mu = rep(0,3), Sigma = diag(1,3))
for (i in 1:N){
  X[i,] <- X[i,]/sqrt(sum(X[i,]^2))
}
scatterplot3js(X,
               xlab = expression(x[1]), ylab = expression(x[2]), zlab = expression(x[3]),
               angle = 90,
               pch = '.',
               size = 0.1)
```



Below, we apply the affine mapping when the columns of $\mathbf{A}$ are orthonormal and $\vec{b} = \vec{0}$ and show a few samples.

```
U <- svd(matrix(rnorm(10*10),nrow = 10))$u[,1:3]
X <- X %*% t(U)
head(round(X,3))
```

```
##      [,1]  [,2]  [,3]  [,4]  [,5]  [,6]  [,7]  [,8]  [,9]  [,10]
## [1,]  0.279  0.678 -0.181  0.252 -0.225  0.173 -0.143 -0.348 -0.284  0.252
## [2,] -0.255 -0.686  0.100 -0.236  0.197 -0.291  0.148  0.318  0.260 -0.292
## [3,]  0.084  0.771  0.043  0.216 -0.415  0.002 -0.124  0.239 -0.243  0.215
## [4,]  0.163 -0.389 -0.500  0.007  0.110 -0.495  0.065 -0.488 -0.033 -0.269
## [5,]  0.254  0.637 -0.321  0.268 -0.365 -0.255 -0.111 -0.186 -0.317  0.097
## [6,]  0.223  0.618 -0.319  0.260 -0.399 -0.347 -0.099 -0.094 -0.310  0.060
```

In the above example, we generated a random orthonormal matrix by applying the SVD to a random 10×10 matrix with $\mathcal{N}(0,1)$ entries. I could continue applying additional transformations (shifts and expansions/contractions along different directions), but we will stop for now with the following important observations. Here is a dataset which has structure we know, 2-dimensional data on a curved surface (the sphere), but we cannot determine this but looking at the data matrix or generating scatterplots of any subset of columns. An effective way of generating data with known but hidden structure which is great to testing algorithms (and students).

3.3.2 Clustered data

The most straightforward way to generate clustered data is to combine realizations from separate data generating mechanisms that tend to create points in disjoint regions of $\mathbb{R}^d$. In Figure 3.3, we show two different cases in $\mathbb{R}^2$.

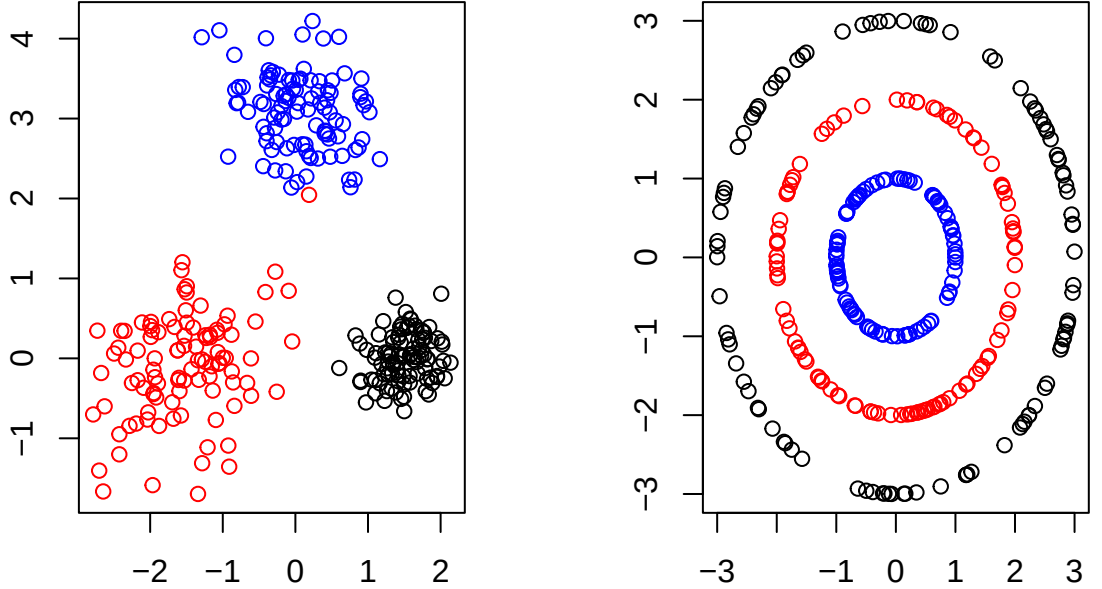


Figure 3.3: In each subfigure below, different subsets of points were generated using different rules and are colored accordingly. Ideally, a clustering algorithm could detect the different cluster shapes (left: ellipsoids, right: concentric rings) and correctly group points depending on how they were generated.

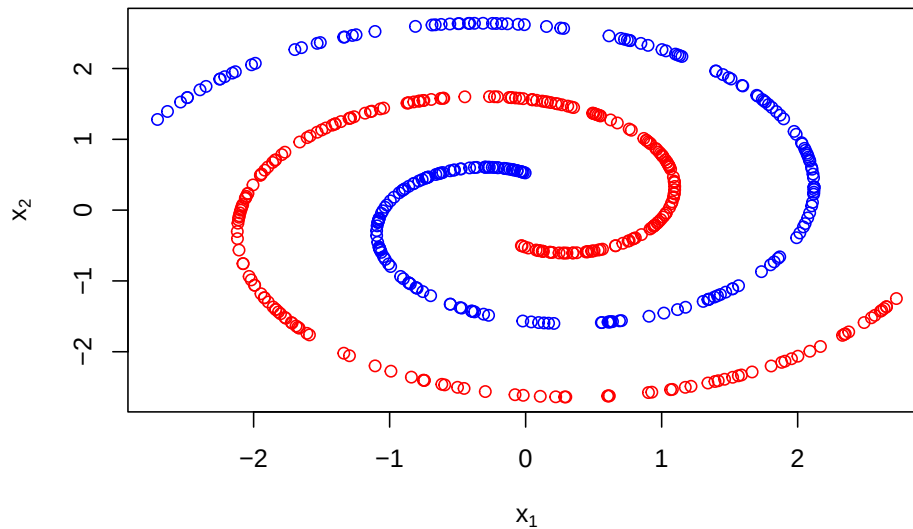
If one did not have access to the actual data generating process (depicted by the different colors), it is still likely that they could recover the correct groupings upon visual inspection. In general, this strategy is not tractable. Naturally, we would like an Unsupervised clustering algorithm that can learn these clusters directly from the data automatically. As we shall see in Chapter 6, certain algorithms will excel at grouping the data contained in disjoint ellipsoids will naturally struggle data clustering in concentric rings because the shape(s) of the different clusters matters has a major impact of the accuracy of the clustering.

In both examples above a two step sampling procedure was used to generate the observations in different clusters. First, a latent cluster *label* was generated. Then based on that label, an associated data point was created. Below we show an example which generates data in one of two spirals. For sample i , we first draw $z_i \sim \text{Ber}(1/2)$. Then if $z_i = 0$, we randomly generate x_i in the first spiral, otherwise we generate x_i in the second.

Example 3.7 (Intertwined spirals).

```
# define functions for different spirals
spiral1 <- function(u){c(u*cos(3*u),u*sin(3*u))}
spiral2 <- function(u){c(u*cos(3*(u+pi)),u*sin(3*(u+pi)))}
# sample latent cluster assignments
z <- sample(c(0,1),size = 1e3,prob = c(1/2,1/2),replace = T)

# generate spirals based on labels
X <- matrix(NA,nrow = N,ncol = 2)
for (i in 1:N){
  if (z[i]==0)
    X[i,] <- spiral1(runif(1,0.5,3))
  else
    X[i,] <- spiral2(runif(1,0.5,3))
}
#plot
plot(X, col = c("blue","red")[z+1], xlab =expression(x[1]),ylab =expression(x[2]))
```



3.4 Summary

Underneath the specific algorithms in the chapters ahead, unsupervised learning almost always rests on one of two geometric beliefs about the data cloud $\vec{x}_1, \dots, \vec{x}_N$: either the points lie near a lower-dimensional surface (the manifold hypothesis, 3.1), or the points separate into visually distinct subgroups (the clustering hypothesis, 3.2). These are genuinely different assumptions – a single, connected Swiss roll satisfies the first without remotely satisfying the second, while several well-separated blobs can satisfy the second without lying near

any interesting low-dimensional surface at all – and much of the difficulty in unsupervised learning comes from not knowing in advance which picture, if either, actually describes your data.

It's worth internalizing that these two pictures are not competitors so much as two different lenses for the same underlying question: what shape does this data cloud have? Dimension reduction asks how thin the cloud is; clustering asks how disconnected it is. A dataset can be well described by neither, by one, or by both simultaneously – the nested rings we'll revisit throughout Chapter 6 are simultaneously a union of two one-dimensional manifolds *and* two well-separated clusters. Since we rarely know the ground truth, we build synthetic examples with a known geometry precisely so that we can watch each method succeed or fail against a shape we understand completely, before ever trusting it on real data whose shape we don't.

3.5 Exercises

1. Generate data on a helix in $\mathbb{R}^3$.
2. Generate data on a 7 dimensional hyperplane in $\mathbb{R}^{50}$. No columns of the associated data matrix should contain all zeros.
3. Generate 2 well separated spherical clusters in $\mathbb{R}^3$. What is the Euclidean distance between the two closest points in different clusters?
4. Repeat problem 4 for $\mathbb{R}^{10}$ and $\mathbb{R}^{100}$. What changes did you have to make to keep the clusters well separated (at least as far apart as problem 3)?

Chapter 4

Linear Methods

4.1 Principal Component Analysis

Principal component analysis (PCA) is arguably the first method for dimension reduction, which dates back to papers by some of the earliest contributors to statistical theory including Karl Pearson and Harold Hotelling [F.R.S., 1901, Hotelling, 1933]. Pearson’s original development of PCA borrowed ideas from mechanics which provides a clear geometric/physical interpretation of the resulting PCA **loadings**, **variances**, and **scores**, which we will define later. This interpretability and an implementation that uses scalable linear algebra methods – allowing PCA to be conducted on massive datasets – is one of the reasons PCA is still used prolifically to this day. In fact, many more modern and complex methods still rely on PCA as an internal step in their algorithmic structure.

There are number of different but equivalent derivations of PCA including the minimization of least squares error, covariance decompositions, and low rank approximations. We’ll revisit these ideas later, but first, let’s discuss PCA through a geometrically motivated lens via a method called iterative projections.

4.1.1 Derivation using Iterative Projections

We begin with a data matrix

$$\mathbf{X} = \begin{bmatrix} \vec{x}_1^T \\ \vdots \\ \vec{x}_N^T \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

Let’s begin with an example of dimension reduction where we’ll seek to replace each vector $\vec{x}_1, \dots, \vec{x}_N$ with corresponding scalars $y_1, \dots, y_N$ which preserve as much of the variability between these vectors as possible. To formalize this idea, let’s introduce a few assumptions.

First, we'll assume the data $\vec{x}_1, \dots, \vec{x}_N$ are centered. To be fair, assumption may be too strong a word. Rather, it's a first step in preprocessing that will simplify the analysis later. We'll discuss how to account for this centering step later, but for now assume $\bar{\vec{x}} = \vec{0}$ so that $\mathbf{H}\mathbf{X} = \mathbf{X}$. More importantly, let's assume that each y_i is constructed in the same way. Specifically, let $y_i = \vec{x}_i^T \vec{w}$ for some common vector $\vec{w}$. Thus, we can view each one-dimensional representation as a dot product of the corresponding observed vector with the same vector $\vec{w}$. We can compactly write this expression as

$$\vec{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} \vec{x}_1^T \vec{w} \\ \vdots \\ \vec{x}_N^T \vec{w} \end{bmatrix} = \mathbf{X}\vec{w}.$$

How do we choose $\vec{w}$? We would like differences in the scalars $y_1, \dots, y_N$ to reflect differences in the vectors $\vec{x}_1, \dots, \vec{x}_N$ so having $y_1, \dots, y_N$ spread out is a natural goal. Thus, if $\vec{x}_i$ and $\vec{x}_j$ are far apart then so will y_i and y_j . To do this, we'll try to maximize the sample variance of the y 's. The sample variance

$$\frac{1}{N} \sum_{i=1}^N (y_i - \bar{y})^2 = \frac{1}{N} \sum_{i=1}^N (\vec{x}_i^T \vec{w} - \bar{y})^2$$

will depend on our choice of $\vec{w}$. In the previous expression,

$$\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i = \frac{1}{N} \sum_{i=1}^N \vec{x}_i^T \vec{w} = \frac{1}{N} \mathbf{1}^T \mathbf{X} \vec{w}$$

is the sample mean of $y_1, \dots, y_N$. Importantly, since we have assumed that $\vec{x}_1, \dots, \vec{x}_N$ are centered, it follows that $\bar{y} = 0$ and the sample variance of $y_1, \dots, y_N$ simplifies to

$$\frac{1}{N} \sum_{i=1}^N (\vec{x}_i^T \vec{w})^2 = \frac{1}{N} \sum_{i=1}^N y_i^2 = \frac{1}{N} \|\vec{y}\|^2 = \frac{1}{N} \vec{y}^T \vec{y}.$$

We can write the above in terms of $\mathbf{X}$ and $\vec{w}$. Using the identity $\vec{y} = \mathbf{X}\vec{w}$, we want to choose $\vec{w}$ to maximize

$$\frac{1}{N} \vec{y}^T \vec{y} = \frac{1}{N} (\mathbf{X}\vec{w})^T \mathbf{X}\vec{w} = \frac{1}{N} \vec{w}^T \mathbf{X}^T \mathbf{X} \vec{w} = \vec{w}^T \left(\frac{\mathbf{X}^T \mathbf{X}}{N} \right) \vec{w}.$$

Since we have assumed that $\mathbf{X}$ is centered it follows that $\mathbf{X}^T \mathbf{X} / N$ is the sample covariance matrix $\hat{\Sigma}$! Thus, we want to make $\vec{w}^T \hat{\Sigma} \vec{w}$ as large as possible.

Naturally, we could increase the entries in $\vec{w}$ and increase the above expression without bound. To make the maximization problem well posed, we will restrict $\vec{w}$ to be unit-length under the Euclidean norm so that $\|\vec{w}\| = 1$. We now have a constrained optimization problem which gives rise to the first **principal component loading**.

Definition 4.1 (First PCA Loading and Scores). The first **principal component loading** is the vector $\vec{w}_1$ solving the constrained optimization problem

$$\begin{aligned} & \text{Maximize } \vec{w}^T \hat{\Sigma} \vec{w} \\ & \text{subject to constraint } \|\vec{w}\| = 1. \end{aligned} \quad (4.1)$$

The first **principal component scores** are the projections, $y_i = \vec{x}_i^T \vec{w}_1$ for $i = 1, \dots, N$, of each sample onto the first loading.

To find the first PCA loading we can make use of Lagrange multipliers (see exercises) to show that $\vec{w}_1$ must also satisfy the equation

$$\hat{\Sigma} \vec{w}_1 = \lambda \vec{w}_1$$

where λ is the Lagrange multiplier. From this expression, we can conclude that the first principal component loading is the unit length eigenvector associated with the largest eigenvalue of the sample covariance matrix $\hat{\Sigma}$ and that the Lagrange multiplier λ is the largest eigenvalue of $\hat{\Sigma}$. In this case, we refer to λ as the first **principal component variance**.

4.1.1.1 Geometric Interpretation of $\vec{w}_1$

Since $\|\vec{w}_1\| = 1$ we may interpret this vector as specifying a direction in $\mathbb{R}^d$. Additionally, we can decompose each of our samples into two pieces: one pointing in the direction specified by $\vec{w}_1$ and a second portion perpendicular to this direction. Thus, we may write

$$\vec{x}_i = \underbrace{\vec{w}_1 \vec{x}_i^T \vec{w}_1}_{\text{parallel}} + \underbrace{(\vec{x}_i - \vec{w}_1 \vec{x}_i^T \vec{w}_1)}_{\text{perpendicular}}.$$

By the Pythagorean theorem,

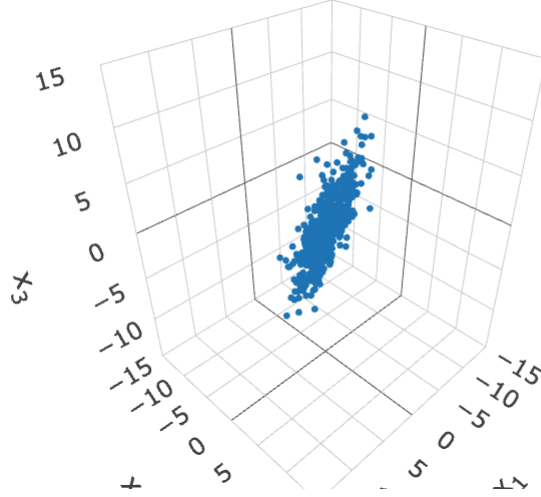
$$\begin{aligned} \|\vec{x}_i\|^2 &= \|\vec{w}_1 \vec{x}_i^T \vec{w}_1\|^2 + \|\vec{x}_i - \vec{w}_1 \vec{x}_i^T \vec{w}_1\|^2 \\ &= (\vec{w}_1^T \vec{x}_i)^2 + \|\vec{x}_i - \vec{w}_1 \vec{x}_i^T \vec{w}_1\|^2 \\ &= y_i^2 + \|\vec{x}_i - \vec{w}_1 \vec{x}_i^T \vec{w}_1\|^2 \end{aligned}$$

for $i = 1, \dots, N$. Averaging over all of samples gives the expression

$$\frac{1}{N} \sum_{i=1}^N \|\vec{x}_i\|^2 = \frac{1}{N} \sum_{i=1}^N y_i^2 + \frac{1}{N} \sum_{i=1}^N \|\vec{x}_i - \vec{w}_1 \vec{x}_i^T \vec{w}_1\|^2.$$

The left-hand side of the above expression is fixed for a given set of data, whereas the first term on the right side is exactly what we sought to maximize when finding the first principal component loading. This quantity is the average squared length of the projection of each sample onto the direction $\vec{w}_1$. As such, we can view the first principal component loading as the direction in which $\vec{x}_1, \dots, \vec{x}_N$ most greatly varies. Let's turn to an example in $\mathbb{R}^3$ to view this.

Example 4.1 (Computing the First PCA Loading and Scores). Below, we show a scatterplot of $N = 500$ points in $\mathbb{R}^3$ drawn randomly from a MVN. These data



have been centered.

Notice the oblong shape of the cloud of points. Rotating this image, it is clear that the data varies more in certain directions than in others.

The sample covariance matrix of these data is

$$\hat{\Sigma} = \begin{bmatrix} 11.4 & 7.12 & 3.22 \\ 7.12 & 9.21 & 9.03 \\ 3.22 & 9.03 & 15.99 \end{bmatrix}$$

We can use the eigendecomposition of this matrix to find the first PCA loading and variance. Given the first loading we can also compute the first PCA scores. A short snippet of code for this calculation is shown below.

```
Sigmahat <- (t(data) %*% data)/N # calculate the sample covariance matrix, recall the
Sigmahat.eigen <- eigen(Sigmahat) # calculate the eigen decomposition of Sigmahat
y <- data %*% Sigmahat.eigen$eigenvectors[,1] # calculate the scores
```

The largest eigenvalue (first PC variance) is $\lambda = 25.58$ with associated eigenvector $(-0.44, -0.57, -0.69)^T$ which is the first PC loading.

We can visualize these results in a few different ways. First, we can add the span of $\vec{w}_1$ (shown in red) to the scatterplot of the data. One can see that $\vec{w}_1$ is oriented along the direction where the data is most spread out.

We could also generate a scatterplot of the scores, but we'll show these scores along the $\vec{w}_1$ axes so that they correspond to the projection of each sample onto $\text{span}\{\vec{w}_1\}$; equivalently, we are plotting the vectors $y_i \vec{w}_1$.

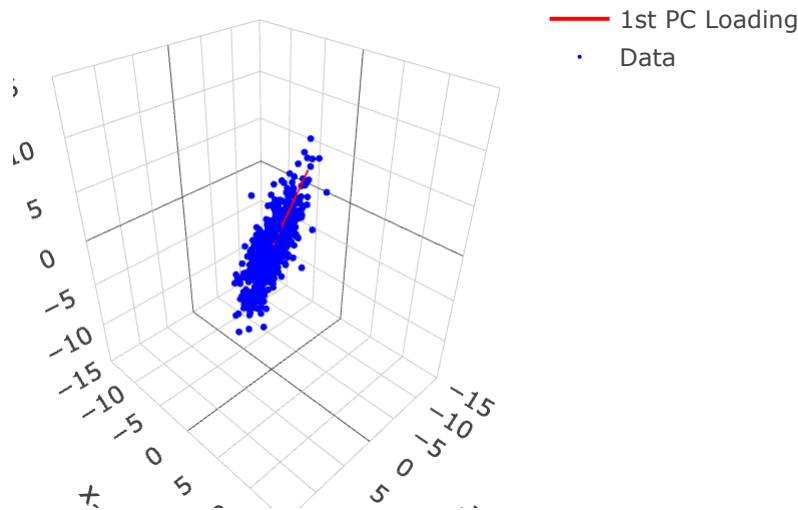


Figure 4.1: Samples with Span of First Loading

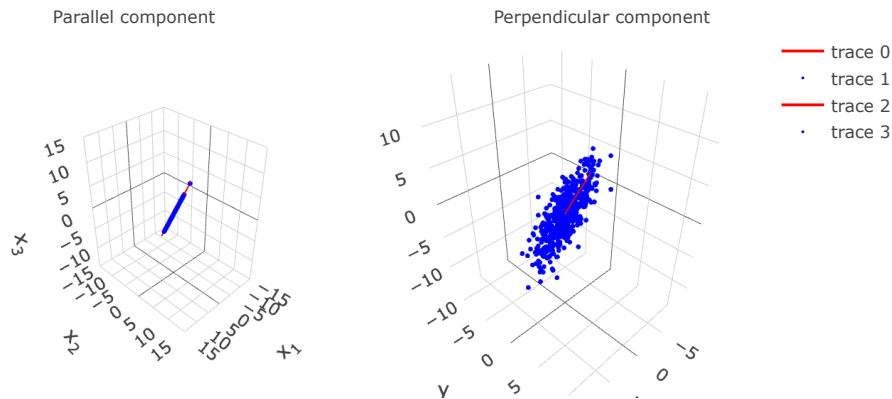


Figure 4.2: Decomposition of Samples into Components Parallel and Perpendicular to First Loading

4.1.1.2 Additional Principal Components

The first PCA loading provides information about the direction in which are data most greatly vary, but it is quite possible that there are still other directions wherein our data still exhibits a lot of variability. In fact, the notion of a *first* principal component loading, scores, and variance suggests the existence of a second, third, etc. iteration of these quantities. To explore these quantities, let's proceed as follows

For each datum, we can remove its component in the direction of $\vec{w}_1$, and focus on the projection onto the orthogonal complement of $\vec{w}_1$. Let

$$\vec{x}_i^{(1)} = \vec{x}_i - \vec{w}_1 \vec{x}_i^T \vec{w}_1 = \vec{x}_i - \vec{w}_1 y_i$$

denote the portion of $\vec{x}_i$ which is orthogonal to $\vec{w}_1$. These points are shown on the right side of Fig @{fig:pca1}. Here, the superscript ⁽¹⁾ indicates we have removed the portion of each vector in the direction of the first loading.

We can organize the orthogonal components into a new data matrix

$$\mathbf{X}^{(1)} = \begin{bmatrix} \left(\vec{x}_1^{(1)}\right)^T \\ \vdots \\ \left(\vec{x}_N^{(1)}\right)^T \end{bmatrix} = \begin{bmatrix} \vec{x}_1^T - \vec{x}_1^T \vec{w}_1 \vec{w}_1^T \\ \vdots \\ \vec{x}_N^T - \vec{x}_N^T \vec{w}_1 \vec{w}_1^T \end{bmatrix} = \mathbf{X} - \mathbf{X} \vec{w}_1 \vec{w}_1^T.$$

Now let's apply PCA to the updated data matrix $\mathbf{X}^{(1)}$ from which we get the second principal component loading, denoted $\vec{w}_2$, the second principal component scores, and the second principal component variance. One can show that the data matrix $\mathbf{X}^{(1)}$ is centered so that its sample covariance matrix is

$$\hat{\Sigma}^{(1)} = \frac{1}{N} (\mathbf{X}^{(1)})^T \mathbf{X}^{(1)}. \quad (4.2)$$

The 2nd PC variance is the largest eigenvalue of $\hat{\Sigma}^{(1)}$ and its associated unit length eigenvector is the 2nd PC loading, denoted $\vec{w}_2$. The second PC scores $\vec{w}_2^T \vec{x}_i^{(1)}$ for $i = 1, \dots, N$.

Continuing 4.1, we have

$$\hat{\Sigma}^{(1)} = \begin{bmatrix} 6.36 & 0.61 & -4.61 \\ 0.61 & 0.81 & -1.07 \\ -4.61 & -1.07 & 3.86 \end{bmatrix}$$

which has largest eigenvalue 10.02 (2nd PC variance) and associated eigenvector $\vec{w}_2 = (0.78, 0.12, -0.61)^T$ (2nd PC Loading). The second set of PC scores are given by $\vec{w}_2^T \vec{x}_i^{(1)}$ for $i = 1, \dots, N$.

Here is one crucial observation. The vector $\vec{w}_2$ gives the direction of greatest variability of the vectors $\vec{x}_1^{(1)}, \dots, \vec{x}_N^{(1)}$. For each of these vectors we have removed

the component in the direction of $\vec{w}_1$. Thus, $\vec{x}_1^{(1)}, \dots, \vec{x}_N^{(1)}$ do not vary at all in the $\vec{w}_1$ direction. What can we say about $\vec{w}_2$? Naturally, it must be perpendicular to $\vec{w}_1$! We can see this geometric relationship if we plot the vectors $\vec{x}_i^{(1)}$ in our previous example along with the span of the first and second loading.

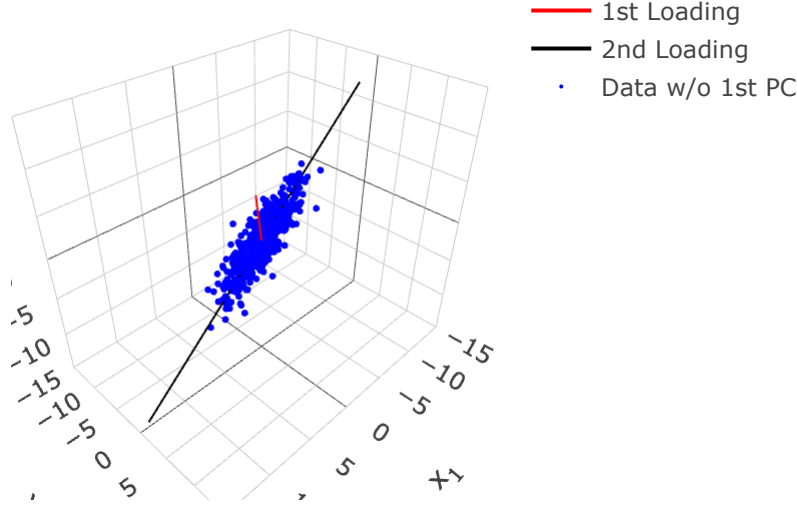


Figure 4.3: First and 2nd Loadings with Data after component along first loading has been removed

We need not stop at the second PCA loading, scores, and variance. We could remove components in the direction of $\vec{w}_2$ and apply PCA to the vectors

$$\begin{aligned}\vec{x}_i^{(2)} &= \vec{x}_i^{(1)} - \vec{w}_2(\vec{x}_i^{(1)})^T \vec{w}_2 \\ &= \vec{x}_i - \vec{w}_1 \vec{x}_i^T \vec{w}_1 - \vec{w}_2(\vec{x}_i - \vec{w}_1 \vec{x}_i^T \vec{w}_1)^T \vec{w}_2 \\ &= \vec{x}_i - \vec{w}_1 \vec{x}_i^T \vec{w}_1 - \vec{w}_2 \vec{x}_i^T \vec{w}_2 + \underbrace{\vec{w}_2 \vec{w}_1^T \vec{x}_i \vec{w}_1^T \vec{w}_2}_{=0}\end{aligned}$$

which gives rise to the centered data matrix

$$\mathbf{X}^{(2)} = \begin{bmatrix} \vec{x}_1^T - \vec{w}_1^T \vec{x}_1 \vec{w}_1^T - \vec{w}_2^T \vec{x}_1 \vec{w}_2^T \\ \vdots \\ \vec{x}_d^T - \vec{w}_1^T \vec{x}_d \vec{w}_1^T - \vec{w}_2^T \vec{x}_d \vec{w}_2^T \end{bmatrix} = \mathbf{X} - \mathbf{X} \vec{w}_1 \vec{w}_1^T - \mathbf{X} \vec{w}_2 \vec{w}_2^T \quad (4.3)$$

with corresponding covariance matrix

$$\hat{\Sigma}^{(2)} = \frac{1}{N} (\mathbf{X}^{(2)})^T \mathbf{X}^{(2)}$$

from which we can obtain a third loading ($\vec{w}_3$), variance (λ_3), and set of scores $\vec{w}_3 \vec{x}_i^{(2)}$ for $i = 1, \dots, N$.

We can continue repeating this argument d times for our d -dimensional data until we arrive at a set of d unit vectors $\vec{w}_1, \dots, \vec{w}_d$ which are the d PCA loadings. Why do we stop after d ? The principal component loadings $\vec{w}_1, \dots, \vec{w}_d$ are all mutually orthogonal and unit length so they form an orthonormal basis for $\mathbb{R}^d$. All of the variability in each sample can be expressed in terms of these d basis vectors.

This iterative approach is admittedly, too intensive for most practical applications. Fortunately, we do not need to follow the sequence of projections and then eigenvalue computations thanks to the following theorem.

Theorem 4.1. *Suppose $\vec{w}_1, \dots, \vec{w}_d$ are the orthonormal eigenvectors of the sample covariance $\hat{\Sigma} = \frac{1}{N} \mathbf{X}^T \mathbf{X}$ with eigenvalues $\lambda_1 \geq \dots \geq \lambda_d \geq 0$ respectively, i.e. $\hat{\Sigma} \vec{w}_j = \lambda_j \vec{w}_j$. Then $\vec{w}_1, \dots, \vec{w}_d$ are also eigenvectors of the matrix $\hat{\Sigma}^{(1)} = \frac{1}{N} (\mathbf{X}^{(1)})^T \mathbf{X}^{(1)}$ with eigenvalues $0, \lambda_2, \dots, \lambda_d$ respectively.*

This result follows nicely from the geometric observation that the loadings are mutually orthogonal. The second loading is then the eigenvector associated with the second largest eigenvalue of the original covariance matrix. Taking the orthogonality of the loadings further, we can get even more from the following corollary.

Corollary 4.1. *Suppose $\vec{w}_1, \dots, \vec{w}_d$ are the orthonormal eigenvectors of the sample covariance $\hat{\Sigma} = \frac{1}{N} \mathbf{X}^T \mathbf{X}$ with eigenvalues $\lambda_1 \geq \dots \geq \lambda_d \geq 0$ respectively. Then for $k = 2, \dots, d-1$, the vectors $\vec{w}_1, \dots, \vec{w}_d$ are also eigenvectors of the matrix $\hat{\Sigma}^{(k)} = \frac{1}{N} (\mathbf{X}^{(k)})^T \mathbf{X}^{(k)}$ with eigenvalues $\underbrace{0, \dots, 0}_k, \lambda_{k+1}, \dots, \lambda_d$ respectively.*

As a result, we can immediately compute the PCA variances and loadings given the full spectral (eigenvalue) decomposition

$$\hat{\Sigma} = [\vec{w}_1 \mid \dots \mid \vec{w}_d] \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \lambda_d \end{bmatrix} \begin{bmatrix} \vec{w}_1^T \\ \vdots \\ \vec{w}_d^T \end{bmatrix}. \quad (4.4)$$

Again, thanks to orthogonality, we can also compute the PCA scores directly from the original data without iteratively removing components of each vector in the direction of the various loadings. To summarize PCA, given data $\mathbf{X}$ we first compute the spectral decomposition of the sample covariance matrix $\hat{\Sigma}$. The eigenvalues (in decreasing magnitude) provide the PC variance and the corresponding unit-length eigenvectors give the corresponding loadings, which form an orthonormal basis in $\mathbb{R}^d$. The PC scores are the inner product of each vector with these loadings (assuming the data are centered). Thus, the PCA scores are $\vec{x}_i^T \vec{w}_1, \dots, \vec{x}_i^T \vec{w}_d$ for $i = 1, \dots, d$. We can compactly (again assuming our data are centered) express the scores in terms of the original data matrix and the loadings as

$$\mathbf{Y} = \mathbf{X}\mathbf{W} \quad (4.5)$$

where $\mathbf{Y} \in \mathbb{R}^{N \times d}$ is a matrix of PC scores and $\mathbf{W}$ is the orthonormal matrix with columns given by the loadings.

4.1.1.3 Geometric interpretation of PCA

We began with an $N \times d$ data matrix $\mathbf{X}$ and now we have an $N \times d$ matrix of PCA scores $\mathbf{Y}$. One may be inclined to ask: where is the dimension reduction? To answer this question, let's revisit some important features of the scores and loadings.

First, since the loadings $\vec{w}_1, \dots, \vec{w}_d$ are orthonormal vectors in $\mathbb{R}^d$, they naturally form a basis! The PCA scores are then the coordinates of our data in the basis $\{\vec{w}_1, \dots, \vec{w}_d\}$. Specifically, $\mathbf{Y}_{ij}$ for $j = 1, \dots, d$ are the coordinates for (centered) vector $\vec{x}_i$ since

$$\vec{x}_i = \mathbf{Y}_{i1}\vec{w}_1 + \dots + \mathbf{Y}_{id}\vec{w}_d.$$

The dimension reduction comes in from an important observations about the PCA scores (aka coordinate). Consider the sample covariance matrix of the PCA scores, which one can show is diagonal (see exercises)

$$\hat{\Sigma}_Y = \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_d \end{bmatrix} \quad (4.6)$$

There are two important perspectives on the using PCA scores. First, the diagonality of the PCA scores indicates that in the new basis, $\{\vec{w}_1, \dots, \vec{w}_d\}$, the coordinates are i) uncorrelated and ii) ordered in decreasing variance. As a result, if we want to construct a $k < d$ dimensional representation our data that minimizes the loss in variability, we should use the first k PCA scores.

Secondly, we can use the first k scores to build approximations,

$$\vec{x}_i \approx \hat{x}_i = \mathbf{Y}_{i1}\vec{w}_1 + \dots + \mathbf{Y}_{ik}\vec{w}_k, \quad i = 1, \dots, N.$$

How good is this approximation? Optimal, in a squared loss sense, thanks to the following theorem.

Theorem 4.2. Fix $k < d$. Given any k -dimensional linear subspace of $\mathbb{R}^d$, denoted $\mathcal{V}$, let $\text{proj}_{\mathcal{V}}(\vec{x})$ denote the orthogonal projection of $\vec{x}$ onto $\mathcal{V}$, then

$$\frac{1}{N} \sum_{i=1}^N \|\vec{x}_i - \text{proj}_{\mathcal{V}}(\vec{x}_i)\|^2 \geq \lambda_{k+1} + \dots + \lambda_d$$

where $\lambda_{k+1}, \dots, \lambda_d$ are the last $(d - k)$ principal component variances of $\vec{x}_1, \dots, \vec{x}_N$.

Furthermore, if $\lambda_k > \lambda_{k+1}$, then $\text{span}\{\vec{w}_1, \dots, \vec{w}_k\}$ is the unique k -dimensional linear subspace for which

$$\frac{1}{N} \sum_{i=1}^N \|\vec{x}_i - \text{proj}_{\mathcal{V}}(\vec{x}_i)\|^2 = \lambda_{k+1} + \dots + \lambda_d.$$

A few notes on the above theorem to add some clarity. First, note that

$$\hat{x}_i = \mathbf{Y}_{i1}\vec{w}_1 + \dots + \mathbf{Y}_{ik}\vec{w}_k$$

is the orthogonal projection of $\vec{x}_i$ onto $\text{span}\{\vec{w}_1, \dots, \vec{w}_k\}$ so that

$$\vec{x}_i - \text{proj}_{\text{span}\{\vec{w}_1, \dots, \vec{w}_k\}}(\vec{x}_i) = \mathbf{Y}_{i,k+1}\vec{w}_{k+1} + \dots + \mathbf{Y}_{id}\vec{w}_d.$$

Using this identity and the properties of the PCA scores allows one to verify the equality at the end of the theorem (see exercises).

The requirement that $\lambda_k \geq \lambda_{k+1}$ is necessary for uniqueness. If $\lambda_k = \lambda_{k+1}$ then the subspace $\text{span}\{\vec{w}_1, \dots, \vec{w}_{k-1}, \vec{w}_{k+1}\}$ would offer an equally good approximation. Fortunately, it is rare that two (nonzero) PCA scores are equal in practice so we rarely have to worry about this issue. In short, PCA learns the k -dimensional linear subspace which best approximates the original data on average!

In all of the previous analysis, we have been operating under the assumption that our data is centered, e.g. $\bar{x} = \vec{0}$ – an preprocessing step that made our analysis easier and is done in practice to avoid potential issue with numerical linear algebra. We can recover all of the nice geometric intuition when working with uncentered data by adding the sample mean back to the approximation and shifting our focus to affine subspaces, which look just like linear subspaces which have been translated by some constant shift away from the origin.

∴ {corollary \$pca-opt-aff} Given any k -dimensional affine subspace $\mathcal{A} \subset \mathbb{R}^d$

$$\frac{1}{N} \sum_{i=1}^N \|\vec{x}_i - \text{proj}_{\mathcal{A}}(\vec{x}_i)\|^2 \geq \lambda_{k+1} + \dots + \lambda_d$$

with equality (assuming $\lambda_k > \lambda_{k+1}$) if and only if $\mathcal{A} = \bar{x} + \text{span}\{\vec{w}_1, \dots, \vec{w}_k\}$. ∴

Note the set $\bar{x} + \text{span}\{\vec{w}_1, \dots, \vec{w}_k\}$ is the set of all vectors

$$\{\vec{v} \in \mathbb{R}^d : \vec{v} - \bar{x} \in \text{span}\{\vec{w}_1, \dots, \vec{w}_k\}\},$$

i.e. the optimal k -dimensional hyperplane found after centering if translated by $\hat{x}$. In this case, the optimal approximation of our data is

$$\vec{x}_i \approx \bar{x} + \mathbf{Y}_{i1}\vec{w}_1 + \dots + \mathbf{Y}_{ik}\vec{w}_k.$$

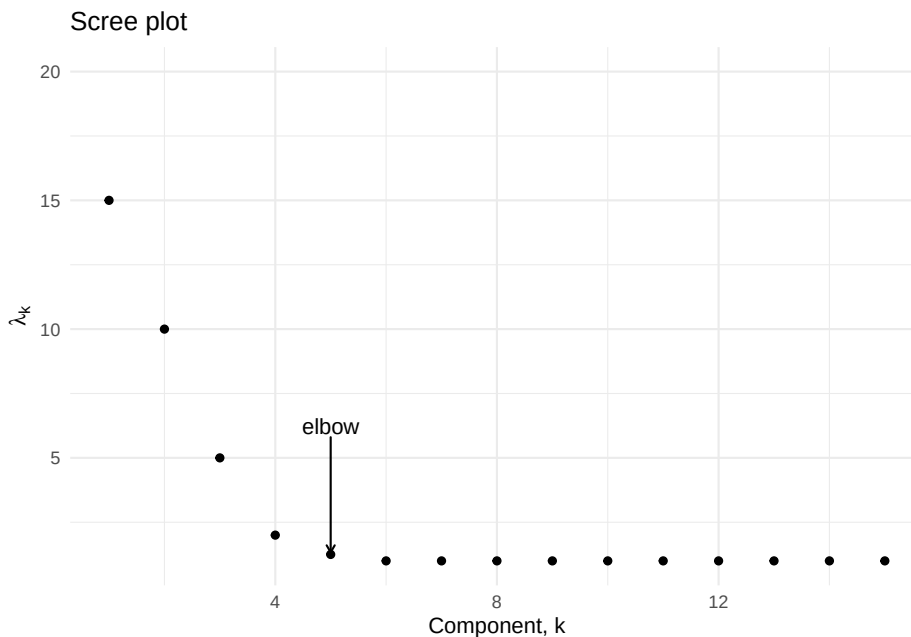
4.1.2 PCA in Practice

4.1.2.1 Choosing the number of components

Now that we have explored the mathematical details of PCA, let us turn to practical guidance when working with real data.

A natural first question for PCA, is how many principal components should I use? Like many modeling decisions in statistics there are a range of answers from simple heuristics to advanced technical answers. For PCA, methods based on random matrix theory are quite strong but also beyond the scope of this book. The curious reader is encouraged to see [Donoho et al., 2023] for more information. Herein, we are going to focus on simpler but more common approaches with an emphasis on inherent trade-offs. Like many statistical modeling choices, decisions are not always so clear cut, and it's best to understand and reason about decisions in the context of a specific problem rather than following hard and fast rules.

The best known method for choosing an optimal number of principal components is the scree plot [Cattell, 1966], which is based on a visual inspection of a plot of the principal component variances in decreasing order. Ideally, one would observe an *elbow* delineating a regions where the variances decrease sharply then saturate near a small value.



In the preceding figure, we would opt to keep the first four components since the variance appears to saturate beginning with the fifth principal component.

Another heuristic suggests keeping the number of components needed to explain a specified percentage of the variance, typically 80 or 90%. Using 80% as a guide

would suggest keeping the first 7 principal components (using the first 6 only gets us to 79.2%) based on the related figure below.

Using the same data as above, we have the following *percentage explained variance* plot below

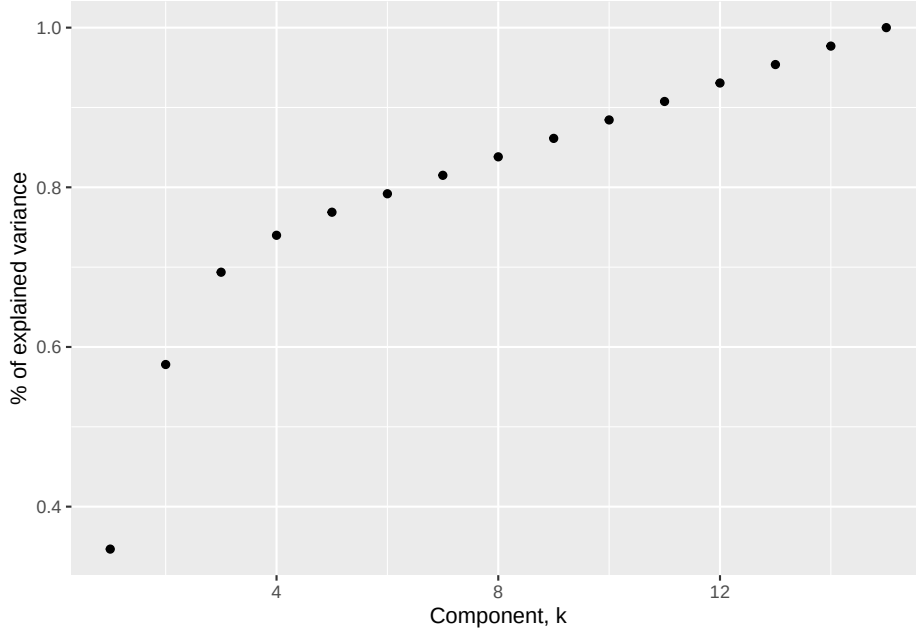


Figure 4.4: Percentage of explained variance

Specifically, we have plotted the cumulative sum of the p.c. variances relative to the total variance

$$\frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^d \lambda_j} = \frac{\sum_{j=1}^k \lambda_j}{\text{tr}(\hat{\Sigma}_x)}$$

as a function of the number of components.

Based on an evaluation of the principal component variances we have two heuristics suggesting different cutoffs (4 vs. 7). Which (if either) is correct? Like many issues in statistics, the answer depends on context. Perhaps there is a data driven reason to choose 4 or 7 or some other number for that matter. The important issue is to recognize the inherent trade-off in the choice so that one can make the best decision in the context of the larger data science problem where PCA is being applied. Choosing a smaller (larger) number of components gives a greater level of dimension reduction at the expense of worse (better) approximation and retention of variability.

Additionally, the variances alone often are not enough to understand the structure of your data. As we will see in the next section, in some cases you may observe

as stark drop in variances that is the result of features of data unrelated to concentration near a lower dimensional hyperplane.

4.1.2.2 Limitations and vulnerabilities

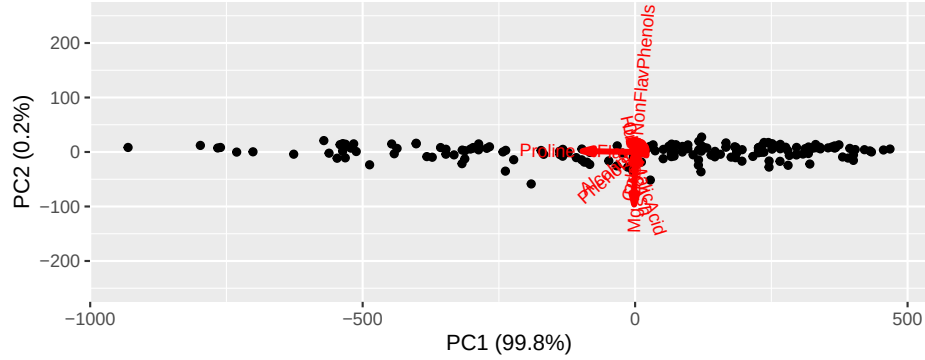
PCA relies on sample variance of data projected in different direction. As such, it is susceptible to outliers and/or imbalanced scales which can greatly influence calculations of sample variance. Furthermore, PCA is optimal at finding the best *linear* subspace as shown in the preceding sections. However, it cannot find (potentially simpler and more compressed) nonlinear structure in the data. Let's investigate each of these limitations in the following subsections.

4.1.2.2.1 Imbalanced scales As a motivating example, let's consider the wine dataset which contains observations of 13 chemical compounds (variables) for 178 different wines (subjects) from three different cultivars. Below we show the first 10 rows of the data matrix.

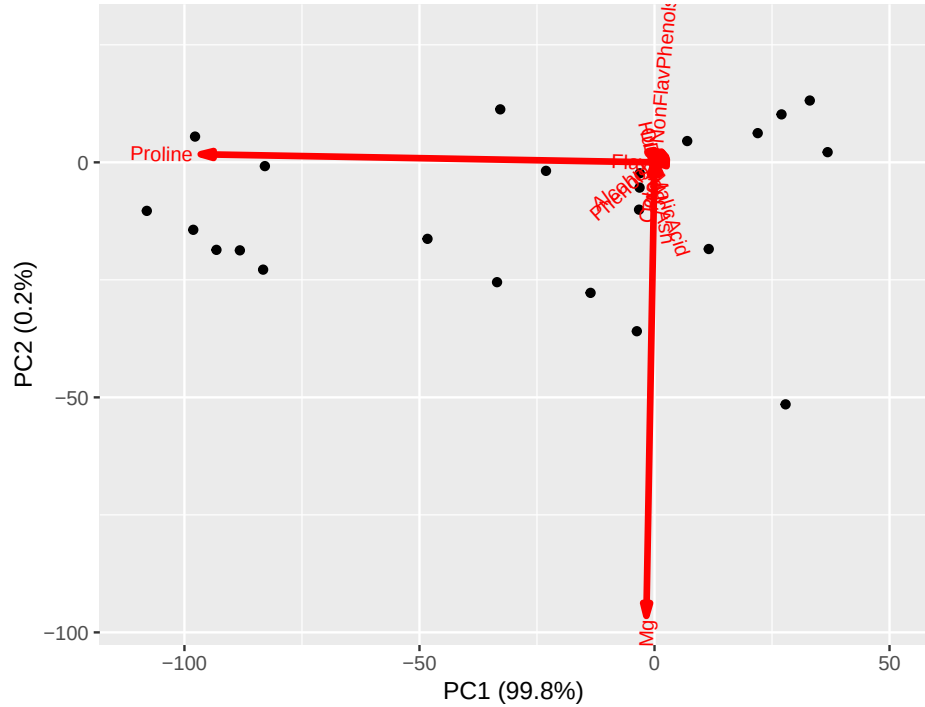
Alcohol	MalicAcid	Ash	AlcAsh	Mg	Phenols	Flav	NonFlavPhenols	Proa	Color	Hue	OD	P
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	
14.20	1.76	2.45	15.2	112	3.27	3.39	0.34	1.97	6.75	1.05	2.85	
14.39	1.87	2.45	14.6	96	2.50	2.52	0.30	1.98	5.25	1.02	3.58	
14.06	2.15	2.61	17.6	121	2.60	2.51	0.31	1.25	5.05	1.06	3.58	
14.83	1.64	2.17	14.0	97	2.80	2.98	0.29	1.98	5.20	1.08	2.85	
13.86	1.35	2.27	16.0	98	2.98	3.15	0.22	1.85	7.22	1.01	3.55	

Prior to running PCA, we might be able to guess which coordinates account for the most variability. Proline (on the scale of 1000s) and magnesium (on the scale of 100s) have much larger values than other coordinates in the data matrix. We can see this observation holds out immediately when looking at the *biplot* summarizing PCA run on the wine data.

```
library(ggbiplot)
out <- prcomp(wine)
ggbiplot(out, scale = 0, varname.color = "red") +ylim(c(-250,250))
```

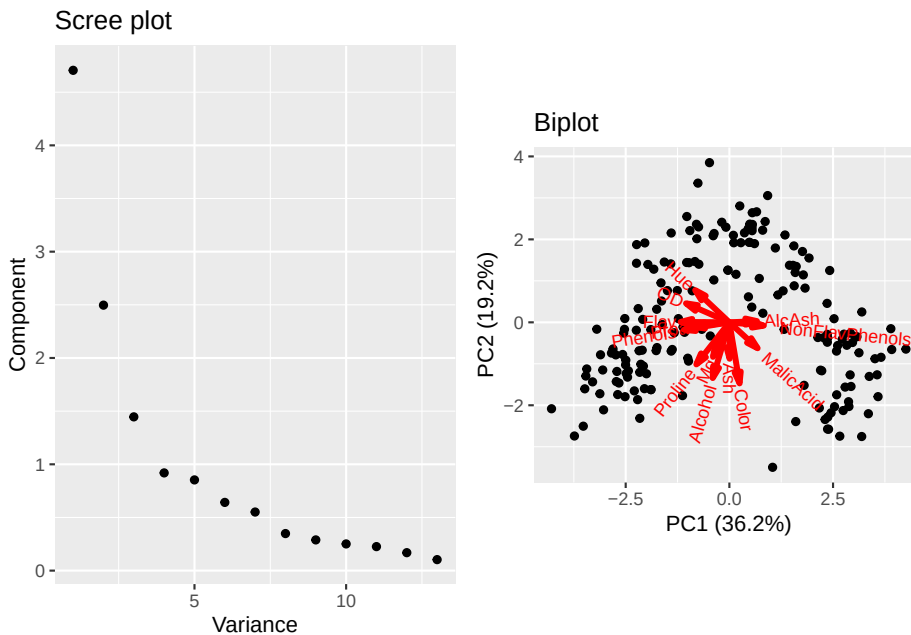


In the figure above, the black dots are scatterplots of the first and second principal component score for each sample in our data set, i.e. $\{(y_{i1}, y_{i2})\}_{i=1}^N$, whereas the arrows correspond to the contribution of each column of the data matrix to the first and second loadings, i.e. $\{(\vec{w}_1)_j, (\vec{w}_2)_j\}_{j=1}^d$. The red vector associated with proline is the only one with a large horizontal component, indicating it is essentially the only coordinate contributing to the first loading. A similar observation indicates magnesium is the only coordinate contributing to the second loading. Admittedly, the arrow corresponding to the variables are difficult to see, but we can zoom in closer to the origin where the results are clear.



A natural way (and generally good idea) to address this issue is to standardize

the columns of the data matrix prior to running PCA. If we use this approach for the wine data, we get the following biplot where there is much more diverse contributions to the first and second loadings from many coordinates.



Outliers

The existence of an outlier in a dataset (as in the univariate case) will alter the calculation of a sample covariance. However, a point can be an outlier because it has an extreme value in one **or more** of its coordinates. Unfortunately, standardization will not do much to alter this issue. Nonetheless, the biplot is a helpful method for identifying this issue. Consider the wine data set, but let us now append a new data vector. It has the same values as the first row, except let's set the malic acid and ash columns to 10^6 (these are unrealistic extreme values only intended to emphasize the effect of an outlier). Below we show the scree plot and biplot resulting from PCA without standardization.

Here, we see one point has an extreme PC score associated with the outlier, which also accounts for a single large variance. It would be inappropriate to assume that a single PC component is sufficient to describe the data. Additionally, observe that the first loading is dominated by Malic Acid and Ash, values for which the outlier is extreme. Standardizing the coordinates can reduce the impact of the outlier but cannot remove it entirely. In such as case, further investigation to see the specific impact the outlier has on the loadings (and the potential overemphasis on some features) warrants further investigation.

4.1.2.2.2 Nonlinear structure Approximations using PCA are built via linear combinations of the loadings. However, there is not reason to *a priori*

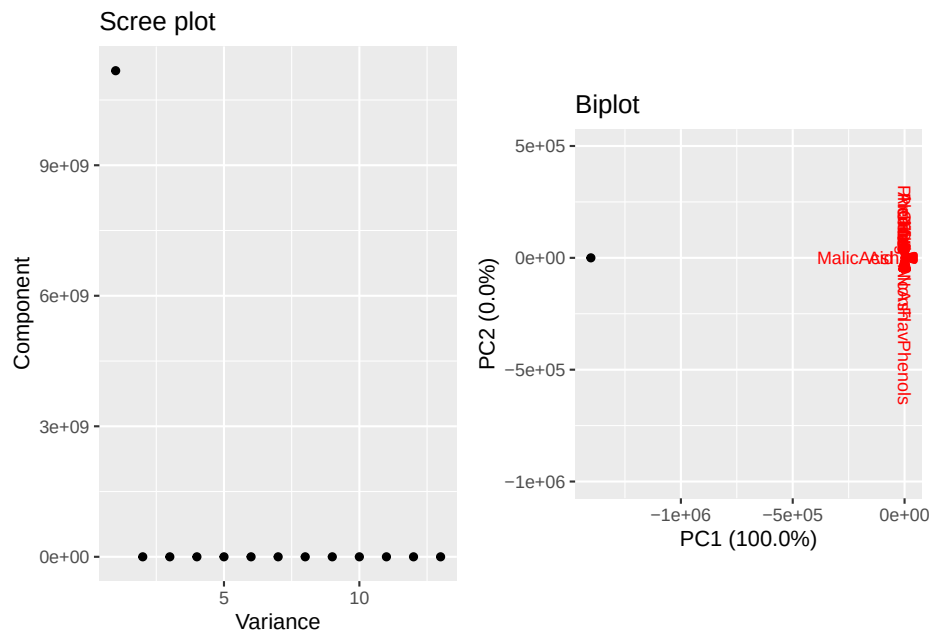


Figure 4.5: PCA on wine data with an outlier

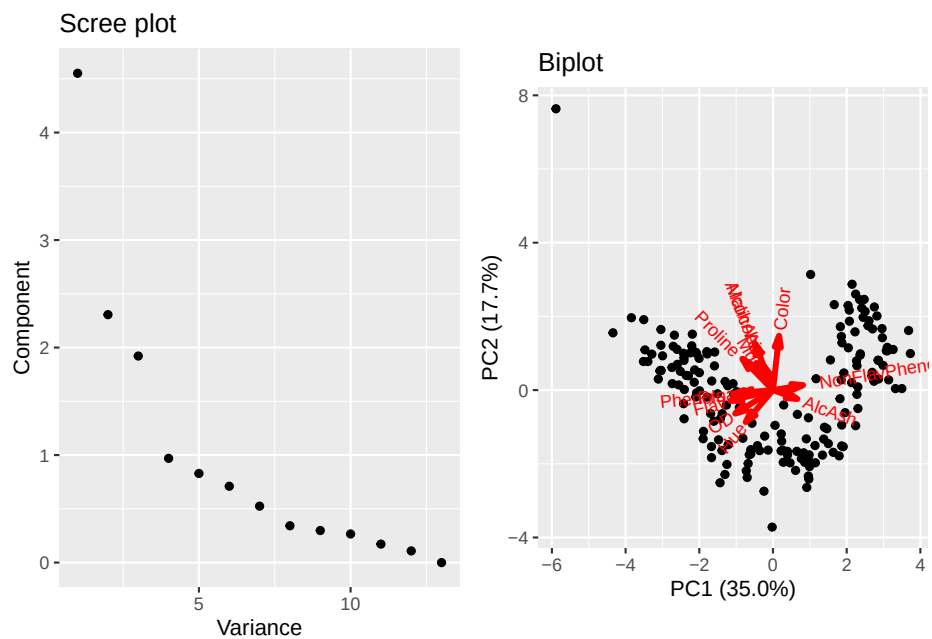
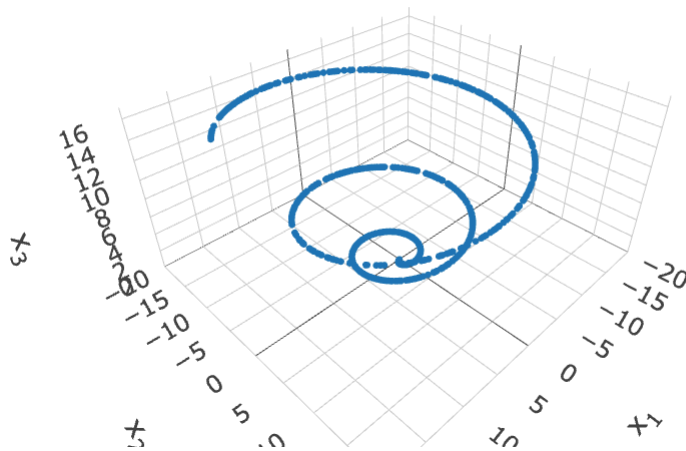


Figure 4.6: PCA on wine data with outlier after standardization

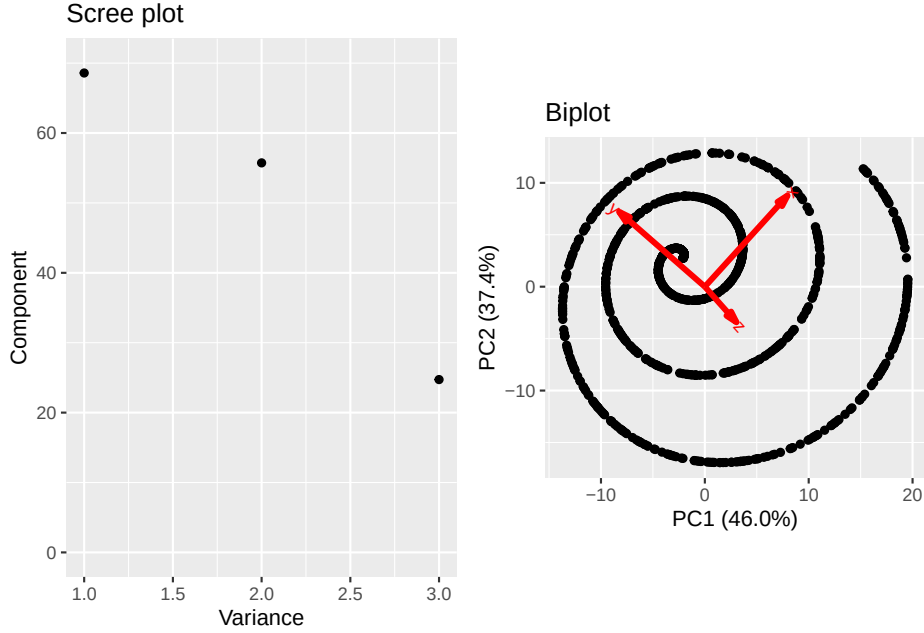
expect that the lower dimensional structure of our data is inherently linear. As such, PCA (and the other linear methods we'll discuss in the remainder of this chapter) are limited in the complexity of structure they can discover in data. As a final example, consider the following data on a curve in $\mathbb{R}^3$.



These data reside on a one-dimensional curve, specifically the curve parameterized by the equation

$$t \mapsto (t \cos(t), t \sin(t), t^2/20)^T.$$

However, if we inspect the principal component variance, we do not see a clear separation in the first PC variance with the second and third. There is a jump between the second and third, and plotting the first two scores (biplot) shows the nonlinear structure.



Ultimately, the fundamental limitation of PCA is a direct byproduct of its primary strength. PCA discovers best approximating hyperplanes but nothing more complex!

4.2 Singular Value Decomposition

4.2.1 Low-rank approximations

In the next two subsections, we are going to focus on *low-rank* matrix approximation methods in which we try to approximate our data matrix $\mathbf{X}$ using a low-rank alternative. In the language of dimension reduction, the idea is to approximate each data with a linear combination of a small number ($k < d$) of latent feature vectors. Briefly, let's discuss how this idea works in the case of PCA.

In PCA, the loadings provide a data-driven orthonormal basis $\vec{w}_1, \dots, \vec{w}_d$ which allow us to compute the PCA scores from the centered data. In matrix notation, these scores are given by

$$\underbrace{\mathbf{Y}}_{\text{PCA scores}} = \underbrace{(\mathbf{H}\mathbf{X})}_{\text{centered data matrix}} \times \underbrace{\mathbf{W}}_{\text{loadings}}.$$

The matrix $\mathbf{W}$ is orthonormal allowing us to write

$$\mathbf{H}\mathbf{X} = \mathbf{Y}\mathbf{W}^T.$$

The i th row of the preceding matrix equality reads

$$(\vec{x}_i - \bar{x})^T = \sum_{j=1}^d \mathbf{Y}_{ij} \vec{w}_j^T.$$

From the PCA notes, an approximation using the first k loadings

$$(\vec{x}_i - \bar{x})^T \approx \sum_{j=1}^k \mathbf{Y}_{ij} \vec{w}_j^T$$

minimizes the average squared Euclidean distance over all vectors. In matrix notation, the approximation over all vectors decomposes as the product of an $N \times k$ matrix and a $k \times d$ matrix as follows.

$$\mathbf{H}\mathbf{X} \approx \underbrace{\begin{bmatrix} \mathbf{Y}_{11} & \cdots & \mathbf{Y}_{1k} \\ \vdots & & \vdots \\ \mathbf{Y}_{N1} & \cdots & \mathbf{Y}_{Nk} \end{bmatrix}}_{N \times k} \underbrace{\begin{bmatrix} \vec{w}_1^T \\ \vdots \\ \vec{w}_k^T \end{bmatrix}}_{k \times d}.$$

Due to the properties of the scores and loadings, the approximation is a rank k matrix. In the following sections, we'll seek similar decompositions of our data matrix.

4.2.2 SVD and Low Rank Approximations

The standard problem for low rank matrix approximations is to solve the following problem. Given a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ and a chosen rank k , we want:

$$\hat{\mathbf{Z}} = \operatorname{argmin}_{\mathbf{Z} \in \mathbb{R}^{N \times d}} \|\mathbf{X} - \mathbf{Z}\|_F^2$$

subject to the constraint that $\mathbf{Z}$ has rank k .

Solving this constrained minimization problem may appear difficult, but the answer is obtainable directly from the SVD of $\mathbf{X}$ due to the following theorem.

Theorem 4.3 (Eckart-Young-Mirsky Theorem). *Suppose matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ has singular value decomposition*

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$$

with singular values

$$\sigma_1 \geq \cdots \geq \sigma_{\min\{N,d\}}.$$

Then 1) For any rank k matrix $\mathbf{Z} \in \mathbb{R}^{N \times d}$,

$$\|\mathbf{X} - \mathbf{Z}\|_F^2 \geq \sigma_{k+1}^2 + \cdots + \sigma_{\min\{N,d\}}^2$$

2) The rank k matrix attained by keeping the first k left singular vectors, right singular vectors, and singular values of the SVD of $\mathbf{X}$ attains this minimum. Specifically, if $\vec{u}_1, \dots, \vec{u}_k$ are the first k left singular vectors and $\vec{v}_1, \dots, \vec{v}_k$ are the first k right singular vectors then

$$\tilde{X} = \sum_{j=1}^k \sigma_j \vec{u}_j \vec{v}_j^T$$

is the best rank k approximation to $\mathbf{X}$ under (squared) Frobenius loss.

There are several important implications of this theorem. First, the direct result indicates that computing the SVD of $\mathbf{X}$ immediately allows us to compute the best approximation under Frobenius loss for a specified rank k . In practice, the full SVD is not required since we will typically consider the case where $k < \min\{N, d\}$. There is another implication as well. In cases where a specific choice of k is not clear, the singular values of $\mathbf{X}$ provide a method for comparing different choices of k . Akin to the scree plot, we can plot the (squared) singular values to look for clear separation or alternatively, plot the ratio

$$\frac{\sum_{j=1}^k \sigma_j^2}{\sum_{j=1}^{\min\{N, d\}} \sigma_j^2}$$

as a function of k to understand the relative error for a specific choice of k .

For a given choice of k , we now approximate our original data by linear combination of the right singular vectors $\vec{v}_1, \dots, \vec{v}_k$. The approximations are

$$\vec{x}_i \approx \vec{z}_i = \sum_{j=1}^k \sigma_j \mathbf{U}_{ij} \vec{v}_j$$

.

4.2.3 Connections with PCA

Suppose that we were to compute the full SVD of the centered data matrix

$$\mathbf{H}\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T.$$

We can express the sample covariance matrix of the original data using the SVD as

$$\hat{\Sigma}_X = \frac{1}{N}(\mathbf{H}\mathbf{X})^T(\mathbf{H}\mathbf{X}) = \frac{1}{N}\mathbf{V}\mathbf{S}^T\mathbf{U}^T\mathbf{U}\mathbf{S}\mathbf{V}^T = \mathbf{V}\left(\frac{1}{N}\mathbf{S}^T\mathbf{S}\right)\mathbf{V}^T. \quad (4.7)$$

The matrix $\frac{1}{N}\mathbf{S}^T\mathbf{S} \in \mathbb{R}^{d \times d}$ is diagonal with entries $\sigma_1^2/N \geq \dots \geq \sigma_d^2/N$. Furthermore, the matrix $\mathbf{V}$ is orthonormal. Thus, from the SVD of $\mathbf{H}\mathbf{X}$ we can immediately compute the spectral decomposition of $\hat{\Sigma}_X$ to attain the principal component variances and loadings. In fact, the principal component loadings

are the right singular vectors of $\mathbf{H}\mathbf{X}$ whereas the principal component variances are the squared singular values divided by N , e.g. $\lambda_j = \sigma_j^2/N$. Using this observation,

$$\mathbf{H}\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T \rightarrow \mathbf{H}\mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{S}$$

from which we may conclude the principal component scores are equal to $\mathbf{U}\mathbf{S}$. This connection is the basis for most numerical implementation of PCA since it is both faster and more numerically stable to compute the SVD of $\mathbf{H}\mathbf{X}$ than to compute both $\hat{\Sigma}_X$ and its eigendecomposition! Thus, computing the best rank k approximation to a centered data matrix is equivalent to the best approximation of the centered data using the first k PC scores.

However, using the SVD to compute a low rank approximation to a *non-centered* data matrix will give a different result than PCA since the SVD of $\mathbf{H}\mathbf{X}$ will be different than the SVD of $\mathbf{X}$. Unlike PCA, which decomposes variability in directions relative to the center of the data, SVD learns an orthonormal basis which decomposes variability relative to the origin. Only when the data is centered (so its mean is the origin) do SVD and PCA coincide. Nonetheless, SVD has similar weaknesses to PCA including a sensitivity to scaling and outliers and an inability to detect nonlinear structure.

SVD can provide one final note of insight regarding PCA. Suppose that $N < d$, which is to say that we have fewer samples than the dimensionality of our data. After centering, the matrix $\mathbf{H}\mathbf{X}$ will have rank at most $N - 1$. (Centering reduces the maximum possible rank from N to $N - 1$). The SVD of $\mathbf{H}\mathbf{X}$ will have at most $N - 1$ non-zero singular values. Thus, $\hat{\Sigma}_X$ will have at most $N - 1$ non-zero PC variances and we can conclude that our data reside on a hyperplane of dimension $N - 1$ (possibly lower if $\mathbf{H}\mathbf{X}$ has rank less than $N - 1$). Since $N - 1 < d$, we are guaranteed to find a lower-dimensional representation of our data! However, this conclusion should be viewed cautiously. Should additional samples be drawn, can we conclude that they would also be constrained to the same hyperplane learned using the first N samples?

4.2.4 Recommender Systems

SVD may also be applied to building recommendation systems which can identify similar items in datasets based on partial observations. As a motivating example, consider the case where we have a dataset of user provided ratings of products, which could be items purchased, songs listened to, or movies watched. In this case, $\mathbf{X}_{ij}$ indicates user i 's rating of item j . Typically, most of the entries of $\mathbf{X}$ will be NA since users have likely interacted with a small number of items. Using a variant of SVD, a simple recommendation system proceeds in two steps. First, we can impute the missing ratings. We can then use this result to infer similar movies which can be used for recommendation.

4.2.4.1 Imputation

Let $\mathbf{X} \in \mathbb{R}^{N \times d}$ be the data matrix of user ratings with rows corresponding to user and columns to items and

$$\mathcal{I} = \{ij \text{ s.t. } \mathbf{X}_{ij} \neq NA\}$$

be the set of all indices of $\mathbf{X}$ for which we have observed ratings. For any approximating matrix $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times d}$, we may define a ‘Frobenius’-like error as

$$\mathbb{L}(\mathbf{X}, \tilde{\mathbf{X}}) = \sum_{ij \in \mathcal{I}} (\mathbf{X}_{ij} - \tilde{\mathbf{X}}_{ij})^2$$

which is the sum-squared error over all observations. Using this definition of loss, here’s a simple algorithm for imputing the missing entries of $\mathbf{X}$ using low rank approximations of a pre-specified rank k .

- 0) We initialize the matrix $\tilde{\mathbf{X}}$ of imputed entries by taking

$$\tilde{X}_{ij} = \begin{cases} X_{ij} & ij \in \mathcal{I} \\ 0 & ij \notin \mathcal{I} \end{cases}.$$

Now let’s use the SVD of $\tilde{\mathbf{X}}$ to compute a rank k approximation, $\tilde{\mathbf{X}}^{(k)}$. Update our imputed matrix $\tilde{\mathbf{X}} = \tilde{\mathbf{X}}^{(k)}$ and compute the error

$$\ell = \mathbb{L}(\mathbf{X}, \tilde{\mathbf{X}})$$

. The low-rank approximation will distort all entries of $\tilde{\mathbf{X}}$ so that $\ell > 0$. We now repeat the following two steps.

- 1) Overwrite the entries of $\tilde{\mathbf{X}}$ corresponding to observations in $\mathbf{X}$, e.g. for all $ij \in \mathcal{I}$, set $\tilde{\mathbf{X}}_{ij} = \mathbf{X}_{ij}$. The entries corresponding to missing observations generated by the low-rank approximation are kept unchanged. Now recompute the SVD of $\tilde{\mathbf{X}}$ to find the rank k approximating matrix $\tilde{\mathbf{X}}^{(k)}$. Update our imputed matrix using the low-rank approximation so that $\tilde{\mathbf{X}} = \tilde{\mathbf{X}}^{(k)}$ and recompute the error $\ell^* = \mathbb{L}(\mathbf{X}, \tilde{\mathbf{X}})$.
- 2) If $\ell^* < \ell$ and $|\ell - \ell^*|/\ell > \epsilon$ then set $\ell = \ell^*$ and return to step (1). Else stop the algorithm and use the matrix $\tilde{\mathbf{X}}$ as our matrix of imputed values.

In summary, after initialization, we are continually overwriting the entries of our matrix of imputed values corresponding to observations then applying a low-rank approximation. We stop the algorithm when the error stops decreasing or when the relative decrease in error is less than a specified threshold ϵ . In addition to the rank k and the stopping threshold ϵ there is one other important ‘tuning’ parameter, the initialization. In the brief description above, we used a standard of 0, but one could also use the average of all entries in the corresponding column (average item rating) or row (average rating given by a user) or some other specification. Many more complicated recommendation systems include user

and item specific initializations and adjustments but still employ a low rank approximation somewhere in their deployment.

In later updates, we show an application of this algorithm to the Movielens 10m dataset containing 10 million ratings from 72,000 viewers for 10,000 movies. To handle the size of this dataset, we use the Fast Truncated Singular Value Decomposition.

```
# library("irlba")
# initialize <- function(mat){
# # get column means ignoring NAs
# ave.rat <- colMeans(mat,na.rm = TRUE)
# # fill NAs by average movie rating
# for(j in 1:ncol(mat)){
#   mat[is.na(mat[,j]),j] <- ave.rat[j]
# }
# return(mat)
# }
#
# maxim <- function(mat,k){
# # temp <- svd(mat)
# temp<- irlba(mat, nv = k)
# return(list(U = temp$u[,1:k],
#             D = temp$d[1:k],
#             V = temp$v[,1:k],
#             mat.hat = temp$u[,1:k] %*% diag(temp$d[1:k]) %*% t(temp$v[,1:k])))
# }
#
# recommender <- function(mat, num_steps, k){
# # initialize loss function tracking
# loss <- rep(NA,num_steps)
# # run EM algorithm and save loss
# ind.known <- !is.na(mat)
# mat2 <- initialize(mat)
# for (j in 1:num_steps){
#   mat2 <- maxim(mat2,k)$mat.hat
#   loss[j] <- sum((mat2[ind.known] - mat[ind.known])^2)
#   mat2[ind.known] <- mat[ind.known]
# }
# return(list(loss= loss, fit = mat2))
# }
#
# k <- 3
# temp <- recommender(ratings,num_steps = 200, k = k)
# plot(temp$loss, xlab = "Step", ylab = expression(ell))
```

4.2.4.2 Recommendation

Suppose now that we have a matrix, $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times d}$ of movie ratings (real or imputed) and its SVD

$$\tilde{\mathbf{X}} = \tilde{\mathbf{U}}\tilde{\mathbf{S}}\tilde{\mathbf{V}}^T$$

where $\tilde{\mathbf{U}} \in \mathbb{R}^{N \times k}$, $\tilde{\mathbf{S}} \in \mathbb{R}^{k \times k}$ and $\tilde{\mathbf{V}} \in \mathbb{R}^{d \times k}$. Then for user i the rating they give to movie j is a linear combination of the elements in the j th column of $\tilde{\mathbf{V}}^T$. Specifically,

$$\tilde{\mathbf{X}}_{ij} \approx \sum_{\ell=1}^k \sigma_{\ell} \tilde{\mathbf{U}}_{i\ell} (\tilde{\mathbf{V}})_{\ell j}^T = \sum_{\ell=1}^k \sigma_{\ell} \tilde{\mathbf{U}}_{i\ell} \tilde{\mathbf{V}}_{j\ell}.$$

For any movie, its rating will always be a linear combination of the elements in the corresponding column of $\tilde{\mathbf{V}}^T$. As such, we may view the k -dimensional vectors in each column of $\tilde{\mathbf{V}}$ as a representation of that movie. We may then use these vectors to identify similar movies; one common approach is the cosine similarity, which for vectors $\vec{x}, \vec{y} \in \mathbb{R}^k$ is the cosine of the angle between them, i.e.

$$\cos \theta = \frac{\vec{x}^T \vec{y}}{\|\vec{x}\| \|\vec{y}\|}.$$

The cosine similarity is bounded between -1 and 1 and two vectors are considered more similar if the cosine of the angle between them is closer to 1. Using this representation, we can take any movie (a column of $\tilde{\mathbf{V}}^T$) and choose the most similar movie (choosing the other columns of $\tilde{\mathbf{V}}^T$ with the largest cosine similarity). Thus, if a user gave a high rating to movie b , we now have a method for recommending one or more similar movies they might enjoy.

4.3 Nonnegative Matrix Factorization

In both PCA and SVD, we learn data-driven orthonormal feature vectors which we can use to decompose our data in an orderly fashion. Nonnegative matrix factorization (NMF) is again focused on learning a set latent vectors which can be used to approximate our data. However, we will add a few restrictions motivated by experimental data and a desire to increase interpretability of the results which will drastically alter the results.

For NMF, we focus on cases where $\mathbf{X}$ is an $N \times d$ data matrix with the added condition that its entries are nonnegative. Notationally, we write $\mathbf{X} \in \mathbb{R}_{\geq 0}^{N \times d}$ to indicate it is composed of nonnegative real values. The nonnegativity condition is a natural constraint for many experimental data sets, but we are also going to impose a similar constraint on our feature vectors and coefficients. More specifically, for a specific rank k , we seek a coefficient matrix $\mathbf{W} \in \mathbb{R}_{\geq 0}^{N \times k}$ and feature matrix $\mathbf{H} \in \mathbb{R}_{\geq 0}^{k \times d}$ such that $\mathbf{WH}$ is as close to $\mathbf{X}$ as possible. The nonnegativity constraint on the elements of $\mathbf{W}$ and $\mathbf{H}$ implies that $\mathbf{WH} \in \mathbb{R}_{\geq 0}^{N \times d}$. There are many different measures of proximity that one may use in NMF which

are greater than zero and equal to zero if and only if $\mathbf{X} = \mathbf{WH}$. The most common measures are

- Frobenius norm: $\|\mathbf{X} - \mathbf{WH}\|_F$.
- Divergence: $D(\mathbf{X} \parallel \mathbf{WH}) = \sum_{i=1}^N \sum_{j=1}^d \left[\mathbf{X}_{ij} \log \frac{\mathbf{X}_{ij}}{(\mathbf{WH})_{ij}} + (\mathbf{WH})_{ij} - \mathbf{X}_{ij} \right]$
- Itakura-Saito Divergence:

$$D_{IS}(\mathbf{X}, \mathbf{WH}) = \sum_{i=1}^N \sum_{j=1}^d \left[\frac{\mathbf{X}_{ij}}{(\mathbf{WH})_{ij}} - \log \frac{\mathbf{X}_{ij}}{(\mathbf{WH})_{ij}} - 1 \right]$$

These loss functions emphasize and prioritize different features of the data and are often coupled with additional penalties or assumptions on the elements of $\mathbf{W}$ and $\mathbf{H}$ which we will discuss at the end of the section. From a statistical perspective, they may also be motivated based on differing beliefs about the data generating process. For example, suppose we assume conditional independence of the entries of $\mathbf{X}$ given $\mathbf{W}$, $\mathbf{H}$, and error variance σ . Under a Gaussian error model

$$\mathbf{X}_{ij} \mid \mathbf{W}, \mathbf{H} \sim \mathcal{N}((\mathbf{WH})_{ij}, \sigma^2)$$

maximizing the associated likelihood w.r.t. $\mathbf{W}$ and $\mathbf{H}$ is equivalent to minimization of the Frobenius norm $\|\mathbf{X} - \mathbf{WH}\|_F^2$. We'll revisit this idea in greater detail in Section 4.3.6.

For now, let us focus on the primary motivation of NMF, which is to create more interpretable feature vectors (the rows of $\mathbf{H}$).

4.3.1 Interpretability, Superpositions, and Positive Spans

Consider the case of SVD where we can approximate a given vector $\vec{x}_i$ using the first k right singular vectors as

$$\vec{x}_i \approx \sum_{j=1}^k \sigma_j \mathbf{U}_{ij} \vec{v}_j.$$

Let us restrict ourselves to the case of nonnegative entries. Suppose for the moment that the ℓ th coordinate of $\vec{x}_i$ is (very close to) zero and this is reflected by our approximation as well so that

$$(\vec{x}_i)_\ell \approx \sum_{j=1}^k \sigma_j \mathbf{U}_{ij} (\vec{v}_j)_\ell \approx 0.$$

The freedom of the coefficients $\sigma_j \mathbf{U}_{ij}$ and the features $(\vec{v}_j)_\ell$ for $j = 1, \dots, k$ to take any value in $\mathbb{R}$ prevents us from concluding that there is a comparable **near-zerosness** in the features or coefficients. It could be that case that $\mathbf{U}_{ij} (\vec{v}_j)_\ell$ is near zero for all $j = 1, \dots, k$ or that some subset are large and positive but

are canceled out by a different subset that is large and negative. If, however, we restrict the coefficients and features to be zero this cancellation effect cannot occur. Features can only then be added but never subtracted. Under this restriction, $\sum_{j=1}^k \sigma_j \mathbf{U}_{ij}(\vec{v}_j)_\ell$ will only be close to zero if the coefficients are (near) zero for any feature vector which has a (large) positive entry in its ℓ th coordinate.

Thus, in a factorization of the form

$$\vec{x}_i \approx \sum_{j=1}^k \underbrace{\mathbf{W}_{ij}}_{\geq 0} \underbrace{\vec{h}_j}_{\in \mathbb{R}_{\geq}^d},$$

we can only superimpose (add) features to approximate our data, so we might expect the features themselves to look more like our data. In matrix notation, we have

$$\mathbf{X} = \mathbf{W}\mathbf{H}$$

where the coefficients for each sample are stored in the rows of $\mathbf{W} \in \mathbb{R}_{\geq 0}^{N \times k}$ and the features are transposed and listed as the rows of $\mathbf{H} \in \mathbb{R}_{\geq 0}^{k \times d}$.

4.3.2 Geometric Interpretation

As a motivating example consider the case, where we have data $\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}_{\geq 0}^3$ for which there is an exact decomposition

$$\mathbf{X} = \mathbf{W}\mathbf{H}$$

which is to say there are nonnegative coefficients, $\mathbf{W}$ and feature vectors $\vec{h}_1, \vec{h}_2 \in \mathbb{R}_{\geq 0}^3$ such that

$$\vec{x}_i = \mathbf{W}_{i1}\vec{h}_1 + \mathbf{W}_{i2}\vec{h}_2 \quad \text{for } i = 1, \dots, N.$$

The nonnegativity condition on data implies that $\vec{x}_1, \dots, \vec{x}_N$ reside in the positive orthant of $\mathbb{R}^3$. The exact decomposition assumptions implies $\{\vec{x}_1, \dots, \vec{x}_N\} \in \text{span}\{\vec{h}_1, \vec{h}_2\}$ and furthermore the following more restricted picture holds.

The data are constrained within the positive span of the vectors, a notion we may now define.

Definition 4.2 (Positive Span). The positive span of a set of vectors $\{\vec{h}_1, \dots, \vec{h}_k\} \in \mathbb{R}^d$ is the set

$$\Gamma(\{\vec{h}_1, \dots, \vec{h}_k\}) = \left\{ \vec{v} \in \mathbb{R}^d \mid \vec{v} = \sum_{j=1}^k a_j \vec{h}_j, a_1, \dots, a_k \geq 0 \right\}.$$

This set is also called the simplicial cone or conical hull of $\{\vec{h}_1, \dots, \vec{h}_k\}$.

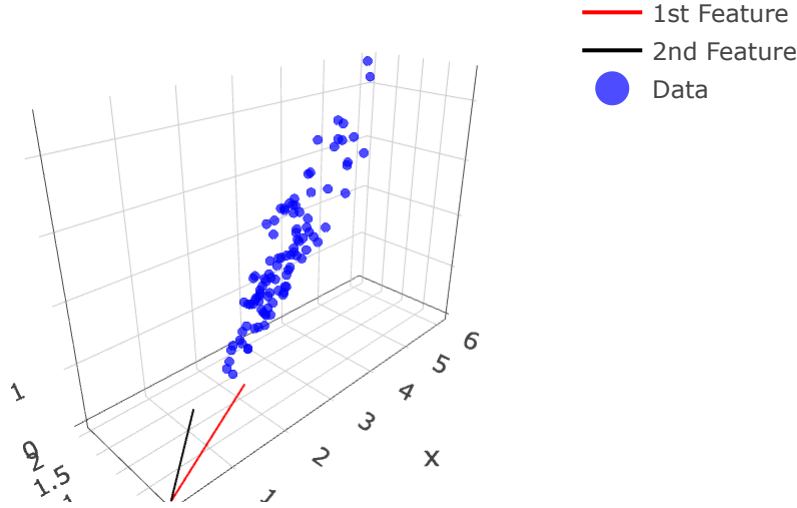


Figure 4.7: Data in the positive span of two vectors

In the motivating example above, our data live in the positive span of the two feature vectors, thus we say the data matrix $\mathbf{X}$ has a nonnegative matrix factorization $\mathbf{WH}$.

Thus, we may view NMF as a restricted version of PCA or SVD where we have moved from spans to positive spans. This seemingly small change gives rise to some big problems. Even in this simple case above we have a uniqueness problem. Up to sign changes and ordering, the feature vectors in PCA and SVD were unique. However, we can find two alternative vectors $\vec{h}'_1$ and $\vec{h}'_2$ which still give an exact NMF. There are trivial cases. First, one can change ordering ($\vec{h}'_1 = \vec{h}_2$ and $\vec{h}'_2 = \vec{h}_1$) which we avoid in PCA and SVD by assuming the corresponding singular values or PC variances are listed in decreasing order. Secondly, we could rescale by setting $\vec{h}'_1 = c\vec{h}_1$ and rescaling the corresponding coefficients by a factor of $1/c$ for some $c > 0$ which PCA and SVD avoid by fixing feature vectors to have unit length. The ordering issue is a labeling concern and may be ignored, whereas the rescaling issue can be addressed by adding constraints on the length of the feature vectors.

A third and far more subtle issue occurs because we do not enforce orthogonality. In 4.7, imagine that the feature vectors are the arms of a folding fan. We could change the angle between our feature vectors by opening or closing the arms of the fan. So long as we do not close the fan too much (and leave our data outside the positive span) or open them too much (so that feature vectors have negative entries), we can still find a perfect reconstruction. This ‘folding fan’ issue can be addressed through additional penalties which we discuss in 4.3.5, but nonuniqueness cannot be eliminated entirely. Thus, we seek **an** NMF for

our data rather than **the** NMF.

4.3.3 Finding a NMF: Multiplicative Updates

For a given choice of error, the lack of a unique solution also means there is no closed form solution for $\mathbf{W}$ and $\mathbf{H}$. Thus, we will need to apply numerical optimization to find a $\mathbf{W}$ and $\mathbf{H}$ which minimizes the selected error,

$$\operatorname{argmin}_{\mathbf{W} \in \mathbb{R}_{\geq 0}^{N \times k}, \mathbf{H} \in \mathbb{R}_{\geq 0}^{k \times d}} D(\mathbf{X}, \mathbf{WH})$$

. The loss is a function of all of the entries of $\mathbf{W}$ and $\mathbf{H}$. To apply any type of gradient based optimization, we need to compute the partial derivative of our loss with respect to each of the entries of these matrices.

As an example, we will focus on the divergence and show the relevant details. For gradient based optimization, we then need to compute $\frac{\partial}{\partial \mathbf{W}_{ij}} D(\mathbf{X} \| \mathbf{WH})$ for $1 \leq i \leq N$ and $1 \leq j \leq k$ and $\frac{\partial}{\partial \mathbf{H}_{j\ell}} D(\mathbf{X} \| \mathbf{WH})$ for $1 \leq j \leq k$ and $1 \leq \ell \leq d$. Note that

$$(\mathbf{WH})_{st} = \sum_{j=1}^k \mathbf{W}_{sj} \mathbf{H}_{jt}$$

so that

$$\frac{\partial (\mathbf{WH})_{st}}{\partial (\mathbf{WH})_{ij}} = \begin{cases} \mathbf{H}_{jt} & s = i \\ 0 & s \neq i \end{cases}$$

Thus, we may make use of the chain rule to conclude that

$$\begin{aligned} \frac{\partial}{\partial \mathbf{W}_{ij}} D(\mathbf{X} \| \mathbf{WH}) &= \frac{\partial}{\partial \mathbf{W}_{ij}} \sum_{st} (\mathbf{X}_{st} \log((\mathbf{X})_{st}) - \mathbf{X}_{st} \log((\mathbf{WH})_{st}) + (\mathbf{WH})_{st} - (\mathbf{X})_{st}) \\ &= \frac{\partial}{\partial \mathbf{W}_{ij}} \sum_{st} (-\mathbf{X}_{st} \log((\mathbf{WH})_{st}) + (\mathbf{WH})_{st}) \\ &= \sum_{st} \left(\frac{\mathbf{X}_{st}}{(\mathbf{WH})_{st}} + 1 \right) \frac{\partial (\mathbf{WH})_{st}}{\partial \mathbf{W}_{ij}} \\ &= \sum_{t=1}^d \left(-\frac{\mathbf{X}_{it}}{(\mathbf{WH})_{it}} + 1 \right) \mathbf{H}_{jt}. \end{aligned}$$

A similar calculation gives

$$\frac{\partial}{\partial \mathbf{H}_{ij}} D(\mathbf{X} \| \mathbf{WH}) = \sum_{s=1}^N \left(-\frac{\mathbf{X}_{sj}}{(\mathbf{WH})_{sj}} + 1 \right) \mathbf{W}_{sj}.$$

In a standard implementation of gradient descent, we choose a step size $\epsilon > 0$

and apply the updates

$$\begin{aligned}\mathbf{W}_{ij} &\leftarrow \mathbf{W}_{ij} - \epsilon \sum_{t=1}^d \left(-\frac{\mathbf{X}_{it}}{(\mathbf{WH})_{it}} + 1 \right) \mathbf{H}_{jt} \\ \mathbf{H}_{ij} &\leftarrow \mathbf{H}_{ij} - \epsilon \sum_{s=1}^N \left(-\frac{\mathbf{X}_{sj}}{(\mathbf{WH})_{sj}} + 1 \right) \mathbf{W}_{sj}\end{aligned}\tag{4.8}$$

to each entry of $\mathbf{W}$ and $\mathbf{H}$ simultaneously. Alternatively, we could consider coordinate descent where we update each entry of $\mathbf{W}$ (holding all other entries of $\mathbf{W}$ and $\mathbf{H}$ constant) separately then do the same for $\mathbf{H}$ then repeat. Each approach will converge to a local mode (though possibly different ones) when ϵ is sufficiently small. However, if ϵ is too small it may take many iterations to converge. Unfortunately, choosing ϵ can create a numerically unstable algorithm (that doesn't converge at all) or one where we lose the nonnegativity condition on the entries of $\mathbf{W}$ or $\mathbf{H}$.

To preserve nonnegativity, [Lee and Seung, 2000] developed a set of state dependent step-sizes (one for each entry of $\mathbf{W}$ and $\mathbf{H}$) which result in multiplicative rather than additive updates. For divergence, they take step sizes

$$\epsilon_{ij}^W = \frac{\mathbf{W}_{ij}}{\sum_t \mathbf{H}_{jt}} \text{ and } \epsilon_{ij}^H = \frac{\mathbf{H}_{ij}}{\sum_s \mathbf{W}_{sj}}.$$

These step sizes are proportional to the entries we are updating so that we take larger steps for larger entries of $\mathbf{W}$ or $\mathbf{H}$. If we substitute these step sizes in, the updates simplify to

$$\begin{aligned}\mathbf{W}_{ij} &\leftarrow \mathbf{W}_{ij} \left[\frac{\sum_t (\mathbf{H}_{jt} \mathbf{X}_{it} / (\mathbf{WH})_{it})}{\sum_t \mathbf{H}_{jt}} \right] \\ \mathbf{H}_{ij} &\leftarrow \mathbf{H}_{ij} \left[\frac{\sum_s (\mathbf{W}_{si} \mathbf{X}_{sj} / (\mathbf{WH})_{sj})}{\sum_s \mathbf{W}_{si}} \right]\end{aligned}\tag{4.9}$$

indicating that we rescale the entries of $\mathbf{W}$ and $\mathbf{H}$ by some nonnegative value which is guaranteed to preserve the nonnegativity of each entry. A short proof showing convergence to a local mode, along with multiplicative update rules for other choices of loss, may be found in the original paper by Lee and Seung [Lee and Seung, 2000]. Deriving these updates is also provided as an exercise for the reader. Furthermore, one can verify that the multiplicative updates simplify to one when $\mathbf{X} = \mathbf{WH}$.

We can more compactly write the updates for each matrix using Hadamard notation, which gives the following multiplicative updates for $\mathbf{W}$ and $\mathbf{H}$

$$\begin{aligned}\mathbf{W} &\leftarrow \mathbf{W} \odot \{ [\mathbf{X} \odot (\mathbf{WH})] \mathbf{H}^T \} \odot [\mathbf{1}_{N \times d} \mathbf{H}^T] \\ \mathbf{H} &\leftarrow \mathbf{H} \odot \{ \mathbf{W}^T [\mathbf{X} \odot (\mathbf{WH})] \} \odot [\mathbf{W}^T \mathbf{1}_{N \times d}]\end{aligned}\tag{4.10}$$

In most implementations of NMF, multiplicative updates for $\mathbf{W}$ and $\mathbf{H}$ are alternated until convergence. However, some packages also implement rescaling following each pair of multiplicative updates to enforce other constraints (such as sum to one constraints on the rows of $\mathbf{W}$ or columns of $\mathbf{H}$). When using NMF it is good to check documentation to see if these additional steps are implemented.

4.3.4 NMF in practice

For a given choice of loss, standard NMF proceeds in the following manner.

- 1) Choose a rank k and initial $\mathbf{W} \in \mathbb{R}_{\geq 0}^{N \times k}$ and $\mathbf{H} \in \mathbb{R}_{\geq 0}^{k \times d}$.
- 2) Apply the multiplicative rule until a local minimum is reached.
- 3) Repeat steps (1) and (2) for different initial conditions then select the final $\mathbf{W}$ and $\mathbf{H}$ which give the lowest overall loss.

Different initial conditions (ICs) will converge to different local modes; repetition over multiple different ICs will help find a better overall minimizer and avoid getting a final NMF which is trapped in a poor local mode. There is no guidance on how many ICs to use. Many packages default to five. More would be better but computational resources are not infinite, and finding even one mode though coordinate ascent can be slow. There are also methods based on nonnegativity constraints of the SVD (aptly named nonnegative SVD or NNSVD) for initialization which have shown good performance [Liu et al., 2008]. However, NNSVD initialization does increase computation burden.

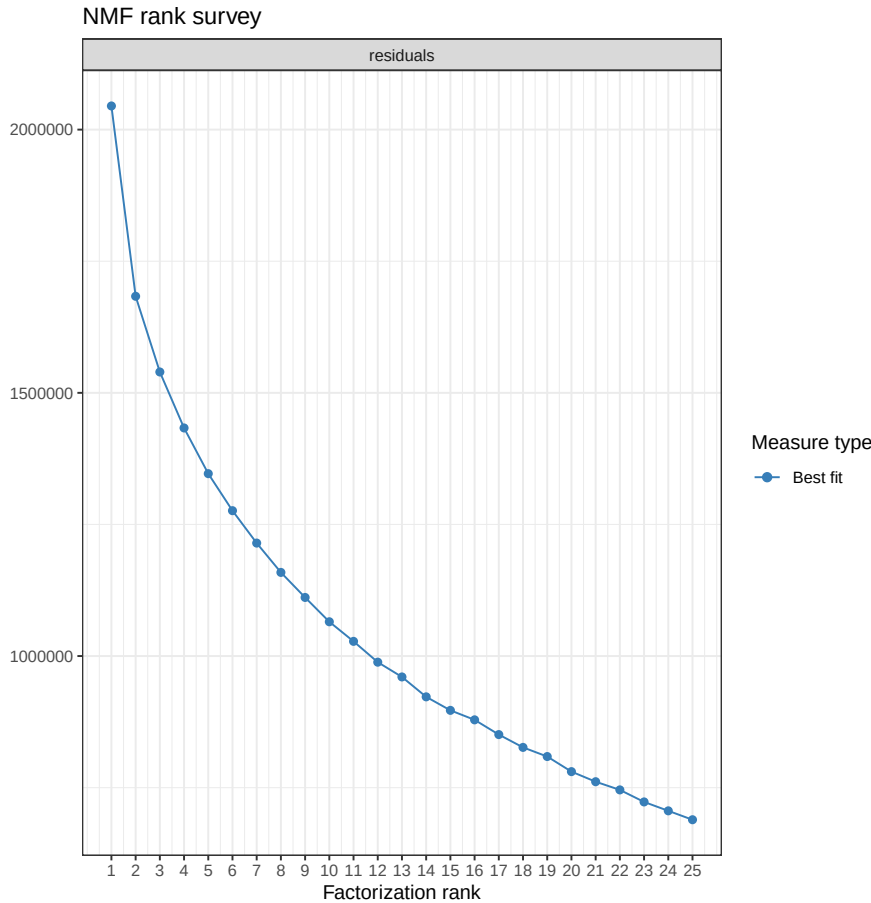
The choice of k is also challenging. If prior information or other project constraints dictate a specific choice of k one should use that value. Alternatively, if visualization is a priority taking $k = 1, 2$, or 3 is a natural choice. Unfortunately, in many cases prior information is typically absent, thus data driven methods for selecting k from a range of possible choices are required. Importantly, since there are no connections between the rank k NMF and rank $k + 1$ NMF, one cannot truncate the vectors and coefficients and attain an optimal solution for a lower-dimensional representation as we can in PCA or SVD. Thus, separate fits at each choice of k must be attained through separate runs of the numerical optimization making NMF far more computationally demanding in practice.

Suppose for now, we have optimal NMF fits for a range of nonnegative ranks obtained by fitting separate NMF models for a range of ranks using multiple initial conditions or NNSVD based initialization at each rank. Given these fits, one natural approach is to follow a **scree plot** like approach similar to what we have used for PCA and SVD. One can then plot the loss as a function of k and look for an elbow where the error appears to saturate.

Much like SVD and PCA, unfortunately, there is often no clear elbow indicating an obvious choice of k . As such, additional approaches have been suggested in the literature such as cross-validation wherein the data is split into test-train sets and the robustness of loss is investigated at each rank. Bayesian methods for balancing low loss with model complexity (small k) are also viable but again

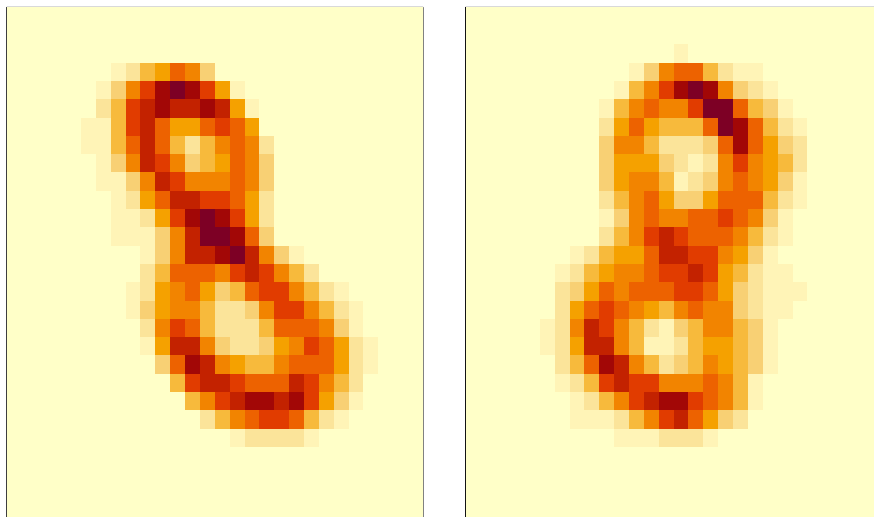
rely on some prior on model complexity. Other authors have used connections between NMF and clustering to adapt methods for assessing the quality of (hierarchical) clustering to the case of finding an optimal k in NMF. For example, one can use the robustness (or lack thereof) of clustering of data at each rank measured by the cophenetic correlation to identify an optimal k [Brunet et al., 2004]. We'll discuss hierarchical clustering and the cophenetic correlation in more detail in 6. For now, we'll restate the advice from [Brunet et al., 2004] and advocate for the value of k at which the cophenetic correlation begins to drop.

Example 4.2 (NMF applied to MNIST). As an example, we show NMF applied to the first 100 eights (to save computation time) from the MNIST dataset [Deng, 2012] using Divergence with five runs per rank. Below we fit separate NMFs for ranks 1 through 25 with 5 random initializations per fit. Divergence (residuals) and the cophenetic correlation are each shown below as a function of rank.

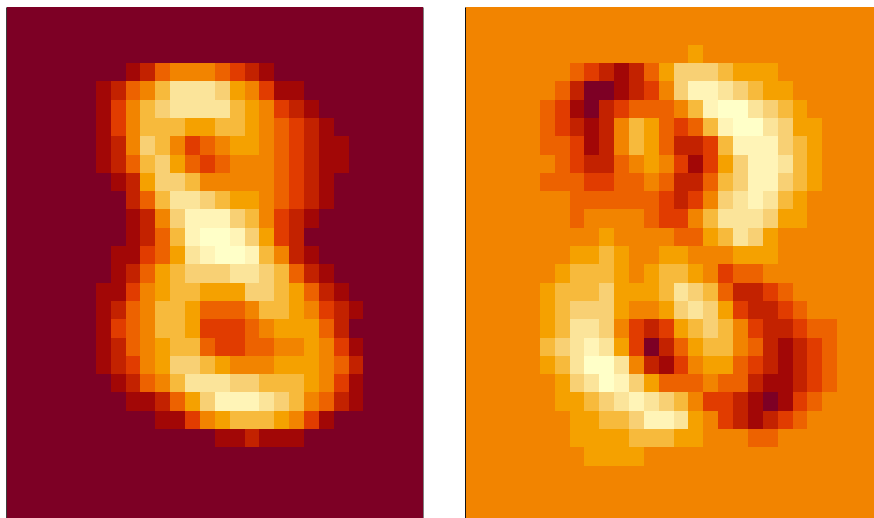


There is no clear elbow in the plot of the model residuals that clearly suggests an optimal rank between 1 and 25. The cophenetic correlation begins to drop after

rank 2. Thus, the method of [Brunet et al., 2004] suggests $k = 2$ is reasonable. In fact if we select a rank 2 NMF, we do get interpretable 8-like images as our feature images(vectors) which correspond to eights slanting in two different directions, a seemingly reasonable compression of structure in the data.



Compare this to the less 8-like images found using the features given by a rank 2 approximation from SVD.



4.3.5 Regularization and Interpretability

For a random initialization, the values of $\mathbf{W}$ and $\mathbf{H}$ obtained when the multiplicative rules are run to convergence do not resemble the original data, which is contrary to our original goal of finding features which are more comparable to the data. More specifically, the feature vectors $\vec{h}_1, \dots, \vec{h}_k$ which are superimposed to approximate the observations may appear quite different from the observations themselves. Many extensions of NMF address this issue through the inclusion of additional penalties on the elements of $\mathbf{W}$ or $\mathbf{H}$ which induce sparsity in the coefficients $\mathbf{W}$ and/or constrain the features to be more similar to the data. In practice, it is insufficient to apply penalties which depend solely on the scaling of either $\mathbf{W}$ or $\mathbf{H}$ as these can typically be made arbitrarily small by increasing corresponding elements of $\mathbf{H}$ or $\mathbf{W}$ respectively. Thus, any penalty which depends on the scaling of one matrix often includes additional constraints or penalties on the other.

Briefly, we discuss two version which are constructed to generate features which strongly mimic the original observations.

4.3.5.1 Archetypes

Before discussing Archetypal Analysis, we need an additional geometric idea.

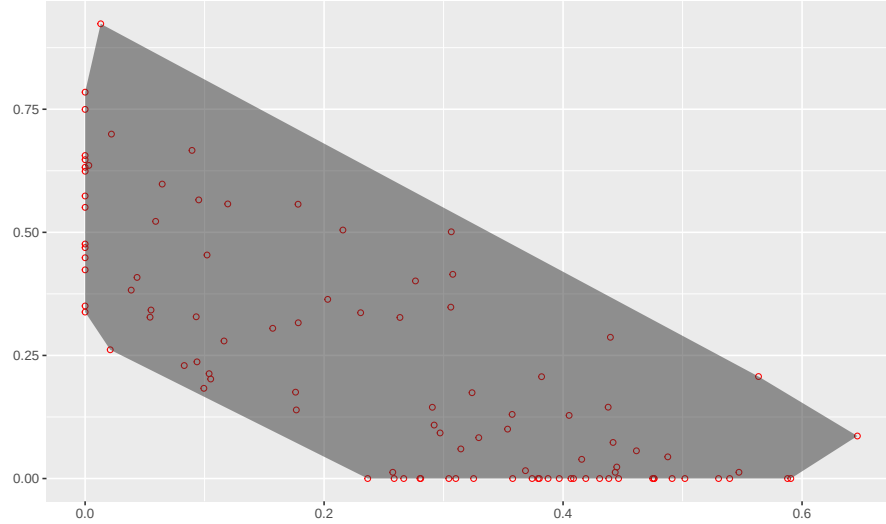
Definition 4.3 (Convex Hull). Given a set of vectors $\mathcal{V} = \{\vec{h}_1, \dots, \vec{h}_k\} \subset \mathbb{R}^d$, the convex hull of $\mathcal{V}$ is the set

$$\text{conv}(\mathcal{V}) = \{b_1\vec{h}_1 + \dots + b_k\vec{h}_k : b_1, \dots, b_k \geq 0, b_1 + \dots + b_k = 1\}$$

The weights in the above definition are constrained to be positive, similar to the definition of positive span, but with the added constraint that they sum to one. Thus, for a set of vectors $\mathcal{V}$

$$\text{conv}(\mathcal{V}) \subset \Gamma(\mathcal{V}) \subset \text{span}(\mathcal{V}).$$

An example of a convex hull (shown in grey) of set of vectors (points in red) in $\mathbb{R}^2$ is shown below. Imagine wrapping a string around the set of points and pulling it tight. Everything on/within the loop is in the convex hull. An analogous interpretation holds in higher dimensions using a “sheet” rather than a string.

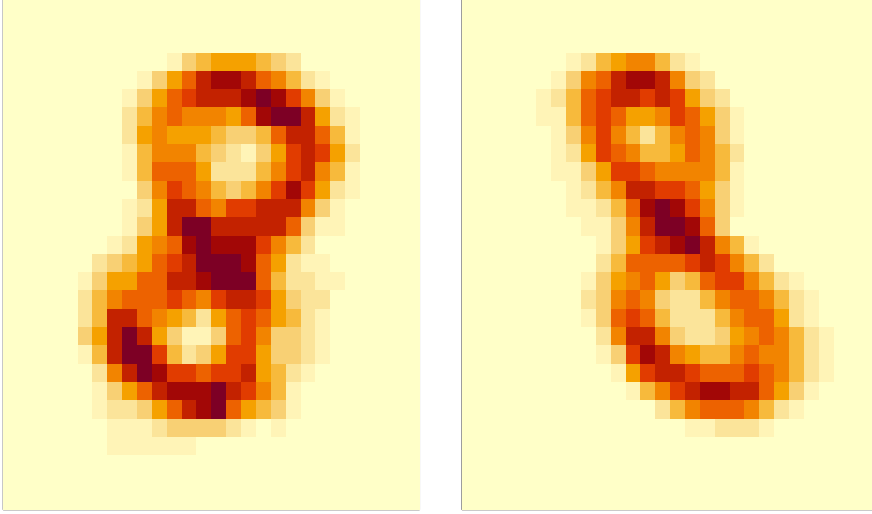


Importantly, we can view the coefficients $b_1, \dots, b_k$ as defining (probability) weights. Thus, every point in the convex hull of $\mathcal{V}$ is a weighted average of the vectors in $\mathcal{V}$ and one then expects any point in the convex hull of $\mathcal{V}$ to resemble the vectors in $\mathcal{V}$. In archetypal analysis [Cutler and Breiman, 1994], we add the constraint that the features (row of $\mathbf{H}$) are themselves elements in the convex hull of the original data so that $\mathbf{H} = \mathbf{B}\mathbf{X}$ where $\mathbf{B} \in \mathbb{R}_{\geq 0}^{k \times N}$ must satisfy the constraint $\mathbf{B}\tilde{\mathbf{1}}_N = \tilde{\mathbf{1}}_k$.

Under this setup, we then want to solve the following constrained optimization problem

$$\operatorname{argmin}_{\mathbf{W} \in \mathbb{R}_{\geq 0}^{N \times k}, \mathbf{B} \in \mathbb{R}_{\geq 0}^{k \times N}, \mathbf{B}\tilde{\mathbf{1}}_N = \tilde{\mathbf{1}}_k} \|\mathbf{X} - \mathbf{W}\mathbf{B}\mathbf{X}\|_F.$$

The rows of $\mathbf{B}\mathbf{X}$, called archetypes, are typically extremal in the sense that they reside on the boundary of the convex hull of the data. We show them below for a rank two approximation to the eights consistent with the choice in the example above.



4.3.5.2 Volume Regularized NMF

Much like Archetype Analysis, Volume regularized NMF creates feature vectors which resemble the observation through a more geometric approach [Lin et al., 2014, Fu et al. [2015]]. Suppose that we are seeking a rank k approximation of data in d dimensions with $k < d$. If the vectors $\vec{h}_1, \dots, \vec{h}_k$ are affinely independent, i.e. they don't live on a hyperplane of dimension less than k , then the volume of the convex hull of the vectors is given by the following formula

$$\text{vol}\left(\text{conv}(\{\vec{h}_1, \dots, \vec{h}_k\})\right) = \frac{\sqrt{\det(\tilde{\mathbf{H}}\tilde{\mathbf{H}}^T)}}{(k-1)!}$$

where

$$\tilde{\mathbf{H}} = \begin{bmatrix} \vec{h}_1^T - \vec{h}_k^T \\ \vdots \\ \vec{h}_{k-1}^T - \vec{h}_k^T \end{bmatrix} \in \mathbb{R}^{(k-1) \times d}.$$

An extension of this result is that for linearly independent vectors $\vec{h}_1, \dots, \vec{h}_k$, the volume of the convex hull of the vectors $\vec{0}, \vec{h}_1, \dots, \vec{h}_k$ can be computed directly from our feature matrix $\mathbf{H}$, i.e.

$$\text{vol}\left(\text{conv}(\{\vec{0}, \vec{h}_1, \dots, \vec{h}_k\})\right) = \frac{\sqrt{\det(\mathbf{H}\mathbf{H}^T)}}{k!}.$$

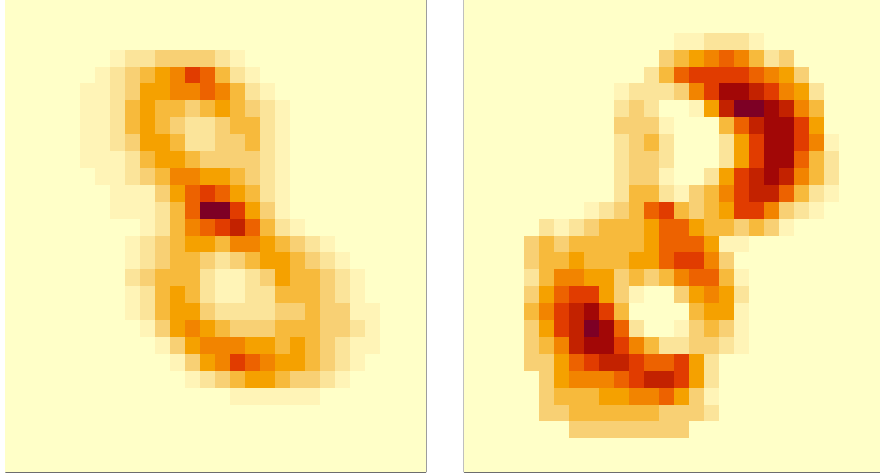
One brief aside on the notion of volume here that is not typically made clear in the related literature. When $k < d$, the convex hull of $\vec{0}, \vec{h}_1, \dots, \vec{h}_k$ is contained in a k -dimensional hyperplane. Thus, its d -dimensional volume must be zero.

When we talk of the volume of a convex hull, we instead mean its k -dimensional volume. As an example, consider the case of a single vector $\vec{h}_1$ in $\mathbb{R}^3$. The convex hull of $\vec{0}, \vec{h}_1$ is the line segment from the origin to $\vec{h}_1$. Line segments have zero 3-dimensional volume, but they do have length (1-dimensional volume). In this case, the 1-dimensional volume of the convex hull is the length of the line segment, e.g. $\|\vec{h}_1\|$.

Incorporating a penalty on the volume of the convex hull of the feature vectors gives rise to the aptly named volume-regularized NMF

$$\operatorname{argmin}_{\mathbf{W} \in \mathbb{R}_{\geq 0}^{N \times k}, \mathbf{W} \vec{1}_N = \vec{1}_k, \mathbf{H} \in \mathbb{R}_{\geq 0}^{k \times d}} \|\mathbf{X} - \mathbf{WH}\|_F^2 + \lambda \sqrt{\det(\mathbf{HH}^T)}$$

here the constraint $\mathbf{W} \vec{1}_N = \vec{1}_k$ requiring that the rows of $\mathbf{W}$ sum to one prevents an arbitrary shrinkage of the feature vectors towards zero. In some numerical implementation, this constraint is relaxed so that the sum of the rows of $\mathbf{W}$ are bounded above by one. The value λ is a tuning parameter that must be specified by the user to balance goodness of fit vs minimal volume. Below we show the results of volume regularized NMF applied to the subset of the eights data studied above which strongly resemble results from the standard NMF implementation. However, this method was far more computationally efficient. Only one run of the VR-NMF was required, and it converged much more quickly than the vanilla NMF.



4.3.6 NMF and Maximum Likelihood Estimation

Throughout this section we have focused on the geometric aspects of NMF, but there are parallel statistical perspectives based on maximum likelihood estimation. For example, consider the model

$$\mathbf{X}_{ij} \mid (\mathbf{WH})_{ij} \sim \mathcal{N}((\mathbf{WH})_{ij}, \sigma^2), \quad 1 \leq i \leq N, 1 \leq j \leq d$$

with the additional assumption that the observations in $\mathbf{X}$ are conditionally independent given $\mathbf{WH} \in \mathbb{R}_{\geq 0}^{N \times d}$, $\mathbf{H} \in \mathbb{R}_{\geq 0}^{k \times d}$ and σ . Under this framework the likelihood of the data is then

$$\mathcal{L}(\mathbf{X} \mid \mathbf{WH}) \propto \frac{1}{\sqrt{2\pi\sigma^2}} \prod_{i=1}^N \prod_{j=1}^d \exp\left(-\frac{(\mathbf{X}_{ij} - (\mathbf{WH})_{ij})^2}{\sigma^2}\right)$$

which has log-likelihood

$$\log \mathcal{L}(\mathbf{X} \mid \mathbf{WH}) = -\frac{N}{2} \log \sigma^2 - \frac{1}{\sigma^2} \sum_{i=1}^N \sum_{j=1}^d (\mathbf{X}_{ij} - (\mathbf{WH})_{ij})^2 = -\frac{N}{2} \log \sigma^2 - \frac{1}{\sigma^2} \|\mathbf{X} - \mathbf{WH}\|_F^2.$$

Thus, minimizing the Frobenius norm is equivalent to maximizing the likelihood in this normal model with homoskedastic errors.

Similarly, divergence arises from a Poisson model

$$\mathbf{X}_{ij} \mid (\mathbf{WH})_{ij} \sim \text{Pois}((\mathbf{WH})_{ij})$$

again with a low rank structure for $\mathbf{W}$ and $\mathbf{H}$ and the conditional independence assumption on the entries of $\mathbf{X}$ given $\mathbf{WH}$.

The IS divergence again uses conditional independence and a low rank structure for $\mathbf{WH}$ under the assumption of multiplicative Gamma distributed errors with mean one, i.e.

$$\mathbf{X}_{ij} \mid (\mathbf{WH})_{ij} = (\mathbf{WH})_{ij} e_{ij}, \quad e_{ij} \sim_{iid} \text{Gamma}(\alpha, \beta)$$

with $\alpha/\beta = 1$. See [Févotte et al., 2009] for additional details.

4.4 Multidimensional Scaling

Multidimensional scaling (MDS) is a broad name for a myriad of different methods which are designed to handle a common problem. Suppose there are N objects of interest in a dataset. Examples include a set of geographic locations or cells from mass spectrometry or census blocks, etc. Importantly, we do not need actual data corresponding to each object. In MDS, we instead require a measure the distance or dissimilarity between each pair of objects. We can organize these distances into a matrix $\Delta \in \mathbb{R}^{N \times N}$, with Δ_{rs} representing the distance/dissimilarity between objects r and s . Thus, $\Delta_{rr} = 0$ and for now, we may assume that $\Delta_{sr} = \Delta_{rs}$. The matrix Δ is often called a *Distance* or *Dissimilarity* matrix. In practice, we may construct a distance matrix from observations, but for the purposes of MDS we only require Δ .

The primary goal of MDS is to find a set of lower-dimensional points $\vec{y}_1, \dots, \vec{y}_N \in \mathbb{R}^t$ corresponding to each of the N objects such that the distance between $\vec{y}_r$ and $\vec{y}_s$ is close to Δ_{rs} . There are numerous different notions of distance one

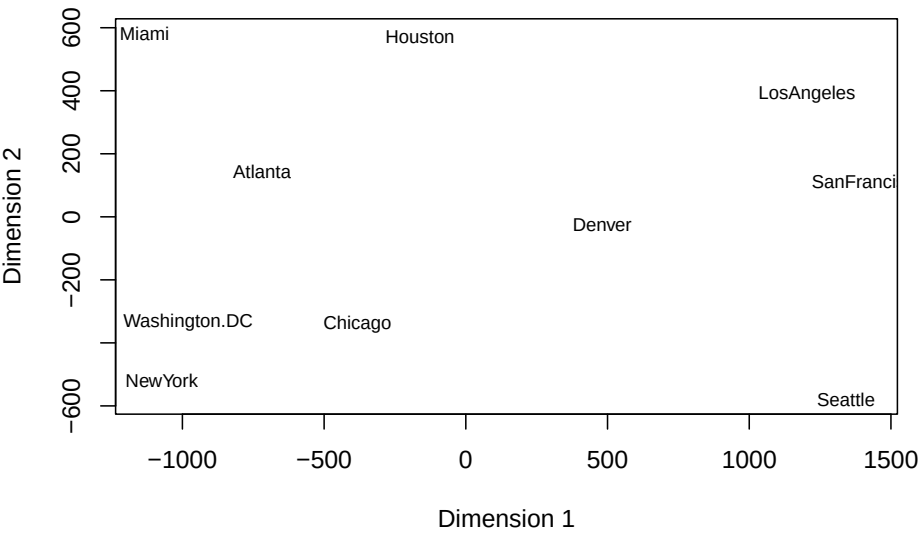
use in $\mathbb{R}^t$. Additionally, we may not know the notion of distance/dissimilarity used when computing Δ or if the dissimilarity corresponds to any well defined notion of distance. Euclidean distance is a common choice for the distance of the lower-dimensional vectors. Fixing this choice still leaves many open questions. Are the original distances Euclidean? If so, can we determine the dimensionality of the original data?

MDS also serves as a data visualization method. In this case, t is typically chosen to be either two or three, and the N points in the two-dimensional (or three-dimensional) representation may be plotted so that one can visualize the relationships between data. Before turning to the most common methods of MDS, let's view a few examples using distances between cities.

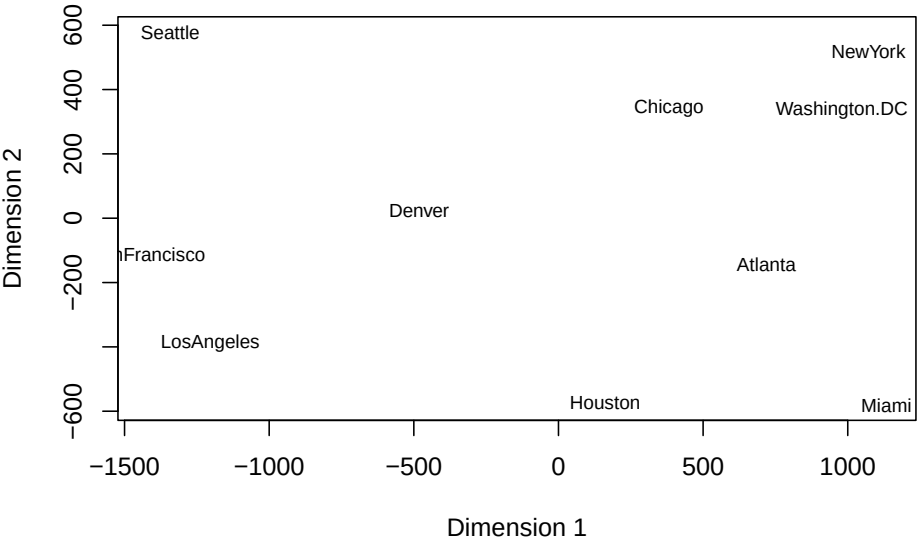
Example 4.3 (Classical scaling applied to distances between cities). The *eurodist* R package provides air travel distances between 21 cities in Europe and 10 cities in the US. For the moment, we will focus on the 10 US cities with names and distances given in the following tables

	Atlanta	Chicago	Denver	Houston	LosAngeles	Miami	NewYork	SanFran
Atlanta	0	587	1212	701	1936	604	748	
Chicago	587	0	920	940	1745	1188	713	
Denver	1212	920	0	879	831	1726	1631	
Houston	701	940	879	0	1374	968	1420	
LosAngeles	1936	1745	831	1374	0	2339	2451	
Miami	604	1188	1726	968	2339	0	1092	
NewYork	748	713	1631	1420	2451	1092	0	
SanFrancisco	2139	1858	949	1645	347	2594	2571	
Seattle	2182	1737	1021	1891	959	2734	2408	
Washington.DC	543	597	1494	1220	2300	923	205	

After conducting classical scaling (a method of MDS, details will be discussed later), we acquire the plot below. The plot is consistent with the geographical relationships between the cities. Miami and Seattle are the farthest apart in our plot. NY and D.C. are quite close on the plot, which is also true for Los Angeles and San Francisco.



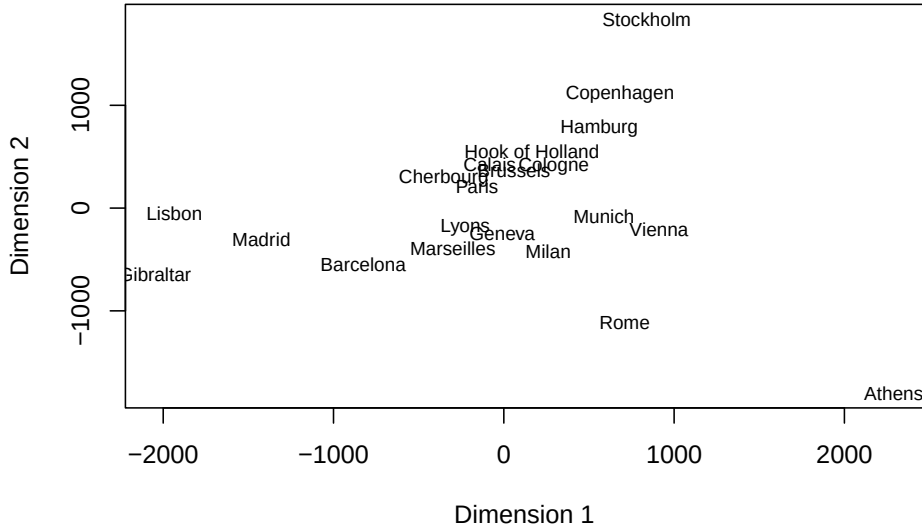
If we rotate the above plot 180 degrees, we recover an representation of the cities consistent with the typical map of the US.



Now let's try MDS on the 21 European cities

	Athens	Barcelona	Brussels	Calais	Cherbourg	Cologne	Copenhagen	Geneva	Gibra
Athens	0	3313	2963	3175	3339	2762	3276	2610	4
Barcelona	3313	0	1318	1326	1294	1498	2218	803	1
Brussels	2963	1318	0	204	583	206	966	677	2
Calais	3175	1326	204	0	460	409	1136	747	2
Cherbourg	3339	1294	583	460	0	785	1545	853	2

	Athens	Barcelona	Brussels	Calais	Cherbourg	Cologne	Copenhagen	Gen
Cologne	2762	1498	206	409	785	0	760	1
Copenhagen	3276	2218	966	1136	1545	760	0	1
Geneva	2610	803	677	747	853	1662	1418	1
Gibraltar	4485	1172	2256	2224	2047	2436	3196	1
Hamburg	2977	2018	597	714	1115	460	460	1
Hook of Holland	3030	1490	172	330	731	269	269	1
Lisbon	4532	1305	2084	2052	1827	2290	2971	1
Lyons	2753	645	690	739	789	714	1458	1
Madrid	3949	636	1558	1550	1347	1764	2498	1
Marseilles	2865	521	1011	1059	1101	1035	1778	1
Milan	2282	1014	925	1077	1209	911	1537	1
Munich	2179	1365	747	977	1160	583	1104	1
Paris	3000	1033	285	280	340	465	1176	1
Rome	817	1460	1511	1662	1794	1497	2050	1
Stockholm	3927	2868	1616	1786	2196	1403	650	2
Vienna	1991	1802	1175	1381	1588	937	1455	1

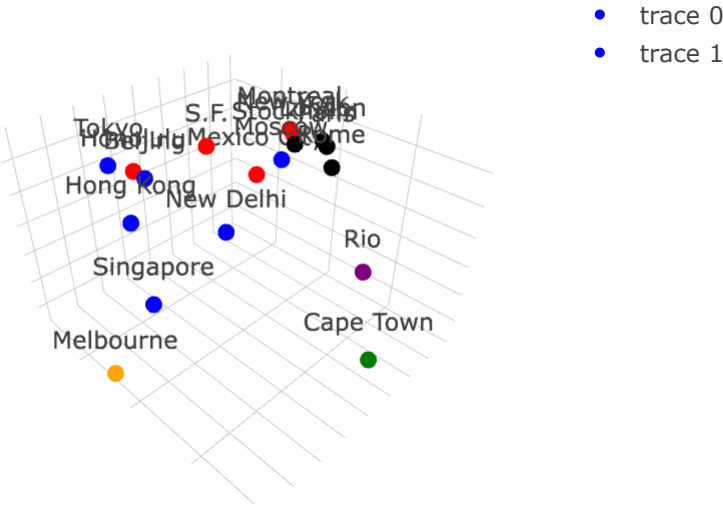


After applying a reflection of the map given by classical scaling to help it comply to the conventional orientation of a European map, the above plot reconstructs the European map quite well. Gibraltar, Lisbon and Madrid are in the south-west corner, the two North European cities Stockholm and Copenhagen are in the north end, and Athens is in the south-west corner.

Finally, let's consider the 18 representative global cities. Their pairwise flight lengths (geodesic distance) are shown in the table below. As we can see, the geodesic distances between the three Southern-Hemisphere cities: Rio, Cape

Town, Melbourne and other Northern-Hemisphere cities are generally large (almost all over 10,000 kilometers)

	Beijing	Cape Town	Hong Kong	Honolulu	London	Melbourne	Mexico City	Montreal	Moscow
Beijing	0	12947	1972	8171	8160	9093	12478	10490	5809
Cape Town	12947	0	11867	18562	9635	10388	13703	12744	10101
Hong Kong	1972	11867	0	8945	9646	7392	14155	12462	7158
Honolulu	8171	18562	8945	0	11653	8862	6098	7915	11342
London	8160	9635	9646	11653	0	16902	8947	5240	2506
Melbourne	9093	10388	7392	8862	16902	0	13557	16730	14418
Mexico City	12478	13703	14155	6098	8947	13557	0	3728	10740
Montreal	10490	12744	12462	7915	5240	16730	3728	0	7077
Moscow	5809	10101	7158	11342	2506	14418	10740	7077	0
New Delhi	2788	9284	3770	11930	6724	10192	14679	11286	3362
New York	11012	12551	12984	7996	5586	16671	3362	533	11930
Paris	8236	9307	9650	11988	341	16793	9213	5522	6724
Rio	17325	6075	17710	13343	9254	13227	7669	8175	10860
Rome	8144	8417	9300	12936	1434	15987	10260	6601	6050
S.F.	9524	16487	11121	3857	8640	12644	3038	4092	16623
Singapore	4465	9671	2575	10824	10860	6050	16623	14816	9603
Stockholm	6725	10334	8243	11059	1436	15593	9603	5900	11319
Tokyo	2104	14737	2893	6208	9585	8159	11319	10409	0



The three-dimensional visualization result of classical MDS is shown above. You can rotate and magnify it on your laptop. The blue points represent Asian cities, the black points represent European cities, and the red points represent North American cities. If you inspect the plot clearly, you may notice that the cities

appear to be constrained to the surface of a sphere, which complies to the true scenario.

In each of the examples, classical MDS was quite successful in generating maps which reflected the geographical configuration with continental or global maps which were (after some reflections/rotations) consistent with conventional maps. Let's discuss the details.

4.4.1 Key features of MDS

MDS can be divided into three major types: Classical Scaling; Metric MDS; and Non-Metric MDS. The choice of method depends on the specifics of the distance/dissimilarity matrix and features which are preferential to preserve. Classical Scaling and Metric MDS require that the provided distances correspond to a metric (more on this below), while Non-metric MDS is usually used when the input data doesn't satisfy the properties of a true distance metric or when the relationships are ordinal (i.e., we only know which distances are larger, but not by how much).

Beyond demonstrating the capacity of MDS to recover meaningful visualization, we also gain two insights from the examples above which hold for any MDS algorithm.

- i) As we can tell from the recovery of US map and European map, the configurations of $\tilde{y}_1, \tilde{y}_2, \dots, \tilde{y}_N$ are not unique, as we can rotate or flip the map. Actually, if $\tilde{y}_1, \tilde{y}_2, \dots, \tilde{y}_N \in \mathbb{R}^{t'}$ is considered as the optimal solution using Euclidean distance, then given any vector $\vec{b} \in \mathbb{R}^{t'}$ and orthogonal matrix $A \in \mathbb{R}^{t' \times t'}$, $\|(A\tilde{y}_r + \vec{b} - (A\tilde{y}_s + \vec{b}))\| = \|\tilde{y}_r - \tilde{y}_s\|$. Rotation, Reflection or Translation don't alter the pairwise distances. So $A\tilde{y}_1 + \vec{b}, A\tilde{y}_2 + \vec{b}, \dots, A\tilde{y}_N + \vec{b}$ is also an optimal solution. Outside of Euclidean distances, reflections are still possible which eliminate the possibility of unique configurations.
- ii) The pairwise distance between two objects need not be Euclidean. In the above example, they are actually great circle distance.

In the following subsections, we will discuss the specifics of classical scaling with a brief discussion on two common methods of MDS, one each in metric and nonmetric MDS.

4.4.2 Classical Scaling

Let us first introduce some important definitions.

Definition 4.4 (Distance Matrix). A matrix $\Delta \in \mathbb{R}^{N \times N}$ is called a distance matrix if it possesses the following properties:

- a) Symmetry: Δ is symmetric, meaning that: $\Delta_{rs} = \Delta_{sr}$ for all r and s

- b) Zero Diagonal: The diagonal entries of the matrix represent the distance of a point to itself, and are thus all zeros: $\Delta_{rr} \equiv 0$ for all $1 \leq r \leq N$
- c) Non-negativity: All distances are non-negative: $\Delta_{rs} \geq 0$
- d) Triangle Inequality: The distances in the matrix respect the triangle inequality:

$$\Delta_{rs} \leq \Delta_{rt} + \Delta_{ts}$$

Distance matrices can be computed in many different ways. We will focus on the following specific case.

Definition 4.5 (Euclidean Distance Matrix). A distance matrix $\Delta \in \mathbb{R}^{N \times N}$ is a Euclidean distance matrix if there exists a configuration $\vec{y}_1, \vec{y}_2, \dots, \vec{y}_N$ s.t. Δ_{rs} represents the Euclidean distance between points r and s , i.e., $\|\vec{y}_r - \vec{y}_s\| = \Delta_{rs} \forall r, s$.

Classical scaling operates under the assumption that Δ is a Euclidean distance matrix though the dimensionality of the configuration $\vec{y}_1, \dots, \vec{y}_N$ which gives rise to the distances specified by Δ is unknown. As we shall see, the minimum dimensionality needed for the configuration to give the specified distances (assuming it exists at all) will be discovered through the classical scaling algorithm. Furthermore, since we are free to translate any configuration without altering pairwise distance, we will seek a configuration $\vec{y}_1, \vec{y}_2, \dots, \vec{y}_N$ which is centered to simplify the following computation. The rotation of the configuration will also be given so that an optimal solution in $k - 1$ dimensions can be attained by dropping the final coordinate in the k dimensional configuration.

4.4.2.1 Recovering Coordinates

After some preprocessing, finding a centered configuration $\vec{y}_1, \vec{y}_2, \dots, \vec{y}_N$ from the Euclidean distance matrix Δ is possible through some of the linear algebra techniques we have discussed so far. Observe that Δ_{rs}^2 can be expressed as

$$\Delta_{rs}^2 = \|\vec{y}_r\|^2 + \|\vec{y}_s\|^2 - 2\vec{y}_r^T \vec{y}_s$$

so that

$$\vec{y}_r^T \vec{y}_s = -\frac{1}{2} (\Delta_{rs} - \|\vec{y}_r\|^2 - \|\vec{y}_s\|^2).$$

We will use this relationship to build an inner product matrix $\mathbf{B}$ with entries $\mathbf{B}_{rs} = \vec{y}_r^T \vec{y}_s$. In matrix form, we may write

$$\mathbf{B} = \mathbf{Y}\mathbf{Y}^T$$

, where $\mathbf{Y}$ is a data matrix containing the configuration $\vec{y}_1, \dots, \vec{y}_N$ with Euclidean distances Δ .

1) Computing the inner product matrix B

The inner product term can be represented as: $B_{ij} = \vec{y}_i^T \vec{y}_j$. As a result,

$$(\Delta_{ij})^2 = B_{ii} + B_{jj} - 2B_{ij}.$$

Key Observation In the previous line, we express the entries of Δ (known) with entries of B (unknown). Our goal is to find a way to express entries of B (unknown) with entries of Δ . Intuitively, we want to cancel out B_{ij} terms in the expression. Considering that $\sum_{i=1}^N \vec{y}_i = \vec{0}$, we sum both sides of the equation over the index i . We can get the following expression:

$$\sum_{i=1}^N (\Delta_{ij})^2 = \text{tr}(B) + NB_{jj}$$

Similarly, we can also sum both sides of the equation over index j , and get the expression:

$$\sum_{j=1}^N (\Delta_{ij})^2 = \text{tr}(B) + NB_{ii}$$

We successfully eliminate all the off-diagonal terms of B through the above steps. Now, we want to take a step further. Sum both sides of the equations over both indexes i and j . We acquire the following expression:

$$\sum_{i=1}^N \sum_{j=1}^N (\Delta_{ij})^2 = 2N \text{tr}(B)$$

Now we can solve the entries of Δ using the entries of B through a backward calculation. From the last equation, we get

$$\text{tr}(B) = \frac{1}{2N} \sum_{i=1}^N \sum_{j=1}^N \Delta_{ij}^2$$

Then substitute the above expression into above formulas, we get the expression of the diagonal entries of B :

$$B_{ii} = \frac{1}{N} \left(\sum_{j=1}^N (\Delta_{ij})^2 - \text{tr}(B) \right)$$

After that, we can finally get the off-diagonal entries of B :

$$\begin{aligned} B_{ij} &= \frac{1}{2} (B_{ii} + B_{jj} - (\Delta_{ij})^2) \\ &= -\frac{1}{2} (\Delta_{ij})^2 + \frac{1}{N} \sum_{i=1}^N (\Delta_{ij})^2 + \frac{1}{N} \sum_{j=1}^N (\Delta_{ij})^2 - \frac{1}{2N^2} \sum_{i=1}^N \sum_{j=1}^N (\Delta_{ij})^2 \end{aligned}$$

We may more compactly express the inner product matrix B in a matrix form, as

$$B = HAH$$

where $A \in \mathbb{R}^{N \times N}$ has entries $A_{ij} = -\frac{1}{2}(\Delta_{ij})$ for $\forall 1 \leq i, j \leq N$ and H is the centering matrix

$$H = \mathbb{I}_N - \frac{1}{N} \mathbb{1}_N \mathbb{1}_N^T$$

2) Recover the coordinates using inner product matrix B

Both diagonal and off-diagonal entries of the inner product matrix B has been shown. We assumed that $B = YY^T$, so B is symmetric and positive semi-definite (all eigenvalues are non-negative). Assuming B is rank t implies that it has t positive eigenvalues and $N - t$ 'zero' eigenvalues.

Our intuition is to apply SVD/diagonalization to B in order to recover the configuration $\vec{y}_1, \vec{y}_2, \dots, \vec{y}_N \in \mathbb{R}^t$ by computing the nonzero eigenvalues and corresponding eigenvectors giving the factorization.

$$\begin{aligned} B &= (\vec{u}_1 | \vec{u}_2 | \dots | \vec{u}_t) \begin{pmatrix} \lambda_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_t \end{pmatrix} \begin{pmatrix} \vec{u}_1^\top \\ \vec{u}_2^\top \\ \vdots \\ \vec{u}_t^\top \end{pmatrix} \\ &= \tilde{U} \begin{pmatrix} \lambda_1^{1/2} & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_t^{1/2} \end{pmatrix} \begin{pmatrix} \lambda_1^{1/2} & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_t^{1/2} \end{pmatrix} \tilde{U}^T \\ &= (\tilde{U} \Lambda^{1/2}) (\tilde{U} \Lambda^{1/2})^T \end{aligned}$$

Let $Y = \tilde{U} \Lambda^{1/2}$, so that rows of $\tilde{U} \Lambda^{1/2}$ correspond to the vectors $\vec{y}_1, \vec{y}_2, \dots, \vec{y}_N$. It satisfies every entry of the inner product matrix B , as well as all pairwise distances Δ_{ij} ! Importantly, the rank of B immediately indicates the minimal dimension needed to recover a configuration which exactly recovers the Euclidean distances in Δ .

For visualization, we may prefer a lower dimensional configuration than the dimensionality t discovered through classical scaling. For a $k < t$ dimensional configuration, we instead use the first k columns of $\tilde{U} \Lambda^{1/2}$ as our k -dimensional vectors. Outside of optimal recovery, we may instead formulate the classical scaling problem in k -dimensions as the following minimization problem.

$$\operatorname{argmin}_{Y \in \mathbb{R}^{N \times k}} \|B - YY^T\|_F$$

from which it follows that YY^T is built from the rank k approximation to B . Using the results from SVD and the equivalence of SVD and eigendecompositions for symmetric positive, semidefinite matrices, this is equivalent to using the first k columns of $U\Lambda^{1/2}$.

Dealing with real data case

For a given distance matrix, the Euclidean condition may not hold. In the previous globe map example, the distance matrix is based on the geodesic distance instead of the Euclidean distance. Under this circumstance, the “inner product” matrix $B = HAH$ is still symmetric but will not be positive definite. As a simpler example, consider the following case

Example 4.4 (Non Euclidean Distances). Suppose we have a distance matrix Δ :

Given the matrix:

$$\Delta = \begin{pmatrix} 0 & 1 & 1 & 2 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 2 & 1 & 1 & 0 \end{pmatrix}$$

```
##      [,1] [,2] [,3] [,4]
## [1,]    0    1    1    2
## [2,]    1    1    0    1
## [3,]    1    1    0    1
## [4,]    2    1    1    0
```

i) Compute matrix A as:

$$A = -\frac{1}{2}\Delta^2$$

```
##      [,1] [,2] [,3] [,4]
## [1,]  0.0 -0.5 -0.5 -2.0
## [2,] -0.5 -0.5  0.0 -0.5
## [3,] -0.5 -0.5  0.0 -0.5
## [4,] -2.0 -0.5 -0.5  0.0
```

ii) Compute matrix B using:

$$H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^T$$

Where I is the identity matrix and n is the number of rows (or columns) in Δ . Then:

$$B = HAH$$

```
##      [,1] [,2] [,3] [,4]
## [1,]  0.9375  0.1875 -0.0625 -1.0625
## [2,]  0.0625 -0.1875  0.0625  0.0625
## [3,]  0.0625 -0.1875  0.0625  0.0625
## [4,] -1.0625  0.1875 -0.0625  0.9375
```

- iii) Finally, perform an eigen-decomposition on matrix B which gives eigenvalues.

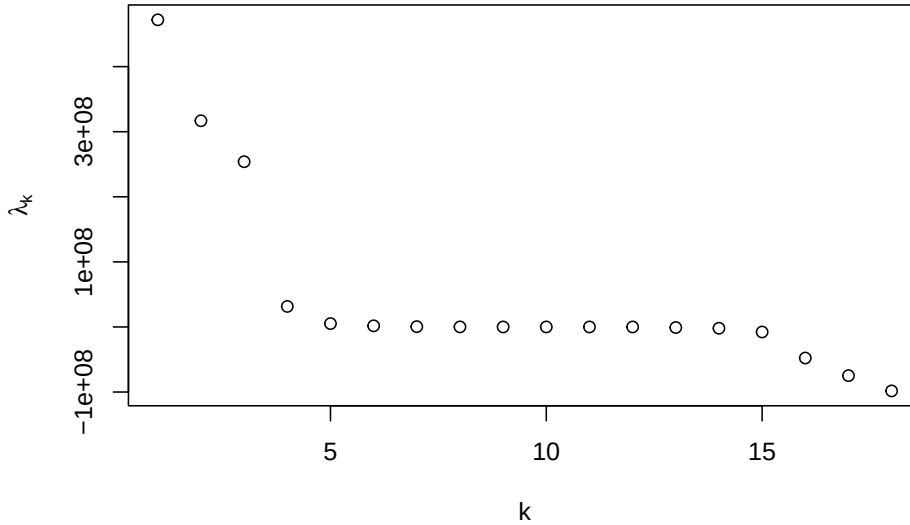
```
## [1]  2.00 -0.25  0.00  0.00
```

Here, we have a negative eigenvalue $-\frac{1}{4}$. This indicates the original distance matrix Δ is not a Euclidean distance matrix; there is no Euclidean space containing four vectors with the given pairwise distances!!

Other practical issues

- 1) **Asymmetric Δ :** There are cases where the distance matrix Δ is not symmetric. In this case, we usually set $\Delta \leftarrow \frac{1}{2}(\Delta + \Delta^T)$ to enforce symmetry.
- 2) When there exist some negative eigenvalues in the inner product matrix B , we usually have two options to deal with it.
 - a) Inflate the original proximity matrix Δ by a small constant factor c , i.e., $\Delta_{ij} \leftarrow \Delta_{ij} + c$, if $i \neq j$ until B is positive semidefinite.
 - b) If there exist several negative eigenvalues with small absolute value (compared to the largest several positive eigenvalues), and there are more positive eigenvalues than our prior estimation (the dimension of the original configuration), we may just pick the largest t eigenvalues and eliminate the rest.

Example 4.5 (Global Cities Revisited). We can also consider the previous global city distance matrix example. Plot the scree plot of the inner product matrix.



We find that the first three eigenvalues are much larger than the rest, so we assume that the dimension of the original configuration is 3, which also complies

to our knowledge about global map.

4.4.2.2 Duality of PCA and Classical Scaling

“There is a duality between a principals components analysis and classical MDS where dissimilarities are given by Euclidean distance”[Cox and Cox, 2000].

Given the matrix expression of the original configuration $\mathbf{X} \in \mathbb{R}^{N \times d}$. Recall that PCA is attained by finding the eigen-vectors of the covariance matrix $\frac{1}{N-1}(HX)^T(HX)$, where H is the centering matrix. Suppose the k eigen-vectors are $\vec{w}_1, \vec{w}_2, \dots, \vec{w}_k$, and the corresponding eigenvalues are $\mu_1, \mu_2, \dots, \mu_k$. Then $HX = YW^T$, where $W = (\vec{w}_1 | \vec{w}_2 | \dots | \vec{w}_N)$ represents PC Loadings, and Y represents PC Scores.

While MDS is attained by first converting X into distance matrix, here, Euclidean distance. In the classical MDS algorithm, the process of converting the Euclidean distance matrix into the inner product matrix gives $B = (HX)(HX)^T$. then we find the eigenvectors of B which are equivalent to the left singular vectors of HX and with nonzero eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_d$ which are the squared singular values of HX .

Recall what you have learned in Linear Algebra. The eigenvalues of $HX(HX)^T$ are the same as those for $X^T X$, together with an extra $n-p$ zero eigenvalues. So the first t PC scores give the t -dimensional configuration for Classical Scaling. **Choosing the number of dimensionality t of the original configuration is equivalent to choosing the number of principal components to keep.**

Furthermore, we have shown that $HX = YW^T$, then $B = (HX)(HX)^T = YW^T W Y^T = Y Y^T$, this means **PC scores are the exact solutions for Classical Scaling!** As a result, classical MDS has the same strengths and weaknesses of PCA.

4.4.3 Metric MDS

Metric MDS is a set of algorithms which seek optimal configuration which minimize a specified loss function. Typically, this loss function is called the stress which has the general form

$$S(\vec{y}_1, \dots, \vec{y}_N) = \sum_{i < j} W_{ij} (\|\vec{y}_i - \vec{y}_j\|^2 - \Delta_{ij})^2 = \frac{1}{2} \sum_{i \neq j} W_{ij} (\|\vec{y}_i - \vec{y}_j\|^2 - \Delta_{ij})^2$$

where the weights $W_{ij} \geq 0$ have different conventions which prioritize the preservation of certain distances in the original coordinates. Heuristically, you can think of $(\|\vec{y}_i - \vec{y}_j\|^2 - \Delta_{ij})^2$ as measuring how far the lower-dimensional Euclidean distances $\|\vec{y}_i - \vec{y}_j\|$ deviate from the specified original distance Δ_{ij} . Importantly, there is no closed form expression for $\vec{y}_1, \dots, \vec{y}_N$ which minimize S so gradient based optimization methods are typically used in practice. When the desired dimension is unknown, one can estimate different $\vec{y}_1, \dots, \vec{y}_N$ in different

dimensions and compare the optimal stress as a data driven method for choosing the dimensionality.

An important method of metric MDS is the Sammon mapping which takes $W_{ij} \propto \frac{1}{\Delta_{ij}}$. Thus, if $\Delta_{ij} \gg \Delta_{kl}$ it follows that $W_{ij} \ll W_{kl}$ and we may infer that the Sammon mapping places a greater emphasis on the preservation of small distances! Briefly, we show the results of the Sammon mapping applied to the global cities data.

Example 4.6 (Sammon mapping and global cities). First, we show the minimal Sammon stress at each dimension.

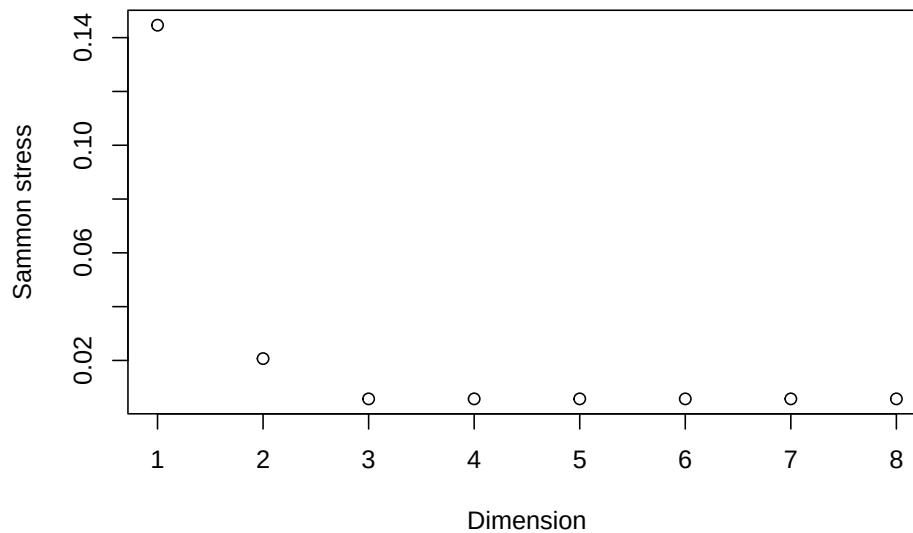
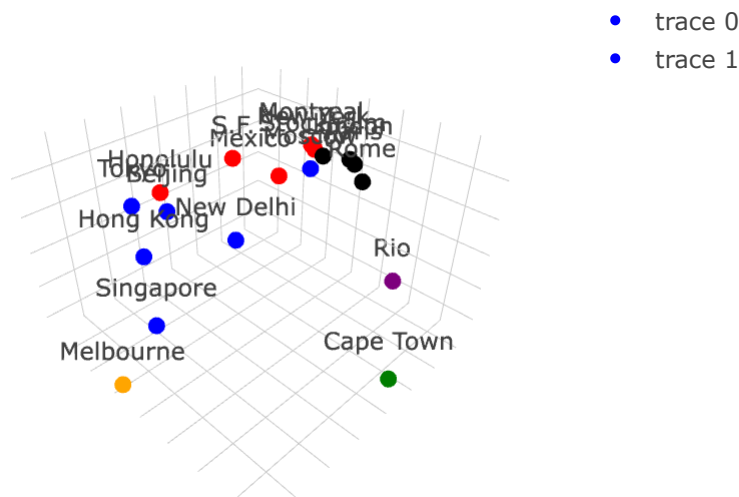


Figure 4.8: Sammon stress for Global Cities Data

```
## A marker object has been specified, but markers is not in the mode
## Adding markers to the mode...
```

This stress saturates near a small value for dimensions three and up suggesting a three dimensional configuration is suitable. We show this result below. Note its similarities to the result of classical scaling (which is default initialization for the Sammon mapping optimization algorithm).

4.4.4 Nonmetric MDS

To be added in the next edition.

4.5 Summary

The four methods in this chapter can feel like a grab bag – PCA, SVD, NMF, and classical MDS solve superficially different problems – but geometrically they are close relatives, each fitting the best *flat* (linear or affine) approximation to the data under a different constraint or a different notion of “best.”

PCA looks for the directions of maximum variance, which is the same as looking for the lower-dimensional hyperplane that the data hug most tightly; the eigenvectors of the covariance matrix are literally the axes of the ellipsoid that best summarizes the data’s spread (4.1). SVD generalizes this same idea to any matrix, not just a centered covariance matrix, and the Eckart-Young-Mirsky Theorem tells us that truncating the SVD gives the best possible low-rank – geometrically, the best low-dimensional – approximation to a matrix in exactly the same least-squares sense PCA uses (4.2). NMF asks the same low-rank question but adds a nonnegativity constraint, which changes the geometry from an unconstrained hyperplane to a convex cone: instead of finding orthogonal axes, NMF looks for a small set of nonnegative “archetype” vectors whose convex hull contains (or nearly contains) the data (4.3). Classical MDS approaches the same underlying question from the opposite direction, starting only from pairwise

distances rather than coordinates, and its duality with PCA (4.4) confirms that these are really two views of one geometric operation: recovering a configuration of points, up to rotation, from either its covariance structure or its distance structure.

The shared limitation is exactly this commitment to flatness. Every method here finds the best hyperplane, cone, or rigid configuration – and when the data’s true structure is curved, as with the Swiss roll, no linear method can do better than a fundamentally flawed approximation. That limitation is precisely the motivation for the nonlinear methods in the next chapter.

4.6 Exercises

1. Show that the PCA scores are centered.
2. Suppose a data matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ has principal component scores $\mathbf{Y} \in \mathbb{R}^{N \times d}$, principal component loading matrix $\mathbf{W} \in \mathbb{R}^{d \times d}$, and principal component variances $\lambda_1, \dots, \lambda_d$. Show that the sample covariance of the PCA scores, $\mathbf{Y}$, is diagonal.
3. Consider the data matrix

$$\mathbf{X} = \begin{bmatrix} 2 & 3 \\ 4 & 5 \\ 6 & 7 \\ 8 & 9 \end{bmatrix}.$$

- a. Compute the principal component scores, variances, and loadings of $\mathbf{X}$.
- b. Does $\mathbf{X}$ exhibit lower dimensional structure? If so, describe it.
4. Given a dataset with outliers:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \\ 4 & 8 \\ 100 & 200 \end{bmatrix}$$

- a. Center the data.
- b. Compute the covariance matrix.
- c. Perform PCA and identify the principal component scores, loadings, and variances.
- d. Discuss how outliers affect PCA and suggest ways to handle them.
5. Consider a dataset:

$$\mathbf{X} = \begin{bmatrix} 1 & 100 \\ 2 & 200 \\ 3 & 300 \\ 4 & 400 \end{bmatrix}$$

- a. Center the data.
 - b. Compute the covariance matrix.
 - c. Perform PCA and find the principal components.
 - d. Discuss the importance of feature scaling in PCA.
6. Given a nonlinear dataset:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 2 & 4 \\ 3 & 9 \\ 4 & 16 \\ 5 & 25 \end{bmatrix}$$

- a. Center the data.
 - b. Compute the covariance matrix.
 - c. Perform PCA and identify the principal components.
 - d. Discuss the limitations of PCA for nonlinear datasets and suggest alternative methods.
7. Load the Iris dataset and standardize the features.
- a. Perform PCA on the standardized data.
 - b. Plot the cumulative explained variance as a function of the number of principal components.
 - c. Choose the number of principal components that explain at least 95% of the variance.
 - d. Project the data onto the chosen principal components and visualize the results.
 - e. Discuss the results and the effectiveness of PCA in reducing the dimensionality of the dataset.

8. Suppose

$$\vec{x} = (x_1, x_2)^T \sim \mathcal{N} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix} \right),$$

i.e. random vector $\vec{x}$ follows a multivariate normal distribution with mean $\vec{0}$ and covariance $\Sigma_X = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$. Let $w = \frac{\sqrt{2}}{2}(x_1 + x_2)$ and $z = \frac{\sqrt{2}}{2}(x_1 - x_2)$. Find the joint distribution of $(w, z)^T$. Hint: linear combinations of normal random variables are also normal.

9. Consider a data matrix $X \in \mathbb{R}^{N \times d}$ with centered columns so that the sample covariance matrix is

$$\hat{\Sigma} = \frac{X^T X}{N}.$$

Assume $\hat{\Sigma}$ has eigenvalues $\lambda_1 > \lambda_2 > \dots > \lambda_d > 0$ with orthonormal eigenvectors $\vec{w}_1, \dots, \vec{w}_d$.

- a. If $X^{(1)} = X - X\vec{w}_1\vec{w}_1^T$ is the data matrix where each row has had its component in the direction of $\vec{w}_1$ removed, show that

$$\hat{\Sigma}^{(1)} = \frac{X^{(1)T}X^{(1)}}{N} = \hat{\Sigma} - \lambda_1\vec{w}_1\vec{w}_1^T.$$

- b. Show that $\hat{\Sigma}$ can be written in the form $\hat{\Sigma} = \sum_{j=1}^d \lambda_j \vec{w}_j \vec{w}_j^T$.
10. For $k < d$, let $\vec{q}_1, \dots, \vec{q}_k \in \mathbb{R}^d$ be fixed orthonormal vectors. Suppose $a_1, \dots, a_k$ are independent Gaussian random variables with mean zero and variances $\lambda_1 > \dots > \lambda_k$ respectively. Let

$$\vec{x} = a_1\vec{q}_1 + \dots + a_k\vec{q}_k + \vec{\epsilon}$$

where $\vec{\epsilon} \sim \mathcal{N}(\vec{0}, \sigma^2 \mathbf{I})$ is independent of $a_1, \dots, a_k$

- Find the mean and covariance of $\mathbf{X}$.
 - Find the eigenvalues and eigenvectors of Σ .
 - For $d = 10$ and $k=3$ and let $\lambda_1 = 25$, $\lambda_2 = 9$, $\lambda_3 = 4$ and $\sigma^2 = 1$. Generate 100 independent realizations of $\vec{x}$ and compute the principal component loadings and variances. How do these compare to your results from b.
 - Repeat c. for 10^4 samples. How have the results changed?
11. Let $\vec{x}_1, \dots, \vec{x}_N$ be vectors in $\mathbb{R}^d$. Assume a PCA of these data has loadings $\vec{w}_1, \dots, \vec{w}_d$ with associated variances $\lambda_1 \geq \dots \geq \lambda_d \geq 0$. Let U be a $d \times d$ orthonormal matrix and set $\vec{y}_i = U\vec{x}_i$. Find expressions for the principal component loadings and variance of $\vec{y}_1, \dots, \vec{y}_N$ in terms of U , $\vec{w}_1, \dots, \vec{w}_d$ and $\lambda_1, \dots, \lambda_d$.
12. Load the *mtcar* dataset containing 11 observations from 32 cars.
- Show the principal component loadings and a biplot of the first and second PC components.
 - Rescale the mpg (miles per gallon) data to feet per gallon, i.e. `mtcars$mpg <- 5280*mtcars$mpg`. Rerun PCA on these modified data and show the loadings and biplot.
 - What is the first loading capturing? Explain this result.
 - Rerun the results using the empirical correlation matrix by setting the option `scale = TRUE` in the `prcomp` command. Compare this result with part (a)
13. Consider the helix data shown below from two different directions.
- What is the dimension of the shape formed by these data?
 - Compute the three principal component variances and show them below.

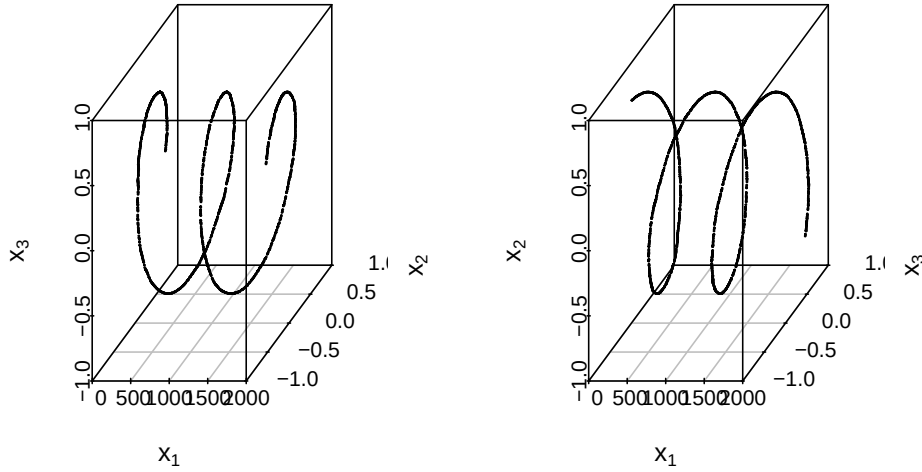


Figure 4.9: Helix data

c. Do the principal component variances reflect the dimension of the data? Why or why not?

1. Consider a dataset with three features:

$$\mathbf{X} = \begin{bmatrix} 2 & 0 & 1 \\ 3 & 2 & 2 \\ 4 & 4 & 3 \\ 5 & 6 & 4 \\ 6 & 8 & 5 \end{bmatrix}$$

- a. Center the data by subtracting the mean of each feature.
 - b. Compute the covariance matrix of the centered data.
 - c. Find the eigenvalues and eigenvectors of the covariance matrix.
 - d. Project the data onto the first two principal components.
 - e. Visualize the original data and the projected data in a 2D plot.
 - f. Discuss how PCA can help in reducing the dimensionality of the data while preserving important information.
2. The dataset *simplex10* contains samples generated from the probability simplex in $\mathbb{R}^{10}$. (This means all entries nonnegative and each random vector has entries which sum to one.)
- a. Run PCA, SVD, and NMF on these data and compare the results. In particular, what lower dimensional structure, if any, do these methods indicate?
 - b. Why do the results differ?
3. Show that the Frobenius norm of a matrix is the square of the sum of its squared singular values. You may use the helpful identity $\|\mathbf{A}\|_F^2 = \text{tr}(\mathbf{A}\mathbf{A}^T)$

Chapter 5

Kernel Methods and Manifold Learning

5.1 Kernels and the Kernel Trick

The techniques considered in the previous chapter (PCA, NMF, SVD, and classical Scaling) are ill suited to identify nonlinear structure and dependence in data. If we wish to most efficiently reduce dimensions without loss of information, we will need techniques which incorporate nonlinear structure. One can expand a data matrix by including specific nonlinear relationships then apply PCA or SVD but there are numerous problems with this approach. In particular, which relationships does one choose to include? Even including simple quadratic or cubic terms (features) can result in a data matrix with a massive increase in the number of columns. Even when the original dimensionality of the data is moderate, the including of polynomial terms can quickly result in a data matrix of nonlinear features with an untenable number of columns which can make application of the linear methods we have discussed much more computationally demanding to implement.

Kernels are an important class of functions which can be used to **kernelize** the methods we have discussed before. In theory, these kernelized versions of the linear methods we have discussed can identify and use nonlinear structure for better dimensionality reduction while circumventing the issue of higher dimensional **featurized** data. This approach follows from an application of the so called 'kernel trick' which we now discuss.

Briefly, a kernel is a function

$$k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$$

which has an associated feature space, $\mathcal{H}$ and (implicitly defined, possibly nonlinear) feature mapping $\varphi : \mathcal{R}^d \rightarrow \mathcal{H}$ such that inner products in the feature space,

denoted $\langle \varphi(\vec{x}), \varphi(\vec{y}) \rangle_{\mathcal{H}}$ can be obtained through an evaluation of the kernel, namely

$$k(\vec{x}, \vec{y}) = \langle \varphi(\vec{x}), \varphi(\vec{y}) \rangle_{\mathcal{H}} \quad (5.1)$$

Any method which can be expressed involving inner products can be kernelized by replacing terms of the form $\vec{x}_i^T \vec{x}_j$ with the quantity $k(\vec{x}_i, \vec{x}_j)$. Thus, we are replacing inner products of our original d -dimensional data with inner products in the associated feature space $\mathcal{H}$. Importantly, if we only need inner products, we never need to explicitly compute the feature map φ for any of our data! At first glance this connection may seem minor, but by using kernels we can turn many linear techniques into nonlinear methods including PCA, SVD, support vector machines, linear regression, and many others.

There are some limits though. Not every choice of k has an associated feature space. A function is only a kernel if it satisfies Mercer's Condition.

Theorem 5.1 (Mercer's Condition). *A function*

$$k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$$

has a an associated feature space $\mathcal{H}$ and feature mapping $\varphi : \mathbb{R}^d \rightarrow \mathcal{H}$ such that

$$k(\vec{x}, \vec{y}) = \langle \varphi(\vec{x}), \varphi(\vec{y}) \rangle_{\mathcal{H}}, \quad \forall \vec{x}, \vec{y} \in \mathbb{R}^d$$

if and only if for any $N \in \{1, 2, \dots\}$ and $\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}^d$ the kernel matrix $\mathbf{K} \in \mathbb{R}^N$ with entries $\mathbf{K}_{ij} = k(\vec{x}_i, \vec{x}_j)$ is positive semidefinite. Equivalently, it must be the case that

$$\int_{\mathbb{R}^d} \int_{\mathbb{R}^d} g(\vec{x}) g(\vec{y}) k(\vec{x}, \vec{y}) d\vec{x} d\vec{y} \geq 0$$

whenever $\int_{\mathbb{R}^2} [g(\vec{x})] d\vec{x} < \infty$.

We will only consider symmetric functions such that $k(\vec{x}, \vec{y}) = k(\vec{y}, \vec{x})$ for all $\vec{x}, \vec{y} \in \mathbb{R}^d$. It may not be immediately obvious if a symmetric function satisfies Mercer's condition, but there are many known examples. A few are shown in the following table.

Name	Equation	Tuning Parameters
Radial Basis Function	$k(\vec{x}, \vec{y}) = \exp(-\sigma \ \vec{x} - \vec{y}\ ^2)$	Scale $\sigma > 0$
Laplace	$k(\vec{x}, \vec{y}) = \exp(-\sigma \ \vec{x} - \vec{y}\)$	Scale $\sigma > 0$
Polynomial	$k(\vec{x}, \vec{y}) = (c + \vec{x}^T \vec{y})^d$	Offset $c > 0$, Degree $d \in \mathbb{N}$

The radial basis function (rbf) is the most commonly used kernel and has an associated feature space $\mathcal{H}$ which is infinite dimensional! The associated feature map φ for the rbf kernel is

$$\varphi(\vec{x}) = e^{-\sigma \|\vec{x}\|^2} \left(a_{\ell_0}^{(0)}, a_1^{(1)}, \dots, a_{\ell_1}^{(1)}, a_1^{(2)}, \dots, a_{\ell_2}^{(2)}, \dots \right)$$

where $\ell_j = \binom{d+j-1}{j}$ and $a_\ell^{(j)} = \frac{(2\sigma)^{j/2} x_1^{\eta_1} \dots x_d^{\eta_d}}{\sqrt{\eta_1! \dots \eta_d!}}$ when $\eta_1 + \dots + \eta_d = j$. The preceding expression is quite cumbersome, but there is one important point to emphasize. Every possible polynomial combination of the coordinates of $\vec{x}$ appears in some coordinate of $\varphi(\vec{x})$ (though higher order terms are shrunk by the factorial factors in the denominator of $a_\ell^{(j)}$). Thus, the rbf kernel is associated with a very expressive feature space which makes it a potent but dangerous choice since risks overfitting. To explore these details more, let's discuss one very important application of kernels in unsupervised learning.

5.2 Kernel PCA

Suppose we have a kernel k and associated feature map φ . In kernel PCA, we want to apply to PCA to the featurized data $\varphi(\vec{x}_1), \dots, \varphi(\vec{x}_N)$ rather than the original data. The idea is that by studying the featurized data, we can identify additional nonlinear structure in the features that provides a better lower-dimensional representation of the data. We have discussed three approaches to computing PC scores to data: (i) diagonalization of the sample covariance, (ii) applying SVD to the centered data, and (iii) using the duality of PCA and classical scaling.

For the rbf kernel and its infinite dimensional feature map, approaches (i) and (ii) are impossible. Why? The centered data matrix of features

$$\mathbf{H}\tilde{\mathbf{X}} = \mathbf{H} \begin{bmatrix} \varphi(\vec{x}_1)^T \\ \vdots \\ \varphi(\vec{x}_N)^T \end{bmatrix} = \begin{bmatrix} \varphi(\vec{x}_1)^T - \bar{\varphi}^T \\ \vdots \\ \varphi(\vec{x}_N)^T - \bar{\varphi}^T \end{bmatrix}$$

has a infinite number of columns so that we cannot compute its SVD. In the above expression, $\bar{\varphi} = \frac{1}{N} \sum_{i=1}^N \varphi(\vec{x}_i)$ is the mean for the feature vectors. The associated sample covariance matrix

$$\Sigma_F = \frac{1}{N} \tilde{\mathbf{X}}\mathbf{H}\tilde{\mathbf{X}} = \frac{1}{N} \sum_{i=1}^N (\varphi(\vec{x}_i) - \bar{\varphi})(\varphi(\vec{x}_i) - \bar{\varphi})^T$$

will have an infinite number of rows and columns so we cannot hope to diagonalize it either.

Fortunately, the third option, using duality of classical scaling and PC, provides a workaround. Observe that the inner product matrix of the centered feature data $\mathbf{H}\tilde{\mathbf{X}}(\mathbf{H}\tilde{\mathbf{X}})^T$ can be written in terms of the kernel since

$$\mathbf{H}\tilde{\mathbf{X}}(\mathbf{H}\tilde{\mathbf{X}})^T = \mathbf{H} \begin{bmatrix} \varphi(\vec{x}_1)^T \\ \vdots \\ \varphi(\vec{x}_N)^T \end{bmatrix} [\varphi(\vec{x}_1) \quad \dots \quad \varphi(\vec{x}_N)] \mathbf{H} = \mathbf{H}\mathbf{K}\mathbf{H}$$

where $\mathbf{K}$ has the inner products in the feature space which we can calculate using the kernel function

$$\mathbf{K}_{ij} = \varphi(\vec{x}_i)^T \varphi(\vec{x}_j) = k(\vec{x}_i, \vec{x}_j).$$

Since k is a symmetric kernel, it follows that $\mathbf{K}$ is positive semidefinite. Using this property, one can argue that $\mathbf{H}\mathbf{K}\mathbf{H}$ will also be positive semidefinite. We can use the eigendecomposition of the doubly centered kernel to compute the kernel principal component scores. Specifically, if $\mathbf{H}\mathbf{K}\mathbf{H}$ rank r with eigenvalues $\lambda_1 \geq \dots \geq \lambda_r > 0$ and corresponding eigenvalues $\vec{u}_1, \dots, \vec{u}_r \in \mathbb{R}^N$, then $\mathbf{H}\mathbf{K}\mathbf{H}$ factorizes as

$$\mathbf{H}\mathbf{K}\mathbf{H} = \underbrace{[\vec{u}_1 \quad \dots \quad \vec{u}_r]}_{\mathbf{U}\mathbf{\Lambda}^{1/2}} \begin{bmatrix} \lambda_1^{1/2} & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \lambda_r^{1/2} \end{bmatrix} (\mathbf{U}\mathbf{\Lambda}^{1/2})^T.$$

The rows of the matrix $\mathbf{U}\mathbf{\Lambda}^{1/2}$ are almost the kernel PC scores. The only issue is an additional the identity

$$\mathbf{H}\mathbf{K}\mathbf{H} = (\mathbf{H}\tilde{\mathbf{X}})(\mathbf{H}\tilde{\mathbf{X}})^T$$

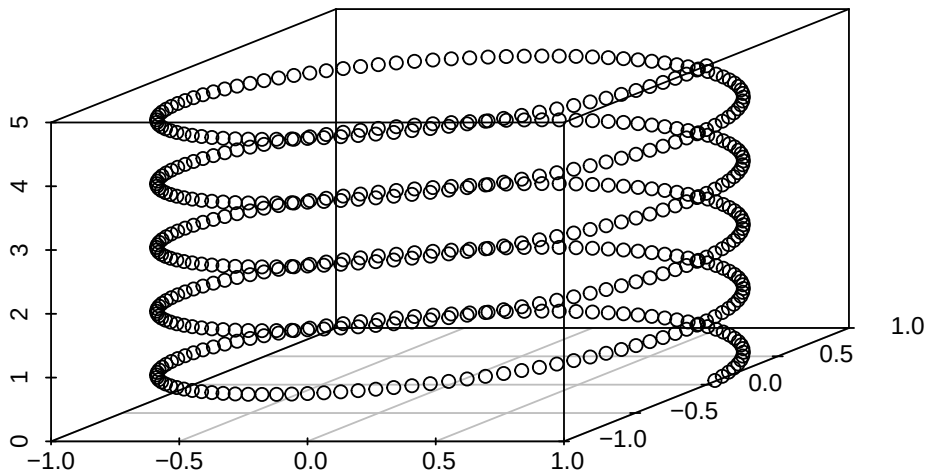
is missing the factor of $1/N$ appearing in the covariance calculation. Accounting for this, the first r non-zero kernel PC scores are the rows of the matrix

$$\frac{1}{\sqrt{N}} \mathbf{U}\mathbf{\Lambda}^{1/2}$$

and the corresponding nonzero PC variances are $\lambda_1/N, \dots, \lambda_r/N$.

Notably, at no point do we compute the PC loadings! However, similar to standard PCA, we use the scores for dimension reduction and the PC variances for choosing a dimension. Without the loadings, we cannot recompute the original data. Below, we show an application of kernel PCA to the helix and demonstrate its ability to identify the one-dimensional structure of the helix and its sensitivity to kernel selection and tuning.

Example 5.1 (Kernel PCA applied to the Helix). First, we show the kPCA variances for three different kernels and tuning parameters. The data are regularly spaced points along the helix.



From these graphs, one would infer very different lower dimensional choices depending on the kernel and parameters. The polynomial kernel provides the most robust estimate of the one-dimensional nature of the data.

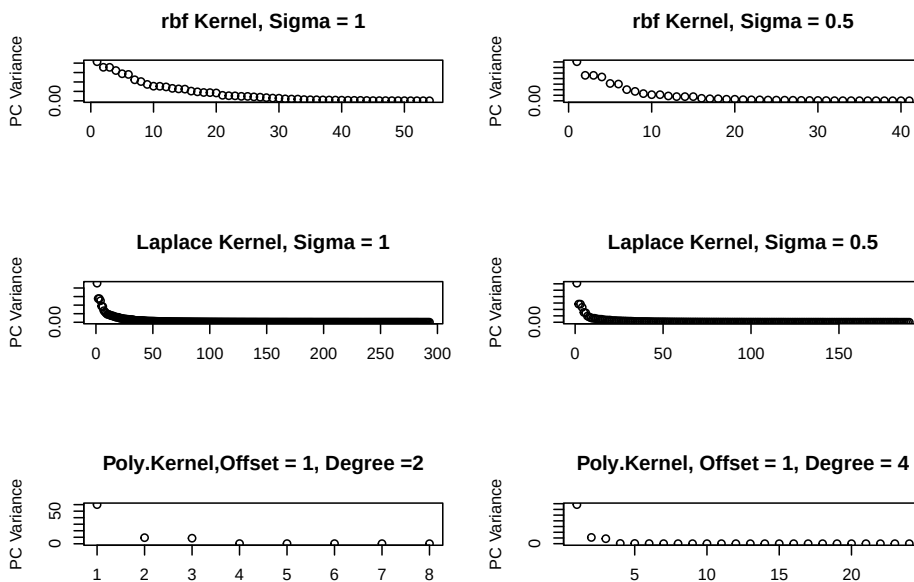
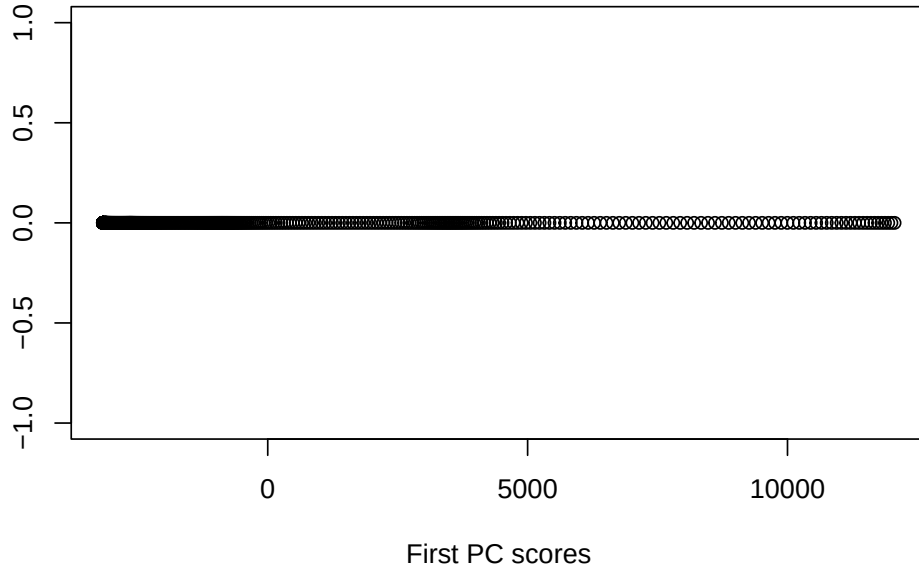


Figure 5.1: kPCA Variances for different Kernels

Below, we show the recovered one-dimensional coordinates for the polynomial kernel with offset 1 and degree 4 shown below, which is good, but do not quite reflect the equal spaced nature of the points.



As the preceding example demonstrates, kernel PCA can identify nonlinear structure, but is quite sensitive to kernel selection and tuning. More advanced implementations make use of cross-validation to aid in the selection and tuning of the kernel [Alam and Fukumizu, 2014, Mika et al. [1998]].

5.3 Brief primer on manifolds and differential geometry

In the preceding section, we focused our attention on linear manifolds and saw cases where this structure was insufficient. Using kernel PCA, we tried to find a workaround by first (implicitly) mapping our data to a higher dimensional feature space then approximating results with linear subspaces (of feature space). We now turn to a family of methods that take a more direct approach, assuming the data live on or near a manifold – the manifold hypothesis introduced in Section 3.1 – rather than relying on a fixed kernel. While our data may exhibit some low-dimensional structure, there is no practical reason to expect such behavior to be inherently linear. In the remaining sections of this chapter, we will explore methods which consider **nonlinear** structure directly. Such methods are referred to as nonlinear dimension reduction or manifold learning. Critical to this discussion is the notion of a manifold.

Definition 5.1 (Informal Definition of a Manifold). A manifold is a (topological) space which locally resembles Euclidean space. Each point on a t -dimensional manifold has a neighborhood that can be mapped continuously to $\mathbb{R}^t$. We call t the intrinsic dimension of the manifold.

We’ll return to mathematical details shortly, but let’s stick with intuition for

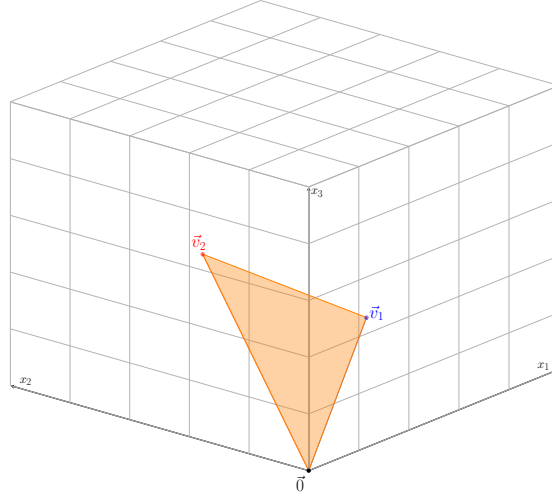
now. If you were to stand at any point on a k -dimensional manifold, the portion of the manifold closest to you look just like a k -dimensional hyperplane – though you might need to be extremely near-sighted for this to be true. The canonical example is the surface of the Earth. If we take all points on the surface of the Earth, they form a sphere in $\mathbb{R}^3$. However, if we focus on the area around any point it looks like a portion of the two-dimensional plane. In fact, we can represent any point on the surface of the Earth in terms of two numbers, latitude and longitude. In the language of manifolds, we can say the surface of the Earth is a manifold with intrinsic dimension two which has been embedded in $\mathbb{R}^3$. Here are a few more concrete examples.

Example 5.2 (Examples of Manifolds). Many familiar geometric objects are manifolds such as lines, planes, and spheres. For example, Let $\vec{w}_1, \dots, \vec{w}_k \in \mathbb{R}^d$ be a set of linearly independent vectors. Then $\text{span}(\vec{w}_1, \dots, \vec{w}_k)$ is a k -dimensional manifold in $\mathbb{R}^d$. When $k = 1$, the span is a line; for $k > 1$ the span is a hyperplane. The sphere unit sphere $\{\vec{x} \in \mathbb{R}^d : \|\vec{x}\| = 1\}$ is a $d - 1$ dimensional manifold. For example, when $d = 3$ manifold resembles the surface of the earth which locally looks like a portion of the 2-dimensional plane. The Swiss roll from Example 3.1 is a two-dimensional manifold in $\mathbb{R}^3$.

In short manifold as a nice smooth, curved surface. There are several reasons why a subset may not be a manifold. The most straightforward examples are cases where the manifold as a self intersection or a sharp point.

Example 5.3 (Non-manifold). Consider a figure eight curve. At the midpoint where the upper and lower circle meet, there is not neighborhood that resembles Euclidean space so the surface is not a manifold.

For a second example, take any two vectors $\vec{h}_1$ and $\vec{h}_2$ and consider there convex hull, that is $\{a\vec{h}_1 + b\vec{h}_2 : a, b > 0, a + b \leq 1\}$ such the orange triangle in the figure below. At the three vertices of the orange triangle, there is no local neighbor that looks flat. However, if we were to exclude the edges and vertices of the triangle, the subset would be a two-dimensional manifold; thus, $\{a\vec{h}_1 + b\vec{h}_2 : a, b > 0, a + b < 1\}$ is a manifold while $\{a\vec{h}_1 + b\vec{h}_2 : a, b > 0, a + b \leq 1\}$ is not.



5.3.1 Charts, atlases, tangent spaces and approximating tangent planes

Hereafter, we will focus on manifolds which are a subset of $\mathbb{R}^d$, which are often referred to as submanifolds. Differential geometry can be made far more abstract, but that is unnecessary for the discussion here. After all, we're dealing with finite dimensional data so any nonlinear surface containing our data must be a submanifold of $\mathbb{R}^d$!

Now, some mathematical foundation. Let $\mathcal{M}$ be a manifold in $\mathbb{R}^d$ with intrinsic dimension $t < d$. For every point $\vec{x} \in \mathcal{M}$, there is a neighborhood $U_x \subset \mathcal{M}$ containing $\vec{x}$ and a function $\phi_x : U_x \rightarrow \phi_x(U_x) \subset \mathbb{R}^t$ which is continuous, bijective, and has continuous inverse (a *homeomorphism* if you like greek). The pair (U_x, ϕ_x) is called a chart and behaves much like a map of the area around $\vec{x}$. There are many choices for ϕ_x , but we can always choose one which maps $\vec{x} \in \mathbb{R}^d$ to the origin in $\mathbb{R}^t$. For vectors $\vec{z} \in U_x$, we call $\phi_x(\vec{z}) \in \mathbb{R}^t$ the *local* coordinates of $\vec{z}$.

Example 5.4 (Charts and Manifold Maps). Let's revisit the Swiss roll example (Example 3.1) where $A = (\pi/2, 9\pi/2) \times (0, 15)$. We defined the map $\Psi : A \rightarrow \mathbb{R}^3$ as follows

$$\Psi(\vec{z}) = \Psi(z_1, z_2) = \begin{bmatrix} z_1 \sin(z_1) \\ z_1 \cos(z_1) \\ z_2 \end{bmatrix}.$$

For other manifolds, we may need multiple charts to cover the manifold, but we only need one chart the swiss roll (or any manifold defined through a homeomorphic manifold map). Let the neighborhood be the entire manifold, i.e. $U = \mathcal{M}$. Given $\vec{x} = (x_1, x_2, x_3)^T$, let $\phi(\vec{x}) = (\sqrt{x_1^2 + x_2^2}, x_3)^T$. This chart essentially unrolls the swiss roll and turns it back into a rectangle in $\mathbb{R}^2$ so that the local coordinates are the original coordinates!

If we take a collection of charts $\{U_x, \phi_x\}_{x \in \mathcal{I}}$ such that $\cup_{x \in \mathcal{I}} U_x = \mathcal{M}$, we have an atlas for the manifold. Here $\mathcal{I}$ is a subset of $\mathcal{M}$ which could be countable or finite. With a chart, we can consider doing differential calculus on the manifold. In particular, if $f : \mathcal{M} \rightarrow \mathbb{R}$, then for a chart (U_x, ϕ_x) , the function $f \circ \phi_x^{-1}$ is a map from a subset of $\mathbb{R}^t$ (namely $\phi_x(U)$) to $\mathbb{R}$ so we might hope that we could apply the typical rules of calculus. However, if we have two charts (U_y, ϕ_y) and (U_x, ϕ_x) which overlap – $(U_x \cap U_y) \neq \emptyset$ – then derivatives $f \circ \phi_x^{-1}$ and $f \circ \phi_y^{-1}$ should agree on $U_x \cap U_y$. More succinctly, the rules of calculus should remain consistent across charts!

Definition 5.2 (Differentiable Manifolds). An atlas $\{U_x, \phi_x\}_{x \in \mathcal{M}}$ is differentiable if the transition maps $\phi_x \circ \phi_y^{-1} : \phi_y(U_y) \rightarrow \mathbb{R}^t$ are differentiable functions. Recall $\phi_y(U_y) \subset \mathbb{R}^t$ so differentiable in this case follows the traditional Euclidean definition from calculus.

With some additional properties and manifold with a differential atlas is a differentiable manifold allowing us to compute derivatives of function from the manifold to the reals. We'll revisit this detail again when discussing Hessian Local Linear Embeddings. For now, let's assume we have a differentiable manifold with intrinsic dimension t . To every point on the manifold, we can attach a t -dimensional tangent space. There are several methods for defining the tangent space, but the most straightforward involves the case where we have a manifold map. We'll restrict our attention to this case.

Definition 5.3 (Tangent Space). Let $A \subset \mathbb{R}^t$ and let $\Psi : A \rightarrow \mathcal{M} \subset \mathbb{R}^d$. Furthermore, suppose Ψ has coordinate functions $\Psi_1, \dots, \Psi_d : A \rightarrow \mathbb{R}$ such that for $\vec{y} = (y_1, \dots, y_t)^T \in A$, $\Psi(\vec{y}) = (\Psi_1(\vec{y}), \dots, \Psi_d(\vec{y}))^T$. The Jacobian of Ψ , denoted $\mathbf{J}_\Psi$ is the $d \times t$ dimensional matrix of partial derivative such that

$$(\mathbf{J}_\Psi)_{ij} = \frac{\partial \Psi_i}{\partial y_j}.$$

The tangent space of the manifold $\mathcal{M}$ at the point $\vec{p} = \Psi(\vec{y})$, denoted $T_p(\mathcal{M})$ is the **column span** of $(\mathbf{J}_\Psi)$ evaluated at $\vec{y}$.

The manifold locally resembles $\mathbb{R}^t$ so the tangent space should also be t -dimensional. As such, we'll require $\mathbf{J}_\Psi$ to be a full rank matrix (with rank t since $t < d$) for every $\vec{y} \in A$. Importantly, the tangent space is a *linear* subspace meaning it passes through the origin. This should not be confused with tangent plane to the manifold which we now define.

Definition 5.4 (Tangent Plane). Suppose a manifold $\mathcal{M}$ has tangent space $T_p(\mathcal{M})$. Then the approximating tangent plane to the manifold is the affine subspace obtained by translations, namely $\{\vec{x} \in \mathbb{R}^d : \vec{x} - \vec{p} \in T_p(\mathcal{M})\}$.

Let's return to the swiss roll to make these details explicit.

Example 5.5 (Jacobians and Tangent Space for the Swiss Roll). The Jacobian

of the swiss roll map is

$$\mathbf{J}_\Psi = \begin{bmatrix} z_1 \cos(z_1) + \sin(z_1) & 0 \\ -z_1 \sin(z_1) + \cos(z_1) & 0 \\ 0 & 1 \end{bmatrix}.$$

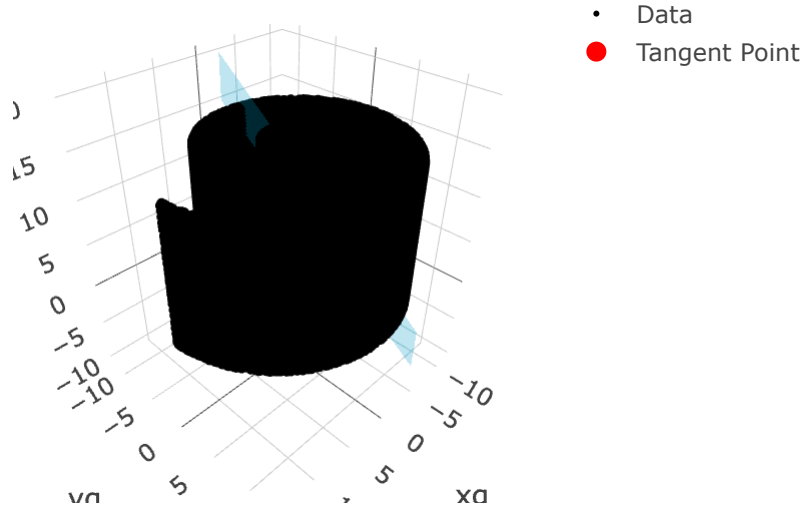
At the point $\Psi(3\pi/2, 5) = (-3\pi/2, 0, 5)^T$ the Jacobian is

$$\mathbf{J}_\Psi|_{(3\pi/2, 5)^T} = \begin{bmatrix} -1 & 0 \\ -3\pi/2 & 0 \\ 0 & 1 \end{bmatrix}$$

so

$$T_{(3\pi/2, 0, 5)^T}(\mathcal{M}) = \text{span}\{(1, 3\pi/2, 0)^T, (0, 0, 1)^T\}.$$

We can view the associated approximating tangent plane (translucent blue) at the point $(3\pi/2, 0, 5)^T$ (in red).



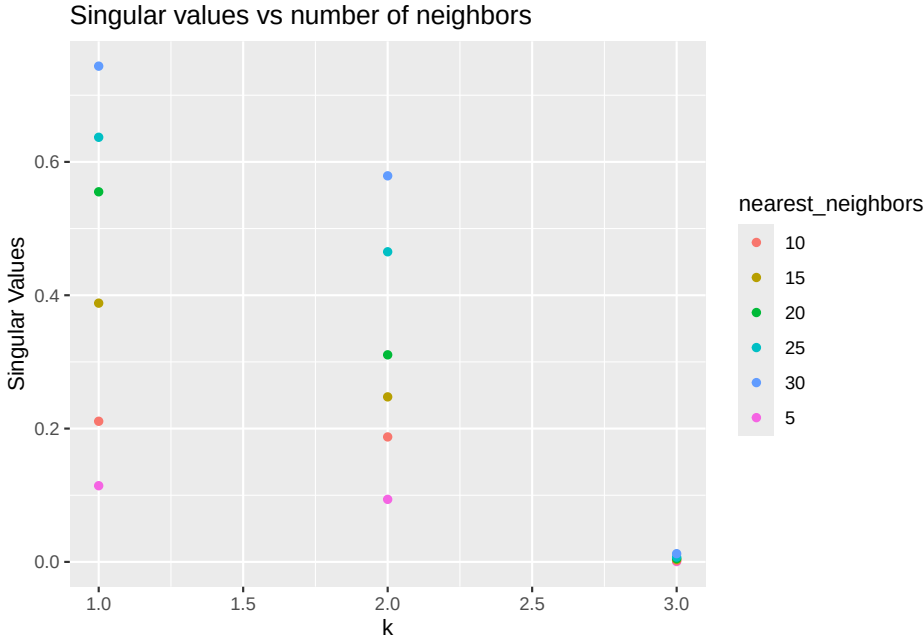
5.3.1.1 Estimating properties from data

In the preceding subsection, we needed to manifold map to define charts, local coordinates, and tangent spaces. However, we do not have access to the manifold map only samples we assume are living near or on an t -dimensional manifold. In fact, we do not typically know t either. Fortunately, ideas we have discussed previously can allow us to estimate intrinsic dimensionality, local coordinates, and tangent spaces from data. The key idea is to zoom in on a sufficiently small neighborhood of the manifold that looks inherently Euclidean.

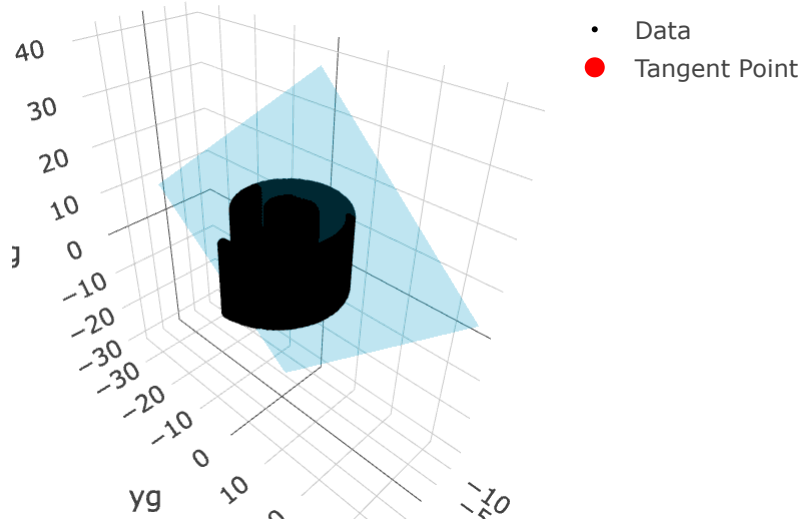
Suppose we have sample $\vec{x}_1, \dots, \vec{x}_N \in \mathcal{M}$. Given a point $\vec{x} \in \mathcal{M}$ – typically a sample in our data set – we can first find k nearest points to $\vec{x}$ using Euclidean distance. Without loss of generality, let's call them $\vec{x}_1, \dots, \vec{x}_k$. If the nearest

neighbors are sufficiently close to $\vec{x}$ they should reside close to a t -dimensional hyperplane! We can then apply PCA to $\vec{x}_1, \dots, \vec{x}_k$ or SVD to the displacements $\vec{x}_1 - \vec{x}, \dots, \vec{x}_k - \vec{x}$. We expect to see a sharp drop after t eigenvalues (or singular values) allowing us to estimate the intrinsic dimension t . Subsequently, the first t PCA scores (or first t columns of $\mathbf{US}$ in the SVD) serve as local coordinates for $\vec{x}_1, \dots, \vec{x}_k$. Finally, the first t PC loadings (or first t right singular vectors) are an approximate basis for the tangent space.

Example 5.6 (Estimating Local Properties from Samples). Suppose we are given $N = 10^4$ samples from the Swiss roll. Let's see how the SVD approach outlined above performs at estimating the tangent space at $\Psi(3\pi/2, 5) = (-3\pi/2, 0, 5)^T = \vec{x}$. As an example, we'll plot the singular values of the SVD of the data matrix with rows $\vec{x}_1^T - \vec{x}^T, \dots, \vec{x}_k^T - \vec{x}^T$. We'll investigate for a range of k .



The sharp drop after $t = 2$ is striking for $k > 5$ nearest neighbors. Furthermore, this method does an excellent job at estimating the tangent space. Below, we show the associated approximating hyperplane which we estimate using the first two right singular vectors of the data matrix using the first 15 neighbors. The results look visually indiscernible from the case where the Jacobian was used.



SVD and PCA still have their uses thanks to the locally Euclidean nature of manifolds. Naturally, the quality of the estimations depends on having sufficiently dense sampling on the manifold. Verifying results are robust to k is a good place to start, but this approach has been formalized into a more reliable method for estimating the intrinsic dimensionality of a manifold [Little et al., 2009].

5.4 Isometric Feature Map (ISOMAP)

5.4.1 Introduction

The first manifold learning method we are going to cover is the Isometric Feature Map (ISOMAP), originally published by Tenenbaum, de Silva, and Langford in 2000 [Tenenbaum et al., 2000]. As suggested by the name, we will see that the assumption of isometry is central to this method. ISOMAP combines the major algorithmic features of PCA and MDS — computational efficiency, global optimality, and asymptotic convergence guarantees. Thanks to these extraordinary features, ISOMAP is capable of learning a broad class of nonlinear manifolds.

5.4.2 Key Definitions

Different notions of pointwise distance

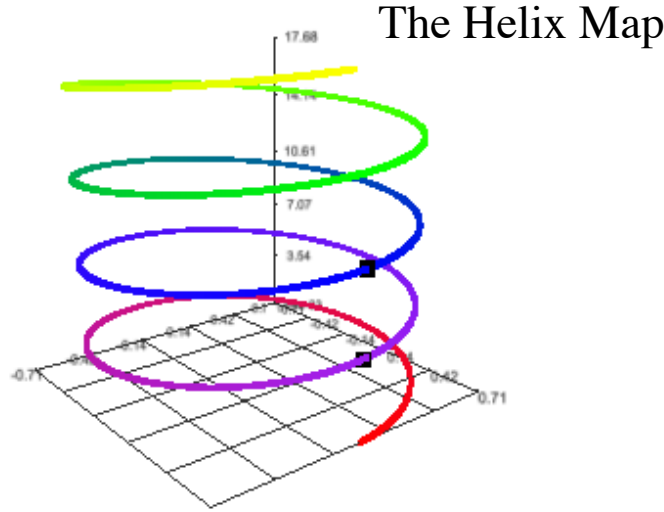
Prior to discussing the ISOMAP algorithm, let's briefly discuss the notion of isometry through an example which motivates different notions of distance between two points.

Example 5.7 (Distance between points on a Helix). Consider the helix map

$\Psi : \mathbb{R} \rightarrow \mathbb{R}^3$ given by the formula

$$\Psi(t) = \begin{bmatrix} \frac{1}{\sqrt{2}} \cos(t) \\ \frac{1}{\sqrt{2}} \sin(t) \\ \frac{1}{\sqrt{2}} t \end{bmatrix} \quad (5.2)$$

Below, we show the result of applying the Helix map to each point in the interval $(0, 25)$. Let's focus on two points $\vec{x}_1 = \Psi(2\pi) = (1/\sqrt{2}, 0, \sqrt{2}\pi)^T$ and $\vec{x}_2 = \Psi(4\pi) = (1/\sqrt{2}, 0, 2\sqrt{2}\pi)^T$ in particular which are shown as large black dots in the figure below.



There are a few different ways we could measure the distance between the two black points. The first approach would be to ignore the helix (manifold) structure viewing them as vectors in $\mathbb{R}^3$ and directly measure their **Euclidean distance** which gives

$$\|\vec{x}_1 - \vec{x}_2\| = \sqrt{2}\pi.$$

However, we also know that these points are images of the one-dimensional coordinate $z_1 = 2\pi$ and $z_2 = 4\pi$ respectively. Thus, we could also consider the Euclidean distance of the lower-dimensional coordinates which is $|2\pi - 4\pi| = 2\pi$, which notably differs from the Euclidean distance.

A third option is to return to the three-dimensional representation but to also account for the manifold structure when considering distance. Recall Euclidean distance gives the length of the shortest, **straightline** path connecting the two points. Instead, let's restrict ourselves to only those paths which stay on the helix (manifold). You may correctly conclude that the curve starting at $\Psi(2\pi)$,

rotating up the helix one rotation, and ending at $\Psi(4\pi)$ is the shortest such path. Fortunately, computing arc-length is relatively friendly in this example since Ψ already parameterizes the path connecting these two points. The arc-length is then

$$\int_{2\pi}^{4\pi} \left\| \frac{d\Psi}{dt} \right\| dt = \int_{2\pi}^{4\pi} dt = 2\pi.$$

Jumping slightly ahead, we then say the manifold distance between $\Psi(2\pi)$ and $\Psi(4\pi)$ is 2π . Importantly, the manifold distance coincides exactly with the Euclidean distance between the **lower-dimensional** coordinates. In fact, for any two points, s and t , on the real line their Euclidean distance, $|s - t|$ will be the same as the manifold distance between $\Psi(s)$ and $\Psi(t)$. Thus, the helix map Ψ above serves as our first example of an isometric (distance preserving) map.

We may generalize this idea to any smooth manifold to define a new notion of distance. Given a manifold $\mathcal{M}$, we define the manifold distance function $d_{\mathcal{M}} : \mathcal{M} \times \mathcal{M} \rightarrow [0, \infty)$ as follows

Definition 5.5 (Manifold Distance Function). Given two points $\vec{x}$ and $\vec{y}$ on a smooth manifold, $\mathcal{M}$, let $\Gamma(\vec{x}, \vec{y})$ be the set of all piecewise smooth curves connecting $\vec{x}$ and $\vec{y}$ constrained to stay on $\mathcal{M}$. Then, we define the manifold distance to be

$$d_{\mathcal{M}}(\vec{x}, \vec{y}) = \inf_{\gamma \in \Gamma(\vec{x}, \vec{y})} L(\gamma) \quad (5.3)$$

where $L(\gamma)$ is the arclength of γ .

As we reviewed above, the helix example with the arclength formula is one example of a manifold and distance function. Additional examples of a manifold and manifold distance include,

- Euclidean space $\mathbb{R}^d$ where standard Euclidean distance gives the manifold distance.
- The sphere in $\mathbb{R}^3$ which is a two-dimensional manifold. Its manifold distance is also called the Great Circle Distance.

We may now define the notion of isometry which is a central assumption of ISOMAP.

Definition 5.6 (Isometry). Let $\mathcal{M}_1$ be a manifold with distance function $d_{\mathcal{M}_1}$ and let $\mathcal{M}_2$ be a second manifold with distance function $d_{\mathcal{M}_2}$. The mapping $\Psi : \mathcal{M}_1 \mapsto \mathcal{M}_2$ is an isometry if

$$d_{\mathcal{M}_1}(x, y) = d_{\mathcal{M}_2}(\Psi(\vec{x}), \Psi(\vec{y})) \quad \text{for all } \vec{x}, \vec{y} \in \mathcal{M}_1.$$

For the purposes of ISOMAP, we will think of $\mathcal{M}_1$ as some subset of a $\mathbb{R}^k$ for k small where we measure distances using the Euclidean norm. Then $\mathcal{M}_2$ will be a k -dimensional manifold in $\mathbb{R}^d$ containing our data. Our first assumption is that the manifold mapping Ψ is an isometry. Unfortunately, in practice we do not know the manifold nor will we have a method for parameterizing curves on the manifold to compute distances.

5.4.3 Algorithm

Instead, ISOMAP makes use of a data-driven approach to estimate the manifold distance between points following a three-step procedure.

1) Construct Weighted Neighborhood Graph:

MDS uses Euclidean distance to measure pairwise distance between points $\vec{x}_i$ and $\vec{x}_j$ (data points in space $\mathcal{M}_2$), while ISOMAP uses the geodesic distance in order to reveal the underlying manifold structure. However, when the data points in the high dimensional space $\mathcal{M}_2$ have a manifold structure, usually the Euclidean pairwise distance is quite different from their pairwise geodesic distance. Fortunately, for small distances on a smoothly embedded manifold, the geodesic path between two close-by points lies nearly flat in the ambient space. So, the length of this path will be very close to the straight line (Euclidean) distance between those points in the ambient space.

The key intuition is that as the density of data points on the manifold increases (i.e., points get closer and closer), the straight line segment in the ambient space connecting two neighboring points becomes a better and better approximation of the shortest path between those points on the manifold. In the limit of the density going to infinity, these distances converge.

Let's elucidate this concept with two illustrative examples. Firstly, imagine a two-dimensional surface, like a Swiss Roll, situated within a three-dimensional space. For an ant journeying across the Swiss Roll, the vast size difference means its immediate surroundings appear flat. From its perspective, the distance between its consecutive steps closely mirrors the distance a human might measure (Euclidean distance) – both virtually equating to the roll's geodesic distance. For a larger-scale analogy, think of Earth. Suppose extraterrestrial beings possessed technology allowing them to traverse straight through Earth's crust and mantle, thus following the shortest Euclidean path. Their journey from Los Angeles to New York might save them hundreds of miles compared to humans. However, when moving between closer landmarks, such as the Science Center to the Smith Center, their advantage diminishes.

As a result, when it comes to the measurement of geodesic distance, it is reasonable to only look at those data points that are close to each other. First, calculate all the pairwise Euclidean distance $d_{ij} = \|\vec{x}_i - \vec{x}_j\|_2$, then determine which points are neighbors on the manifold by connecting each point to Either (i) All points that lie within a ball of radius ϵ of that point; OR (ii) all points which are K-nearest neighbors with it. (Two different criteria, K and ϵ are tuning parameters)

According to this rule, a weighted neighborhood graph $G = G(V, E)$ can be built. The set of vertices (data points in space $\mathcal{M}_2$): $V = \{\vec{x}_1, \dots, \vec{x}_N\}$ are the input data points, and the set of edges $E = \{e_{ij}\}$ indicate neighborhood relationships between the points. $e_{ij} = d_{ij}$ if (i) $\|\vec{x}_i - \vec{x}_j\|_2 \leq \epsilon$; OR (ii) $\vec{x}_j$ is one of the K-nearest neighbors of $\vec{x}_i$, otherwise $e_{ij} = \infty$. Sometimes, the tuning of ϵ (or

K) is quite decisive in the output of ISOMAP, we will explain this later with a simulation example.

2) Compute graph distances

In this step, we want to estimate the unknown true geodesic distances $\{d_{ij}^{\mathcal{M}}\}$ between all pairs of points with the help of the neighborhood graph G we have just built. We use the graph distances $\{d_{ij}^{\mathcal{G}}\}$ —the shortest distances between all pairs of points in the graph G to estimate $\{d_{ij}^{\mathcal{M}}\}$. For $\vec{x}_i$ and $\vec{x}_j$ that are not connected to each other, we try to find the shortest path that goes along the connected points on the graph. Following this particular sequence of neighbor-to-neighbor links, the sum of all the link weights along the path is defined as $\{d_{ij}^{\mathcal{G}}\}$. In other words, we use a number of short Euclidean distances (representing the local structure of the manifold) to approximate the geodesic distance $\{d_{ij}^{\mathcal{M}}\}$.

This path finding step is usually done by Floyd-Warshall algorithm, which iteratively tries all transit points k and find those that $\tilde{d}_{ik} + \tilde{d}_{kj} < \tilde{d}_{ij}$, and updates $\tilde{d}_{ij} = \tilde{d}_{ik} + \tilde{d}_{kj}$ for all possible combination of i, j . The algorithm works best in dense neighboring graph scenario, with a computational complexity of $O(n^3)$.

The theoretical guarantee of this graph distance computation method is given by Bernstein et al. [Bernstein et al., 2001] one year after they first proposed ISOMAP in their previous paper. They show that asymptotically (as $n \rightarrow \infty$), the estimate $d^{\mathcal{G}}$ converges to $d^{\mathcal{M}}$ as long as the data points are sampled from a probability distribution that is supported by the entire manifold, and the manifold itself is flat.

The distance matrix Δ can be expressed as:

$$\Delta_{ij} = d_{ij}^{\mathcal{G}}$$

Simulation Example

Here we provide a randomly generated Neighborhood Graph for six data points, it uses the K-nearest neighbor criteria (can easily tell this since the matrix is not symmetric, $K = 2$)

```
# Define the matrix
matrix <- matrix(c(
  0, 3, 4, Inf, Inf, Inf,
  7, 0, Inf, 2, Inf, Inf,
  6, Inf, 0, Inf, 7, Inf,
  Inf, 5, Inf, 0, Inf, 10,
  Inf, Inf, 8, Inf, 0, 13,
  Inf, Inf, Inf, 9, 14, 0
), byrow = TRUE, nrow = 6)
print(matrix)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]    0    3    4 Inf  Inf  Inf
## [2,]    7    0 Inf    2 Inf  Inf
## [3,]    6 Inf    0 Inf    7 Inf
## [4,] Inf    5 Inf    0 Inf  10
## [5,] Inf Inf    8 Inf    0  13
## [6,] Inf Inf Inf    9  14    0
```

Shown below is the implementation of Floyd-Warshall algorithm in R. As you can see from the three for loops, its computation complexity is $O(n^3)$.

```
# Adjusting the matrix to set d_ij and d_ji to the smaller value
n <- dim(matrix)[1]

for (i in 1:n) {
  for (j in 1:n) {
    if (i != j && is.finite(matrix[i, j]) && is.finite(matrix[j, i])) {
      min_val <- min(matrix[i, j], matrix[j, i])
      matrix[i, j] <- min_val
      matrix[j, i] <- min_val
    }
  }
}

# Floyd-Warshall Algorithm
floyd_warshall <- function(mat) {
  n <- dim(mat)[1]
  dist <- mat

  for (k in 1:n) {
    for (i in 1:n) {
      for (j in 1:n) {
        dist[i, j] <- min(dist[i, j], dist[i, k] + dist[k, j])
      }
    }
  }

  return(dist)
}

# Get the result
result <- floyd_warshall(matrix)

# Print the result
print(result)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6]
```

## [1,]	0	3	4	5	11	14
## [2,]	3	0	7	2	14	11
## [3,]	4	7	0	9	7	18
## [4,]	5	2	9	0	16	9
## [5,]	11	14	7	16	0	13
## [6,]	14	11	18	9	13	0

3) Applying MDS to Δ

As mentioned before, ISOMAP can be viewed as the application of classical MDS in a non-linear case. As a result, the reconstruction of $\{\vec{z}_i\}$ in the k dimensional $\mathcal{M}_1$ follows similar steps as that of classical MDS. The main goal is to preserve the geodesic distance of the manifold in $\mathcal{M}_2$ as much as possible.

Without any additional information, there are infinite $\{\vec{z}_i\}$ that can be viewed as the optimal solution. For some invertible function $\Phi : \mathbb{R}^k \rightarrow \mathbb{R}^k$, a new manifold mapping $\Psi \circ \Phi^{-1}$ can be constructed. $\vec{x}_i = \Psi \circ \Phi^{-1}(\Phi(\vec{z}_i))$, which proves that $\{\Phi(\vec{z}_i)\}$ is equivalent to $\{\vec{z}_i\}$ when it comes to the reconstruction of the lower dimensional configuration.

Without loss of generality, we assume that $\{\vec{z}_i\}$ are actually centered. So the distance matrix of $\{\vec{z}_i\}$ can be expressed as $B = Z^T Z$, so that $B_{ii} = \|\vec{z}_i\|_2^2$ and $B_{ij} = \vec{z}_i^T \vec{z}_j$.

The embedding vectors $\{\hat{z}_i\}$ (estimate of points in lower dimensional feature space $\mathcal{M}_1$) are chosen in order to minimize the objective function:

$$\sum_{i,j} (\|\vec{z}_i - \vec{z}_j\|_2 - \Delta_{ij})^2$$

Following the same procedure explained in classical MDS chapter, we can compute each entry of B :

$$B_{ij} = -\frac{1}{2} \left(\Delta_{ij}^2 - \frac{1}{N} \sum_{k=1}^N \Delta_{kj}^2 - \frac{1}{N} \sum_{k=1}^N \Delta_{ik}^2 + \frac{1}{N^2} \sum_{k=1}^N \sum_{l=1}^N \Delta_{kl}^2 \right)$$

To express it in matrix form, it is actually, $B = -\frac{1}{2} H \Delta^{(2)} H$, where $\Delta^{(2)}$ is the matrix of elementwise-squared geodesic distances, i.e. $\Delta_{ij}^{(2)} = \Delta_{ij}^2$, and $H = I_N - \frac{1}{N} \mathbb{1} \mathbb{1}^T$.

The next step is just a PCA problem. Implement eigen decomposition on matrix B , $B = U \Lambda U^T = (\Lambda^{1/2} U)^T (\Lambda^{1/2} U)$, then arrange the singular value in descending order, find the first k' ones. We acquire $\Lambda_{k'}$ and $U_{k'}$.

$$(\hat{z}_1 | \hat{z}_2 | \dots | \hat{z}_N) = \Lambda_{k'} U_{k'}$$

Since we don't know the dimension of the underlying feature space, here k' is a tuning parameter. Usually, we use a scree plot (k' against the sum of the omitted eigenvalues) and find the elbow point.

5.4.4 Limitations of ISOMAP

Though ISOMAP is a powerful manifold learning method that works well under most circumstances. It still has some limitations in certain scenarios.

- 1) If the noises $\{\epsilon_i\}$ is not negligible, then ISOMAP may fail to identify the manifold. Also, ISOMAP is quite sensitive to the tuning parameters. To alleviate the negative impact, it's highly suggested to start with a relatively small ϵ or K , and increase them gradually.
- 2) When data points are sparse in certain areas or directions of the manifold, the integrity of the learned manifold structure can be compromised. The following example will clarify this notion:
- 3) One of the two major assumptions of ISOMAP is the convexity of the manifold, that is to say, if the manifold contains many holes and concave margins, then the result of ISOMAP will probably be not ideal.

5.5 Locally Linear Embeddings (LLEs)

5.5.1 Introduction

Locally linear embedding (LLE) is an unsupervised learning algorithm first introduced in 2000 by Sam T. Roweis and Lawrence K. Saul [Roweis and Saul, 2000]. In the original four-page paper, the two authors introduced the LLE algorithm and demonstrated its effectiveness in dimensional reduction, manifold learning, and in handling real-world high-dimensional data. Unlike clustering methods for local dimensional reduction, LLE maps its inputs into a single global coordinate system of lower dimensionality, and its optimizations do not involve local minima. By exploiting the local symmetries of linear reconstructions, LLE is able to learn the global structure of nonlinear manifolds, such as those generated by images of faces or documents of text. Thanks to its great mathematical properties and relatively low computing cost (compared to other manifold learning methods, like ISOMAP), LLE quickly became attractive to researchers after its emergence due to its ability to deal with large amounts of high dimensional data and its non-iterative way of finding the embeddings [Chen and Liu, 2011]. Compared to ISOMAP and some other previous manifold learning methods, LLE is computationally simpler and can give useful results on a broader range of manifolds [Saul and Roweis, 2003].

For dimensional reduction, most methods introduced before LLE need to estimate pairwise distances between even two remote data points, no matter it is the simple Euclidean distance (classical MDS) or more sophisticated manifold distance (ISOMAP). The underlying main idea of these methods is actually finding a configuration that recovers all pairwise distances of original data points as much as possible. LLE, however, is quite different from these previous methods as it focuses on preserving **locally linear relationships**.

5.5.2 Algorithm

LLE algorithm is actually built on very simple geometric intuitions. As explained in MDS Part, if we consider a small enough region on a manifold in D dimensional space, in most cases, it can be regarded as a d dimensional hyperplane ($d \ll D$). LLE also makes use of this intuition and assumes that the whole manifold consist of numerous d -dimensional patches that have been stitched together. Assuming that there exists sufficient data (data points are compact), it is reasonable to expect that each data point and its neighbors lie on or close to a locally linear patch of the manifold.

Following this idea, LLE approximates each data point by a weighted linear combination of its neighbors and proceeds to find a lower-dimensional configuration of data points so that the linear approximations of all data points are best preserved.

Specifically speaking, LLE algorithm consists of three steps. The initial step involves selecting a certain number of each data point's nearest neighbors based on Euclidean distance. Following this, the second step calculates the optimal reconstruction weights for each point using its nearest neighbors. The final step carries out the embedding while maintaining the local geometry depicted by the reconstruction weights.

5.5.2.1 Construct Neighborhood Graph

This step is actually very similar to that of ISOMAP. The process of finding neighbors in LLE is typically conducted using grouping methods like k -nearest neighbors (KNN) or selecting neighbors within a fixed radius ball (ϵ -neighborhoods), based on the Euclidean distance for each data point, in the provided data set. The KNN method is predominantly utilized for its straightforwardness and ease of implementation. The following explanations are based on KNN method.

Denote N data points in original D dimensional space as $\vec{x}_1, \vec{x}_2, \dots, \vec{x}_N \in \mathbb{R}^D$. For a point $\vec{x}_i$, $1 \leq i \leq N$, its neighbor set is defined as $N_i^k \subseteq \{1, 2, 3, \dots, i-1, i+1, \dots, N\}$, where N_i^k can also be called as the indices of k nearest neighbors of $\vec{x}_i$. The tuning parameter k is chosen small enough so that the patch around $\vec{x}_i$ is flat. However, k should also be strictly larger than d so as to let the algorithm work.

As we can tell from these, LLE works well only if data points are dense and hopefully evenly distributed, which will be explained in detail in later examples. The parameter tuning of the appropriate number of neighbors, k , faces challenges of complexity, non-linearity, and diversity of high-dimensional input samples. A larger k value might cause the algorithm to overlook or even lose the local nonlinear features on the manifold. This issue is exacerbated as neighbor selection, typically based on Euclidean distance, can result in distant neighbors when considering the intrinsic geometry of the data, akin to a short circuit. Conversely, an overly small k value may lead the LLE algorithm to fragment the

continuous manifold into isolated local pieces, losing global characteristics.

5.5.2.2 Reconstruct with Linear Weights

As put before, we try to reconstruct each $\vec{x}_i$ using an almost convex weighted combination of its neighbors. The respective weights of all its neighbors $\vec{x}_j$, $j \neq i$ for each $\vec{x}_i$ is quite essential in the later reconstruction of the underlying intrinsic configuration, as we consider these weights to remain invariant before and after mapping.

To explain it in mathematical formulas, the approximate of $\vec{x}_i$: $\tilde{x}_i$ is defined as $\tilde{x}_i = \sum_{j=1}^N w_{ij} \vec{x}_j$. There are two constraints for this formula: First, $w_{ij} \equiv 0$, if $j \notin N_i^k$ (consistent with the assumption of k nearest neighbors); Second, the sum of weights for each $\vec{x}_i$ is always one, i.e., $\sum_{j=1}^N w_{ij} = 1$.

Then, the problem of finding the optimal w_{ij} , $1 \leq i, j \leq N$ is equivalent to solving the following constrained Least Squares problem for $\forall 1 \leq i \leq N$:

$$\begin{aligned} \min \quad & \left\| \vec{x}_i - \sum_{j \in N_i^k} w_{ij} \vec{x}_j \right\|^2 \\ \text{s.t.} \quad & \sum_{j \in N_i^k} w_{ij} = 1. \end{aligned} \quad (\#eq : \text{opt}_{prob}) \quad (5.4)$$

It is worth noting that the weights can be negative theoretically, though in practice, we don't expect that to happen.

Invariance to Rotation, Rescaling and Translation

Define $\epsilon(w) = \sum_{i=1}^N \left\| \vec{x}_i - \sum_{j \in N_i^k} w_{ij} \vec{x}_j \right\|^2$, which is the cost function.

- 1) $\epsilon(w)$ is unchanged by rotation or rescaling by common factor

Actually $\sum_{i=1}^N \left\| aU\vec{x}_i - \sum_{j \in N_i^k} w_{ij} aU\vec{x}_j \right\|^2 = a^2 \epsilon(w)$, where a is a non-zero scalar and U is an orthonormal matrix.

- 2) $\epsilon(w)$ is unchanged by translations

Thanks to the constraint that $\sum_{j=1}^N w_{ij} = 1$, for any translation $\vec{x}_i \rightarrow \vec{x}_i + \vec{y}$, the cost function does not change.

$$\sum_{i=1}^N \left\| (\vec{x}_i + \vec{y}) - \sum_{j \in N_i^k} w_{ij} (\vec{x}_j + \vec{y}) \right\|^2 = \sum_{i=1}^N \left\| \vec{x}_i - \sum_{j \in N_i^k} w_{ij} \vec{x}_j \right\|^2 = \epsilon(w)$$

From the expressions, we develop a strategy that optimizes one row of matrix w at a time. Now let's try to rewrite $\epsilon(\vec{w}_i) = \left\| \vec{x}_i - \sum_{j \in N_i^k} w_{ij} \vec{x}_j \right\|^2$.

$$\epsilon(\vec{w}_i) = \left\| \vec{x}_i - \sum_{j \in N_i^k} w_{ij} \vec{x}_j \right\|^2 \quad (5.5)$$

$$= \left[\sum_{j=1}^N w_{ij} (\vec{x}_i - \vec{w}_j) \right]^T \left[\sum_{l=1}^N w_{il} (\vec{x}_i - \vec{w}_l) \right]^T \quad (5.6)$$

$$= \sum_{j=1}^N \sum_{l=1}^N w_{ij} w_{il} (\vec{x}_i - \vec{x}_j)^T (\vec{x}_i - \vec{x}_l) \quad (5.7)$$

$$= \vec{w}_i^T G_i \vec{w}_i \quad (5.8)$$

$\vec{w}_i^T = (w_{i1}, w_{i2}, \dots, w_{iN})$ is the i^{th} row of W . Here $G_i \in \mathbb{R}^{N \times N}$, where entry $G_i(j, l)$, $1 \leq j, l \leq N$ can be represented as:

$$G_i(j, l) = \begin{cases} (\vec{x}_i - \vec{x}_j)^T (\vec{x}_i - \vec{x}_l) & j, l \in N_i^k \\ 0 & j \text{ or } l \notin N_i^k \end{cases}$$

The (j, l) entry of G_i is actually the inner product of $\vec{x}_j$ and $\vec{x}_l$ when centered around $\vec{x}_i$. From this expression, we know that actually G_i is a sparse matrix and can be reduced to a compact matrix $\tilde{G}_i \in \mathbb{R}^{k \times k}$ that eliminates those empty columns and rows.

$$\tilde{G}_i = (\vec{x}_{i[1]} - \vec{x}_i, \dots, \vec{x}_{i[k]} - \vec{x}_i)^T (\vec{x}_{i[1]} - \vec{x}_i, \dots, \vec{x}_{i[k]} - \vec{x}_i) \quad (5.9)$$

$$= Q_i^T Q_i \quad (5.10)$$

where $[1]$ denotes the first entry in N_i^k . So $\tilde{G}_i$ is actually a real symmetric and positive semi-definite matrix.

Now let's go back to deal with the optimization function — Equation @ref(eq:opt_prob) can be solved with Lagrange multiplier given that it has only equality constraints. (More details about the use of Lagrange multiplier can be found in Lagrange multiplier)

Optimizing Equation @ref(eq:opt_prob) is equivalent to minimizing (for $\forall 1 \leq i \leq N$)

$$f(\vec{w}_i, \lambda) = \vec{w}_i^T G_i \vec{w}_i - \lambda(\vec{w}_i^T \mathbf{1}_k - 1)$$

which has the result:

$$\vec{w}_i^* = \frac{\tilde{G}_i^{-1} \mathbf{1}_k}{\mathbf{1}_k^T \tilde{G}_i^{-1} \mathbf{1}_k}$$

Complement:

As discussed before, we can only prove that $\tilde{G}_i$ is positive semi-definite, however, we cannot ensure that it is positive definite, which means $\tilde{G}_i$ is not necessarily invertible. That is why we use the generalized inverse sign here. In practice, it can be done through performing SVD on $\tilde{G}_i$ and select the first few large singular values and eliminate the rest. Then when computing $\tilde{G}_i^-$, just do the reciprocal of these retained singular values.

Actually $\tilde{G}_i$ only has d (the intrinsic original dimension if you forget) relatively large eigenvalues. The rest are either very small or zeros. So it is very likely that $\tilde{G}_i$ is singular, making the computation result highly unstable. In [Saul and Roweis, 2003], the authors proposed to address this issue through regularizing $\tilde{G}_i$.

$$\tilde{G}_i \leftarrow \tilde{G}_i + \left(\frac{\Delta^2}{k} \right) \text{Tr}(\tilde{G}_i) \mathbf{I}$$

Here $\text{Tr}(\tilde{G}_i)$ denotes the trace of $\tilde{G}_i$ and $\Delta \ll 1$.

5.5.2.3 Embedding

In the previous step, we have recovered the optimal weight matrix

$$\mathbf{W} = \begin{pmatrix} \vec{w}_1^T \\ \vdots \\ \vec{w}_N^T \end{pmatrix}$$

The optimal weights $\mathbb{W}$ reflects local, linear geometry around each $\vec{x}_i$, thus if the configuration $\vec{y}_1, \vec{y}_2, \dots, \vec{y}_N \in \mathbb{R}^d$ are the lower dimensional representation, they should also “match” the local geometry.

Following this idea, the objective function is:

$$\begin{aligned} & \underset{\mathbf{Y}}{\text{argmin}} \sum_{i=1}^N \left\| \vec{y}_i - \sum_{j=1}^N w_{ij} \vec{y}_j^T \right\|^2 \\ &= \underset{\mathbf{Y}}{\text{argmin}} \sum_{i=1}^N \left\| \sum_{j=1}^N w_{ij} (\vec{y}_i - \vec{y}_j)^T \right\|^2 \\ &= \underset{\mathbf{Y}}{\text{argmin}} \|\mathbf{Y} - \mathbf{WY}\|_F^2 \\ &= \underset{\mathbf{Y}}{\text{argmin}} \|(\mathbf{I}_N - \mathbf{W})\mathbf{Y}\|_F^2 \\ &= \underset{\mathbf{Y}}{\text{argmin}} \text{Tr}[\mathbf{Y}^T (\mathbf{I}_N - \mathbf{W})^T (\mathbf{I}_N - \mathbf{W}) \mathbf{Y}] \end{aligned} \tag{5.11}$$

where $\mathbf{Y} = (\vec{y}_1 | \vec{y}_2 | \dots | \vec{y}_N)^T$

There are two constraints:

- a) $\mathbf{1}_N^T \mathbf{Y} = \vec{0}$. This forces $\mathbf{Y}$ to be centered
- b) $\frac{1}{N} \mathbf{Y}^T \mathbf{Y} = \mathbf{I}_d$. This fixes rotation and scaling.

Key Observation

Considering the final expression of Equation (5.11), the optimization function is now equivalent to finding $\vec{y}_i$ s that minimizes $\mathbf{Y}^T (\mathbf{I}_N - \mathbf{W})^T (\mathbf{I}_N - \mathbf{W}) \mathbf{Y}$.

Here we introduce $\mathbf{M} = (\mathbf{I}_N - \mathbf{W})^T (\mathbf{I}_N - \mathbf{W})$, which is a positive semi-definite matrix. Since $\mathbf{M} \mathbf{1}_N = (\mathbf{I} - \mathbf{W})^T (\mathbf{1}_N - \mathbf{W} \mathbf{1}_N) = \vec{0}$, $\mathbf{1}_N$ is an eigen-vector of $\mathbf{M}$ with eigenvalue zero.

$$\mathbf{Y}^T (\mathbf{I}_N - \mathbf{W})^T (\mathbf{I}_N - \mathbf{W}) \mathbf{Y} = \mathbf{Y}^T \mathbf{M} \mathbf{Y}$$

From constraint (b), we know that columns of $\mathbf{Y}$ are orthogonal to each other. As a result, this whole problem can be simplified to finding the eigen-vectors of $\mathbf{M}$ with the smallest eigenvalues.

Compute eigen-vectors with the smallest $d + 1$ eigenvalues $0 = \lambda_1 \leq \lambda_2 < \dots < \lambda_{d+1}$, eliminate $\mathbf{1}_N$ (the first one). The remaining d vectors are respectively $\vec{v}_2, \vec{v}_3, \dots, \vec{v}_{d+1} \in \mathbb{R}^N$. So $\mathbf{Y} = (\vec{v}_2 | \vec{v}_3 | \dots | \vec{v}_{d+1})$, we successfully recover the corresponding $\vec{y}_1, \vec{y}_2, \dots, \vec{y}_N \in \mathbb{R}^d$.

An illustration of the algorithm

In the original paper [Roweis and Saul, 2000], the authors provide a very intuitive plot that summarizes the above three steps.

Parameter Tuning

There are two parameters to tune in LLE, i.e. (the number of neighbors: k ; the dimension of the recovered configuration: d).

- 1) For selection of d , we usually use a reverse scree plot and find the elbow point. It is worth noting that we are choosing the smallest $d + 1$ eigenvalues and compute their corresponding eigen-vectors here. Since the eigen-vectors and eigenvalues of a particular matrix is super sensitive to any sort of noises or perturbations, especially for those small eigenvalues, it is hard to accurately derive the corresponding eigen-vectors $\vec{v}_2, \dots, \vec{v}_{d+1}$. This is called ill-conditioned eigen-problem.
- 2) Choose the optimal k

LLE seeks to preserve local structure through nearest neighbor connections. This is the key point to LLE. As a result, we may use the neighbor set of the original $\vec{x}_1, \vec{x}_2, \dots, \vec{x}_N \in \mathbb{R}^D$ and $\vec{y}_1, \vec{y}_2, \dots, \vec{y}_N \in \mathbb{R}^d$ as a criteria.

As explained before, we use N_i^k to denote the indices of k -nearest neighbors to $\vec{x}_i$. Similarly, we can also use V_i^k to denote the indices of k -nearest neighbors to $\vec{y}_i$. They should be as close as possible.

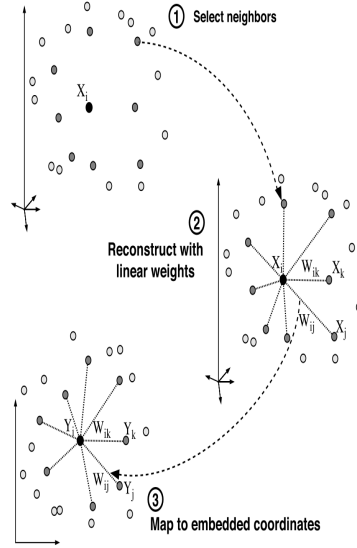


Figure 5.2: LLE_illustration

So our objective function here is:

$$Q(k) = \frac{\sum_{i=1}^N |N_i^k \cap V_i^k|}{Nk}$$

Plot $Q(k)$ against k , select k^* where the increase of $Q(k)$ becomes negligible.

5.5.3 Strengths and Weaknesses of LLE

5.5.3.1 Strengths

1) High Computation Efficiency

The low computation cost of LLE algorithm may be its most shining advantage over other manifold learning methods, and it is actually one of its biggest selling points when it was first introduced. The LLE algorithm involves solving a sparse eigen problem, with computational complexity of roughly $O(N^2 d^2 + N d^3)$ where N is the number of data points and d is the dimension of the recovered configuration.

In comparison, ISOMAP requires computing shortest paths between all pairs of points, which is typically done using Dijkstra's or Floyd-Warshall algorithm, leading to a complexity of $O(N^2 \log N)$ or $O(N^3)$ respectively. Then, it involves eigen decomposition similar to classical MDS which is $O(N^3)$.

In practice, $d \ll N$, hence the computation cost of LLE is lower than that of ISOMAP in most cases.

2) Few parameters to tune

There are only two parameters to tune, respectively the number of neighbors included in the map: k , and the dimension of the recovered configuration: d . In addition, there exist clear methods to find the optimal k and d , as stated in the previous part. This makes LLE algorithm easy to find the optimal parameters.

5.5.3.2 Weaknesses

1) Sensitivity to tuning parameters

The result of LLE is quite sensitive to its two control parameters: the number of neighbors k and the dimension of the recovered configuration: d .

Here we use the Swiss Roll example to illustrate this. LLE is optimal at $k = 45$. However, when $k = 40$, the recovered lower-dimensional configuration is wrong (Green points and yellow points overlap, which is not the case in the Swiss Roll); and when we slightly increase k to 50, the recovered two-dimensional expression is not necessarily a rectangle.

2) Vulnerable to sparse or unevenly-distributed samples

The vulnerability towards sparsity and uneven distribution exists in almost all manifold learning methods, including ISOMAP, as we have illustrated in the previous section. LLE is not immune to this either. When a data set is unevenly distributed, since LLE relies on the original Euclidean distance metric, it tends to select neighbors from a singular direction where these neighbors are densely clustered. Clearly, using these selected neighbors to reconstruct the reference point results in significant redundancy in that specific direction. Concurrently, essential information from other directions or regions is not retained for the reconstruction of the reference point. As a result, these selected neighbors are inadequate for accurately representing and reconstructing the reference point. Consequently, much of the intrinsic structure and internal features will be lost after dimension reduction using LLE.

3) Sensitivity to noise

LLE is extremely sensitive to noise. Even a small noise would cause failure in deriving low dimensional configurations. Justin Wang, et al. utilize various visualization examples to illustrate this drawback in their paper [lle, 2019], you may take a look if you are interested. Various algorithms have been developed to address this issue, i.e., Robustly Locally Linear Embedding (RLLE) [Chang and Yeung, 2006], and Locally Linear Embedding with Additive Noise (LLEAN) [lle, 2019]. The former works well when outliers exist, while the latter has a satisfactory performance when the original points are distorted with noises.

5.6 Laplacian Eigenmap (LEs)

The Laplacian eigenmap [Belkin and Niyogi, 2001] is method of manifold learning with algorithmic and geometric similarities to LLEs. Like LLEs and ISOMAP, Laplacian eigenmaps make use of k -nearest neighbor relationships and the solution of an eigenvalue problem to reconstruct the low-dimensional manifold. As suggested by the name, we will be using the graph Laplacian matrix and emphasize the preservation of nearby points on the manifold making Laplacian Eigenmaps a local method with a different emphasis than LLEs.

The graph Laplacian is an important matrix representation of our data which we will revisit later when discussing spectral clustering. In practice, Laplacian eigenmaps use sparse version of the graph Laplacian. For now, we will briefly introduce this matrix without sparsity and an important identity relating the graph Laplacian and the loss function we will minimize when constructing our low-dimensional representation.

Given data $\vec{x}_1, \dots, \vec{x}_N$, we are going to build a weighted graph $\mathcal{G} = (V, \mathcal{E}, \mathbf{W})$, with one node per sample and weighted, undirected edges connecting the nodes. The weights, $\mathbf{W}_{ij} \geq 0$, $1 \leq i, j \leq N$, correspond to a notion of affinity, which is typically a decreasing function of the distance between our data. Two common choices are (i) binary weights:

$$\mathbf{W}_{ij} = \begin{cases} 1 & \|\vec{x}_i - \vec{x}_j\| \leq \epsilon \\ 0 & \text{else} \end{cases}$$

and (ii) weights based on the radial basis function:

$$\mathbf{W}_{ij} = \exp\left(-\frac{\|\vec{x}_i - \vec{x}_j\|^2}{2\sigma^2}\right).$$

The symmetric matrix $\mathbf{W}$ is called the (weighted) adjacency matrix of the graph, $\mathcal{G}$, encodes of all the pairwise relationships. We always set the diagonal entries of $\mathbf{W}$ to zero to preclude self-connections in the graph.

From the adjacency matrix, we can compute the total affinity of each node (sample) to all other nodes (samples) by summing along the rows. Specifically, the i th entry of the vector $\mathbf{W}\vec{1}_N$, has the total affinity of the i th node ($\vec{x}_i$) to all other nodes (data). Using these summed affinities, we then create the graph Laplacian

$$\mathbf{L} = \mathbf{D} - \mathbf{W} \tag{5.12}$$

where $\mathbf{D}$ is a diagonal matrix with entries $\mathbf{W}\vec{1}_N$ along its diagonal. The graph Laplacian is a symmetric matrix, and while not obvious at first glance, it is also positive semidefinite thanks to the following important identity. Given any vector $\vec{y} \in \mathbb{R}^N$,

$$\vec{y}^T \mathbf{L} \vec{y} = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \mathbf{W}_{ij} (y_i - y_j)^2 \tag{5.13}$$

However, it is not full rank. The previous equation shows that $\vec{1}$ is an eigenvector with eigenvalue 0.

In the previous statement we can view the entries of vector $\vec{y}$ as N separate scalars. However, we can also extend the preceding expression to include Euclidean distances between vectors $\vec{y}_1, \dots, \vec{y}_N \in \mathbb{R}^t$ to give the following important expression

$$\sum_{i=1}^N \sum_{j=1}^N \mathbf{W}_{ij} \|\vec{y}_i - \vec{y}_j\|^2 = \frac{1}{2} \text{tr}(\mathbf{Y}^T \mathbf{L} \mathbf{Y}) \quad (5.14)$$

where $\mathbf{Y} \in \mathbb{R}^{N \times t}$ has rows $\vec{y}_1^T, \dots, \vec{y}_N^T$. We will return to this identity and its implications for Laplacian Eigenmaps after discussing the algorithmic details of the method

5.6.1 Algorithm

5.6.1.1 Compute neighbor relationships

Fix either a number of nearest neighbors $k > 0$ or maximum distance $\epsilon > 0$. If using ϵ , then $\vec{x}_i$ and $\vec{x}_j$ (correspondingly nodes i and j in the graph) are neighbors if $\|\vec{x}_i - \vec{x}_j\| \leq \epsilon$. Alternatively, if using the nearest neighbor parameter k , then we consider $\vec{x}_i, \vec{x}_j$ to be neighbors if $\vec{x}_i$ is one the k closest points to $\vec{x}_j$ **and** $\vec{x}_j$ is one of the k closest points to $\vec{x}_i$. The construction of neighbors, like the use of pairwise distance alone, results in symmetric neighbor relationship. We then connect nodes i and j with an edge if $\vec{x}_i$ and $\vec{x}_j$ are neighbors.

5.6.1.2 Compute weights and build graph Laplacian

Nodes which are not connected immediately receive an edge weight equal to zero. For all connected nodes, we compute the edge weight

$$\mathbf{W}_{ij} = \exp\left(-\frac{\|\vec{x}_i - \vec{x}_j\|^2}{2\sigma^2}\right).$$

Here we have shown weights based on the radial basis function, which is motivated by theoretical connections to the heat kernel and an approximation of the Laplacian on the manifold $\mathcal{X}$ [Belkin and Niyogi, 2001]. As such, this method is the default in most implementation of Laplacian eigenmaps. The parameter σ^2 does require tuning which can have a large impact on the performance of the algorithm.

When k or ϵ are small, which is typically the case in practice, the (weighted) adjacency matrix $\mathbf{W}$ will be sparse (most entries equal to 0). From this adjacency matrix, we then construct the graph Laplacian as above. The graph Laplacian built from these weights will also be sparse. By preserving only those connections between nearest points, we have only maintained the pairwise, local relationships on the manifold. We will now use $\mathbf{L}$ to construct a lower-dimensional representation of the data.

5.6.1.3 Solve generalized eigenvalue problem

Consider the loss function

$$\mathcal{L}(\vec{y}_1, \dots, \vec{y}_N) = \sum_{i=1}^N \sum_{j=1}^N \mathbf{W}_{ij} \|\vec{y}_i - \vec{y}_j\|^2. \quad (5.15)$$

This loss function is most sensitive to large pairwise distance $\|\vec{y}_i - \vec{y}_j\|$ when $\mathbf{W}_{ij}$ is also large (our original data were close). Thus, minimizing the preceding penalty prioritizes keeping $\vec{y}_i$ and $\vec{y}_j$ close when $\vec{x}_i$ and $\vec{x}_j$ have a high affinity (weight). As a result, Laplacian eigenmaps emphasize local geometry.

Vectors $\vec{y}_1, \dots, \vec{y}_N$ which minimizes this loss function are not unique. First, there is an issue of translation. To address this issue, we will add a constraint that $\mathbf{D}\mathbf{Y}$ is a centered data matrix, i.e. $\mathbf{Y}^T \mathbf{D} \vec{1} = \vec{0}$. We can view the matrix $\mathbf{D}\mathbf{Y}$ as a reweighting of the data matrix $\mathbf{Y}$ with higher weights $\mathbf{D}_{ii}$ for data $\vec{x}_i$ which are closer to more points. Here $\mathbf{D}$ is the diagonal matrix used in the definition of the graph Laplacian. Note that

$$\mathbf{D}\mathbf{Y} = \begin{bmatrix} \mathbf{D}_{11} \vec{y}_1^T \\ \vdots \\ \mathbf{D}_{NN} \vec{y}_N^T \end{bmatrix}.$$

Requiring $\mathbf{D}\mathbf{Y}$ to be centered results in configurations where those points with highest affinity are constrained close to the origin in our lower dimensional representation.

However, solving the optimization problem with this modified centering constraint is still ill-posed, namely we could take $\vec{y}_1 = \dots = \vec{y}_N = \vec{0}$ giving a configuration which is collapsed onto the origin. In fact, given any configuration $\vec{y}_1, \dots, \vec{y}_N$ we could decrease the loss by rescaling all of our data by some constant scalar $0 < c < 1$ since $\mathbf{D}(c\mathbf{Y}) = c\mathbf{D}\mathbf{Y}$ will still be centered. To address this scaling issue and give a meaningful t -dimensional configuration, we also add the constraint $\mathbf{Y}^T \mathbf{D}\mathbf{Y} = \mathbf{I}_t$. This constraint eliminates the collapse of the t -dimensional configuration onto a $t-1$ dimensional hyperplane, and in particular eliminates the cases where the 1-dimensional configuration collapses onto a point.

Thus, we seek a data matrix $\mathbf{Y} \in \mathbb{R}^{N \times t}$ solving the following constrained optimization problem

$$\operatorname{argmin}_{\mathbf{Y}^T \mathbf{D}\mathbf{Y} = \mathbf{I}_t, \mathbf{Y}^T \mathbf{D} \vec{1} = \vec{0}} \operatorname{tr}(\mathbf{Y}^T \mathbf{L} \mathbf{Y}). \quad (5.16)$$

To solve this problem, we first introduce the change of variable $\tilde{\mathbf{Y}} = \mathbf{D}^{1/2} \mathbf{Y}$ so that the constraints become

$$\mathbf{Y}^T \mathbf{D} \vec{1}^T = (\mathbf{D}^{1/2} \mathbf{Y})^T \mathbf{D}^{1/2} \vec{1}^T = \tilde{\mathbf{Y}}^T \mathbf{D}^{1/2} \vec{1}^T = \vec{0}$$

and

$$\mathbf{Y}^T \mathbf{D} \mathbf{Y} = \mathbf{Y}^T \mathbf{D}^{1/2} \mathbf{D}^{1/2} \mathbf{Y} = (\mathbf{D}^{1/2} \mathbf{Y})^T (\mathbf{D}^{1/2} \mathbf{Y}) = \tilde{\mathbf{Y}}^T \tilde{\mathbf{Y}} = \mathbf{I}$$

implying that the columns of $\tilde{\mathbf{Y}}$ are orthonormal. After the change of variable, our optimization problem becomes

$$\operatorname{argmin}_{\tilde{\mathbf{Y}}^T \tilde{\mathbf{Y}} = \mathbf{I}, \tilde{\mathbf{Y}}^{1/2} \mathbf{D}^{1/2} \tilde{\mathbf{I}} = \tilde{\mathbf{0}}} \operatorname{tr} \left(\tilde{\mathbf{Y}}^T \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2} \tilde{\mathbf{Y}} \right). \quad (5.17)$$

We can minimize this equation by making use of the eigenvalues and eigenvectors of the (symmetric) normalized graph Laplacian

$$\mathbf{L}_{sym} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{W} \mathbf{D}^{-1/2}.$$

Importantly, note that $\mathbf{L}_{sym}$ is symmetric. Furthermore, it is positive semidefinite since it can be viewed as the graph Laplacian of a graph with weights $\mathbf{W}_{ij} / \sqrt{\mathbf{D}_{ii} \mathbf{D}_{jj}}$ thus subject to the identity (5.13). Thus, it is diagonalizable with orthonormal eigenvectors $\vec{v}_1, \dots, \vec{v}_N \in \mathbb{R}^N$ and associated nonnegative eigenvalues $\lambda_1 \leq \dots \leq \lambda_N$ which we list in **increasing order** in this case. This leads to the first important observation

- 1) We could use any t of these vectors as the columns of $\tilde{\mathbf{Y}}$ and immediately satisfy the constraint $\tilde{\mathbf{Y}}^T \tilde{\mathbf{Y}} = \mathbf{I}$.

However, we have an additional constraint and the minimization to consider. Note that $\vec{1}_N$ is an eigenvector of original graph Laplacian with eigenvalue 0. Thus, $\mathbf{L}_{sym}$ also has eigenvalue 0 with associated eigenvector $\vec{v}_1 = \mathbf{D}^{1/2} \vec{1}$ since

$$\mathbf{L}_{sym}(\mathbf{D}^{1/2} \vec{1}) = \mathbf{D}^{-1/2} (\mathbf{D}^{1/2} - \mathbf{W} \mathbf{D}^{-1/2}) \mathbf{D}^{1/2} \vec{1} = \mathbf{D}^{-1/2} (\mathbf{D} - \mathbf{W}) \vec{1} = \mathbf{D}^{-1/2} \mathbf{L} \vec{1} = \vec{0}.$$

As a result, all other eigenvectors of $\mathbf{L}_{sym}$ must be orthogonal to $\vec{v}_1 = \mathbf{D}^{1/2} \vec{1}$.

- 2) This suggests that if we drop the first eigenvector associated with eigenvalue $\lambda_1 = 0$ and use t the remaining eigenvectors of $\mathbf{L}_{sym}$ as the columns of $\tilde{\mathbf{Y}}$ we will satisfy the constraint $\tilde{\mathbf{Y}}^T \mathbf{D}^{1/2} \vec{1} = \vec{0}$.

The final observation is that we should choose the eigenvectors to minimize the objective.

- 3) We make use of the eigenvalues themselves and take

$$\tilde{\mathbf{Y}} = [\vec{v}_2 \quad \dots \quad \vec{v}_{t+1}]$$

so that

$$\operatorname{tr} (\tilde{\mathbf{Y}}^T \mathbf{L}_{sym} \tilde{\mathbf{Y}}) = \lambda_2 + \dots + \lambda_{t+1}$$

is minimized.

After undoing the change of variables, we use the rows of

$$\mathbf{Y} = \mathbf{D}^{-1/2} \tilde{\mathbf{Y}} \in \mathbb{R}^{N \times t}$$

as our t -dimensional configuration.

5.7 Hessian Eigenmaps (HLLes)

5.7.1 Introduction

This method derives its name from the Hessian of a scalar valued function, properties of which we'll briefly revisit in the context of two examples

$$g_1(x, y) = x^2 + y^2$$

$$g_2(x, y) = \sin(2\pi x) \sin(2\pi y).$$

Let's visualize these functions using surface plots.

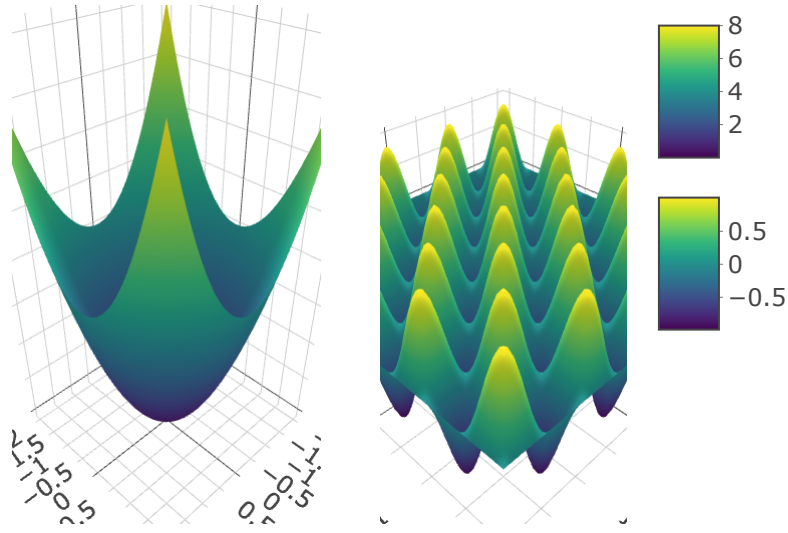


Figure 5.3: Surface plots of g_1 (left) and g_2 (right) over the rectangle $[-2, 2]^2$

Which of these two functions is more curved? Informally, which function is wigglier? The answer is clearly g_2 , but let's revisit ideas from calculus to quantify the difference.

Suppose $g : \mathbb{R}^t \rightarrow \mathbb{R}$. The second degree Taylor expansion of a function g around $\vec{x}$ is

$$g(\vec{z}) \approx g(\vec{x}) + \nabla g(\vec{x})^T (\vec{z} - \vec{x}) + \frac{1}{2} (\vec{z} - \vec{x})^T (\nabla^2 g(\vec{x})) (\vec{z} - \vec{x}).$$

The gradient provides information for orienting the approximating tangent plane of the surface whereas the Hessian provides information on additional variations from the tangent plane which captures curvature of the surface near $\vec{x}$. For g_1 and g_2 above we have

$$\nabla g_1 = \begin{bmatrix} 2x \\ 2y \end{bmatrix} \text{ and } \nabla g_2 = \begin{bmatrix} 2\pi \cos(2\pi x) \sin(2\pi y) \\ 2\pi \sin(2\pi x) \cos(2\pi y) \end{bmatrix}$$

and

$$\nabla^2 g_1 = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \text{ and } \nabla^2 g_2 = \begin{bmatrix} -4\pi^2 \sin(2\pi x) \sin(2\pi y) & 4\pi^2 \cos(2\pi x) \cos(2\pi y) \\ 4\pi^2 \cos(2\pi x) \cos(2\pi y) & -4\pi^2 \sin(2\pi x) \sin(2\pi y) \end{bmatrix}.$$

The Hessian of g_1 is constant whereas the Hessian of g_2 is both varying but generally comprised of larger values.

Subsequently, the squared Frobenius norms of the Hessians are

$$\|\nabla^2 g_1\|_F^2 = 8 \text{ and } \|\nabla^2 g_2\|_F^2 = 16\pi^4 [1 + \cos(4\pi x) \cos(4\pi y)]$$

where the later expression follows after some trig identities. Importantly, the squared Frobenius norm of the Hessian for g_2 ranges from 0 to $32\pi^4$. On average (over the rectangle $[-2, 2]^2$) $\|\nabla^2 g_2\|_F^2$ is much larger than $\|\nabla^2 g_1\|_F^2$. Mathematically, we have

$$\int_{-2}^2 \int_{-2}^2 \|\nabla^2 g_2\|_F^2 dx dy = 16\pi^2 > 8 = \int_{-2}^2 \int_{-2}^2 \|\nabla^2 g_1\|_F^2 dx dy.$$

To adapt the notion of gradients and Hessians to manifolds, let us revisit the idea of derivatives of real valued functions on a manifold. Suppose we have a manifold $\mathcal{M} \subset \mathbb{R}^d$ and function $f : \mathcal{M} \rightarrow \mathbb{R}$. Assuming that $\mathcal{M}$ has intrinsic dimension $t < d$, to each point $\vec{x} \in \mathcal{M}$, we can associate a t dimensional tangent space $T_{\vec{x}}(\mathcal{M})$ which we equip with a choice of orthonormal basis vectors and a corresponding chart (U, ϕ) which maps a neighborhood, U containing $\vec{x}$ to $\mathbb{R}^t$. For now, it is sufficient to think of the ϕ as the projection of neighboring points of $\vec{x}$ onto the approximating tangent plane (if we then subtract $\vec{x}$ from these projections, we get projections onto the tangent space). For $\vec{z} \in U$, we referred to $\phi(\vec{z})$ as the local coordinates of $\vec{z}$.

From the perspective of calculus, the function $f \circ \phi^{-1}$ is a function mapping an open subset of $\mathbb{R}^t$ to $\mathbb{R}$. We can then compute gradients and Hessians of $f \circ \phi^{-1}$ using standard multivariable calculus and $\nabla^{tan} f := \nabla(f \circ \phi^{-1})$ and $(\nabla^{tan})^2(f) := \nabla^2(f \circ \phi^{-1})$. We'll refer to $\nabla^{tan} f$ and $(\nabla^{tan})^2 f$ as the gradient and Hessian of f in *local coordinates*. For a choice of f , the specific values in gradient or Hessian depends on our choice of ϕ^{-1} (or equivalently the basis we choose for the tangent space). However, the (squared) Euclidean length of the gradient or the (squared) Frobenius norm of the Hessian do not depend on this choice since these quantities are unchanged by orthonormal transformations. We'll be using this fact for the remainder of this section.

In principal, we could take every point $\vec{x} \in \mathcal{M}$ and, using this approach, compute the local Hessian of f at $\vec{x}$. If we take an average of the local Hessian over the manifold, denoted

$$\mathbb{H}(f) = \int_{\mathcal{M}} \|(\nabla^{tan})^2 f\|_F^2 d\mu(\vec{x})$$

we have a measure of the average curvature of the function f . Here, $\mu(\vec{x})$ is the probability distribution of data on the manifold fold (a measure for those

with the requisite background). The utility of this quantity of f is given in the following theorem.

Theorem 5.2 (Null Space of Hessian Operator on Manifolds). *Suppose $\mathcal{M} = \Psi(A)$ where A is an open, connected subset of $\mathbb{R}^t$, and Ψ is a locally isometric embedding of A into $\mathbb{R}^d$. Then $\mathbb{H}(f)$ has a $t + 1$ dimensional null space spanned by the constant function and a t -dimensional space of functions spanned by the original coordinates in A .*

In short, if $\mathbb{H}(f) = 0$ then there is some $\alpha \in \mathbb{R}$ and $\vec{\beta} \in \mathbb{R}^t$ such that $f(\vec{x}) = \alpha + \beta^T \vec{z}$ where $\vec{z} \in A$ is the preimage of $\vec{x}$, e.g. $\Psi(\vec{z}) = \vec{x}$. By looking for functions in the null space of $\mathbb{H}(f)$, we can then find functions which help us recover the original low-dimensional coordinates in A !

Unfortunately, we cannot calculate $\mathbb{H}f$ for a general function f . However, we can estimate $\mathbb{H}(f)$ using observed data $\vec{x}_1, \dots, \vec{x}_N \in \mathcal{M}$, which is the central idea behind HLES. Specifically, we will construct a discretized estimate of $\mathbb{H}(f)$ which we can write in the quadratic form

$$\vec{f}^T \mathbf{H} \vec{f} \text{ where } \vec{f} = (f(\vec{x}_1), \dots, f(\vec{x}_N))^T \in \mathbb{R}^N.$$

The matrix $\mathbf{H} \in \mathbb{R}^{N \times N}$ is symmetric and positive semidefinite and depends only on the observed data $\vec{x}_1, \dots, \vec{x}_N$. Its construction will be discussed in greater detail in the algorithms section below.

For now, assume that we have access to $\mathbf{H}$. Suppose that f is in the null space of $\mathbb{H}(f)$ then i) $\vec{f}^T \mathbf{H} \vec{f} \approx 0$ and ii) from Theorem 5.2 it must be the case that $f(\vec{x}_i) = \alpha + \beta^T \vec{z}_i$ where $\vec{z}_i$ is a point in the lower dimensional configuration A . Importantly, if we can identify the null space of the matrix $\mathbf{H}$ then we can use this to recover the original low-dimensional coordinates (up to rigid motion and coordinate rescaling).

Similar to LLEs and ISOMAP, this algorithm begins with a k-nearest neighbor search and finishes with an eigenvalue decomposition. As inputs we provide the data, the intrinsic dimension, t , of the manifold, and a specified number of nearest neighbors k . For reasons we'll discuss later, we must select $k > t(t+3)/2$. This constraint can be problematic in practice. For example, if we believe the intrinsic dimension of $\mathcal{M}$ is 100, then we need to pick $k > 5150$ meaning we need at least 5151 samples in our dataset! Such a constraint may be difficult to satisfy in practice, but for dimension reduction focused on visualization with the ambition assumption that $t = 1, 2$, or 3 , $k \geq 10$ is sufficient.

5.7.1.1 Compute nearest neighbors and local coordinates

For each $\vec{x}_i$ compute its k -nearest neighbors which we'll denote $\mathcal{N}_i$. We'll use each of these neighbors to estimate the Hessian of f at $\vec{x}_i$. Next we'll estimate local coordinates of the neighbors in $\mathcal{N}_i$ by using SVD to approximate the tangent plane $T_{\vec{x}_i}(\mathcal{M})$. One important observation to recall. We expect the points in $\mathcal{N}_i$ to all be close together and centered around $\vec{x}_i$. In this neighborhood around

$\vec{x}_i$ we expect the manifold $\mathcal{M}$ to look like a small patch of $\mathbb{R}^d$. As such, they should be (nearly) contained in a t -dimensional affine subspace of $\mathbb{R}^d$.

Let $\mathbf{M}_i \in \mathbb{R}^{k \times d}$ be the matrix of centered nearest neighbors of $\vec{x}_i$. Specifically, the j th row of $\mathbf{M}_i$ is $(\vec{x}_{i_j} - \vec{x}_i)^T$ where $\vec{x}_{i_j}$ is the j th nearest neighbor of $\vec{x}_i$. We apply a singular value decomposition to $\mathbf{M}_i$ giving factorization

$$\mathbf{M}_i = \mathbf{U}_i \mathbf{S}_i \mathbf{V}_i^T$$

where $\mathbf{U}_i \in \mathbb{R}^{k \times k}$ and $\mathbf{V}_i \in \mathbb{R}^{d \times d}$ have orthonormal columns and $\mathbf{S}_i \in \mathbb{R}^{k \times d}$ is diagonal. If the neighbors in $\mathcal{N}_i$ were fully contained in a t -dimensional affine subspace we expect that $\mathbf{S}_i$ has t large singular values with the remainder equal to zero. Additionally in this case, the tangent space $T_{\vec{x}_i}$ is $\vec{x}_i + \text{span}\{\vec{v}_1, \dots, \vec{v}_t\}$ where $\vec{v}_1, \dots, \vec{v}_t$ are the first t columns of $\mathbf{V}$.

In practice, we should only expect the first t singular values to be large with the remainder smaller but non-zero. However, we'll still use the first t columns of $\mathbf{V}$ as an (approximate) orthonormal basis for the tangent space. The first t columns of $\mathbf{U}_i \mathbf{S}_i$ give approximations of the local coordinates of the neighbors $\mathcal{N}_i$ (the j th row gives the local coordinates for the j th nearest neighbor). Hereafter, we'll let $\vec{u}_j^i = (u_{j1}^i, \dots, u_{jt}^i)^T$ denote the local coordinates of the j th nearest neighbor.

5.7.1.2 Estimate the Hessian

Recall from section 5.3, that we can define derivatives for a function f at $\vec{x}_i$ using the local coordinates.

Now, we can use the local gradients and Hessians to construct a Taylor approximation to $f(\vec{x}_j)$ for each point in the neighbor $\mathcal{N}_j$. In the tangent space $T_{\vec{x}_i}(\mathcal{M})$, we associate the origin with $\vec{x}_i$. Taylor expanding around the origin to second order we have the following approximation

$$\begin{aligned} f(\vec{x}_{i_j}) &\approx f(\vec{x}_i) + [\nabla^{\text{tan}} f(\vec{x}_i)]^T \cdot \vec{u}_j^i + \frac{1}{2} (\vec{u}_j^i)^T ((\nabla^{\text{tan}})^2 f(\vec{x}_i)) \vec{u}_j^i \\ &= f(\vec{x}_i) + \sum_{\ell=1}^t [\nabla^{\text{tan}} f(\vec{x}_i)]_{\ell} \vec{u}_{j\ell}^i + \frac{1}{2} \sum_{\ell=1}^t ((\nabla^{\text{tan}})^2 f(\vec{x}_i))_{\ell\ell} (u_{j\ell}^i)^2 + \sum_{\ell < s} ((\nabla^{\text{tan}})^2 f(\vec{x}_i))_{\ell,s} u_{j\ell}^i u_{js}^i \end{aligned}$$

To estimate the entries of the Hessian, we use quadratic regression. Build design matrix $\mathbf{X}_i \in \mathbb{R}^{k \times (1+d+d(d+1)/2)}$ including all terms up to second order of the local coordinates such that

$$\mathbf{X}_i = \begin{bmatrix} 1 & u_{11}^i & \dots & u_{1t}^i & \frac{1}{2}(u_{11}^i)^2 & \dots & \frac{1}{2}(u_{1t}^i)^2 & \frac{\sqrt{2}}{2}u_{11}^i u_{12}^i & \frac{\sqrt{2}}{2}u_{11}^i u_{13}^i & \dots & \frac{\sqrt{2}}{2}u_{1,t-1}^i u_{1t}^i \\ \vdots & \vdots & & \vdots & \vdots & \dots & \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & u_{k1}^i & \dots & u_{kt}^i & \frac{1}{2}(u_{k1}^i)^2 & \dots & \frac{1}{2}(u_{kt}^i)^2 & \frac{\sqrt{2}}{2}u_{k1}^i u_{k2}^i & \frac{\sqrt{2}}{2}u_{k1}^i u_{k3}^i & \dots & \frac{\sqrt{2}}{2}u_{k,t-1}^i u_{kt}^i \end{bmatrix}$$

If we let $\vec{f}_i = (f(\vec{x}_{i_1}), \dots, f(\vec{x}_{i_k}))^T$ be a vector containing the function values at the nearest neighbors of $\vec{x}_i$, then we can estimate the coefficients of the Taylor

expansion using the following regression formula

$$\vec{f}_i = \mathbf{X}_i \vec{\beta}$$

where the last $t+t(t+1)/2$ terms of $\vec{\beta}_i$ correspond to the entries of the $(\nabla^{tan})^2(f)$ at $\vec{x}_i$. We then estimate $\vec{\beta}_i$ using the Moore-Penrose pseudoinverse

$$\vec{\beta}_i \approx (\mathbf{X}_i^T \mathbf{X}_i)^{-1} \mathbf{X}_i^T \vec{f}_i.$$

For the pseudoinverse to be invertible, $\mathbf{X}_i$ must have at least as many rows as columns. Thus, we need $k \geq 1 + t + t(t+1)/2 = 1 + t(t+3)/2 > t(t+3)/2$.

We can drop the first $t+1$ entries which contain the intercept and the first order regression terms corresponding to the gradient. Let $\vec{\beta}_{i,drop}$ denote the final $t(t+1)/2$ entries of $\vec{\beta}$. If we square then sum the entries of $\vec{\beta}_{i,drop}$, we now have an estimate of $\|(\nabla^{tan})^2(f)\|_F^2$ at $\vec{x}_i$.

One final note on this issue. The $1/2$ and $\sqrt{2}/2$ terms in $\mathbf{X}_i$ are introduced to ensure we are correctly estimating (and single counting) the diagonal entries of the Hessian while double counting its upper triangular elements. This detail is missing from the original paper [Donoho and Grimes, 2003]. As such, many implementations may also replicate the mistake. Be cautious when choosing a package for implementing HLLSs!

5.7.1.3 The Eigenvalue Problem

Define $\mathbf{S}_i \in \mathbb{R}^{k \times N}$ as follows

$$(\mathbf{S}_i)_{j\ell} = \begin{cases} 1 & \vec{x}_\ell \text{ is the } j\text{th nearest neighbor of } \vec{x}_i \\ 0 & \text{else.} \end{cases}$$

Using this notation, we can express $\vec{f}_i$ (the function values from the k nearest neighbors of $\vec{x}_i$) as the product of $\mathbf{S}_i$ and $\vec{f}$ (the vector containing the function value at all samples $\vec{x}_1, \dots, \vec{x}_N$). Specifically, $\vec{f}_i = \mathbf{S}_i \vec{f}$.

Now, we will use an empirical average of our Hessian estimates at each data point to approximate the operator $\mathbb{H}(f)$ as follows

$$\begin{aligned} \mathbb{H}(f) &= \int_{\mathcal{M}} \|(\nabla^{tan})^2(f)\|_F^2 dm \\ &\approx \frac{1}{N} \sum_{i=1}^N \|(\nabla^{tan})^2(f)(\vec{x}_i)\|_F^2 \\ &\approx \frac{1}{N} \sum_{i=1}^N \|\mathbf{H}_i \vec{f}_i\|^2 \end{aligned}$$

Now, we can make use of the identity $\vec{f}_i = \mathbf{S}_i \vec{f}$ to write this approximation as a quadratic function of $\vec{f}$.

$$\begin{aligned} \approx \frac{1}{N} \sum_{i=1}^N \|\mathbf{H}_i \vec{f}_i\|^2 &= \frac{1}{N} \sum_{i=1}^N \vec{f}_i^T \mathbf{H}_i^T \mathbf{H}_i \vec{f}_i \\ &= \frac{1}{N} \sum_{i=1}^N \vec{f}^T \mathbf{S}_i^T \mathbf{H}_i^T \mathbf{H}_i \mathbf{S}_i \vec{f} \\ &= \vec{f}^T \left(\frac{1}{N} \sum_{i=1}^N \mathbf{S}_i^T \mathbf{H}_i^T \mathbf{H}_i \mathbf{S}_i \right) \vec{f} \end{aligned}$$

Letting

$$\mathbf{H} = \frac{1}{N} \sum_{i=1}^N \mathbf{S}_i^T \mathbf{H}_i^T \mathbf{H}_i \mathbf{S}_i$$

our approximation becomes

$$\vec{f}^T \mathbf{H} \vec{f}.$$

Here is the final critical observation. If f is a vector in the null space of $\mathbf{H}$ then it must be in the span of the coordinate functions, *and* we would expect the approximation to be small (close to zero except for sampling variability and estimation error). As such, we can look eigenvector(s) associated with the smallest eigenvalue(s) of $\mathbf{H}$ to give approximations to the coordinate functions!

Like Laplacian Eigenmaps and LLEs, $\mathbf{H}$ will have one eigenvalue which is exactly zero corresponding to the constant function. We'll take the next t eigenvectors associated with the next smallest t eigenvalues and use them to build a data matrix. The rows of this data matrix give the corresponding low dimensional coordinates of our data.

Since $\mathbf{H}$ is symmetric and positive semidefinite (it is the sum of symmetric, psd matrices), its eigenvectors will be orthogonal. Selecting these eigenvectors as approximates for functions $f_1, \dots, f_t$ is equivalent to imposing an orthogonality constraint on these vectors. We'll also take the vectors to be unit length. As a result, we can expect HLLE to recover original coordinates up to rigid motion (scaling and rotation/reflection) and coordinate rescaling.

5.7.2 Assumptions, Strength, and Weaknesses

HLLEs rely on two assumptions: the manifold map is locally isometric and the lower-dimensional set A is connected. These are far more flexible than ISOMAP, which requires an isometric manifold map and convex A . Additionally, HLLEs have fewer tuning parameters than Laplacian eigenmaps, which also require one to specify the bandwidth σ^2 is the rbf kernel.

However, there are also drawbacks. The theoretical justification for HLLEs relies on a quality approximation to the local Hessian, which is difficult without a

large number of samples. Furthermore, for a manifold with intrinsically high dimension t , a prohibitively large number of nearest neighbors is required.

5.8 t-Distributed Stochastic Neighbor Embedding (t-SNE)

5.8.1 Introduction

t-SNE was introduced by van der Maaten and Hinton in 2008 [van der Maaten and Hinton, 2008] and, alongside UMAP, remains one of the two most widely used tools for visualizing high-dimensional data. Like UMAP, t-SNE is a local method built from a notion of neighborhood rather than the pairwise geodesic distances used by ISOMAP, and like UMAP, it produces its embedding by directly optimizing the positions of low-dimensional points rather than solving a single eigenvalue problem. The two methods share this same basic recipe – build an affinity between points in $\mathbb{R}^d$, build a comparable affinity between candidate points in $\mathbb{R}^t$, and move the candidate points to make the two affinities agree – and, as we will see later in this chapter, this shared recipe is not a coincidence (5.12). The interesting differences between the methods are in the details of how each affinity is built and what notion of “agreement” each one optimizes.

5.8.2 Algorithm

5.8.2.1 Affinities in the original space

As with Laplacian eigenmaps and UMAP, we begin by turning distances in $\mathbb{R}^d$ into affinities. For each $\vec{x}_i$, t-SNE defines a conditional probability that $\vec{x}_i$ would pick $\vec{x}_j$ as its neighbor,

$$p_{j|i} = \frac{\exp(-\|\vec{x}_i - \vec{x}_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\vec{x}_i - \vec{x}_k\|^2 / 2\sigma_i^2)}, \quad p_{i|i} = 0,$$

using a Gaussian centered at $\vec{x}_i$ with its own bandwidth σ_i . As in UMAP, this bandwidth is calibrated locally rather than shared across the whole dataset: σ_i is chosen (by binary search) so that the Shannon entropy of $\{p_{j|i}\}_{j \neq i}$ equals $\log_2(\text{Perp})$ for a user-specified *perplexity*, which is best understood as a smooth stand-in for the number of effective neighbors of $\vec{x}_i$ – points in dense regions get a small σ_i and points in sparse regions get a large one, so that every point ends up with roughly the same amount of “neighborhood mass” regardless of local density. This is precisely the same density-adaptivity problem that motivated UMAP’s local bandwidth σ_i (5.9), solved with a different calibration target.

Since $p_{j|i}$ need not equal $p_{i|j}$, we symmetrize by averaging,

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}.$$

5.8.2.2 Affinities in the embedding and the crowding problem

Given candidate low-dimensional points $\vec{y}_1, \dots, \vec{y}_N \in \mathbb{R}^t$, the most direct approach would define an analogous Gaussian affinity q_{ij} in the embedding and ask that q_{ij} match p_{ij} as closely as possible. This is essentially what earlier methods (including the original SNE that t-SNE improves on) did, and it runs into a real geometric obstruction.

Think about how much “room” there is around a point at a given distance r in $\mathbb{R}^d$: the volume of a thin shell of radius r grows like r^{d-1} , so in high dimensions there is an enormous amount of space available at moderate distances – room enough for many points to sit at a similar, moderate similarity to $\vec{x}_i$. A two- or three-dimensional embedding has nowhere near this much room. If we insist on reproducing every moderate high-dimensional affinity with an equally moderate low-dimensional distance, the embedding is forced to crowd unrelated points close together simply because there isn’t enough space to spread them all out – distinct clusters end up squeezed into the middle of the picture. This is known as the **crowding problem**.

t-SNE’s fix is to give the embedding a different, heavier-tailed notion of affinity than the original space uses. Instead of a Gaussian, we use a Student-t distribution with one degree of freedom (a Cauchy distribution),

$$q_{ij} = \frac{(1 + \|\vec{y}_i - \vec{y}_j\|^2)^{-1}}{\sum_{k \neq \ell} (1 + \|\vec{y}_k - \vec{y}_\ell\|^2)^{-1}}, \quad q_{ii} = 0.$$

The Cauchy density decays polynomially ($\propto \|\vec{y}_i - \vec{y}_j\|^{-2}$) rather than the Gaussian’s exponential decay, so a moderate affinity p_{ij} can now be matched by q_{ij} at a *larger* embedded distance than a same-shaped Gaussian would have allowed. Geometrically, this hands moderately dissimilar points an escape route: they are permitted to drift apart in the embedding while genuinely close neighbors – which still require a small $\|\vec{y}_i - \vec{y}_j\|$ to produce a large q_{ij} – remain tightly packed. The mismatch between the two tail shapes is doing real work here, actively pushing weakly related points apart to make room for the points that matter.

5.8.2.3 Optimizing the embedding

With both sets of affinities in hand, t-SNE chooses $\vec{y}_1, \dots, \vec{y}_N$ to minimize the Kullback-Leibler divergence between them,

$$C(\vec{y}_1, \dots, \vec{y}_N) = \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right).$$

This loss is asymmetric in a useful way: it penalizes using a small q_{ij} to represent a large p_{ij} far more heavily than the reverse, so the optimization prioritizes keeping genuine neighbors close over faithfully preserving large distances. Minimizing C via gradient descent produces a force not unlike UMAP’s attractive and

repulsive forces (5.9): pairs with $p_{ij} > q_{ij}$ pull $\vec{y}_i$ and $\vec{y}_j$ together, while pairs with $p_{ij} < q_{ij}$ push them apart. In practice, the optimization is initialized (often from a small random configuration or a PCA projection), run with momentum to speed convergence, and typically uses an “early exaggeration” phase which temporarily inflates the p_{ij} to encourage well-separated clusters to form before fine details are refined.

Example 5.8 (t-SNE applied to the Swiss Roll). Let’s revisit the swiss roll one more time and compare t-SNE’s embedding to what we’ve seen from ISOMAP, LLE, and UMAP.

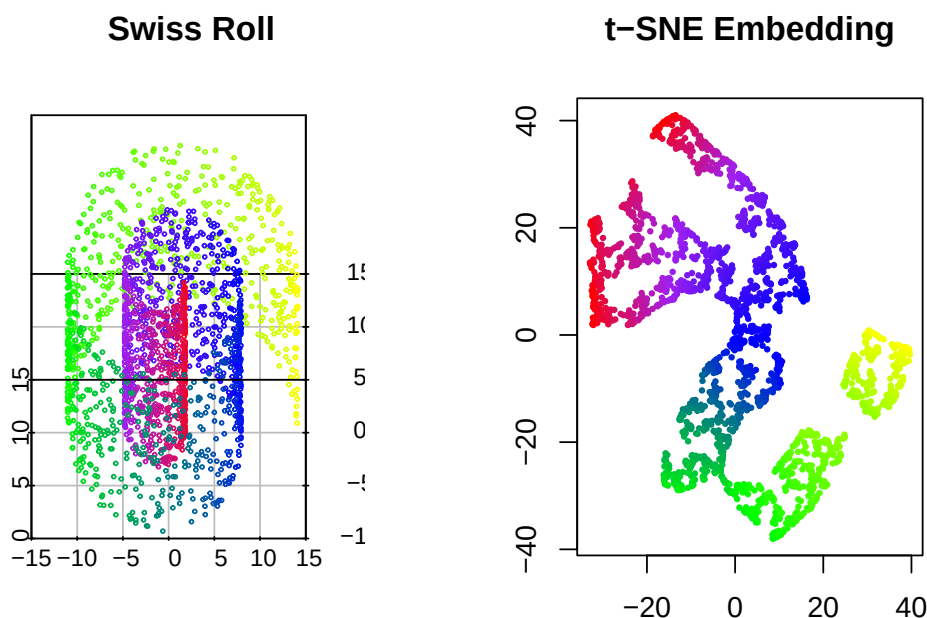


Figure 5.4: The Swiss Roll (left) and its t-SNE embedding (right)

Like UMAP, t-SNE successfully separates the roll along its length, and the coloring confirms that nearby points on the manifold remain nearby in the embedding. Also like UMAP, the result is not a faithful rectangle – t-SNE has stretched, compressed, and torn the manifold into disconnected pieces in a few places, a direct consequence of the crowding problem discussion above: the algorithm is actively trading distance-fidelity for the breathing room needed to keep local neighborhoods intact.

5.8.3 Strengths and Weaknesses

5.8.3.1 Strengths

- 1) **Excellent at revealing cluster structure**

The combination of the asymmetric KL loss and the heavy-tailed embedding affinity makes t-SNE particularly good at pulling apart genuinely distinct clusters, which is a large part of why it became a default first step for visualizing datasets such as single-cell genomic data or the internal representations learned by neural networks.

2) Few conceptual moving parts

Beyond the perplexity, there are relatively few choices to make, and the perplexity itself has a fairly direct interpretation (an effective neighborhood size) that makes it easier to reason about than some of the tuning parameters of other methods.

3) Directly addresses a real geometric obstruction

Unlike methods that inherit a single fixed notion of similarity from the ambient space, t-SNE's use of mismatched tail behavior between p_{ij} and q_{ij} is a deliberate, principled response to the crowding problem, not an incidental design choice.

5.8.3.2 Weaknesses

1) Distances and cluster sizes are not meaningful

As with UMAP, t-SNE prioritizes neighborhoods over distances. The size of a cluster in a t-SNE plot and the gap between two clusters are both driven largely by the optimization and the perplexity, not by the true geometry of the data, and should not be over-interpreted.

2) No natural out-of-sample extension

t-SNE optimizes the positions of a fixed set of points directly; there is no manifold map Ψ or its inverse to apply to a new observation, so embedding new data typically requires rerunning the entire optimization (or falling back on a separate out-of-sample heuristic).

3) Sensitive to perplexity, and computationally demanding

Very different perplexities can produce visually different embeddings of the same data, and unlike UMAP's spectral initialization, t-SNE's optimization has no principled starting point, so different runs (or different random initializations) can converge to different local optima. The $O(N^2)$ cost of the exact pairwise affinities above also makes t-SNE slower than UMAP on large datasets in practice, though accelerated approximations (e.g. Barnes-Hut and FFT-based variants) are commonly used to mitigate this.

5.9 Uniform Manifold Approximation and Projection (UMAP)

5.9.1 Introduction

UMAP was introduced by McInnes, Healy, and Melville in 2018 [McInnes et al., 2018] and has quickly become one of the most widely used tools for visualizing high-dimensional data. Like Laplacian eigenmaps, UMAP is a local method built from a k -nearest neighbor graph rather than the pairwise (geodesic) distances used by ISOMAP. Unlike every method we have discussed so far, however, UMAP does not produce its embedding by solving a single eigenvalue problem. Instead, it builds a weighted graph representing the neighborhood structure of the data in $\mathbb{R}^d$, builds a comparable weighted graph on a set of candidate points in $\mathbb{R}^t$, and then directly optimizes the positions of those points so that the two graphs are as similar as possible. This shift from an eigenvalue problem to a direct optimization problem is the central idea that distinguishes UMAP from ISOMAP, LLE, Laplacian eigenmaps, and HLLEs.

5.9.2 Algorithm

5.9.2.1 Constructing a fuzzy neighborhood graph

As with Laplacian eigenmaps, we begin by fixing a number of nearest neighbors $k > 0$ and, for each $\vec{x}_i$, finding its k -nearest neighbors $\mathcal{N}_i$. Rather than a single bandwidth σ^2 shared across the entire dataset, UMAP calibrates a *local* bandwidth σ_i for each point $\vec{x}_i$, allowing the method to adapt to regions of the manifold with different sampling density. Let $\rho_i = \min_{j \in \mathcal{N}_i} \|\vec{x}_i - \vec{x}_j\|$ be the distance from $\vec{x}_i$ to its nearest neighbor. We then choose σ_i so that

$$\sum_{j \in \mathcal{N}_i} \exp \left(-\frac{\max(0, \|\vec{x}_i - \vec{x}_j\| - \rho_i)}{\sigma_i} \right) = \log_2(k),$$

which can be solved numerically for each i . Subtracting ρ_i guarantees that $\vec{x}_i$ is connected to its closest neighbor with full weight, while the choice of σ_i ensures that every point contributes the same total weight to the graph regardless of how densely sampled its neighborhood happens to be. These choices give a directed, weighted graph with entries

$$\tilde{\mathbf{W}}_{ij} = \begin{cases} \exp \left(-\frac{\max(0, \|\vec{x}_i - \vec{x}_j\| - \rho_i)}{\sigma_i} \right) & \vec{x}_j \in \mathcal{N}_i \\ 0 & \text{else.} \end{cases}$$

5.9.2.2 Symmetrization

The matrix $\tilde{\mathbf{W}}$ need not be symmetric, since $\vec{x}_j$ being a neighbor of $\vec{x}_i$ does not guarantee $\vec{x}_i$ is a neighbor of $\vec{x}_j$. UMAP symmetrizes $\tilde{\mathbf{W}}$ using the fuzzy set union

$$\mathbf{W}_{ij} = \tilde{\mathbf{W}}_{ij} + \tilde{\mathbf{W}}_{ji} - \tilde{\mathbf{W}}_{ij} \tilde{\mathbf{W}}_{ji},$$

so that $\vec{x}_i$ and $\vec{x}_j$ receive a high weight $\mathbf{W}_{ij}$ if *either* point considers the other a close neighbor. The resulting $\mathbf{W}$ plays the same role as the adjacency matrix $\mathbf{W}$ from Laplacian eigenmaps, just calibrated with this locally adaptive procedure.

5.9.2.3 Optimizing the low-dimensional embedding

Given the high-dimensional affinities $\mathbf{W}$, UMAP next defines an analogous affinity between candidate low-dimensional points $\vec{y}_1, \dots, \vec{y}_N \in \mathbb{R}^t$,

$$v_{ij} = (1 + a\|\vec{y}_i - \vec{y}_j\|^{2b})^{-1},$$

where $a, b > 0$ are constants chosen so that v_{ij} closely matches a target curve controlled by the tuning parameter *min_dist*, which specifies how tightly points are allowed to pack together in the embedding. Initializing $\vec{y}_1, \dots, \vec{y}_N$ using the spectral embedding from Laplacian eigenmaps (a convenient reuse of a method we have already discussed), UMAP then adjusts the points to minimize the cross-entropy between $\mathbf{W}$ and $\mathbf{V}$,

$$C(\vec{y}_1, \dots, \vec{y}_N) = \sum_{i \neq j} \left[\mathbf{W}_{ij} \log \left(\frac{\mathbf{W}_{ij}}{v_{ij}} \right) + (1 - \mathbf{W}_{ij}) \log \left(\frac{1 - \mathbf{W}_{ij}}{1 - v_{ij}} \right) \right].$$

Minimizing this loss via stochastic gradient descent produces an attractive force which pulls $\vec{y}_i$ and $\vec{y}_j$ together whenever $\mathbf{W}_{ij}$ is large, and a repulsive force which pushes $\vec{y}_i$ and $\vec{y}_j$ apart otherwise. Since most entries of $\mathbf{W}$ are (close to) zero, evaluating the repulsive force for every pair at every iteration would be wasteful; in practice, UMAP approximates it using a small number of randomly sampled non-neighbor pairs at each step, a trick known as negative sampling. The final coordinates $\vec{y}_1, \dots, \vec{y}_N$ after optimization are the UMAP embedding.

Example 5.9 (UMAP applied to the Swiss Roll). Let's revisit the swiss roll from earlier in this chapter and see how UMAP recovers its two-dimensional structure.

UMAP successfully unrolls the swiss roll, and the coloring shows that nearby points on the manifold remain nearby in the embedding. Notice, however, that the embedding is not a faithful rectangle in the way that ISOMAP's embedding is – distances within the embedding, and the overall aspect ratio, need not match the original data closely. UMAP prioritizes preserving neighborhoods, not distances.

5.9.3 Strengths and Weaknesses

5.9.3.1 Strengths

1) Speed and scalability

Because UMAP relies on approximate nearest neighbor search and stochastic gradient descent with negative sampling rather than the eigendecomposition of a dense $N \times N$ matrix, it scales far more gracefully to large datasets than ISOMAP, Laplacian eigenmaps, or HLLs.

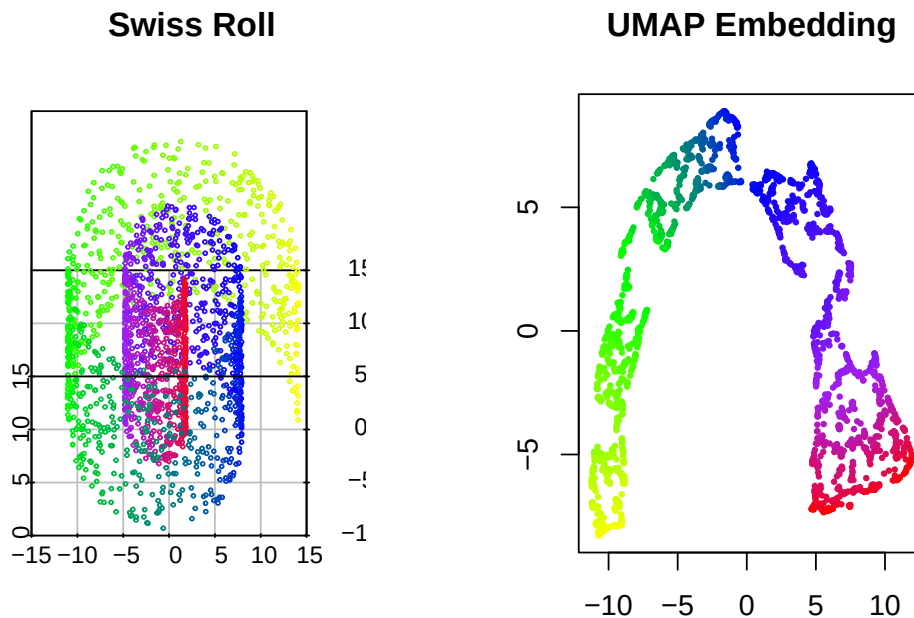


Figure 5.5: The Swiss Roll (left) and its UMAP embedding (right)

2) Balances local and global structure

The spectral initialization (itself a Laplacian eigenmap of $\mathbf{W}$) gives the optimization a sensible global starting point before the attractive and repulsive forces refine local neighborhoods, so UMAP tends to preserve more of the manifold's coarse, global organization than LLE or Laplacian eigenmaps alone.

3) Few tuning parameters

The two main tuning parameters, the number of neighbors k and min_dist , play a similar role to the tuning parameters of LLE and Laplacian eigenmaps and have reasonably interpretable effects on the resulting embedding.

5.9.3.2 Weaknesses

1) No uniqueness guarantee

Because the embedding is the result of a nonconvex optimization rather than an eigenvalue problem, different random initializations or optimization runs can produce noticeably different embeddings, unlike the (essentially) unique solutions returned by ISOMAP, LLE, Laplacian eigenmaps, and HLLEs.

2) Distances between (and sizes of) clusters are not meaningful

UMAP is explicitly designed to preserve neighborhoods, not distances. As a result, the relative sizes of clusters and the gaps between them in a UMAP

embedding often do not reflect the true geometry of the data and should not be over-interpreted.

3) No natural inverse mapping

Like ISOMAP, LLE, and Laplacian eigenmaps, UMAP does not directly provide a mapping from the embedding back to the original space. We turn to a class of methods, autoencoders, which are built explicitly to provide such an inverse.

5.10 Autoencoders (AEs)

5.10.1 Introduction

In each of the previously discussed manifold learning techniques, we have never made an attempt to explicitly model the manifold mapping (or its inverse). Rather, we have focused on methods which recover lower-dimensional representations of the data assuming certain properties of the manifold map (e.g. isometry) and its preimage (convexity). As a result, the journey from high-dimensional data to lower-dimensional representations has been a one-way trip.

Suppose now, that we take a more ambitious approach and try to model the manifold mapping and its inverse. In practice, we know nothing about these functions – their mere existence is assumed via the manifold hypothesis – but at minimum we can expect them to be complicated nonlinear functions. Enter autoencoders (AEs) which are specially designed neural networks that simultaneously model the manifold map and its inverse.

Generally, an autoencoder is comprised of three main components.

- 1) **Encoder:** A neural network $\mathcal{E}(\vec{x})$ that compresses the input $\vec{x}$ into a latent representation. The encoder is acting like the inverse manifold map.
- 2) **Latent Space:** The reduced dimensionality representation, often denoted as $\vec{z}$, where $\vec{z} = \mathcal{E}(\vec{x})$.
- 3) **Decoder:** A neural network $\mathcal{D}(\vec{z})$ that aims to reconstruct the original input from the latent representation. This function acts like the manifold map.

5.10.2 Neural networks: design and training

In their simplest form, neural networks iterate between affine transformations and elementwise nonlinear operations. If $\vec{x} \in \mathbb{R}^d$ is the input to a neural network, the affine transformation is determined by the equation $\mathbf{W}\vec{x} + \vec{b}$ where $\mathbf{W} \in \mathbb{R}^{k_1 \times d}$, called weights, and $\mathbf{b} \in \mathbb{R}^{k_1}$, called the bias, are tuneable parameters. We then apply a nonlinear function, $\sigma_1 : \mathbb{R} \rightarrow \mathbb{R}$, to $\mathbf{W}\vec{x} + \vec{b}$ resulting in a k_1 dimensional vector, $\sigma_1(\mathbf{W}\vec{x} + \vec{b})$ where the application of σ_1 is interpreted elementwise. The nonlinear function is also called an activation. Together, the

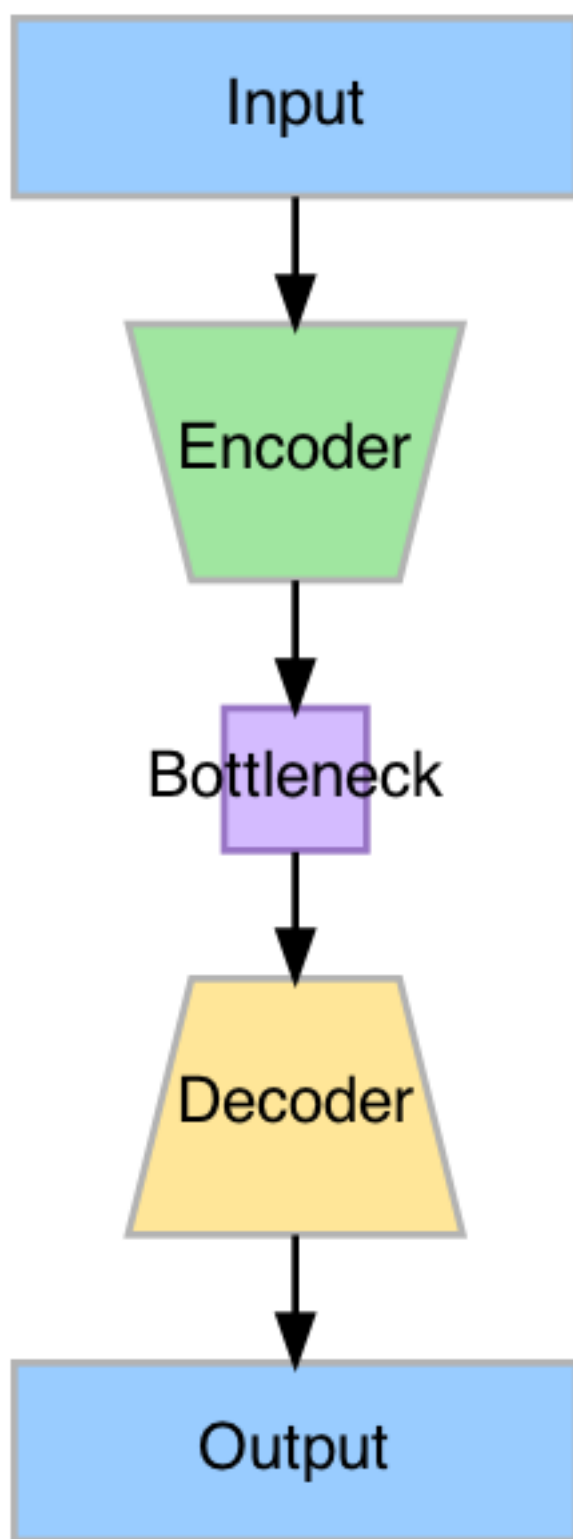


Figure 5.6: Informal schematic of an autoencoder. The encoder and decoder are neural networks which map the original data into a lower dimensional latent space.

affine transformation and nonlinear transformation are called a layer of the network with width k_1 reflecting the dimension of the output of the layer.

We can then iterate this idea, using the output of the previous layer as the input to the subsequent layers resulting in the sequence of operations,

$$\begin{aligned}\vec{\ell}_0 &= \vec{x} \\ \vec{\ell}_1 &= \sigma_1(\mathbf{W}_1 \vec{\ell}_0 + \vec{b}_1) \\ \vec{\ell}_2 &= \sigma_2(\mathbf{W}_2 \vec{\ell}_1 + \vec{b}_2) = \sigma_2(\mathbf{W}_2 \sigma_1(\mathbf{W}_1 \vec{\ell}_0 + \vec{b}_1) + \vec{b}_2) \\ &\vdots \\ \vec{\ell}_t &= \sigma_t(\mathbf{W}_t \vec{\ell}_{t-1} + \vec{b}_t).\end{aligned}$$

In practice, we are free to choose the number of layers (depth of the network) and the width and activation within each layer providing innumerable configurations. Choosing the structure of the network – its architecture – has an enormous impact on how well this model can perform as a method of dimension reduction. Given the importance of these choices, building a neural network can appear overwhelming.

And while there is still some art to the practice there are several conventions specific to autoencoders which offer guidance. First, we want to compress our high dimensional data into a lower dimensional space. To this end, we'll have an internal layer which maps to a lower dimensional space, referred to as the bottleneck. We refer to all layers between the input and the bottleneck as the encoder. The layers following the bottleneck then map back to the original high-dimensional space. We call this portion of the network the decoder. By convention, the network is symmetric around the bottleneck so the encoder and decoder have the same number of layers with widths in the opposite order. See the MNIST example below for a specific case.

There are also other data specific insights from geometric deep learning that motivate more complicated neural network architecture beyond the traditional, dense, feed-forward approach we have outlined here. For now, we'll focus common activations and methods for training the model (tuning the weights and biases in layers) to accomplish dimension reduction.

5.10.2.1 Activation Functions

Activation functions play a vital role in neural networks, determining the output of a neuron based on its input. They introduce non-linear properties into the network, enabling it to learn from the error and make adjustments, which is essential for learning complex, nonlinear patterns.

Two common activation functions we can choose from.

1) Sigmoid (Logistic) Function:

Equation: $f(x) = \frac{1}{1+e^{-x}}$

Range: Between 0 and 1

Pros: Smooth gradient, preventing “jumps” in output values

Cons: Can cause vanishing gradient problems in deep networks

2) Rectified Linear Unit (ReLU) Function:

Equation: $f(x) = \max(0, x)$

Range: From 0 to infinity

Pros: Helps mitigate the vanishing gradient problem, leading to faster convergence

Cons: Neurons can sometimes “die”, especially with a large learning rate

There are many other choices, but the ReLU activation is a fantastic first choice since it results in a network which can be trained more easily.

5.10.2.2 Training

At its core, an autoencoder seeks to learn a compressed, efficient representation of input data by nonlinearly encoding the data into a lower dimension space and subsequently decoding it to reconstruct the original data [Hinton and Salakhutdinov, 2006]. This translates the training of a neural network into a self-supervised problem, wherein we want to approximate $\vec{x}$ by its reconstruction $\mathcal{D}(\mathcal{E}(\vec{x}))$. The primary objective during the training phase of an autoencoder is to minimize the reconstruction error, often quantified using metrics such as Mean Squared Error (MSE) between the input data and its reconstructed counterpart. Mathematically, this gives rise to the loss function

$$L(\theta_e, \theta_d) = \frac{1}{N} \sum_{i=1}^N \|\vec{x}_i - \mathcal{D}_{\theta_d}(\mathcal{E}_{\theta_e}(\vec{x}_i))\|^2.$$

Here θ_e and θ_d are the weights and biases of the encoder and decoder respectively.

Importantly, since the network is comprised of iterated affine transformations and nonlinear operations, it is a differentiable function of its parameters. Computing these derivatives is called backpropagation in the ML literature and relies on repeated applications of the chain rule. With gradients in hand, gradient based optimization is a natural choice for tuning the parameters of the model to minimize the MSE. At present, the most common choice of gradient based optimization is an algorithm called Adam [Kingma and Ba, 2015] which uses an accelerated version of stochastic gradient descent to fit the autoencoder. The details of Adam and stochastic gradient descent are left to the curious reader.

Once the autoencoder has been trained and we have identified optimal parameters, we can use the encoder for dimension reduction. As a motivating example, let’s revisit MNIST.

Example 5.10 (AEs and MNIST). Compared to other manifold learning methods we have introduced, the implementation and training of an Autoencoder is much more complicated as we’ve outlined above. Here, we focus again on MNIST. Following the setup of other methods, we continue to flatten each 28×28 image into a 784-dimensional vector, but we take one more step. Pixel intensities take values in the interval $[0, 255]$, which we divide by 255 so that every entry is between zero and one. We then apply the following dense neural network, which we train on the full MNIST data set. This network has 10 layers (we don’t count the input layer). All layers use a ReLu activation except for the last layer of the encoder (so the bottleneck is in $[0, 1]^2$) and the output (so that pixel intensities in the reconstruction are bounded between $[0, 1]$ matching the division by 255 done in preprocessing)

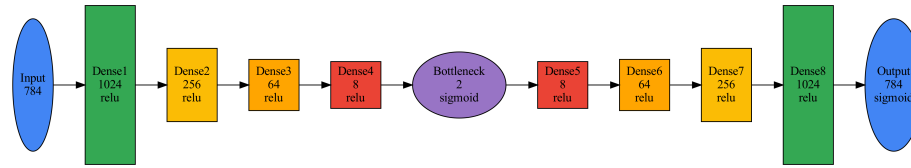


Figure 5.7: A deep, dense neural network for modeling MNIST data

Between the encoder and decoder, this model has 1,892,954 parameters, which we train via the Adam optimizer [Kingma and Ba, 2015] using 200 epochs, batch size = 256, and a 80%/20% test/train split. After training, we can run each image through the encoder portion of the network to view where each image is mapped in the latent space. As we can see from the image below, the neural network does a far better job of separating the images of different digits into separate groups.

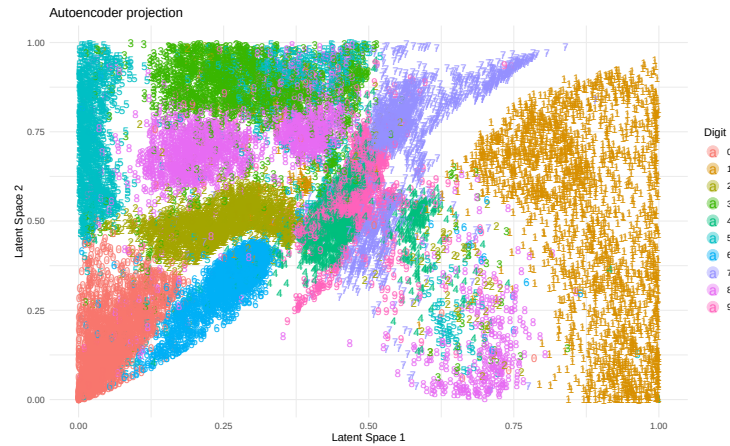


Figure 5.8: Latent space representation of MNIST

For comparison, here is the two-dimensional configuration of points obtained by PCA.



Figure 5.9: 2D Representation of MNIST via PCA

5.10.3 Strengths and weaknesses

Neural networks are incredibly powerful tools for approximating nonlinear functions, so it is natural to hope that we might be able to approximate the manifold map using the decoder. Like previous methods, however, the lower dimensional space is not unique so we can hope to learn the manifold map up to some invertible transformation without a lot more information. Furthermore, AEs are focused on minimizing the reconstruction error rather than directly using properties of the manifold containing our data. The greedy gradient based approach means we relinquish control on which relationships within the dataset we would like to preserve and what other relationships that might exist between the bottleneck and manifold (isometry for example). In particular, there are no constraints that an AE prioritize the neighborhood structure between points. Samples $\vec{x}_i$ and $\vec{x}_j$ may be close originally, but the nonlinear nature of a neural network could result in $\mathcal{E}(\vec{x}_i)$ and $\mathcal{E}(\vec{x}_j)$ being far apart.

Nonetheless, the utility of Autoencoders has been demonstrated in a wide array of applications beyond dimensionality reduction. Anomaly detection, denoising [Vincent et al., 2008], and generative modeling [Kingma and Welling, 2013] are a few exciting extensions and applications. The capacity of an AE to generalize and generate new samples is by far its greatest strength, which delineates it from the preceding methods. LLMs and many other generative models are based on the self-supervised approach central to AEs. The specifics that go into training state of the art language models are beyond the scope of this text. However, a basic mechanism is built into the decoder portion of the network. In principle, we could choose a point in the bottleneck (perhaps one between the lower-dimensional representation of two sample arising from the encoder), then pass it through the decoder to generate a new data point. For an interactive

Table 5.2: Manifold learning examples and their properties

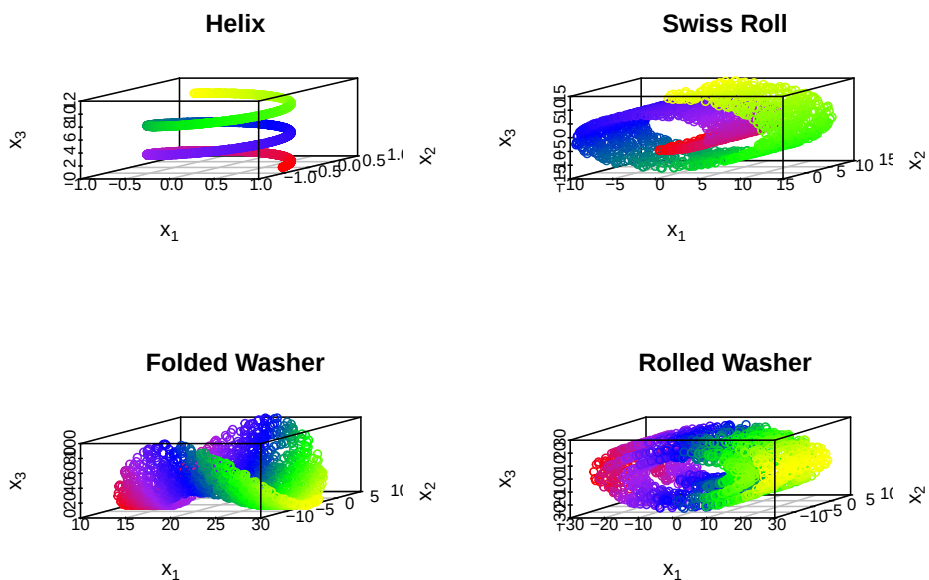
Manifold	Intrinsic Dimension	Preimage	Preimage connected?	Preimage convex?	Is
Helix	One	Line segment	Yes	Yes	
Swiss Roll	Two	Rectangle	Yes	Yes	
Folded Washer	Two	Annulus	Yes	No	
Rolled Washer	Two	Annulus	Yes	No	

demonstration of this idea applied to MNIST, check out this java applet hosted by n8programs. The autoencoder architecture in that demo is different than the method we used in the example above hence the different organization of points in the bottleneck.

5.11 Comparison of Manifold Learning Methods

As an initial comparison, we'll look at several 3-dimensional examples with known lower dimensional latent spaces where we can visualize the data. Importantly, these examples fail to satisfy some of the conditions of the preceding methods so we can assess how susceptible the methods we have discussed to violations in their assumptions. Table 5.2 summarizes the results.

For comparison, we provide scatterplots of 2000 samples from each manifold which we will use for analysis. The points have been color coded to improve visualization and to aid comparison with recovered lower dimensional coordinates.



5.11.1 Visual comparison of results

5.11.1.1 Helix

The helix is the simplest example and satisfies all assumptions we have discussed in the preceding sections. As such, it is not surprising that each of the methods discussed does a reasonable job compressing the data to a line segment. For the helix, we have used the isometric manifold mapping

$$\Psi(t) = \left(\frac{\sqrt{2}}{2} \cos(t), \frac{\sqrt{2}}{2} \sin t, \frac{\sqrt{2}}{2} t \right)^T$$

where $t \in [0, 15]$ is the original 1-dimensional coordinate for the data. Below we plot the recovered 1-d representation of the data against the corresponding value of t . As we have discussed, a perfect recovery of the original t is impossible. The best we can hope for is an affine relationship reflecting a one-to-one correspondence (up to translation, rescaling, and rigid motion) between the original low dimension coordinates and those recovered by our manifold learning methods.

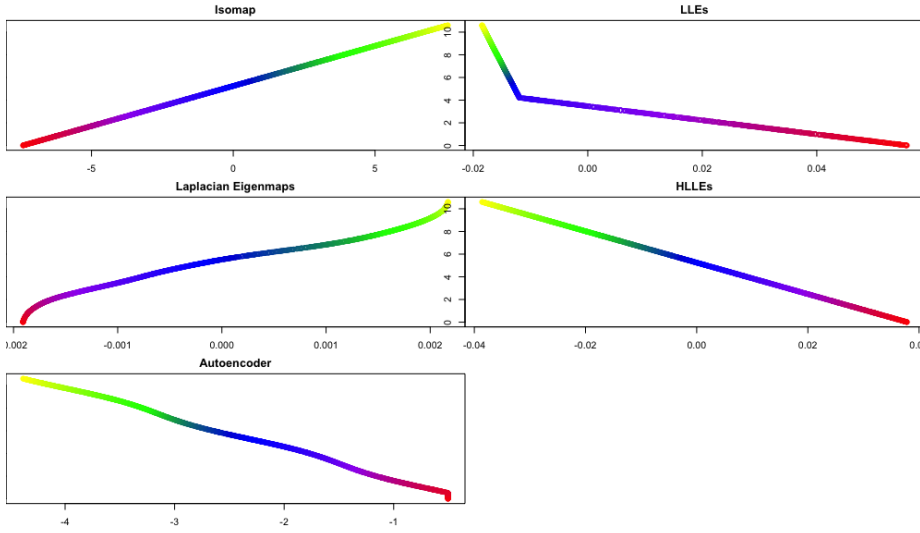


Figure 5.10: Recovered latent space vs original coordinates

5.11.1.2 Swiss Roll

For the Swiss roll, we use the manifold map

$$\Psi : \left[\frac{3\pi}{2}, \frac{9\pi}{2} \right] \times [0, 15] \rightarrow \mathbb{R}^3$$

given by the equation

$$\Psi(s, t) = (s \cos s, t, s \sin s)^T.$$

Below, we compare the two-dimensional configurations recovered by each of the methods discussed in this chapter.

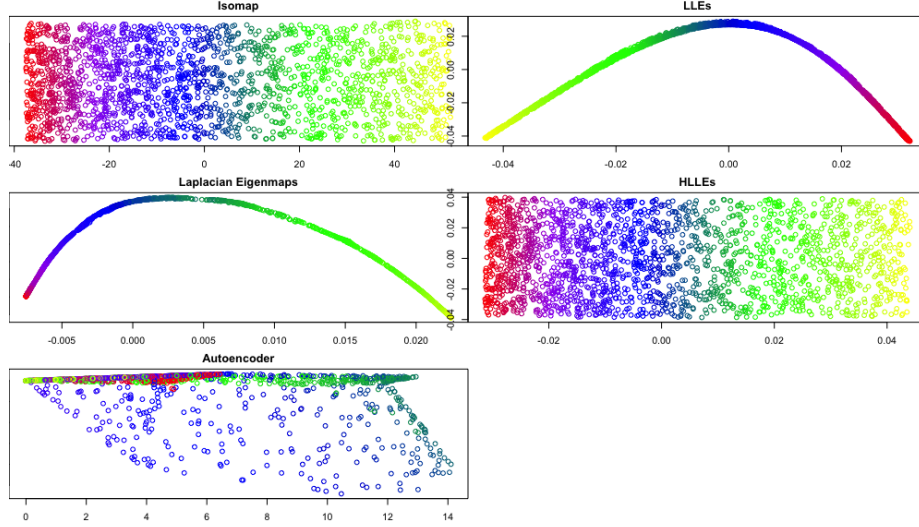


Figure 5.11: Recovered latent space vs original coordinates

5.11.1.3 Folded Washer

5.11.1.4 Rolled Washer

5.12 A Unifying View: Manifold Learning as Kernel PCA

Having covered ISOMAP, LLE, and Laplacian eigenmaps, it is worth stepping back to notice a common thread. Each method feels different – one preserves geodesic distances, one preserves local linear reconstructions, one preserves a weighted graph structure – yet each method ultimately produces its embedding the same way: by eigendecomposing a particular symmetric matrix built from the data and taking the leading eigenvectors (suitably scaled) as coordinates. This is exactly the recipe used by kernel PCA. It turns out that this similarity is not a coincidence. ISOMAP, LLE, and Laplacian eigenmaps can each be written as kernel PCA applied to a specific, data-dependent “kernel” matrix in place of the doubly centered kernel matrix $\mathbf{H}\mathbf{K}\mathbf{H}$ from earlier in this chapter [Ham et al., 2004].

ISOMAP. Recall that ISOMAP applies classical scaling directly to the matrix of squared geodesic distances, forming $B = -\frac{1}{2}\mathbf{H}\Delta^{(2)}\mathbf{H}$ where $\Delta_{ij}^{(2)} = (d_{ij}^{\mathcal{G}})^2$. By the duality of PCA and classical scaling (4.4), this is precisely kernel PCA using the kernel matrix $\mathbf{K}_{\text{ISOMAP}} = B$. Since geodesic distances are only estimated

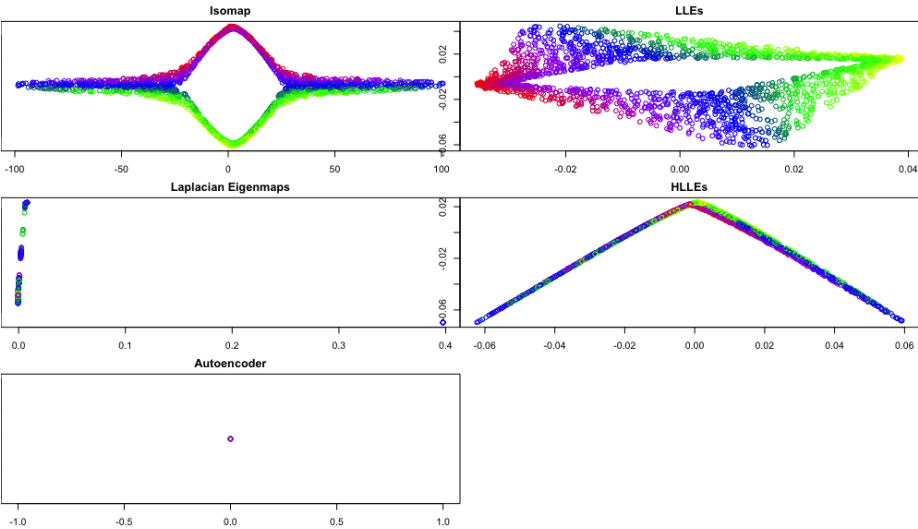


Figure 5.12: Recovered latent space vs original coordinates

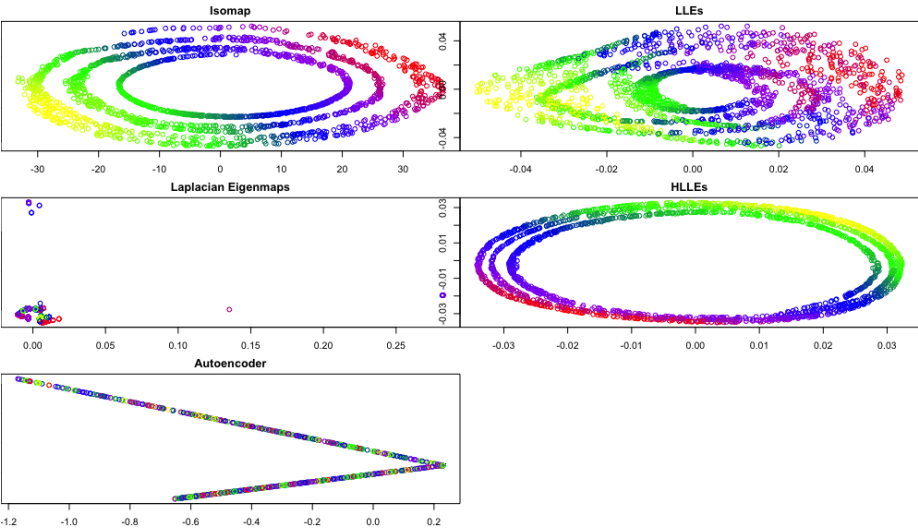


Figure 5.13: Recovered latent space vs original coordinates

from a finite neighborhood graph, $\mathbf{K}_{\text{ISOMAP}}$ is not guaranteed to be positive semidefinite in practice (unlike a kernel matrix built from a valid Mercer kernel); small negative eigenvalues are typically discarded.

LLE. LLE embeds the data using the *smallest* nontrivial eigenvectors of $\mathbf{M} = (\mathbf{I} - \mathbf{W})^T(\mathbf{I} - \mathbf{W})$, whereas kernel PCA always uses the *largest* eigenvectors of a kernel matrix. The two views are reconciled by a simple eigenvalue flip. With $\lambda_{\max}$ the largest eigenvalue of $\mathbf{M}$, the top nontrivial eigenvectors of $\mathbf{K}_{\text{LLE}} = \lambda_{\max}\mathbf{I} - \mathbf{M}$ coincide with the bottom nontrivial eigenvectors of $\mathbf{M}$, so the LLE embedding is exactly the kernel PCA embedding using $\mathbf{K}_{\text{LLE}}$.

Laplacian eigenmaps. By the same flip argument, the Laplacian eigenmap embedding built from the smallest nontrivial eigenvectors of $\mathbf{L}_{\text{sym}}$ is the kernel PCA embedding using the (Moore-Penrose) pseudoinverse $\mathbf{K}_{\text{LE}} = \mathbf{L}_{\text{sym}}^\dagger$ as the kernel matrix.

Two consequences follow from this unifying view. First, it explains why these methods behave so differently from kernel PCA with a fixed kernel like the rbf kernel. The matrices $\mathbf{K}_{\text{ISOMAP}}$, $\mathbf{K}_{\text{LLE}}$, and $\mathbf{K}_{\text{LE}}$ are all built from the entire training sample (through a k -nearest-neighbor graph), rather than from a function $k(\vec{x}, \vec{y})$ that can be evaluated on two arbitrary points. This is precisely why ordinary kernel PCA readily extends to new, out-of-sample points while ISOMAP, LLE, and Laplacian eigenmaps do not – doing so requires a Nyström-type approximation of the relevant kernel [Ham et al., 2004]. Second, it foreshadows a connection we revisit in the next chapter. The graph Laplacian $\mathbf{L}_{\text{sym}}$ used above to build $\mathbf{K}_{\text{LE}}$ is the same graph Laplacian at the heart of spectral clustering (6.5), and kernel k -means with a graph-based kernel recovers the same spectral clustering objective. Dimension reduction and clustering, at least for these methods, are two sides of the same eigenvalue problem.

5.13 Summary

This chapter’s methods all attack the same limitation identified at the end of the last chapter – linear methods can only fit flat structure – but they do so along two genuinely different geometric strategies, and it’s worth keeping the distinction straight.

The first strategy, the kernel trick, keeps the machinery of linear methods entirely intact and instead changes the space they operate in: by (implicitly) mapping the data into a higher-, possibly infinite-dimensional feature space $\mathcal{H}$ where the structure straightens out, kernel PCA lets us keep using inner products and eigendecompositions while still capturing curved structure back in the original coordinates (5.1, 5.2). The second strategy, manifold learning, instead works directly with the local geometry of the data – tangent spaces, neighborhood graphs, geodesic distances – and asks what a nonlinear surface would have to look like to produce the neighborhood structure we observe. ISOMAP preserves geodesic distances; LLE preserves each point’s local linear reconstruction from

its neighbors; Laplacian and Hessian eigenmaps preserve smoothness and local flatness through a graph Laplacian or an estimated local Hessian; t-SNE and UMAP abandon a single eigenvalue problem altogether and instead directly optimize an embedding to match an affinity built from local neighborhoods, using this extra flexibility to explicitly counteract the crowding problem (5.8) that a fixed, single-shot eigendecomposition cannot address.

The unifying view (5.12) is, in a sense, the real punchline of the chapter: it shows that ISOMAP, LLE, and Laplacian eigenmaps are not a separate family of techniques from kernel PCA at all, but kernel PCA applied to a cleverly constructed, data-dependent kernel matrix. The two strategies we started with turn out to be the same strategy, differing only in whether the “kernel” comes from a fixed function $k(\vec{x}, \vec{y})$ or from the neighborhood structure of the specific dataset in front of us. Every method in this chapter, in the end, is still doing exactly what PCA did – finding the leading eigenvectors of a symmetric matrix – just choosing that matrix far more cleverly.

5.14 Exercises

1. Let k_1 and k_2 be valid kernels on $\mathbb{R}^d \times \mathbb{R}^d$ (i.e. each satisfies Mercer’s Condition 5.1).
 - a. Show that $k(\vec{x}, \vec{y}) = k_1(\vec{x}, \vec{y}) + k_2(\vec{x}, \vec{y})$ is also a valid kernel. Hint: use the fact that the sum of two positive semidefinite matrices is positive semidefinite.
 - b. For three points $\vec{x}_1, \vec{x}_2, \vec{x}_3 \in \mathbb{R}^d$, suppose a candidate kernel gives the matrix

$$\mathbf{K} = \begin{bmatrix} 1 & 0.9 & 0.1 \\ 0.9 & 1 & 0.9 \\ 0.1 & 0.9 & 1 \end{bmatrix}.$$

Is $\mathbf{K}$ a valid kernel matrix? Justify your answer using Mercer’s Condition.

2. Consider the polynomial kernel $k(\vec{x}, \vec{y}) = (1 + \vec{x}^T \vec{y})^2$ for $\vec{x}, \vec{y} \in \mathbb{R}^2$. Find an explicit feature map $\varphi : \mathbb{R}^2 \rightarrow \mathbb{R}^m$ (for some finite m) such that $k(\vec{x}, \vec{y}) = \varphi(\vec{x})^T \varphi(\vec{y})$.
3. Recall that kernel PCA computes scores from the eigendecomposition of the doubly centered kernel matrix $\mathbf{H}\mathbf{K}\mathbf{H}$ where $\mathbf{K}_{ij} = k(\vec{x}_i, \vec{x}_j)$. Suppose we use the linear kernel $k(\vec{x}, \vec{y}) = \vec{x}^T \vec{y}$.
 - a. Show that $\mathbf{K} = \mathbf{X}\mathbf{X}^T$ where $\mathbf{X}$ is the (uncentered) data matrix with rows $\vec{x}_1^T, \dots, \vec{x}_N^T$.
 - b. Using the duality of PCA and classical scaling, explain why kernel PCA with the linear kernel recovers exactly the same scores as ordinary PCA on $\vec{x}_1, \dots, \vec{x}_N$.

4. Let $\vec{x}_1 = -1$, $\vec{x}_2 = 0$, $\vec{x}_3 = 1 \in \mathbb{R}$, and consider the rbf kernel $k(x, y) = \exp(-(x - y)^2)$ (i.e. $\sigma = 1$).
 - a. Compute the 3×3 kernel matrix $\mathbf{K}$ by hand.
 - b. Using R, compute the doubly centered matrix $\mathbf{H}\mathbf{K}\mathbf{H}$ and its eigendecomposition. Report the first kernel PC scores for $\vec{x}_1, \vec{x}_2, \vec{x}_3$.
5. For each of the following subsets of $\mathbb{R}^2$, state whether it is a manifold. If it is, give its intrinsic dimension; if it is not, explain which point(s) fail Definition 5.1 and why.
 - a. Two lines through the origin with different slopes, i.e. $\{(t, t) : t \in \mathbb{R}\} \cup \{(t, -t) : t \in \mathbb{R}\}$.
 - b. The open unit disk $\{\vec{x} \in \mathbb{R}^2 : \|\vec{x}\| < 1\}$.
 - c. A “T”-shaped union of two segments, $\{(0, y) : -1 \leq y \leq 1\} \cup \{(x, 0) : -1 \leq x \leq 1\}$.
 - d. The unit circle $\{\vec{x} \in \mathbb{R}^2 : \|\vec{x}\| = 1\}$.
6. Let $A = \mathbb{R}$ and define the manifold map $\Psi : A \rightarrow \mathbb{R}^3$ by $\Psi(z) = (z, z^2, z^3)^T$.
 - a. Compute the Jacobian $\mathbf{J}_\Psi$ as a function of z .
 - b. Find the tangent space $T_{\vec{p}}(\mathcal{M})$ at the point $\vec{p} = \Psi(1) = (1, 1, 1)^T$.
 - c. Find the approximating tangent plane to $\mathcal{M}$ at $\vec{p}$.
7. Consider the map $\Psi(t) = (\cos t, \sin t)^T$ for $t \in (0, 2\pi)$, which traces out (most of) the unit circle.
 - a. Show that $\|d\Psi/dt\| = 1$ for all t , and conclude that Ψ is a *local* isometry, i.e. for t_1, t_2 close together, the manifold distance between $\Psi(t_1)$ and $\Psi(t_2)$ equals $|t_1 - t_2|$.
 - b. Show that Ψ is not a *global* isometry by comparing the manifold distance between $\Psi(0.1)$ and $\Psi(2\pi - 0.1)$ (measured by traveling around the circle the short way) to $|0.1 - (2\pi - 0.1)|$.
8. Consider four points connected in a path, with edge weights $d_{12} = 1$, $d_{23} = 2$, $d_{34} = 1$ (there is no direct edge between any other pair of points).
 - a. Write down the initial 4×4 distance matrix $\Delta^{(0)}$ used to initialize the Floyd-Warshall algorithm (using ∞ for pairs with no direct edge).
 - b. Run the Floyd-Warshall algorithm by hand to find the geodesic distance matrix Δ with entries $\Delta_{ij} = d_{ij}^{\mathcal{G}}$.
9. Recall from Table 5.2 that the preimage of the folded washer is an annulus, which is not convex. Explain, in terms of the neighborhood graph construction in ISOMAP’s first step, why a non-convex preimage can cause

the graph distance $d_{ij}^{\mathcal{G}}$ to badly overestimate the true manifold distance $d_{ij}^{\mathcal{M}}$ for some pairs of points.

10. Suppose $\vec{x}_i \in \mathbb{R}$ has exactly two neighbors, $\vec{x}_j$ and $\vec{x}_l$, so that $N_i^k = \{j, l\}$.

- a. Using the constraint $w_{ij} + w_{il} = 1$, show that the LLE reconstruction problem $\min(\vec{x}_i - w_{ij}\vec{x}_j - w_{il}\vec{x}_l)^2$ has the closed-form solution

$$w_{ij} = \frac{\vec{x}_i - \vec{x}_l}{\vec{x}_j - \vec{x}_l}, \quad w_{il} = \frac{\vec{x}_j - \vec{x}_i}{\vec{x}_j - \vec{x}_l}.$$

- b. For $\vec{x}_i = 2$, $\vec{x}_j = 0$, $\vec{x}_l = 5$, compute w_{ij} and w_{il} numerically and verify that $\vec{x}_i = w_{ij}\vec{x}_j + w_{il}\vec{x}_l$.

11. Let $\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}^D$ have optimal LLE reconstruction weight matrix $\mathbf{W}$. Show directly (i.e. without invoking the general argument for $\epsilon(w)$ in the text) that if every point is reflected through the origin, i.e. $\vec{x}_i \mapsto -\vec{x}_i$ for all i , the optimal reconstruction weights are unchanged.

12. Let $\vec{x}_1 = 0$, $\vec{x}_2 = 1$, $\vec{x}_3 = 5$, $\vec{x}_4 = 6 \in \mathbb{R}$. Use $\epsilon = 2$ and binary weights to build the neighborhood graph for Laplacian eigenmaps.

- a. Write down the adjacency matrix $\mathbf{W}$, the degree matrix $\mathbf{D}$, and the graph Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{W}$.

- b. How many eigenvalues of $\mathbf{L}$ are exactly zero? Relate your answer to the number of connected components of the graph.

- c. For $\vec{y} = (1, 2, 3, 4)^T$, verify the identity $\vec{y}^T \mathbf{L} \vec{y} = \frac{1}{2} \sum_{i=1}^4 \sum_{j=1}^4 \mathbf{W}_{ij} (y_i - y_j)^2$ ((5.13)) by computing both sides directly.

13. In HLLEs, the local Hessian is estimated using quadratic regression with design matrix $\mathbf{X}_i \in \mathbb{R}^{k \times (1+t+t(t+1)/2)}$, and we require k to be at least the number of columns of $\mathbf{X}_i$ for the pseudoinverse to be well defined.

- a. Explain why $\mathbf{X}_i$ has $1 + t + t(t+1)/2$ columns by counting the intercept, linear, and quadratic/cross terms in the local quadratic Taylor expansion.

- b. Show that the requirement $k \geq 1 + t + t(t+1)/2$ is equivalent to $k > t(t+3)/2$.

- c. For an intrinsic dimension of $t = 5$, what is the minimum number of neighbors k required?

14. Let $g(x, y) = xy$.

- a. Compute ∇g and $\nabla^2 g$.

- b. Compute $\|\nabla^2 g\|_F^2$. Is it constant over $\mathbb{R}^2$?

- c. Compare your answer in (b) to $\|\nabla^2 g_1\|_F^2 = 8$ from the text's example with $g_1(x, y) = x^2 + y^2$. Which function is “more curved” on average over $[-2, 2]^2$?
15. Laplacian eigenmaps use a single bandwidth σ^2 shared by every point, while UMAP calibrates a local bandwidth σ_i for each point $\vec{x}_i$ individually. Suppose a dataset has one very densely sampled region and one very sparsely sampled region.
- Explain what is likely to go wrong if a single, shared σ^2 is tuned to work well in the dense region and then applied to the sparse region (or vice versa).
 - Explain how UMAP's local bandwidth calibration, via the condition $\sum_{j \in \mathcal{N}_i} \exp(-\max(0, \|\vec{x}_i - \vec{x}_j\| - \rho_i)/\sigma_i) = \log_2(k)$, addresses this issue.
16. Consider an autoencoder for data $\vec{x} \in \mathbb{R}^{500}$ with encoder layer widths $500 \rightarrow 100 \rightarrow 20 \rightarrow 3$ and a symmetric decoder $3 \rightarrow 20 \rightarrow 100 \rightarrow 500$ (each arrow indicates one layer with weights and a bias). Using the formula for the number of parameters in an affine layer, compute the total number of trainable parameters in the encoder, the decoder, and the full autoencoder.
17. Suppose the encoder and decoder of an autoencoder are both single linear layers with no bias and no activation function, i.e. $\mathcal{E}(\vec{x}) = \mathbf{W}\vec{x}$ for $\mathbf{W} \in \mathbb{R}^{t \times d}$ and $\mathcal{D}(\vec{z}) = \mathbf{V}\vec{z}$ for $\mathbf{V} \in \mathbb{R}^{d \times t}$, so that the reconstruction of $\vec{x}_i$ is $\mathbf{V}\mathbf{W}\vec{x}_i$.
- Explain why the reconstruction loss

$$L(\mathbf{W}, \mathbf{V}) = \frac{1}{N} \sum_{i=1}^N \|\vec{x}_i - \mathbf{V}\mathbf{W}\vec{x}_i\|^2$$

can be written as $\frac{1}{N} \|\mathbf{X} - \mathbf{X}(\mathbf{V}\mathbf{W})^T\|_F^2$ for the data matrix $\mathbf{X}$ with rows $\vec{x}_1^T, \dots, \vec{x}_N^T$.

- The matrix $\mathbf{V}\mathbf{W} \in \mathbb{R}^{d \times d}$ has rank at most t . Using the Eckart-Young-Mirsky Theorem 4.3, explain why the optimal choice of $\mathbf{V}\mathbf{W}$ is the best rank- t approximation to $\mathbf{X}$ (up to centering), and conclude that a linear autoencoder's bottleneck spans the same subspace as the leading t PCA loadings.
18. Suppose $\mathbf{M} = (\mathbf{I} - \mathbf{W})^T(\mathbf{I} - \mathbf{W})$ has eigenvalues $0 = \mu_1 < \mu_2 = 1 < \mu_3 = 3 < \mu_4 = 4$ with corresponding eigenvectors $\vec{v}_1, \vec{v}_2, \vec{v}_3, \vec{v}_4$.
- Using $\lambda_{\max} = 4$, compute the eigenvalues of $\mathbf{K}_{\text{LLE}} = \lambda_{\max} \mathbf{I} - \mathbf{M}$.
 - Confirm that the ordering of the eigenvalues flips, i.e. $\vec{v}_1$ (the *trivial* eigenvector of $\mathbf{M}$, associated with $\mu_1 = 0$) becomes the eigenvector associated with the *largest* eigenvalue of $\mathbf{K}_{\text{LLE}}$.

- c. Since $\vec{v}_1$ corresponds to the trivial, constant-function direction, it must still be discarded even when working with $\mathbf{K}_{\text{LLE}}$. Which eigenvector should actually be used as the top coordinate of the LLE embedding, and to which eigenvalue of $\mathbf{K}_{\text{LLE}}$ does it correspond?
19. The *S-curve* is another standard manifold learning benchmark, defined by the map $\Psi(s, t) = (\sin(s), t, \text{sign}(s)(\cos(s) - 1))^T$ for $s \in (-3\pi/2, 3\pi/2)$ and $t \in (0, 10)$.

```
N <- 2000
s <- runif(N, min = -3*pi/2, max = 3*pi/2)
t <- runif(N, min = 0, max = 10)
s_curve <- cbind(sin(s), t, sign(s)*(cos(s)-1))
```

- a. Generate $N = 2000$ samples from the S-curve using the code above, and produce a 3-dimensional scatterplot color-coded by s .
- b. Apply ISOMAP, LLE, Laplacian eigenmaps, HLLEs, and UMAP to recover a two-dimensional embedding of these data.
- c. Which method(s) best recover the rectangular structure of the original (s, t) coordinates? Which assumptions of each method are (or are not) satisfied by the S-curve, and how does this explain the differences you observe?

Chapter 6

Clustering

We now turn to clustering, which is the branch of UL focused on partitioning our data into subgroups of similar observations. Hereafter, we use the terms clusters and subgroups interchangeably. Recall from Section 3.2 of Chapter 3 that clustering rests on the assumption that our data organize into well defined subsets, which raises two linked questions: how many clusters (if any) exist, and what do we mean by similar observations? As the visual examples there showed, the answer to the second question – our notion of similarity – directly shapes the answer to the first, and this will not be so easy to resolve visually once we move beyond two dimensions. We now discuss data driven ways to estimate the appropriate number of clusters for a specific algorithm.

There is one main commonality (and associated set of notation) that will persist throughout this section. The ultimate goal of clustering is to separate data $\vec{x}_1, \dots, \vec{x}_N$ into k groups. The choice of k is critical in clustering. Some algorithms proceed by iteratively merging observations until only k clusters remain (hierarchical methods). Others (center- and model-based and spectral methods) treat clustering as a partitioning problem, where each element of the partition corresponds to a pre-specified number of clusters. Algorithm dependent methods for choosing k can be used for deciding optimal number of clusters without a priori knowledge.

6.1 Center-Based Clustering

Center-based clustering algorithms are typically formulated as optimization problems. Throughout this section we'll use, $C_1, \dots, C_k \subset \{\vec{x}_1, \dots, \vec{x}_N\}$ to denote the partitioning of our data into clusters. As such, $C_i \cap C_j, i \neq j$ and $C_1 \cup \dots \cup C_k = \{\vec{x}_1, \dots, \vec{x}_N\}$. If $\vec{x}_i, \vec{x}_j \in C_\ell$ then we will say that $\vec{x}_i, \vec{x}_j$ are in cluster ℓ . Importantly, for each cluster we will also have an associated center points, denoted $\vec{c}_1, \dots, \vec{c}_k$ respectively. A point $\vec{x}_i$ is in cluster ℓ if $\vec{x}_i$ is the

closest center to $\vec{x}_i$ under our chosen notion of distance/dissimilarity. To find a clustering of our data, we minimize a loss function dependent on the partition and/or the set of centers.

We will begin with k -means, the most well known method in this class of clustering algorithms (and perhaps all of clustering). In many ways, k -means is to clustering what PCA is to dimension reduction: a default algorithm used as a first step of analysis. Additional methods of center-based clustering include k -center and k -medoids, which we'll discuss later. Importantly, in all of these methods, the user pre-specifies a desired number of clusters, and the algorithms proceed to find an (approximately) optimal clustering. When the number of clusters is unknown, separate runs of an algorithm can be run for a range of cluster numbers. Post-processing techniques to compare different clusterings can then be used to find an 'optimal' number of clusters.

6.1.1 k -means

Given a partitioning $C_1, \dots, C_k$, we define the intracluster variation in cluster ℓ to be

$$V(C_\ell) = \frac{1}{2|C_\ell|} \sum_{\vec{x}_i, \vec{x}_j \in C_\ell} \|\vec{x}_i - \vec{x}_j\|^2$$

be the average squared Euclidean distance between all observations in cluster ℓ . Here $|C_\ell|$ is the number of samples in cluster ℓ . If $V(C_\ell)$ is small, all points within the cluster are close together, which indicates high similarity as measured by squared Euclidean distance.

In k -means, we want all clusters to have small intracluster variation so a natural loss function is the sum of all intracluster variations

$$L_{kmeans}(C_1, \dots, C_k) = \sum_{\ell=1}^k V(C_\ell).$$

With a little algebra, one can show that

$$V(C_\ell) = \sum_{\vec{x}_i \in C_\ell} \|\vec{x}_i - \vec{\mu}_\ell\|^2$$

where $\vec{\mu}_\ell = \frac{1}{|C_\ell|} \sum_{\vec{x}_i \in C_\ell} \vec{x}_i$ is the sample mean of points in cluster ℓ . Thus, the k means loss function simplifies to

$$T_{kmeans}(C_1, \dots, C_k) = \sum_{\ell=1}^k \sum_{\vec{x}_i \in C_\ell} \|\vec{x}_i - \vec{\mu}_\ell\|^2.$$

Before we turn to algorithms for minimizing this loss, let's highlight a few potential issues with k -means. Like PCA, k -means relies on squared Euclidean

distance, thus it is sensitive to outliers and imbalances in scale. These issues can be ameliorated with standardization (and potentially, the removal of outliers). However, the choice of squared Euclidean distance implicitly emphasizes a specific type of cluster shape: spheres. As we will show with an example later, k -means works well when the clusters are spherical in shape and/or far apart as measured by Euclidean distance. More complicated structure can be quite vexing for k -means.

6.1.2 k -center and k -medoids

In k -center and k -medoids, we require the cluster centers to be data points as opposed to sample means of data. Note that a given choice of centers $\vec{c}_1, \dots, \vec{c}_k \subset \{\vec{x}_1, \dots, \vec{x}_N\}$ implies a partition where we take

$$C_\ell = \{\vec{x}_i : \|\vec{x}_i - \vec{c}_\ell\| < \|\vec{x}_i - \vec{c}_j\|, j \neq \ell\}.$$

Using this fact, we can specify the loss function for k -center and k -medoids clustering.

$$\begin{aligned} L_{k\text{-center}}(\vec{c}_1, \dots, \vec{c}_k) &= \max_{\ell=1, \dots, k} \max_{x_i \in C_\ell} \|\vec{x}_i - \vec{c}_\ell\| \\ L_{k\text{-medoids}}(\vec{c}_1, \dots, \vec{c}_k) &= \sum_{\ell=1, \dots, k} \sum_{x_i \in C_\ell} \|\vec{x}_i - \vec{c}_\ell\| \end{aligned}$$

In the preceding expression we have used (non-squared) Euclidean distance, which immediately makes these algorithms less sensitive to outliers and scale imbalance compared to k -means. Furthermore, alternative distance/dissimilarity measures can be used instead of Euclidean distance which adds an additional level of flexibility. In fact, we do not even need the original data in practice to implement the k -center or k -medoids. A distance matrix alone is sufficient.

Unfortunately, squared Euclidean distance has the added advantage of being much faster to implement in practice.

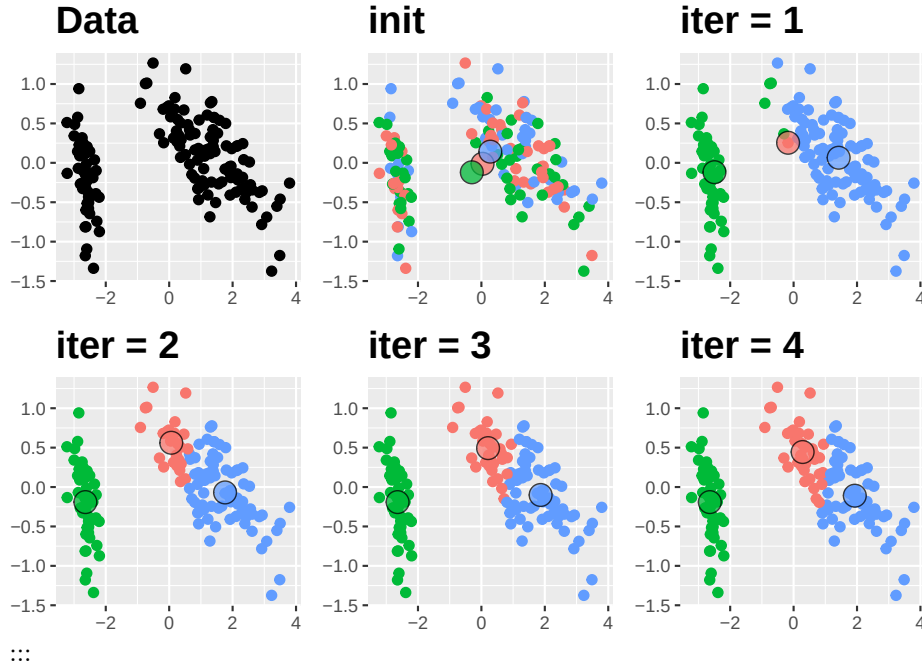
6.1.3 Minimizing clustering loss functions

For any of the three preceding methods, we want to find an optimal partitioning and/or set of centers which minimize their associated loss. One naive approach would be to search over all possible partitions of our data into k clusters (k -means) or all possible choices of k center points from our data, but one can imagine how computationally unfeasible this approach becomes for even moderate amounts of data. Instead, we will turn to greedy, iterative algorithms which converge to (locally) optimal solutions. The standard approach for k -means is based on Lloyd's algorithm with a special initialization to avoid convergence to local minima. Later versions of this text will discuss greedy approaches for k -center and the standard partitioning around medians (PAM) algorithm for k -medoids.

6.1.3.1 Lloyd’s Algorithm for k -Means

1. **Initialize:** Choose initial centers randomly from the data.
2. **Cluster Assignment:** Assign each point to the nearest center.
3. **Center Update:** Recompute cluster centers based on current assignments.
4. **Repeat** steps 2-3 until convergence.

...{example name=“Lloyd’s algorithm and Iris Data”} In this example, we’ll consider the three dimensional **iris** data. For visualization, we’ll plot the first two PC. Centers will be outlined in black.



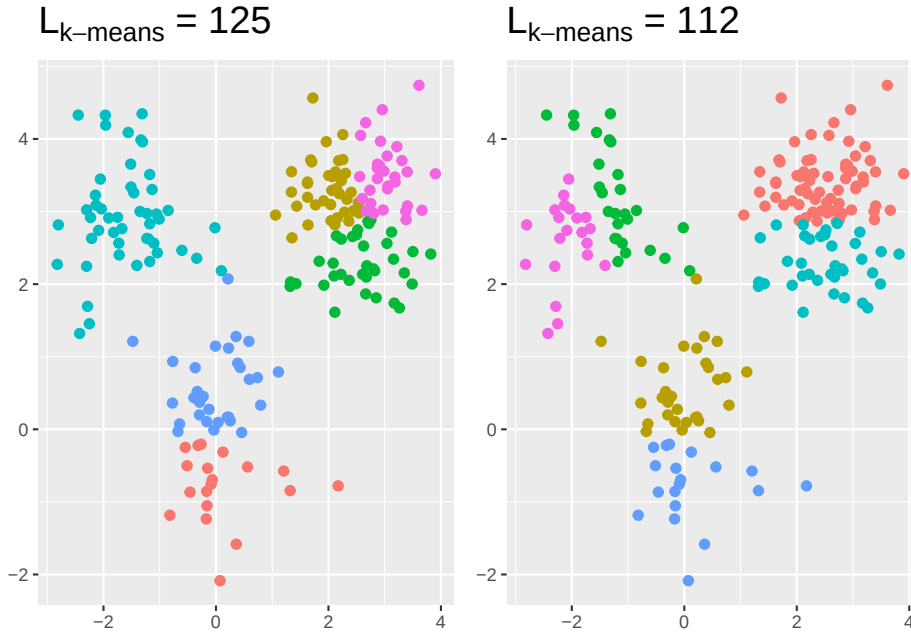
6.1.4 Strengths and Weaknesses

The advantages and disadvantages of center-based clustering algorithms can be separated into two main categories: computational and geometric. For now, we’ll focus on the computational aspects.

In the theoretical worst case, Lloyd’s algorithm can take $\propto N^{k+1}$ iterations to converge, but it is typically very fast in practice. Most implementations allow the user to set a maximum number of iterations and converge to approximate suboptimal solutions if the maximum number of iterations is reached. PAM and greedy k -center algorithms are typically much, much slower. However, once any of the clustering algorithms are complete, new data can be merged into the cluster with nearest center. Other methods we’ll discuss later cannot incorporate new data without running the algorithm from scratch.

There are other issues that can increase the computational demands of center-based clustering. Each of the methods requires one to choose the number of clusters in advance. We discuss this issue in more depth shortly, but for now we want to emphasize that there is no connection between the k -cluster solution and $(k + 1)$ -cluster solution. Thus, when investigating a suitable number of clusters, we need to run our clustering algorithm for a range of potential values of k .

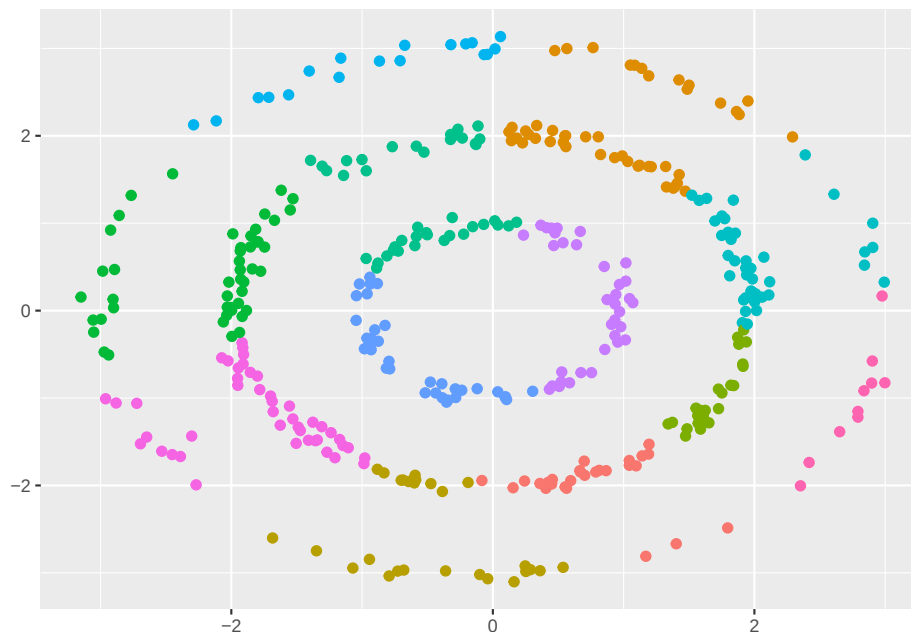
Convergence is also dependent on initial condition. In the figure below, we see two separate clustering arrangements resulting from different initial choices of the centers for k -means.



Thus, we also need to avoid convergence to local minima either by running many different initial conditions to convergence and picking the best solution or by using better initialization such as k -means++, which picks the initial points iteratively to maximize distance between the centers before running Lloyd's algorithm. Using k -means++ can reduce the number of initializations that one should use, but does not eliminate the potential for convergence to a poor clustering. Thus, center-based algorithms can become quite computationally demanding to analyze appropriately.

We have already discussed the impact of scale imbalance and outliers so as we turn to the geometric aspects of center based clustering, let's first focus on the centers, which we often treat as **representative** examples of the cluster. However, the sample averages in k -means may not resemble actual data points. Though k -means may be much faster in practice, the centers in k -center and k -medoids are restricted to observations making them more representative of the data.

Clusters are based on an interpretable notion of (Euclidean) distance so that points in a cluster are closer to one another than to points in other clusters. However, this feature biases center-based methods to clusters which are spherical in shape. If (dis)similarity not best captured through (squared) Euclidean distance, poor clustering is inevitable, and many of the methods used to estimate the number of clusters tend to overestimate the true value. As an example, consider the three rings shown at the beginning of this chapter. Silhouette scores (see 6.1.5.3) suggest 12 clusters which is much larger than the three rings we see in the figure.



6.1.5 Choosing the number of clusters

There are many different methods which tend to give (slightly) different results. Direct methods such as the Elbow plot and silhouette diagnostics, are based on balancing a score measuring goodness of fit with a minimal number of clustering. Alternatively, one can consider testing methods such as the Gap Statistic which compare the clustering performance to a null model where clustering is absent.

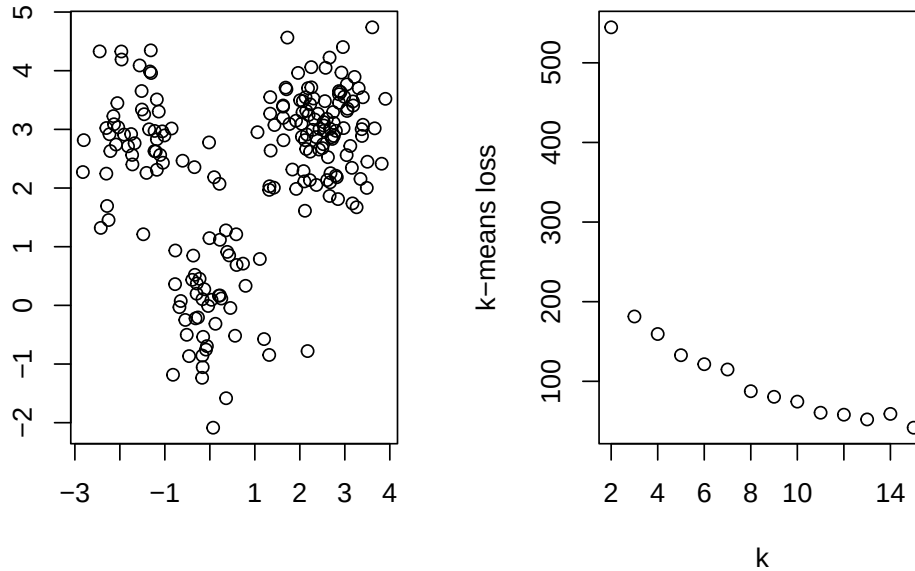
6.1.5.1 Elbow plot

The center-based loss function will decrease as the number of clusters increases. Thus, we can plot the optimal loss found at different numbers of clusters and look for a sudden drop, much like we did with the scree plot in PCA. Below, we show an example of this method for data with three well separated spherical clusters

```

k <- 2:15
inertia <- rep(NA,length(k))
for (j in 1:length(k)){
  inertia[j] <- kmeans(data1,k[j])$tot.withinss
}
par(mfrow = c(1,2))
plot(data1$V1,data1$V2,xlab = '', ylab = '')
plot(k,inertia, ylab="k-means loss" )

```



6.1.5.2 Gap Statistic

The Gap statistic (Tibshirani et al., (JRSS-B, 2001)), takes a specified number of clusters and compares the total within cluster variation to the expected within-cluster variation under the assumption that the data have no obvious clustering (i.e., randomly distributed). This method can be used to select an optimal number of clusters or as evidence that there is no clustering structure. Though best suited to center based methods (particularly k -means), one can apply it to the output of any clustering algorithm in practice so long as there is some quantitative measure of the quality of the clustering.

The algorithm proceeds through the following six steps.

1. Cluster the data at varying number of total clusters k . Let $L_{k\text{-means}}(k)$ be the total within-cluster sum of squared distances using k clusters.
2. Generate B reference data sets of size N , with the simulated values of variable j uniformly generated over the range of the observed variable j . Typically $B = 500$.

3. For each generated data set $b = 1, \dots, B$ perform the clustering for each K . Compute the total within-cluster sum of squared distances $T_K^{(b)}$.
4. Compute the Gap statistic

$$Gap(K) = \bar{w} - \log(T_K), \quad \bar{w} = \frac{1}{B} \sum_{b=1}^B \log(T_K^{(b)})$$

5. Compute the sample variance

$$var(K) = \frac{1}{B-1} \sum_{b=1}^B \left(\log(T_K^{(b)}) - \bar{w} \right)^2,$$

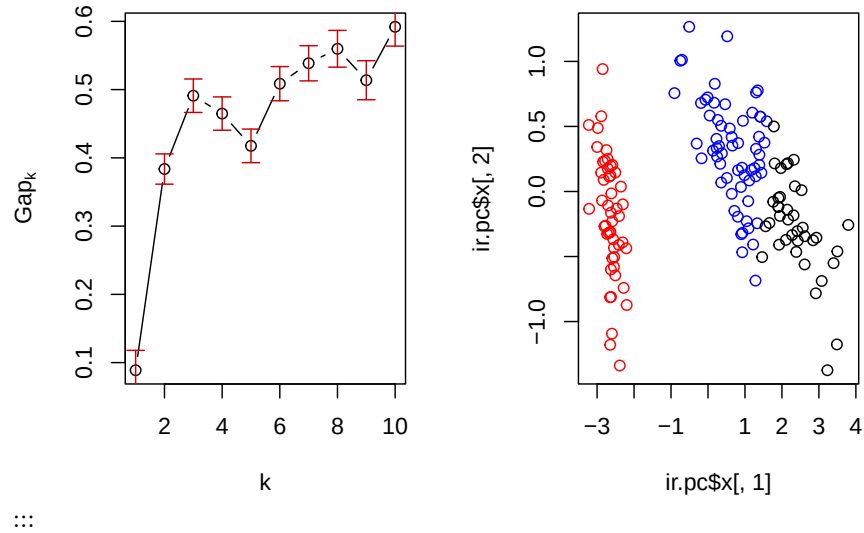
and define $s_K = \sqrt{var(K)(1 + 1/B)}$

6. Finally, choose the number of clusters as the smallest K such that

$$Gap(K) \geq Gap(K+1) - s_{K+1}$$

∴ {example name=“GAP statistic and iris data”} Below, we show a plot of the Gap Statistic (left) which indicates that three clusters is correct. The associated clustering is used to color a plot of the first two PC scores (right)

Gap statistic for Iris Data



6.1.5.3 Silhouette plots and coefficients

For a given clustering, we would like to determine how well each sample is clustered.

a_i = avg. dissimilarity of $\vec{x}_i$ with all other samples in same cluster

b_i = avg. dissimilarity of $\vec{x}_i$ with samples in *the closest* cluster

We then define

$$s_i = \frac{b_i - a_i}{\max\{a_i, b_i\}} \in (-1, 1)$$

as the silhouette for $\vec{x}_i$.

- Observations with $s_i \approx 1$ are well clustered
- Observations with $s_i \approx 0$ are between clusters
- Observations with $s_i < 0$ are probably in wrong cluster

We can use any dissimilarity!

- Can use silhouettes as diagnostics of any method!
- A great clustering will have high silhouettes for all samples.
- To compare different values of K (and different methods), we can compute the average silhouette

$$S_K = \frac{1}{N} \sum_{i=1}^N s_i$$

over a range of values of K and choose the K which maximizes S_K .

6.2 Kernel k -means

The kernel trick is not limited to PCA. Recall from 5 that the kernel trick lets us replace inner products $\vec{x}_i^T \vec{x}_j$ with a kernel evaluation $k(\vec{x}_i, \vec{x}_j)$, giving many linear methods access to nonlinear structure without ever explicitly computing the feature map φ . The k -means loss function introduced above is a natural candidate, since expanding the squared norm shows that it only involves inner products of the data,

$$\|\vec{x}_i - \vec{\mu}_\ell\|^2 = \vec{x}_i^T \vec{x}_i - \frac{2}{|C_\ell|} \sum_{\vec{x}_j \in C_\ell} \vec{x}_i^T \vec{x}_j + \frac{1}{|C_\ell|^2} \sum_{\vec{x}_j, \vec{x}_{j'} \in C_\ell} \vec{x}_j^T \vec{x}_{j'}, \quad \vec{\mu}_\ell = \frac{1}{|C_\ell|} \sum_{\vec{x}_j \in C_\ell} \vec{x}_j.$$

Replacing each inner product $\vec{x}_i^T \vec{x}_j$ with a kernel evaluation $k(\vec{x}_i, \vec{x}_j)$ gives the squared distance from $\varphi(\vec{x}_i)$ to the (never explicitly computed) feature-space cluster mean $\bar{\varphi}_\ell = \frac{1}{|C_\ell|} \sum_{\vec{x}_j \in C_\ell} \varphi(\vec{x}_j)$,

$$\|\varphi(\vec{x}_i) - \bar{\varphi}_\ell\|_{\mathcal{H}}^2 = k(\vec{x}_i, \vec{x}_i) - \frac{2}{|C_\ell|} \sum_{\vec{x}_j \in C_\ell} k(\vec{x}_i, \vec{x}_j) + \frac{1}{|C_\ell|^2} \sum_{\vec{x}_j, \vec{x}_{j'} \in C_\ell} k(\vec{x}_j, \vec{x}_{j'}).$$

Kernel k -means then proceeds just like Lloyd's Algorithm above, but replaces the assignment step's (squared) Euclidean distance with this kernelized distance.

1. **Initialize:** Choose an initial partition $C_1, \dots, C_k$ (e.g. by randomly assigning points to k groups).
2. **Cluster Assignment:** Reassign each $\vec{x}_i$ to the cluster ℓ minimizing $\|\varphi(\vec{x}_i) - \bar{\varphi}_\ell\|_{\mathcal{H}}^2$ given the current partition.
3. **Repeat** step 2, recomputing the expression above from the updated partition, until cluster assignments stop changing.

Notice there is no analog of the “center update” step from ordinary k -means. Since $\bar{\varphi}_\ell$ typically cannot be computed or even represented explicitly, the cluster membership itself is the only state carried between iterations.

As with kernel PCA (5), the choice and tuning of the kernel is critical. A poorly tuned kernel gives a result no better (and no worse) than ordinary k -means, while a well chosen kernel can recover cluster structure that is not spherical in the original coordinates.

Example 6.1 (Kernel k -means applied to Nested Rings). Consider $N = 400$ points sampled uniformly around two concentric circles with additive noise, a shape ordinary k -means cannot separate correctly since neither ring is convex (let alone spherical).

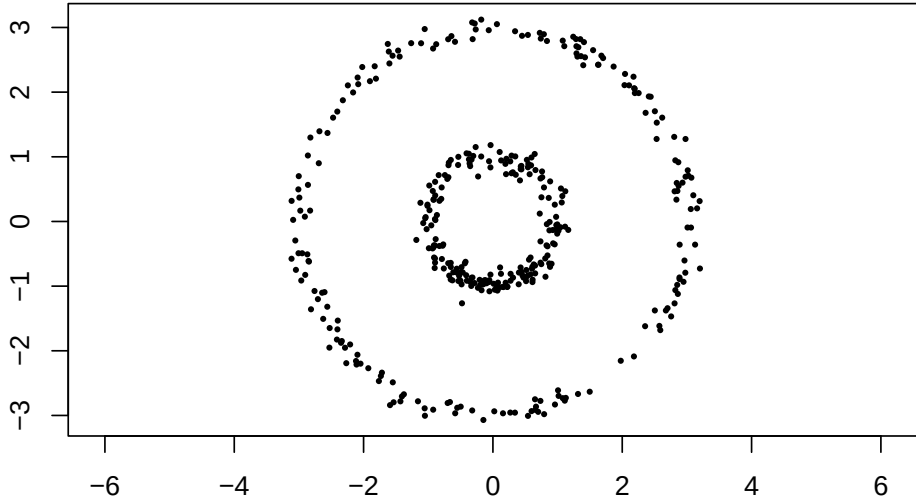


Figure 6.1: Nested rings which are not separable by ordinary k -means

Below, we compare the partition found by ordinary k -means to the partition found by kernel k -means using an rbf kernel.

Ordinary k -means splits the data by a hyperplane, cutting each ring in half. Kernel k -means, by first (implicitly) mapping the data to a feature space where the rings become linearly separable, recovers the two rings exactly.

Kernel k -means also has an important connection to a clustering method discussed later in this chapter: with an appropriately chosen (graph-based) kernel,

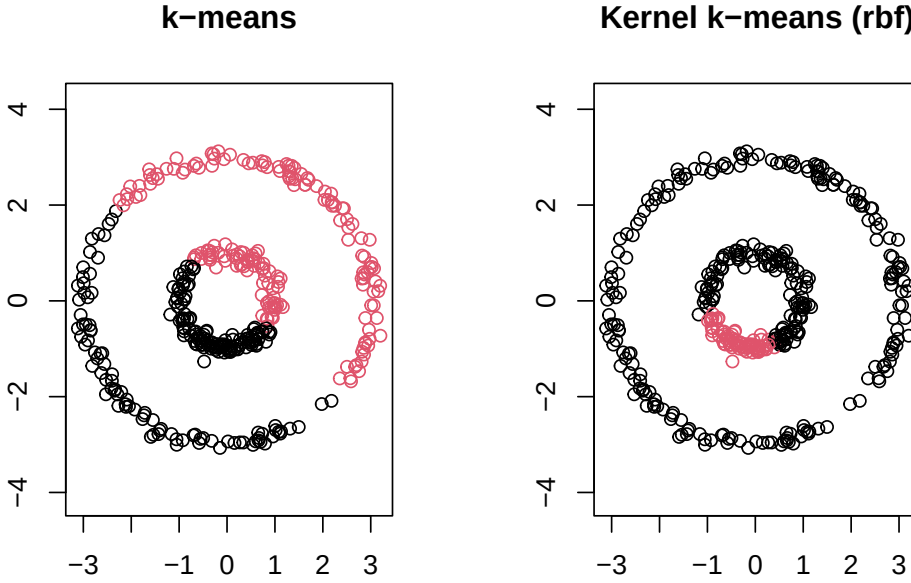


Figure 6.2: Ordinary k -means (left) vs. kernel k -means with an rbf kernel (right)

minimizing the kernel k -means loss is equivalent to minimizing a normalized graph cut, which is exactly the objective used by spectral clustering (6.5) [Dhillon et al., 2004]. In other words, kernel k -means and spectral clustering are two algorithmic routes to the same underlying idea.

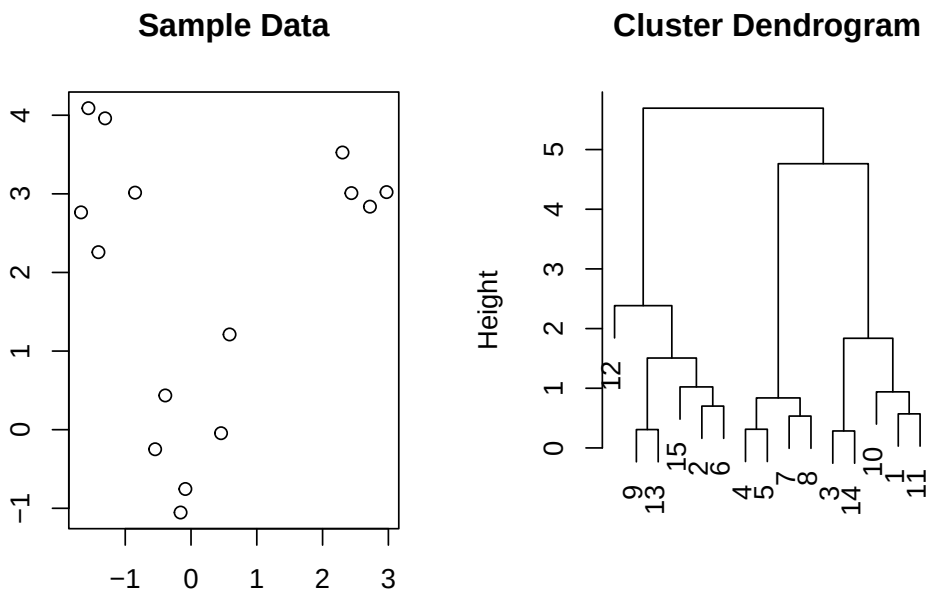
6.3 Hierarchical Clustering

Hierarchical clustering views clustering from an alternative perspective compared to the partitioning viewpoint of center-based methods. Rather than finding an optimal partitioning of data into a pre-specified number of groups, hierarchical methods treat clustering as an iterative process either merging data into larger and larger groups (agglomerative methods) or dividing the full dataset into smaller and smaller clusters (divisive methods). As a result, there are strong connections between the clustering of data into k vs $k + 1$ groups. In particular, when using hierarchical methods, the optimal clustering of data into k ($k + 1$) groups can be obtained by merging (splitting) the optimal clustering of data into $k + 1$ (k) groups.

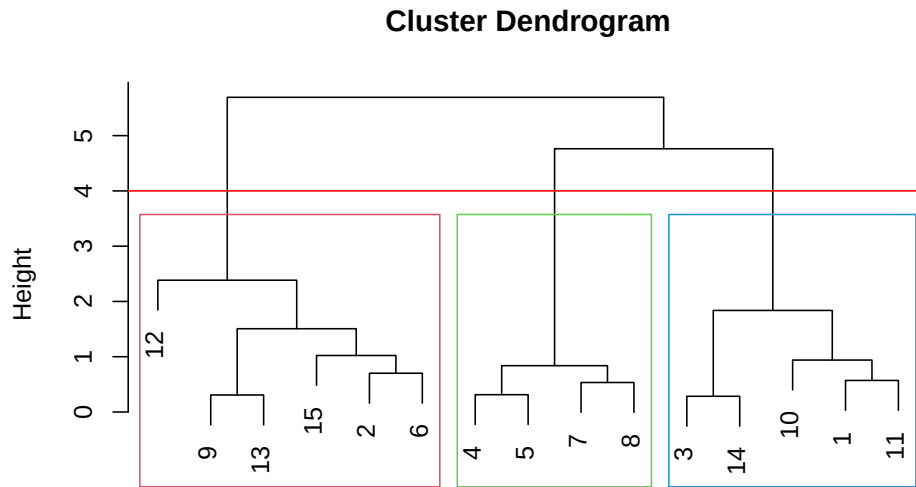
6.3.1 Dendrograms

The key output of hierarchical clustering is a dendrogram (tree diagram) depicting the subsequent merges/divisions of data, where each merge/division is shown by a horizontal line, with height indicating dissimilarity between merged or divided clusters. Before discussing the generation of a dendrogram and the many choices

In the figure below, samples 8 and 15 are merged into a cluster at height ≈ 0.5 indicating they have an original dissimilarity (Euclidean distance in this case) of 0.5. Sample 4 is then merged into the cluster with samples 8 and 15 at height ≈ 0.75 , indicating the dissimilarity of sample 4 with the cluster of samples 8 and 15.



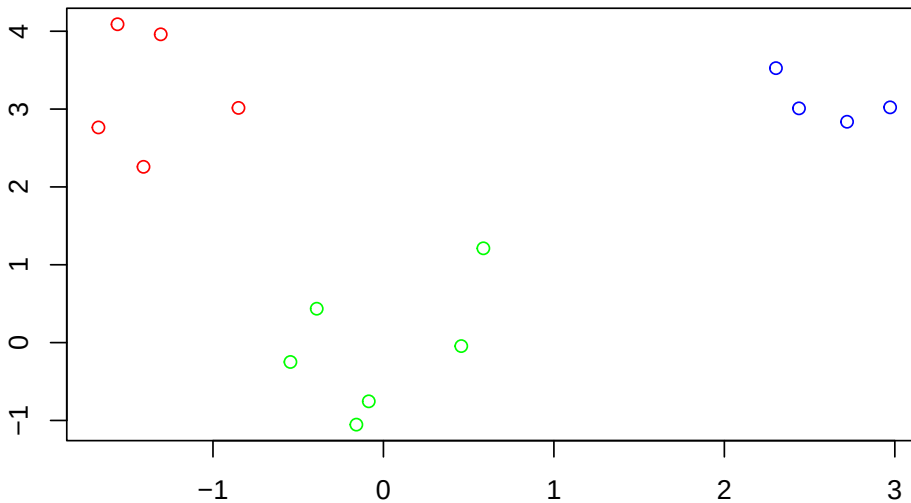
In addition to indicating the subsequent mergings/divisions of our data, the dendrogram can be used to determine a clustering of the data into a chosen number of clusters. For example, to cluster the data into three groups, we draw a horizontal line at a height (four in this case) which only cuts the tree into three branches.



Samples on the same branch are in the same cluster. Thus, the branch cut above suggest the following clustering:

- Cluster 1, samples: 12, 9, 13, 15, 2, 6
- Cluster 2, samples: 4, 5, 7, 8
- Cluster 3, samples: 3, 14, 10, 1, 11

which appears to match the clusters shown in the original data quite well.



6.3.2 Building a dendrogram

As suggested above there are two primary methods for building a dendrogram. The first is an agglomerative approach which begins with N clusters (one per data point) then iteratively merges clusters based on the minimum dissimilarity until a final cluster containing all data remains. Alternatively, there is a divisive

approach which begins with all data points in one cluster and splits iteratively until it finishes with N clusters (one per data point). Divisive clustering is less common, so we will focus on agglomerative methods hereafter.

Agglomerative clustering proceeds as follows.

1. **Initialize:** Start with N clusters, one per data point.
2. **Identify Closest Clusters:** Find the pair with the smallest dissimilarity.
3. **Merge Clusters:** Combine the clusters and recalculate distances.
4. **Repeat** until only one cluster remains.

Importantly, the initial dissimilarity or distance is a tuneable choice made by the practitioners. While Euclidean distance is the default, the performance of the hierarchical agglomerative clustering is highly dependent on this choice. Like k -center and k -medoids clustering, you can use non-Euclidean dissimilarities or distances such as Manhattan distance or cosine (dis)similarity if they would be a better choice depending on the application.

Additionally, we have also have a choice for specifying how the pairwise dissimilarities/distances are used to compute the distance between clusters containing more than one data point. The method for computing dissimilarity/distance between clusters is called **linkage** and common methods include

- **Single Linkage:** The distance between cluster A and cluster B is the smallest distance between a sample in A and a sample in B . Using C_A and C_B to denote clusters A and B , the distance between the clusters is

$$d(C_A, C_B) = \min_{\vec{x} \in C_A, \vec{y} \in C_B} d(\vec{x}, \vec{y}).$$

- **Complete Linkage:** The largest distance between a sample in A and a sample in B is used to define the distance between clusters A and B . Mathematically, the distance between the clusters is

$$d(C_A, C_B) = \max_{\vec{x} \in C_A, \vec{y} \in C_B} d(\vec{x}, \vec{y}).$$

- **Average Linkage:** Average distance between all pairs of points in cluster A and B defines the cluster distance. Then, the distance between the clusters is

$$d(C_A, C_B) = \frac{1}{|C_A||C_B|} \sum_{\vec{x} \in C_A, \vec{y} \in C_B} d(\vec{x}, \vec{y}).$$

- **Ward's Linkage:** This method uses an iterative formula based on the squared distances to minimize the within cluster variation at each merge akin to k -means. When the input to agglomerative clustering is Euclidean distance, Ward's Linkage is proportional to the squared Euclidean distance between the sample means in each cluster [Roux] though an explicit formula using the original dissimilarities is often used so that the original data is not required. For a complete discussion on this method and how it fits into the infinite family of Lance-Williams algorithms, see additional references [Szekely et al., 2005, Everitt, 2001].

In the above, the notation $d(\cdot, \cdot)$ represents the distance or dissimilarity of observations and/or clusters. Both the choice of original distances/dissimilarities and the type of linkage have a strong impact on the quality of the clustering.

6.3.2.1 Comparing Linkage Methods

The four linkages methods above have different properties with single and complete representing extreme cases and average linkage and Ward linkage somewhere in between. Specifically, single linkage only requires two points to be close for clusters to merge, whereas complete linkage requires all pairs of points to be close. This “friend of my friend is my friend” feature of single linkage can generate clusters which are formed by long chains of singletons merged together at short heights and can in some cases capture manifold structure within a cluster. Conversely, complete linkage favors many small, compact clusters which get merged at larger heights.

Comparing and choosing a linkage method can use the above heuristics as a guide, but for a more data driven approach, Gap statistics or silhouette coefficients are suitable. Unique to hierarchical clustering, we can also use the cophenetic correlation as a quantitative measure of how well a clustering preserves pairwise distances/dissimilarities in the original data.

Definition 6.1 (Cophenetic Correlation Coefficient). For compactness, let Δ_{ij} , $1 \leq i < j \leq N$, denote the user supplied distances/dissimilarities between observations i and j . For a given choice of linkage, let h_{ij} be the height that $\vec{x}_i$ and $\vec{x}_j$ are merged into the same cluster. The *cophenetic correlation* is the sample correlation of the $\binom{N}{2}$ pairs

$$(h_{ij}, \Delta_{ij}), \quad 1 \leq i < j \leq N.$$

The cophenetic correlation can be computed for different linkage methods. Typically, the method closest to one (optimal value of the cophenetic correlation) should be selected, though care should be used as is the case with any method to balance additional factors.

6.3.2.2 Determining the number of clusters

In addition to selecting the number of clusters, Gap statistics and silhouette coefficients can also be used to select the optimal number of clusters. Other methods, such as the Mojena coefficient [Mojena, 1977] are designed specifically for hierarchical clustering.

6.4 Model-Based Clustering

Unlike the preceding methods, model-based clustering assumes a probabilistic model for the data generating process. This model implies a likelihood (or posterior if we want to take a fully Bayesian approach) which we can optimize

over the model parameters. Using the optimal parameters, we can construct probability weights for cluster membership (e.g. we assign sample one to cluster one with probability 0.6 and cluster two with probability 0.4) or make a discrete decision by assigning each sample to the cluster for which it has the highest probability of membership.

For now, fix the number of clusters as K . Let us turn our attention to a standard two-step approach for generating a data set clustering structure which relies on two key components

1. **Cluster Densities:** Each cluster is associated with a probability density $f_j : \mathbb{R}^d \rightarrow [0, \infty)$, for $j = 1, \dots, K$. (In the related literature, these densities are often referred to as kernels which should be confused with the kernels we have discussed previously.)
2. **Cluster Probabilities:** Probabilities $\{p_j\}_{j=1}^K$ represent the *a priori* probability that a sample will be assigned to a given cluster

Now, suppose random vector $\vec{x} \in \mathbb{R}^d$ is generated by first choosing a cluster label Z with probability p_j , then conditional on $Z = j$ we sample x from f_j . This method implies the following joint density of $(\vec{x}, z)$

$$p(\vec{x}, Z = j) = p_j f_j(\vec{x}) \quad (6.1)$$

If we marginalize over z , we then obtain the following mixture density for $\vec{x}$,

$$p(\vec{x}) = \sum_{j=1}^K p_j f_j(\vec{x}) \quad (6.2)$$

We then repeat this approach independently N times to generate our data set. Hereafter, we'll use $Z_1, \dots, Z_n \in \{1, \dots, K\}$ to denote the **latent** cluster labels for each observation and continue using the notation $\vec{x}_1, \dots, \vec{x}_N$ for our observed data. Under an independent sampling assumption, we then obtain the following joint distribution for the latent cluster labels and observed data

$$p(\vec{x}_1, Z_1 = z_1, \dots, \vec{x}_N, z_N) = \prod_{i=1}^N p_{z_i} f_{z_i}(\vec{x}_i) \quad (6.3)$$

When we turn to likelihood estimation, a slightly different expression of the likelihood will be helpful. We can rewrite (6.3) in the following form:

$$p(\vec{x}_1, Z_1 = z_1, \dots, \vec{x}_N, z_N) = \prod_{i=1}^N p_{z_i} f_{z_i}(\vec{x}_i) \quad (6.4)$$

$$= \prod_{i=1}^N \sum_{j=1}^K \mathbb{I}(z_i = j) p_j f_j(\vec{x}_i) \quad (6.5)$$

$$= \prod_{i=1}^N \prod_{j=1}^K [p_j f_j(\vec{x}_i)]^{\mathbb{I}(z_i = j)} \quad (6.6)$$

Marginalizing over $Z_1, \dots, Z_N$ we then obtain the following joint density for the observed data

$$p(\vec{x}_1, \dots, \vec{x}_N) = \prod_{i=1}^N \left(\sum_{j=1}^K p_j f_j(\vec{x}_i) \right). \quad (6.7)$$

As in any modeling approach, performance is strongly influenced by the flexibility of the model to capture the observed data. There are many decisions which one can use to encode beliefs of the inherent structure of the data including the number of clusters, their shape, and the proportion of samples we expect in each cluster. For remainder of this section, we will focus on Gaussian mixture models (wherein $f_1, \dots, f_K$ are Gaussian densities) which are the most common application of model-based clustering.

6.4.1 Gaussian Mixture Models (GMM)

For a fixed number of clusters k , we assume that each cluster follows a Gaussian distribution, which are specified through their associated means and covariances. Following the notational convention from Chapter 2, we have

$$f_j(\vec{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma_j|} \exp \left(-\frac{1}{2} (\vec{x} - \vec{\mu}_j)^T \Sigma_j^{-1} (\vec{x} - \vec{\mu}_j) \right).$$

Following the notational convention from chapter 2, we use the shorthand

$$f_j(\vec{x}) = \mathcal{N}(\vec{x}; \vec{\mu}_j, \Sigma_j).$$

A Gaussian Mixture model (GMM) with K cluster is thus specified by the associated means, $\{\vec{\mu}\}_{j=1}^K$, covariances matrices $\{\Sigma_j\}_{j=1}^K$ and probabilities $p_1, \dots, p_K$. For brevity, we'll use

$$\Phi = \{ \{\vec{\mu}\}_{j=1}^K, \{\Sigma_j\}_{j=1}^K, \{p_j\}_{j=1}^K \}$$

to denote the set of model parameters. We'll use $p(\vec{x}_1, z_1, \dots, \text{vec} x_N, z_N \mid \Phi)$ to denote the joint distribution of the latent cluster labels and observations for a given set of parameters Φ . Similarly, we'll use $p(\vec{x}_1, \dots, \vec{x}_N \mid \Phi)$ to denote the marginal density of the observations given parameters Φ . Later, we'll consider fixing $\vec{x}_1, \dots, \vec{x}_N$ and treating $p(\vec{x}_1, \dots, \vec{x}_N \mid \Phi)$ as a likelihood for Φ . First, let's visualize data that can be generated by a GMM to demonstrate what shapes of clusters are possible for GMMs.

Example 6.2 (Samples from a GMM). Briefly, we'll demonstrate the type of data one can generate from a Gaussian mixture model. We focus on a two-dimensional case with $K = 3$ clusters. For the sampling process, we'll use means

$$\vec{\mu}_1 = \begin{bmatrix} 6 \\ 0 \end{bmatrix}, \vec{\mu}_2 = \begin{bmatrix} 0 \\ 3 \end{bmatrix}, \vec{\mu}_3 = \begin{bmatrix} -3 \\ 0 \end{bmatrix}$$

and covariance matrices

$$\Sigma_1 = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}, \Sigma_3 = \begin{bmatrix} 5 & -3 \\ -3 & 2 \end{bmatrix}$$

for the Gaussian densities. And we'll use probabilities $\vec{p} = (0.5, 0.3, 0.2)$ for the cluster label assignment. Below, we show 1000 samples from this GMM with points color-coded by sampled cluster label z_i .

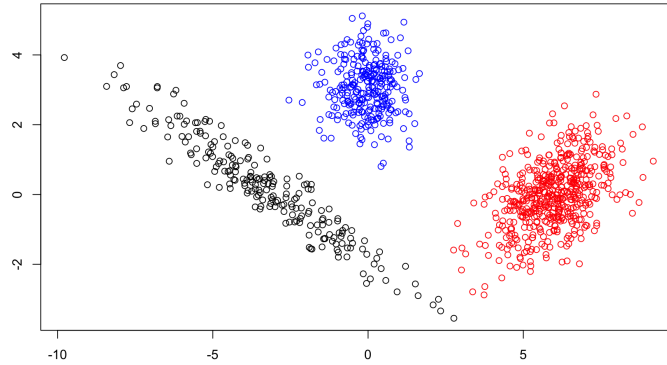
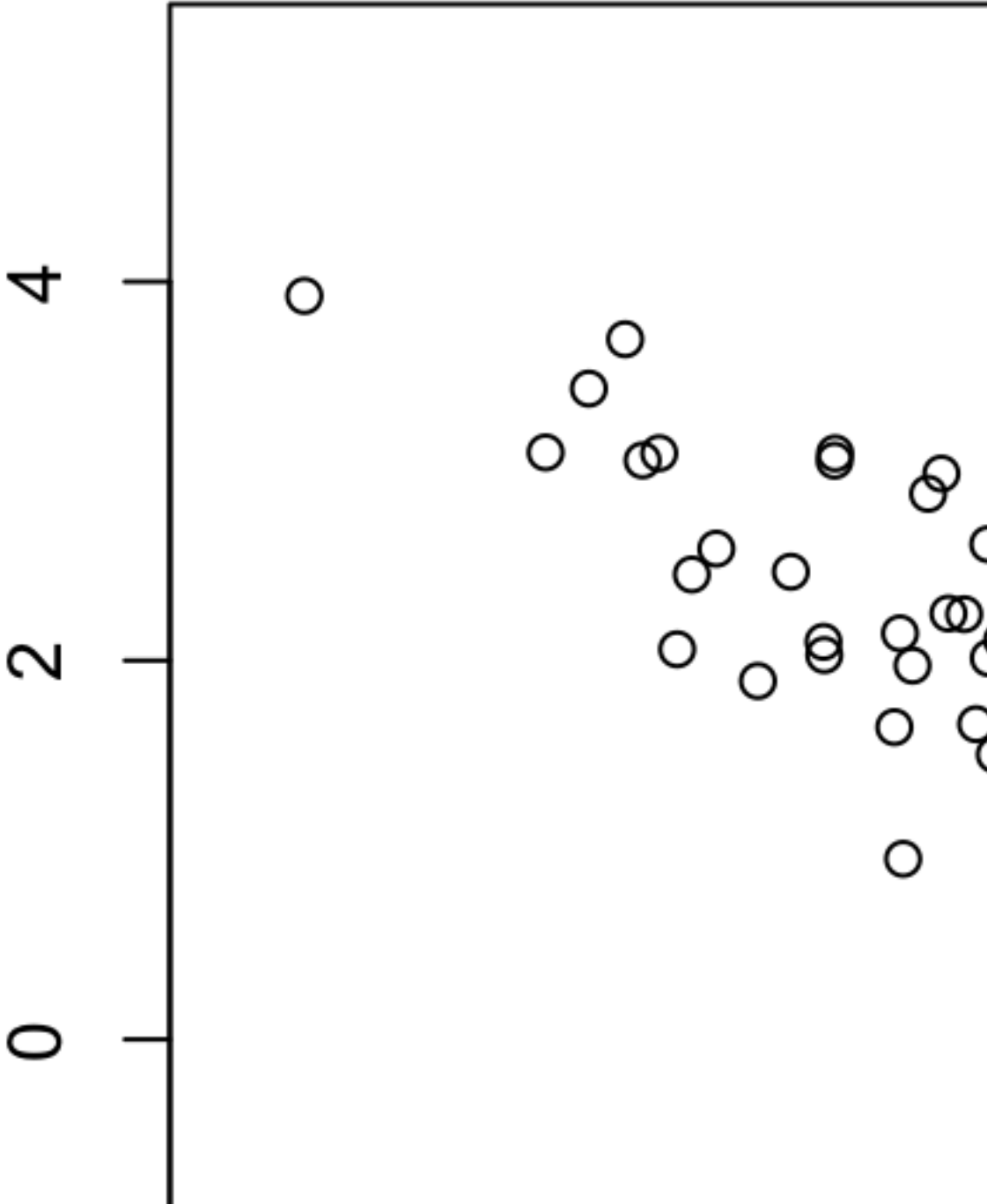


Figure 6.3: Samples from a GMM

In the preceding example, the clusters had an elliptical shape which is a distinct variation from k -means clustering which presumes spherical clusters. The location and shape/orientation of each cluster is controlled by its mean vector and covariance matrix respectively. As such, GMMs can still generate spherical clusters if the eigenvalues of each covariance matrix are (approximately) equal. This is a natural consequence of the Gaussian distribution and diagonalization of the covariance matrix. In high dimensions, Gaussian distributions concentrate near the surface of ellipsoids (rather than on the interior) but the general observation regarding the increased flexibility of GMMs over k -means holds true. Ignoring computational demands, one might expect GMMs to outperform k -means in many settings.

Suppose you were asked to assign a sample at the location marked with a purple x below.



Do you expect the x was generated from the same density as the points in the red cluster or from the density that generated the point in the black cluster? In center based clustering, this is a binary choice. However, it is unclear from the picture so perhaps that most honest answer is that there is a 50/50 chance that it was generated from either (we can likely agree there is vanishingly small chance it belongs to the blue cluster). The option to make such *fuzzy* clustering is a natural consequence of the probabilistic modeling approach behind GMMs. For a given parameter vector Φ we can compute a posterior $p(z_i|\vec{x}_i, \Phi)$ reflecting our belief about the clustering labeling.

Ultimately, clustering via GMMs requires estimation of the parameter Φ for a given model. If we had access to the cluster labels, $z_1, \dots, z_N$ this would be an easy problem. We could simply take sample means and covariances for the samples assigned to cluster j to estimate $\vec{\mu}_j$ and Σ_j and we could use the proportion of samples given cluster label j as our estimate for p_j . Mathematically, we would use the estimates

$$\hat{\mu}_j = \frac{\sum_{i=1}^N \mathbb{I}(Z_i = j) \vec{x}_i}{\sum_{i=1}^N \mathbb{I}(Z_i = j)}, \hat{\Sigma}_j = \frac{\sum_{i=1}^N \mathbb{I}(Z_i = j) (\vec{x}_i - \hat{\mu}_j)(\vec{x}_i - \hat{\mu}_j)^T}{\sum_{i=1}^N \mathbb{I}(Z_i = j)}, \hat{p}_j = \frac{\sum_{i=1}^N \mathbb{I}(z_i = j)}{N} (\#eq : simple) \quad (6.8)$$

where $\mathbb{I}(z_i = j)$ is the indicator function taking value one if $z_i = j$ and zero otherwise.

Unfortunately, we do not have access to $z_1, \dots, z_N$. (If we did, there would be no need to estimate the clusters.) So we need an algorithm for optimizing $p(\vec{x}_1, \dots, \vec{x}_N | \Phi)$ over Φ . One approach is to use a gradient based method to optimize the log-likelihood

$$\log p(\vec{x}_1, \dots, \vec{x}_N | \Phi) = \sum_{i=1}^N \log \left(\sum_{j=1}^K p_j \mathcal{N}(\vec{x}_i; \vec{\mu}_j, \Sigma_j) \right). \quad (6.9)$$

However, this method easily suffers from computation limitations (underflow) and additional care is required to enforce constraints on the parameters: the covariance matrices $\{\Sigma_j\}_{j=1}^K$ must be positive definite and the probabilities $p_1, \dots, p_K$ must be non-negative and sum to one. Instead, we'll use the powerful Expectation-Maximization (EM) algorithm which was named in the 1977 paper by Dempster, Laird, and Rubin [Dempster et al., 1977]. In addition to proving convergence to a local maxima in our GMM setting, that paper also discusses many use cases for EM beyond mixture models.

6.4.2 Expectation-Maximization (EM) Algorithm

EM can be opaque when one is first exposed to the method. We'll discuss the mathematical details shortly. For now, let's discuss the algorithm from a high level. It begins with an initial guess for Φ . The EM algorithm is comprised of two aptly named steps.

1. **E-Step:** We use our current guess for the model parameters to estimate the cluster labels $z_1, \dots, z_N$. These estimates are probabilistic in the sense that we compute $p(z_i | \vec{x}_i, \Phi)$ rather than discrete choices of the labels.
2. **M-Step:** Maximize the expected log-likelihood with respect to parameters, Φ , using the estimated label probabilities. In this step, we modify the estimates in `@ref(eq:simple_gmm_estimates)` to reflect our uncertainty in the cluster labels.

The EM algorithm then proceeds by iterating the E and M steps until convergence to a local maximum is achieved. Let's explore these steps in greater detail for GMMs. Let $\Phi^{(t)}$ to denote the estimate of Φ after t iterations of the E and M steps. We'll use the superscript $^{(t)}$ above individual parameters as well.

In the E-step, we want to find $p(z_i | \vec{x}_i, \Phi^{(t)})$ which follows from an application of Bayes' formula

$$\begin{aligned} p(z_i = j | \vec{x}_i, \Phi^{(t)}) &= \frac{p(\vec{x}_i, z_i = j | \Phi^{(t)})}{p(\vec{x}_i | \Phi^{(t)})} \\ &= \frac{p(\vec{x}_i | z_i = j, \Phi^{(t)})p(z_i = j | \Phi^{(t)})}{p(\vec{x}_i | \Phi^{(t)})} \\ &= \frac{p_j^{(t)} \mathcal{N}(\vec{x}_i; \vec{\mu}_j^{(t)}, \Sigma_j^{(t)})}{\sum_{\ell=1}^K p_\ell^{(t)} \mathcal{N}(\vec{x}_i; \vec{\mu}_\ell^{(t)}, \Sigma_\ell^{(t)})} \end{aligned}$$

We'll adopt the notation

$$\gamma_{ij}^{(t)} = \frac{p_j^{(t)} \mathcal{N}(\vec{x}_i; \vec{\mu}_j^{(t)}, \Sigma_j^{(t)})}{\sum_{\ell=1}^K p_\ell^{(t)} \mathcal{N}(\vec{x}_i; \vec{\mu}_\ell^{(t)}, \Sigma_\ell^{(t)})}$$

to compress our notation. Importantly, the quantities $\gamma_{ij}^{(t)}$ give a conditional distribution of $z_1, \dots, z_N$ given $\vec{x}_1, \dots, \vec{x}_N$ and $\Phi^{(t)}$

Now, let's turn to the handy form of the likelihood of the observations and the latent clusters labels in (6.6) which has associated log-likelihood

$$\log p(\vec{x}_1, z_1, \dots, \vec{x}_N, z_N | \Phi) = \sum_{i=1}^N \sum_{j=1}^K \mathbb{I}(z_i = j) \log(p_j \mathcal{N}(\vec{x}_i; \vec{\mu}_j, \Sigma_j)).$$

Treating $\vec{x}_1, \dots, \vec{x}_N$ and Φ as fixed, we take the expectation of the above expression w.r.t. the cluster label probabilities $\gamma_{ij}^{(t)}$. From the linearity of expectation, we have

$$\begin{aligned} Q(\Phi | \Phi^{(t)}) &= E_{z_1, \dots, z_N | \vec{x}_1, \dots, \vec{x}_N, \Phi^{(t)}} [\log p(\vec{x}_1, z_1, \dots, \vec{x}_N, z_N | \Phi)] \\ &= \sum_{i=1}^N \sum_{j=1}^K \gamma_{ij}^{(t)} \log(p_j \mathcal{N}(\vec{x}_i; \vec{\mu}_j, \Sigma_j)) \\ &= \sum_{i=1}^N \sum_{j=1}^K \gamma_{ij}^{(t)} \left[\log p_j - \frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma_j| - \frac{1}{2} (\vec{x}_i - \vec{\mu}_j)^T \Sigma_j^{-1} (\vec{x}_i - \vec{\mu}_j) \right] \end{aligned}$$

The indicators $\mathcal{I}(z_i = j)$ are replaced with the quantities $\gamma_{ij}^{(t)}$ when we take the expectation! Once we have computed $Q(\Phi \mid \Phi^{(t)})$ the E-step is complete.

For the M -step, we then optimize $Q(\Phi \mid \Phi^{(t)})$ over Φ so that

$$\Phi^{(t+1)} = \operatorname{argmax}_{\Phi} Q(\Phi \mid \Phi^{(t)}).$$

For a GMM, there are closed form expressions for the optimal values of parameters. One can compute them directly using partial derivatives (and enforcing the constraints). We omit the calculation here and cite the result instead

$$\begin{aligned} p_j^{(t+1)} &= \frac{\sum_{i=1}^N \gamma_{ij}^{(t)}}{\sum_{i=1}^N \sum_{\ell=1}^K \gamma_{i\ell}^{(t)}} = \frac{\sum_{i=1}^N \gamma_{ij}^{(t)}}{N} \\ \vec{\mu}_j^{(t+1)} &= \frac{\sum_{i=1}^N \gamma_{ij}^{(t)} \vec{x}_i}{\sum_{i=1}^N \gamma_{ij}^{(t)}} \\ \Sigma_j^{(t+1)} &= \frac{\sum_{i=1}^N \gamma_{ij}^{(t)} (\vec{x}_i - \vec{\mu}_j^{(t+1)}) (\vec{x}_i - \vec{\mu}_j^{(t+1)})^T}{\sum_{i=1}^N \gamma_{ij}^{(t)}} \end{aligned}$$

The above quantities resemble the simple estimates from @ref(eq:simple_gmm_estimates). Rather than taking average using the known labels, we take weighted estimates using the relative probability of vectors have associated cluster labels!

Once we have computed $\Phi^{(t+1)}$ when then return to the E-step and continuing iterating until convergence. Convergence of the EM algorithm to a local maximum follows directly from Jensen's inequality [Dempster et al., 1977]. Since the algorithm is only guaranteed to converge to a local maximum, one should typically investigate multiple initial conditions then select the parameter value among all optima that attains the highest log-likelihood $p(\vec{x}_1, \dots, \vec{x}_N \mid \Phi)$.

Given a final estimate Φ^* we can now determine a clustering of the data. Let

$$\gamma_{ij}^* = \frac{p_j^* \mathcal{N}(\vec{x}_i; \vec{\mu}_j^*, \Sigma_j^*)}{\sum_{\ell=1}^K p_{\ell}^* \mathcal{N}(\vec{x}_i; \vec{\mu}_{\ell}^*, \Sigma_{\ell}^*)}$$

denote our estimate cluster label probabilities for each observation given parameter Φ^* . Each γ_{ij}^* reflects our belief that observation i was generated by the j th cluster density. To make a discrete clustering, we then assign observation $\vec{x}_i$ to the cluster maximizing γ_{ij}^* .

6.4.3 Model Selection and Parameter Constraints

The EM-algorithm provides a method for a finding the optimal parameters and subsequent clustering for a fixed number of clusters K . If we wish to estimate

the number of clusters, we can use the maximized log-likelihood as a measure of the goodness of fit to the data. Let

$$\log L_k^* = \log p(\vec{x}_1, \dots, \vec{x}_N \mid \Phi^*)$$

denote the maximum log-likelihood assuming k clusters. Larger values of $\log L_k^*$ reflect a better fit to the observed data so it natural to expect $\log L_k^*$ to increase as k increases. To avoid overfitting, we use the Bayesian Information Criterion (BIC). Let $\mathcal{P}_k$ denote the number of parameters in a GMM for k clusters. BIC is defined as

$$BIC_k = 2 \log L_k^* - \mathcal{P}_k \log N$$

which balances goodness of fit with model complexity. Under this framework, we select the value of k which maximizes BIC. (Note, some sources define BIC as the negative of our formula above and select k opt to minimize. Be mindful of the convention when using pre-existing packages!)

We can use the BIC approach to investigate additional constraints on the model parameters. Recall from the 6.2, the shape of clusters is controlled by the covariance matrices. By applying addition constraints to $\Sigma_1, \dots, \Sigma_K$ we can reduce the number of model parameters. The standard convention for covariance constraints comes from Banfield and Raftery (1993) who observed that every covariance matrix can be decomposed via its eigenvalue decomposition

$$\Sigma_j = \lambda_j \mathbf{W}_j \mathbf{\Lambda}_j \mathbf{W}_j$$

where λ_j is a scalar parameter controlling the volume of the cluster, $\mathbf{W}_j$ is an orthonormal matrix controlling the orientation of the cluster, and $\mathbf{\Lambda}_j$ is a diagonal matrix with diagonal entries in the interval $(0, 1]$ giving the relative lengths of the semi-major axes of the ellipsoid [Banfield and Raftery, 1993]. We can apply constraints on one or more of the corresponding components of the covariance matrices summarized in the table below

	Volume	Shape	Orientation
Equal	$\lambda = \lambda_1 = \dots = \lambda_K$	$\Lambda = \Lambda_1 = \dots = \Lambda_K$	$W = W_1 = \dots W_K$
Variable	$\lambda_1, \dots, \lambda_K$ can differ	$\Lambda_1, \dots, \Lambda_K$ can differ	$W_1, \dots W_K$ can differ
Identity	Not applicable	$\Lambda_1 = \dots \Lambda_K = I$	$W_1 = \dots W_K = I$

We then use triplets of Equal, Variable, and Intity as shorthand for covariance constraints. For example, a GMM with covariance constraint EVE indicates covariances with equal volume, varying shape, and equal direction so that

$$\Sigma_j = \lambda \mathbf{W} \mathbf{\Lambda}_j \mathbf{W}^T.$$

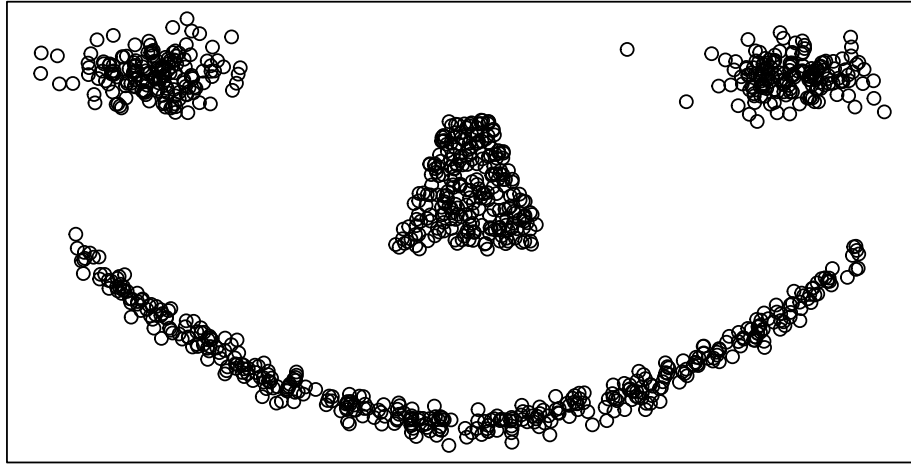
This results in a total of $2 \times 3 \times 3 - 4 = 14$ covariance options. We have subtracted 4 since some triplets coincide such as EII, EIE, and EIV.

For each covariance model, we can count the associated number of parameters and compute BIC accordingly. As a result, we have quantitative method for choosing the number of clusters and the optimal covariance model (for a given number of clusters). GMMs (with the help of the EM algorithm for fitting) provide a self-contained clustering framework with easy to interpret model selection criteria. However, there are inherent limitations which should be addressed.

6.4.4 Weaknesses and Limitations

Unlike center-based methods, GMMs allow for a greater level of flexibility in the shapes of clusters (ellipses vs spheres). However, ellipsoids may still be inadequate choices a cluster supported on or near a nonlinear manifold. For example, in the **smile** data below.

Example 6.3 (GMMs fail to capture complex nonlinear structure). Consider the following data in $\mathbb{R}^2$ which exhibits four clusters upon visual inspection



The eyes and nose are each compact and approximately elliptical in shape, but the mouth is concentrated around a nonlinear parabolic curve. Applying the GMM model above for a range $K = 2, \dots, 15$ clusters gives the following BIC curves

Using BIC, the optimal GMM one should select contains nine clusters with EVV covariance structure. In the associated clustering shown below, the eyes are appropriately clustered, but the nose is split into two separate clusters. Most egregiously though, we can see the mouth was broken apart into five thin elliptic pieces reflecting the inherent limitations of the cluster shapes that can be identified by GMMs.

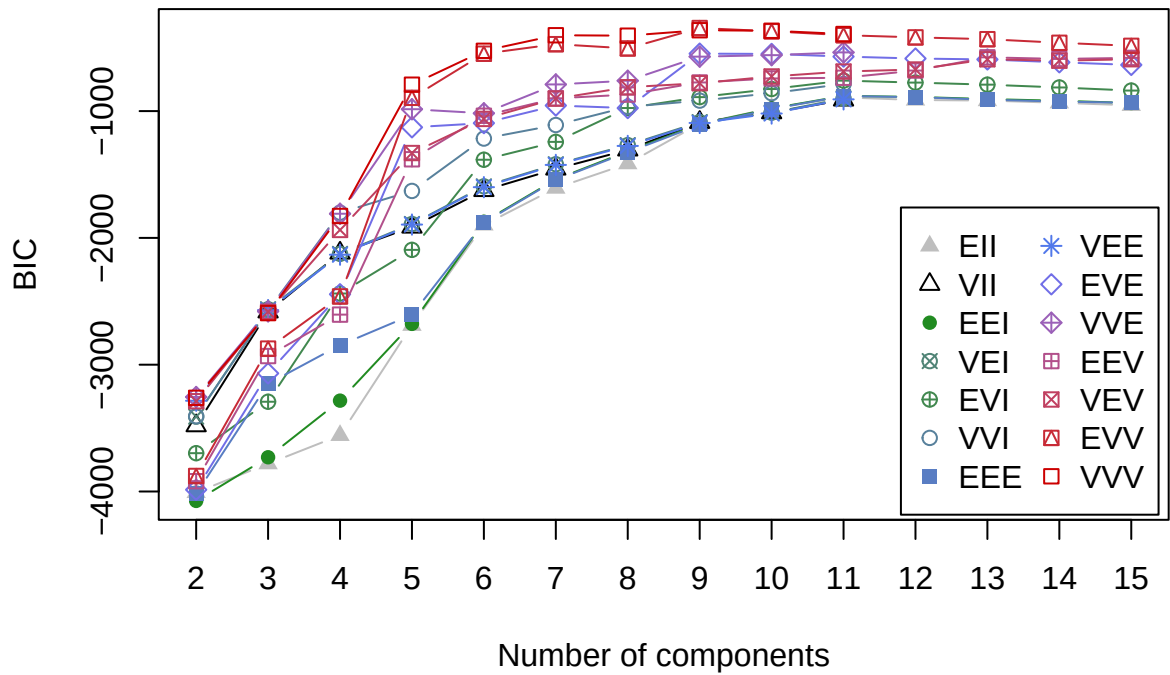
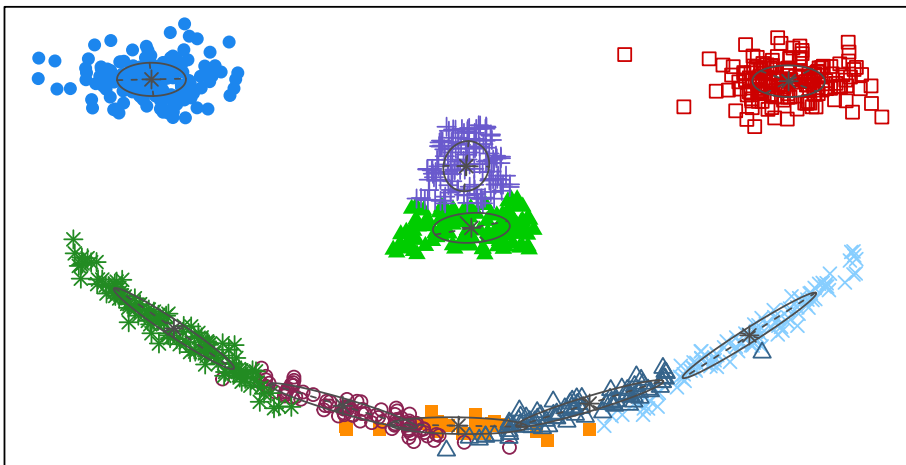


Figure 6.4: BIC curves for GMMs applied to the smile data

```
plot(smile.gmm, what = "classification",
     xaxt='n', yaxt='n',
     xlab = '', ylab = '')
```



Given this limitation, there has been considerable work dedicated to improving model-based clustering including the design of more expressive mixture densities

[Karlis and Santourian [2009]; O’Hagan et al. [2016]; Dang et al. [2023]] and methods which produce more parsimonious clustering arrangements by iteratively merging nearby clusters [Dombowsky and Dunson, 2025].

6.5 Spectral Clustering

6.5.1 Introduction

Spectral Clustering represents a significant leap in the evolution of clustering techniques. Distinguished from traditional methods like K-means, it excels in detecting complex structures and patterns within data. It’s based on ideas from graph theory and simple math concepts, mainly focusing on how to use information from graphs. Imagine each piece of data as a point on a graph, with lines connecting the points that are similar. Spectral Clustering uses these connections to figure out how the data should be grouped, which is especially handy when the groups are twisted or oddly shaped.

The key step in Spectral Clustering is breaking down a special graph matrix (called the Laplacian matrix) to find its eigen values and eigen vectors. These eigen vectors help us see the data in a new way that makes the groups more obvious. This makes it easier to use simple grouping methods like K-means to sort the data into clusters. This approach is great for finding hidden patterns in the data.

However, Spectral Clustering comes with its own challenges. Choosing the right number of groups can be tricky, and it might not work as well with very large sets of data because of the relatively large computing cost. Nevertheless, its robustness and adaptability have cemented its role across various domains, from image processing to bioinformatics.

6.5.2 Algorithm

- 1) **Similarity Graph Construction:** Start by constructing a similarity graph G from your data (similar to the steps in the Laplacian Eigenmap section). Each data point is represented as a node in the graph.

Define the edges of the graph. There are three common ways to do this:

- i) **ϵ -neighborhood graph:** Connect all points whose pairwise distances are smaller than ϵ .
- ii) **K-nearest neighbors:** For each point, connect it to its k nearest neighbors.
- iii) **Fully connected graph:** Connect all points with each other. Typically, the Gaussian similarity function (also known as the Radial Basis Function or RBF) is used to calculate the weights of the edges:

$w_{ij} = \exp(-\frac{\|\vec{x}_i - \vec{x}_j\|^2}{2\sigma^2})$, where $\vec{x}_i$ and $\vec{x}_j$ are two points in the dataset and σ is a tuning parameter.

Note: It is worth noting that there exists a slight difference in the construction of similarity graph matrix compared to Laplacian Eigenmap we mentioned in manifold learning chapter. For the fully connected graph, after using Radial basis to depict all the pair-wise distances, we don't need to set a threshold and sparsify the matrix (set some entries to zero) like we did in Laplacian Eigenmap, we just keep all the original radial basis distances.

- 2) **Graph Laplacian Matrix:** Similar to corresponding parts in Laplacian Eigenmap. Calculate the adjacency matrix W , where W_{ij} represents the weight of the edge between nodes i and j . Calculate the degree matrix D , which is a diagonal matrix where each diagonal element D_{ii} is the sum of the weights of the edges connected to node i . Compute the unnormalized Graph Laplacian matrix L as $L = D - W$.
- 3) **Eigen Decomposition:** Perform the eigen decomposition on the Laplacian matrix L to find its eigenvalues and eigenvectors. Select k smallest eigenvalues and their corresponding eigen vectors. k is the number of clusters you want to identify.

Mathematical Proof behind this step

As we have stated in Laplacian Eigenmap section, the Graph Laplacian matrix L is positive semi-definite.

Given any vector $\vec{y} \in \mathbb{R}^N$

$$\vec{y}^T \mathbf{L} \vec{y} = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \mathbf{W}_{ij} (y_i - y_j)^2$$

Obviously, it is non-negative. Besides, we can always find a vector $\vec{y} = \mathbf{1}_N$ that makes it zero, which means the smallest eigen value must be zero, with the corresponding eigen vector being $\mathbf{1}$.

From the above equation, we can also justify our eigen-decomposition approach in finding the number of clusters. For either ϵ -neighborhood graph or K-nearest neighbors approach, if point i and j are not connected, then $\mathbf{W}_{ij} = 0$, however, if they are connected, $\mathbf{W}_{ij} = 1 > 0$. With some careful observation, we find that as long as we set $y_i = y_j$ for $\forall \mathbf{W}_{ij} > 0$, then we are able to get zero in the above equation. Since in cluster $\Omega_1 = \{\vec{x}_p, \dots, \vec{x}_q\}$, all the points are connected, and $\mathbf{W}_{ij} > 0 \forall \{\vec{x}_i, \vec{x}_j\} \in \Omega_1$, we can simply set $y_i = 1$ for $\forall i \in \Omega_1$, and $y_j = 0$ for $\forall j \notin \Omega_1$. So the eigen-vector that corresponds to cluster Ω_1 is $\vec{y}_1 = \sum \vec{e}_i \in \mathbb{R}^N$, $\forall i$ s.t. $\vec{x}_i \in \Omega_1$, where $\vec{e}_i$ is a vector with all zero except the i^{th} entry being one.

So when we perform eigen decomposition on Graph Laplacian matrix $\mathbf{L}$: $\mathbf{L} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$. Then for $\vec{y}$ s.t. $\vec{y}^T \mathbf{L} \vec{y} = 0$, we can rewrite it as

$$\vec{y}^T \mathbf{L} \vec{y} = \vec{y}^T \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T \vec{y} = (\mathbf{\Lambda}^{1/2} \mathbf{Q}^T \vec{y})^T (\mathbf{\Lambda}^{1/2} \mathbf{Q}^T \vec{y}) = 0$$

As a result, we know $\mathbf{\Lambda}^{1/2} \mathbf{Q}^T \vec{y} = \vec{0}$, in other words

$$\mathbf{L} \vec{y} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T \vec{y} = (\mathbf{Q} \mathbf{\Lambda}^{1/2}) (\mathbf{\Lambda}^{1/2} \mathbf{Q}^T \vec{y}) = \vec{0}$$

So we know that $\vec{y}$ is just an eigen-vector of $\mathbf{L}$, with the corresponding eigen-value being zero.

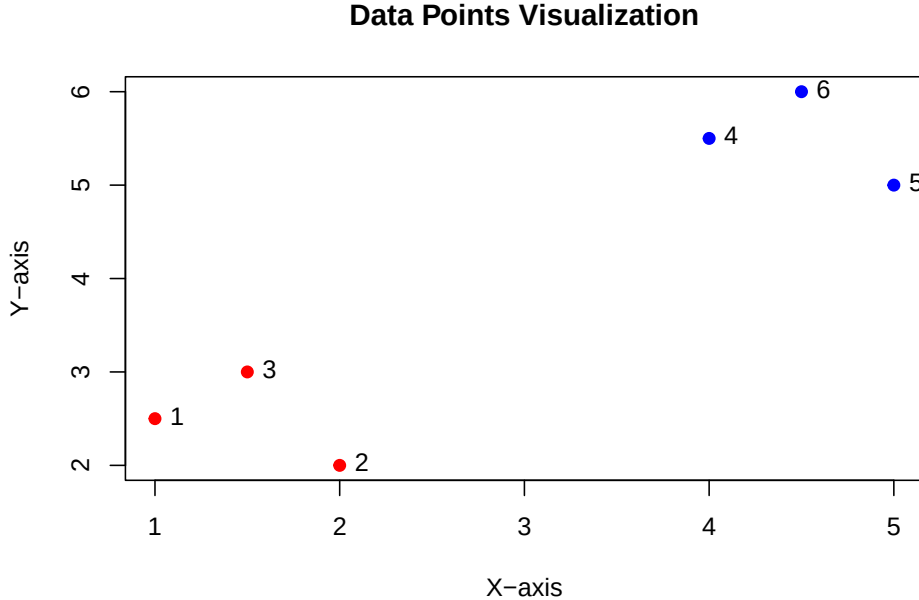
In reality, especially when we use Fully-connected graph, we can't get k exact zero-eigenvalues with corresponding k eigenvectors. (Different clusters are not necessarily completely separate, and Fully-connected graph even allows every $\mathbf{W}_{ij} > 0$). So we will just conduct eigen decomposition and choose k smallest eigenvalues together with their corresponding eigen-vectors.

A toy example

First, we'll create the simulation data with two distinct clusters.

Visualize the data points to make it more intuitive.

```
plot(data, col = c(rep("red", nrow(cluster1)), rep("blue", nrow(cluster2))), pch = 19,
text(data, labels = 1:nrow(data), pos = 4, col = "black") # Adding labels
title("Data Points Visualization")
```



We use the ϵ -neighborhood approach to construct the similarity graph.

```
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]  0   1   1   0   0   0
```

```
## [2,] 1 0 1 0 0 0
## [3,] 1 1 0 0 0 0
## [4,] 0 0 0 0 1 1
## [5,] 0 0 0 1 0 1
## [6,] 0 0 0 1 1 0
```

Compute the Laplacian matrix.

```
degree_matrix <- diag(apply(similarity_matrix, 1, sum))
laplacian_matrix <- degree_matrix - similarity_matrix
print(laplacian_matrix)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,] 2 -1 -1 0 0 0
## [2,] -1 2 -1 0 0 0
## [3,] -1 -1 2 0 0 0
## [4,] 0 0 0 2 -1 -1
## [5,] 0 0 0 -1 2 -1
## [6,] 0 0 0 -1 -1 2
```

Perform eigen decomposition on the Laplacian matrix. We choose the smallest two eigenvalues here since we want to identify two clusters.

```
eigen_result <- eigen(laplacian_matrix)

# Sort eigenvalues and their corresponding eigenvectors
sorted_indices <- order(eigen_result$values)
sorted_eigenvalues <- eigen_result$values[sorted_indices]
sorted_eigenvectors <- eigen_result$vectors[, sorted_indices]

# Select the smallest two eigenvalues and their corresponding eigenvectors
smallest_eigenvalues <- sorted_eigenvalues[1:2]
smallest_eigenvectors <- sorted_eigenvectors[, 1:2]
print(smallest_eigenvalues)
```

```
## [1] 1.776357e-15 1.776357e-15
```

```
print(smallest_eigenvectors)
```

```
##      [,1] [,2]
## [1,] 0.0000000 -0.5773503
## [2,] 0.0000000 -0.5773503
## [3,] 0.0000000 -0.5773503
## [4,] -0.5773503 0.0000000
## [5,] -0.5773503 0.0000000
## [6,] -0.5773503 0.0000000
```

We find that the smallest two eigen-values are 0 (not exact zero here because of computational precision issue), and their corresponding eigen-vectors give us

information about the clustering. The first cluster contains data point 4, 5, 6; while the second cluster contains the rest three data points 1, 2, 3.

We try **fully-connected graph** with radial basis function to construct the adjacency matrix this time.

```
gamma <- 1 # Scale parameter for the RBF kernel
n <- nrow(data)
similarity_matrix <- matrix(0, n, n)

for (i in 1:n) {
  for (j in 1:n) {
    if (i != j) {
      distance <- dist(rbind(data[i, ], data[j, ]))^2
      similarity_matrix[i, j] <- exp(-gamma * distance)
    }
  }
}

print(similarity_matrix)
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 0.000000e+00 2.865048e-01 6.065307e-01 1.522998e-08 2.172440e-10
## [2,] 2.865048e-01 0.000000e+00 2.865048e-01 8.764248e-08 1.522998e-08
## [3,] 6.065307e-01 2.865048e-01 0.000000e+00 3.726653e-06 8.764248e-08
## [4,] 1.522998e-08 8.764248e-08 3.726653e-06 0.000000e+00 2.865048e-01
## [5,] 2.172440e-10 1.522998e-08 8.764248e-08 2.865048e-01 0.000000e+00
## [6,] 2.289735e-11 2.172440e-10 1.522998e-08 6.065307e-01 2.865048e-01
##           [,6]
## [1,] 2.289735e-11
## [2,] 2.172440e-10
## [3,] 1.522998e-08
## [4,] 6.065307e-01
## [5,] 2.865048e-01
## [6,] 0.000000e+00
```

```
degree_matrix <- diag(apply(similarity_matrix, 1, sum))
laplacian_matrix <- degree_matrix - similarity_matrix
print(laplacian_matrix)
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 8.930355e-01 -2.865048e-01 -6.065307e-01 -1.522998e-08 -2.172440e-10
## [2,] -2.865048e-01 5.730097e-01 -2.865048e-01 -8.764248e-08 -1.522998e-08
## [3,] -6.065307e-01 -2.865048e-01 8.930393e-01 -3.726653e-06 -8.764248e-08
## [4,] -1.522998e-08 -8.764248e-08 -3.726653e-06 8.930393e-01 -2.865048e-01
## [5,] -2.172440e-10 -1.522998e-08 -8.764248e-08 -2.865048e-01 5.730097e-01
## [6,] -2.289735e-11 -2.172440e-10 -1.522998e-08 -6.065307e-01 -2.865048e-01
##           [,6]
```

```
## [1,] -2.289735e-11
## [2,] -2.172440e-10
## [3,] -1.522998e-08
## [4,] -6.065307e-01
## [5,] -2.865048e-01
## [6,]  8.930355e-01

eigen_result <- eigen(laplacian_matrix)

# Sort eigenvalues and their corresponding eigenvectors
sorted_indices <- order(eigen_result$values)
sorted_eigenvalues <- eigen_result$values[sorted_indices]
sorted_eigenvectors <- eigen_result$vectors[, sorted_indices]

# Select the smallest two eigenvalues and their corresponding eigenvectors
smallest_eigenvalues <- sorted_eigenvalues[1:2]
smallest_eigenvectors <- sorted_eigenvectors[, 1:2]
print(smallest_eigenvalues)

## [1] 2.564067e-16 2.632047e-06

print(smallest_eigenvectors)

##           [,1]      [,2]
## [1,] -0.4082483  0.4082488
## [2,] -0.4082483  0.4082494
## [3,] -0.4082483  0.4082467
## [4,] -0.4082483 -0.4082467
## [5,] -0.4082483 -0.4082494
## [6,] -0.4082483 -0.4082488
```

As we can see, this time the two smallest eigenvalues are not both zero, with one being a little bit more than zero. This has something to do with the properties of the new adjacency matrix $\mathbf{W}$. In addition, we observe that the two eigen-vectors are not something we expected. It seems weird at first glance, but since $\mathbf{L}\vec{y} = \vec{0}$, $\vec{y} = \vec{y}_2 \pm \vec{y}_1$ is also an eigen-vector with the eigen-value being zero, we are still able to recover the two clusters. This suggests that we may need to do some computation ourselves after getting the eigen-vectors to recover the clustering situation.

In this situation, it may seem that fully-connected graph is not as straightforward as the other two adjacency matrix construction methods, and the result is also not as optimal. But we should realize that in real-data situations, different clusters are not totally separate, as a result, a soft-threshold can be a better choice in most situations.

6.6 Density-Based Spatial Clustering (DBSCAN)

6.6.1 Introduction

Every clustering method we have discussed so far commits to a particular geometric picture of what a cluster looks like. Center-based methods look for clusters shaped like balls (or, for GMMs, ellipsoids) around a small number of centers; hierarchical methods build clusters by repeatedly merging whichever pair of points or subgroups is closest; spectral clustering and kernel k -means look for clusters that are well separated by cuts in a similarity graph. DBSCAN (Density-Based Spatial Clustering of Applications with Noise), introduced by Ester et al. [1996], commits to a different picture entirely: a cluster is a region of high point density, separated from other clusters by regions of low density, and a cluster's shape is whatever shape that dense region happens to be. This makes DBSCAN a natural fit for exactly the kind of non-convex structure (nested rings, elongated arcs, and other shapes with no meaningful “center”) that causes center-based methods to fail.

DBSCAN also breaks from the other methods we've seen in a second, equally important way: it does not assign every point to a cluster. Points sitting in low-density regions are explicitly labeled **noise** and left out of every cluster entirely, rather than being forced into whichever cluster happens to be nearest. Together with never having to specify k in advance, this makes DBSCAN especially useful for exploratory analysis of data one suspects contains real outliers.

6.6.2 Algorithm

DBSCAN's entire notion of a cluster is built from just two ingredients: a radius ϵ and a minimum count minPts . Given these, every point in the data is classified into one of three types.

- 1) **Core point:** $\vec{x}_i$ is a core point if its ϵ -neighborhood $N_\epsilon(\vec{x}_i) = \{\vec{x}_j : \|\vec{x}_i - \vec{x}_j\| \leq \epsilon\}$ contains at least minPts points (including $\vec{x}_i$ itself). Core points sit in the interior of a dense region.
- 2) **Border point:** $\vec{x}_i$ is a border point if it is not itself a core point, but $N_\epsilon(\vec{x}_i)$ contains a core point. Border points sit on the edge of a dense region.
- 3) **Noise point:** any point that is neither a core point nor a border point. Noise points are not close enough to any dense region to belong to one.

Clusters are then built directly out of the core points using two linked notions of reachability.

Definition 6.2 (Density-Reachability and Density-Connectivity). A point $\vec{x}_j$ is **directly density-reachable** from a core point $\vec{x}_i$ if $\vec{x}_j \in N_\epsilon(\vec{x}_i)$. More generally, $\vec{x}_j$ is **density-reachable** from $\vec{x}_i$ if there is a chain of core points

$\vec{x}_i = \vec{x}_{(1)}, \vec{x}_{(2)}, \dots, \vec{x}_{(m)} = \vec{x}_j$ such that each $\vec{x}_{(\ell+1)}$ is directly density-reachable from $\vec{x}_{(\ell)}$. Two points $\vec{x}_i$ and $\vec{x}_j$ are **density-connected** if there is some core point $\vec{x}_k$ from which both are density-reachable.

A DBSCAN cluster is then a maximal set of density-connected points – geometrically, think of every core point as an overlapping ball of radius ϵ ; a cluster is exactly the (possibly very elongated or curved) region swept out by chaining these overlapping balls together, plus whatever border points happen to fall along its edge. Since this chain can bend, it is what allows a single DBSCAN cluster to trace out an arbitrarily shaped, non-convex region, in sharp contrast to the balls and ellipsoids that center-based methods are restricted to.

- 1) **Choose ϵ and minPts.** Every point is not yet visited.
- 2) **Classify and expand.** Pick an unvisited point $\vec{x}_i$ and compute $N_\epsilon(\vec{x}_i)$.
 - i) If $|N_\epsilon(\vec{x}_i)| < \text{minPts}$, temporarily label $\vec{x}_i$ as noise (it may later be reclaimed as a border point).
 - ii) Otherwise, $\vec{x}_i$ is a core point and starts a new cluster. Add every point in $N_\epsilon(\vec{x}_i)$ to the cluster, and for each of those points that is *itself* a core point, recursively add its ϵ -neighborhood as well – exactly the chaining process in Definition 6.2.
- 3) **Repeat** step 2 until every point has been visited. Any point never reached by this expansion remains permanently labeled as noise.

Notice that k , the number of clusters, is never specified – it falls out naturally as the number of separate dense regions the data happens to contain. The real tuning parameters are ϵ and minPts, and the two trade off against each other: raising minPts demands denser evidence before calling a point core, while raising ϵ makes it easier for nearby dense regions to bridge together into a single cluster. A common rule of thumb, used below, is to fix minPts (often around twice the data dimension) and choose ϵ by plotting each point’s distance to its minPts-th nearest neighbor in increasing order and looking for a “knee” – a sharp jump marking the transition from typical within-cluster spacing to the much larger gaps between clusters.

Example 6.4 (DBSCAN applied to Nested Rings with Noise). Let’s return to the nested rings from the Kernel k -means section (6.2), a shape that defeats ordinary k -means precisely because neither ring is convex. This time, we scatter a handful of uniformly random noise points across the plane as well, to see how DBSCAN handles points that don’t belong to either ring at all.

To choose ϵ , we plot the distance from each point to its 5th nearest neighbor.

```
kNNdistplot(data, minPts = 5)
abline(h = 0.5, col = "red", lty = 2)
```

The knee near $\epsilon = 0.5$ separates the small distances typical of points inside a ring from the much larger distances of the scattered noise, so we use $\epsilon = 0.5$ and

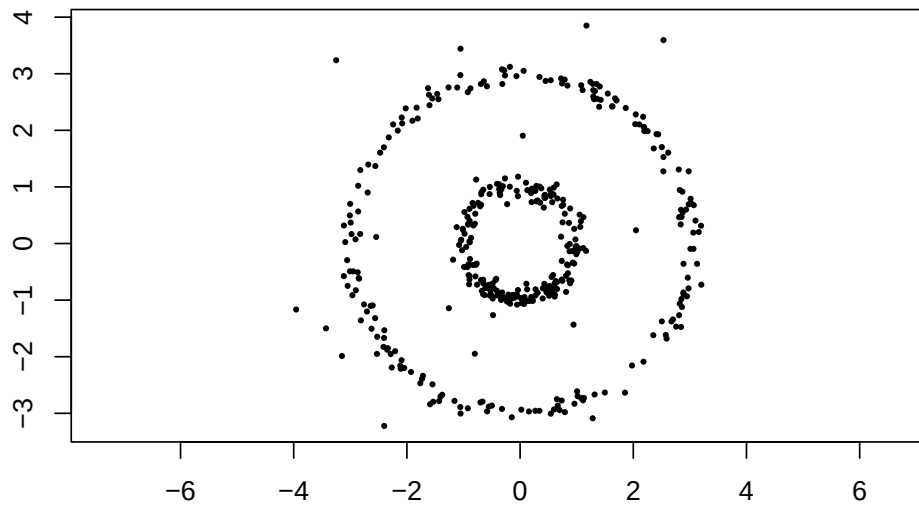


Figure 6.5: Nested rings with scattered noise points

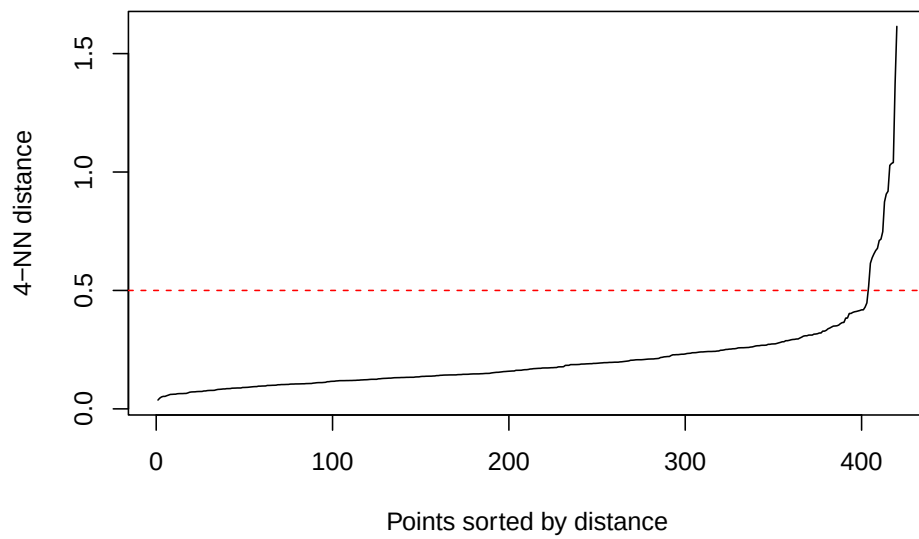


Figure 6.6: kNN distance plot used to choose epsilon

minPts = 5.

```
out <- dbscan(data, eps = 0.5, minPts = 5)
plot(data, col = out$cluster + 1, pch = ifelse(out$cluster == 0, 1, 16),
      xlab = "", ylab = "", asp = 1)
```

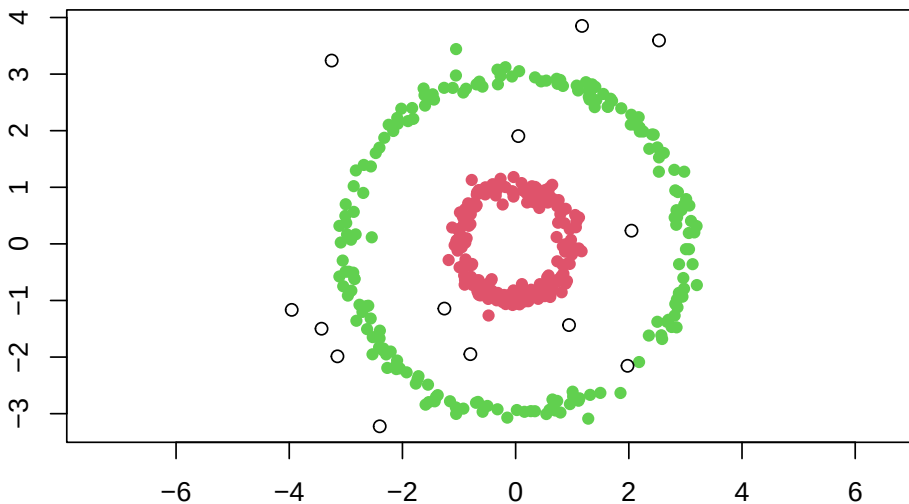


Figure 6.7: DBSCAN clustering of the nested rings; open circles are labeled noise

DBSCAN recovers both rings exactly, correctly refusing to merge them into a single cluster despite sharing the same center, and it labels essentially all of the scattered points as noise (open circles) rather than forcing them into whichever ring happens to be closest. Kernel k -means (6.1) can also separate the two rings, but only by first mapping the data into a feature space where they become linearly separable; DBSCAN reaches the same partition by reasoning about density directly in the original coordinates, and gets the noise points “for free” as a byproduct of that reasoning.

6.6.3 Strengths and Weaknesses

6.6.3.1 Strengths

1) No assumption on cluster shape

Because clusters are built by chaining overlapping neighborhoods rather than measuring distance to a center, DBSCAN can recover clusters of essentially any shape, including the non-convex rings above.

2) The number of clusters is discovered, not specified

Unlike k -means, hierarchical clustering, and GMMs, DBSCAN never requires k as an input; the number of clusters emerges from the density structure of the

data itself.

3) Built-in outlier detection

Noise points are identified automatically as part of the clustering process, rather than requiring a separate outlier-detection step beforehand.

6.6.3.2 Weaknesses

1) A single global ϵ struggles with varying density

DBSCAN uses one ϵ (and minPts) for the entire dataset. If some true clusters are much denser than others, a single ϵ that works well for the dense cluster may be too small to connect the sparse one, splitting it into fragments or noise – the same shared-bandwidth issue we saw motivate UMAP’s *local* bandwidth calibration (5.9), here appearing as a limitation rather than a solved problem.

2) Sensitive to the choice of ϵ and minPts

Small changes to ϵ can noticeably change which points are labeled noise versus border, and whether two nearby clusters merge into one.

3) No natural cluster representative

Because clusters are not built around a center, DBSCAN provides no direct analog of a cluster mean or medoid, which some downstream tasks may expect.

4) Struggles in high dimensions

Like other methods relying on local neighborhoods (5.9, 6.5), the notion of a meaningful density estimate degrades as dimension grows, since pairwise distances tend to concentrate around a common value in high dimensions, leaving little contrast between “close” and “far” for a fixed ϵ to distinguish.

6.7 Cluster Validation

Throughout this chapter, we’ve seen several ways to judge a clustering using only the data itself: the elbow plot and Gap statistic (which compare within-cluster variation to what we’d expect under no clustering structure at all), the silhouette score (6.1.5.3) (which measures how much better each point fits its own cluster than the next closest one), and for hierarchical clustering specifically, the cophenetic correlation coefficient (Definition 6.1). These are all **internal** validation methods: they never use any information beyond the data $\vec{x}_1, \dots, \vec{x}_N$ and the clustering itself.

Occasionally, we have access to a second source of information: a known grouping of the same data that we’d like our clustering to recover. This could be a set of true class labels available for benchmarking purposes (as in the example below), a previously published clustering we want to reproduce or compare against, or even the output of a second clustering algorithm we’d like to compare to a

first. When such a reference partition is available, we can measure how well our clustering agrees with it directly, rather than relying on an internal proxy for quality. This is called **external** validation.

It is worth emphasizing why this differs from supervised learning. We are not using the labels to fit the clustering itself, nor could we in most genuinely unsupervised applications where no such labels exist. Instead, external validation is primarily a tool for methodology: comparing algorithms against each other on benchmark data, sanity-checking a new method, or – as in the example below – better understanding what a clustering has and has not recovered when a partial or imperfect reference grouping happens to be available.

6.7.1 The Rand Index and Adjusted Rand Index

Suppose we have two partitions of the same N samples into groups: a clustering $C = \{C_1, \dots, C_k\}$ we've produced and a reference partition $C' = \{C'_1, \dots, C'_{k'}\}$ we'd like to compare it to (the number of groups k and k' need not match). The most natural way to compare two partitions is to ask, for every pair of samples, whether the two partitions agree on whether that pair belongs together.

Definition 6.3 (Rand Index). For each of the $\binom{N}{2}$ pairs of samples $\{\vec{x}_i, \vec{x}_j\}$, $i < j$, say the pair is

- **concordant** if $\vec{x}_i$ and $\vec{x}_j$ are in the same group under both C and C' , or in different groups under both C and C' ;
- **discordant** otherwise, i.e. the two partitions disagree on whether $\vec{x}_i$ and $\vec{x}_j$ belong together.

Letting a denote the number of concordant pairs and $\binom{N}{2}$ the total number of pairs, the *Rand Index* is

$$RI(C, C') = \frac{a}{\binom{N}{2}} \in [0, 1].$$

The Rand Index is easy to interpret – it is simply the fraction of all pairs of samples on which the two partitions agree – but it has an unfortunate flaw. Even two *independent, random* partitions of the data tend to agree on a large fraction of pairs, especially when k or k' is large, since most pairs of samples end up in different groups under both partitions purely by chance and are counted as concordant. As a result, the Rand Index is typically far from zero even when the clustering has learned nothing about the reference partition, and it is not straightforward to compare Rand Index values across different choices of k .

To fix this, we adjust the Rand Index so that its expected value under a random partition (with the same cluster sizes as C and C') is exactly zero.

Definition 6.4 (Adjusted Rand Index). Let n_{ij} denote the number of samples in both C_i and C'_j , let $n_{i\cdot} = \sum_j n_{ij}$ be the size of C_i , and let $n_{\cdot j} = \sum_i n_{ij}$ be

the size of C'_j . The *Adjusted Rand Index* is

$$ARI(C, C') = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \left[\sum_i \binom{n_{i\cdot}}{2} \sum_j \binom{n_{\cdot j}}{2} \right] / \binom{N}{2}}{\frac{1}{2} \left[\sum_i \binom{n_{i\cdot}}{2} + \sum_j \binom{n_{\cdot j}}{2} \right] - \left[\sum_i \binom{n_{i\cdot}}{2} \sum_j \binom{n_{\cdot j}}{2} \right] / \binom{N}{2}}.$$

While the formula above looks intimidating, the interpretation is simple: $ARI(C, C') = 1$ when the two partitions agree perfectly (up to relabeling of the groups), $ARI(C, C') \approx 0$ when C is no better than a random partition with the same cluster sizes, and $ARI(C, C')$ can be (mildly) negative when C agrees with C' *less* than a random partition would. Crucially, unlike the Rand Index, values of the ARI are comparable across different numbers of clusters k and different sample sizes N , which makes it the more commonly reported of the two in practice.

6.7.2 Normalized Mutual Information

A second, quite different way to compare two partitions comes from information theory rather than pairwise agreement. If we think of a random sample's cluster membership under C as a random variable $\mathcal{C}$ and its group membership under C' as a random variable $\mathcal{C}'$, we can ask how much knowing $\mathcal{C}$ tells us about $\mathcal{C}'$.

Definition 6.5 (Normalized Mutual Information). Using the notation of Definition 6.4, let $p_{ij} = n_{ij}/N$, $p_{i\cdot} = n_{i\cdot}/N$, and $p_{\cdot j} = n_{\cdot j}/N$ denote the empirical joint and marginal distributions of $(\mathcal{C}, \mathcal{C}')$. The *mutual information* between C and C' is

$$MI(C, C') = \sum_{i,j} p_{ij} \log \left(\frac{p_{ij}}{p_{i\cdot} p_{\cdot j}} \right),$$

and letting $H(C) = -\sum_i p_{i\cdot} \log p_{i\cdot}$ and $H(C') = -\sum_j p_{\cdot j} \log p_{\cdot j}$ denote the (empirical) entropies of C and C' , the *Normalized Mutual Information* is

$$NMI(C, C') = \frac{2 MI(C, C')}{H(C) + H(C')} \in [0, 1].$$

The mutual information $MI(C, C')$ measures (in nats, if using the natural logarithm) how much the uncertainty in a sample's group under C' is reduced once we know its group under C ; it is zero exactly when $\mathcal{C}$ and $\mathcal{C}'$ are independent. Dividing by the average of the two entropies rescales MI to lie in $[0, 1]$: $NMI(C, C') = 0$ when the partitions share no information and $NMI(C, C') = 1$ when each partition is a deterministic function of the other (in particular, when they agree perfectly). Unlike the Rand Index, mutual information is already comparatively well behaved under random partitions, but it – like the Rand Index – has no built-in preference for a particular k , so it should still be compared across values of k with some caution, a point the example below illustrates directly.

6.7.3 Worked Example: Validating Clusterings of Tumor Gene Expression Data

Let's put both of these tools to use on a genuinely messy, real dataset: the NCI60 microarray data compiled in Hastie et al. [2001]. The data record the expression levels of 6,830 genes ($d = 6830$) measured via microarray for $N = 64$ human tumor samples. Each tumor's expression level for a given gene is reported as a log-ratio relative to a reference sample, so entries of the data matrix $\mathbf{X} \in \mathbb{R}^{64 \times 6830}$ can be positive or negative. Each of the 64 tumor samples also comes with a label recording the type of cancer it was taken from – BREAST, COLON, LEUKEMIA, RENAL, and so on – giving us 14 labels to use as an external reference partition. A handful of these labels (e.g. K562A-repro and K562B-repro) are technical replicates of the same cell line rather than biologically distinct cancer types, so we should not expect even a great clustering to recover exactly 14 well-separated groups.

Before clustering, it's worth checking whether the genes (columns of $\mathbf{X}$) are on comparable scales.

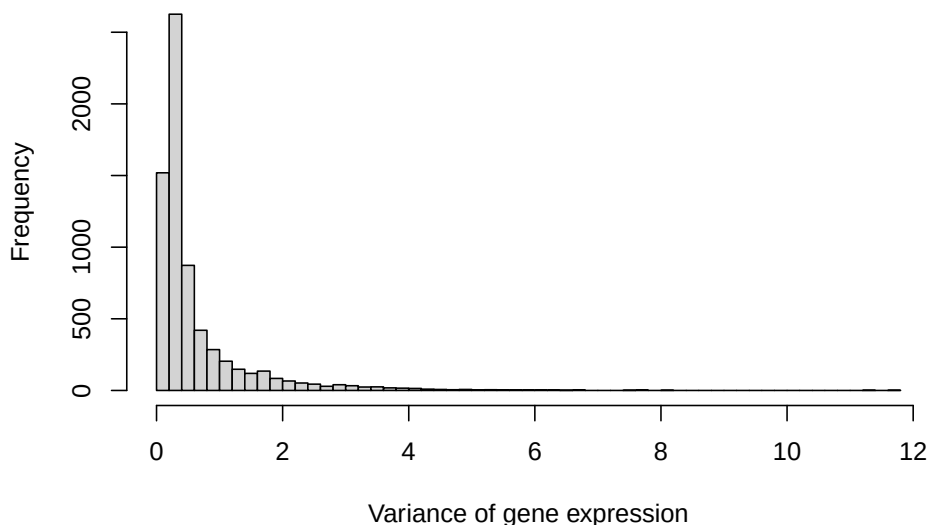


Figure 6.8: Variance of expression level across the 6,830 genes in the NCI60 data

The variances span nearly a 300-fold range. Since every clustering method we've discussed relies on Euclidean (or a closely related) distance, genes with the largest raw variance would otherwise dominate every pairwise distance calculation regardless of their actual biological relevance. We'll standardize each gene to have mean zero and variance one before clustering, matching the convention used elsewhere in this book when features are not already on a common scale.

Now let's cluster the (standardized) data using a few methods we've covered in this chapter – k -means, and hierarchical clustering with both average and

Table 6.2: Internal (Silhouette) vs. external (ARI, NMI) validation on the NCI60 data

k	Method	Silhouette	ARI	NMI
2	k-means	0.086	0.111	0.339
2	Hier., average linkage	0.121	0.055	0.274
2	Hier., Ward linkage	0.080	0.118	0.350
4	k-means	0.080	0.221	0.503
4	Hier., average linkage	0.079	0.050	0.300
4	Hier., Ward linkage	0.070	0.199	0.525
14	k-means	0.105	0.416	0.706
14	Hier., average linkage	0.094	0.161	0.570
14	Hier., Ward linkage	0.101	0.473	0.723

Ward linkage – at a few different choices of k , and compare the silhouette score (an internal measure) to the ARI and NMI against the true tumor type labels (external measures).

Two things stand out. First, the silhouette score barely moves across k and stays quite low throughout (≈ 0.07 to 0.12) – unsurprising for real, noisy, high-dimensional biological data without the well-separated blobs of our earlier synthetic examples, but also not very informative: silhouette alone gives us almost no reason to prefer $k = 14$ over $k = 2$. Second, and in sharp contrast, both ARI and NMI increase substantially as k grows, roughly tripling from $k = 2$ to $k = 14$. Knowing the true tumor types reveals that the finer partitions are, in fact, capturing considerably more of the real structure in the data than an internal diagnostic alone would have suggested. Ward linkage is the strongest performer at every k , consistent with our discussion of linkage methods.

We can also look at this qualitatively. Below, we project the data onto its first two principal components and compare the true tumor types to the $k = 14$ Ward clustering, the best-performing configuration in the table above.

The cluster of points near the bottom of both panels is recovered almost identically whether we use the true labels or the clustering, which matches its high visual separation from the rest of the data. Elsewhere, the two panels agree less – a reminder that even our best-performing configuration achieved an ARI of only 0.473, far from perfect agreement.

This example highlights the central lesson of cluster validation: internal and external methods can, and sometimes do, disagree. When ground truth is genuinely unavailable (the ordinary case in unsupervised learning), internal diagnostics like the ones from earlier in this chapter are all we have, but this example is a useful reminder that a clustering with a mediocre internal score is not necessarily a poor clustering, and vice versa.

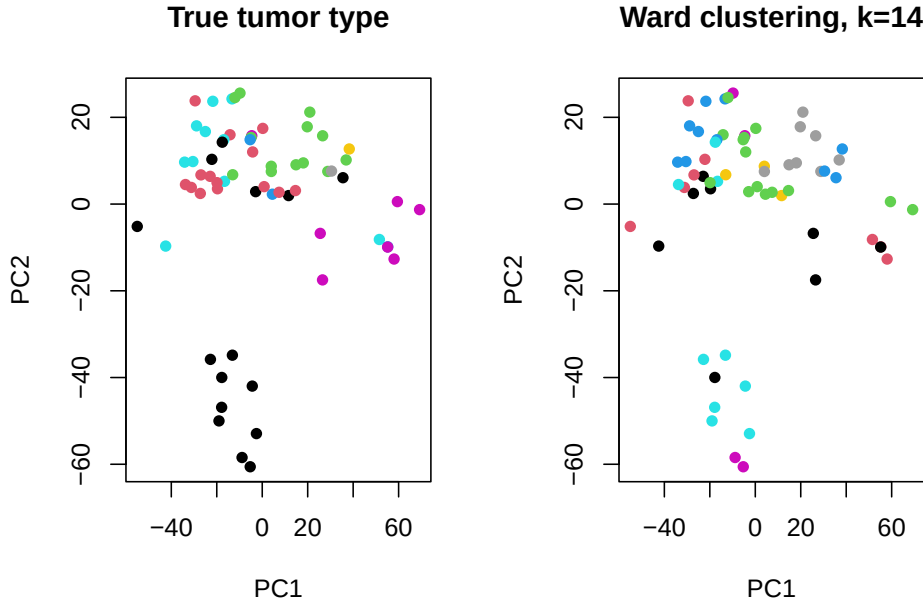


Figure 6.9: Left: samples colored by true tumor type. Right: samples colored by Ward-linkage clustering ($k=14$). Colors are not matched between panels, but the well-separated group near the bottom is recovered nearly identically by both.

6.8 Summary

If Chapter 5 was organized around two strategies for handling curved structure, this chapter is organized around competing geometric definitions of the word “cluster” itself, and choosing a clustering method is really choosing which definition matches your data.

Center-based methods (k -means, k -medoids, k -center) define a cluster as a set of points close to a single representative center, which implicitly commits to roughly ball-shaped clusters (6.1). Kernel k -means keeps this same center-based picture but relocates it to an implicit feature space, letting it recover clusters – like the nested rings – that are not ball-shaped in the original coordinates but become so after an implicit nonlinear mapping (6.2). Hierarchical clustering drops the notion of a center entirely and instead builds structure bottom-up from pairwise distances, with the choice of linkage function encoding a different geometric rule for how two whole clusters, not just two points, should be compared (6.3). Model-based clustering (GMMs) replaces hard geometric boundaries with soft, probabilistic ones, representing each cluster as an ellipsoidal region of high density and allowing points to belong fractionally to more than one cluster (6.4). Spectral clustering steps back from distances in the original space altogether, instead defining clusters as well-connected regions of a similarity graph – a picture that, as we saw, is secretly equivalent to kernel k -means with a graph-based

kernel (6.5). And DBSCAN commits to yet another picture, defining a cluster as a connected region of high density with no assumption on shape at all, and – unlike every other method in this chapter – allowing points to belong to no cluster whatsoever (6.6).

None of these definitions is more “correct” than the others; each is simply a different geometric lens, well suited to some datasets and poorly suited to others, which is exactly why the nested rings example recurs throughout this chapter as a stress test. Since we generally don’t know in advance which geometric picture fits our data, the cluster validation tools at the end of the chapter (6.7) become essential: they let us check, after the fact, whether the geometric assumptions a method made were actually the right ones for the data we gave it.

6.9 Exercises

1. Recall that the intraclass variation is defined as $V(C_\ell) = \frac{1}{2|C_\ell|} \sum_{\vec{x}_i, \vec{x}_j \in C_\ell} \|\vec{x}_i - \vec{x}_j\|^2$. Show algebraically that

$$V(C_\ell) = \sum_{\vec{x}_i \in C_\ell} \|\vec{x}_i - \vec{\mu}_\ell\|^2$$

where $\vec{\mu}_\ell$ is the sample mean of the points in C_ℓ . Hint: expand $\|\vec{x}_i - \vec{x}_j\|^2 = \|(\vec{x}_i - \vec{\mu}_\ell) - (\vec{x}_j - \vec{\mu}_\ell)\|^2$ and use the fact that $\sum_{\vec{x}_i \in C_\ell} (\vec{x}_i - \vec{\mu}_\ell) = \vec{0}$.

2. Let $\vec{x}_1, \dots, \vec{x}_4 = 0, 1, 9, 10 \in \mathbb{R}$ and consider two candidate partitions into $k = 2$ clusters: Partition A, $C_1 = \{0, 1\}, C_2 = \{9, 10\}$; and Partition B, $C_1 = \{0, 9\}, C_2 = \{1, 10\}$.
 - a. Compute T_{kmeans} for Partition A and for Partition B.
 - b. Which partition does the k -means loss prefer? Does this match your intuition?
3. Let $\vec{x}_1, \dots, \vec{x}_5 = -2, -1, 0, 1, 10 \in \mathbb{R}$, and consider a single cluster ($k = 1$) containing all five points.
 - a. Find the center c minimizing the k -means loss $\sum_i (x_i - c)^2$, and compute the resulting loss.
 - b. Find the center $c \in \{x_1, \dots, x_5\}$ minimizing the k -medoids loss $\sum_i |x_i - c|$, and compute the resulting loss.
 - c. The outlier at $x = 10$ pulls the k -means center far from the tight cluster of the other four points, while the k -medoids center stays inside it. Explain why, using the difference between squared and non-squared distance.
4. Consider $\vec{x}_1, \dots, \vec{x}_6 = 1, 2, 3, 8, 9, 10 \in \mathbb{R}$ and $k = 2$. Run Lloyd’s algorithm by hand starting from initial centers $c_1 = 1, c_2 = 4$.

- a. Find the cluster assignments and updated centers after one iteration.
 - b. Find the cluster assignments and updated centers after a second iteration.
 - c. Has the algorithm converged? How does the final partition compare to what you'd guess visually?
5. Consider four points in $\mathbb{R}^2$: $A = (0, 0)$, $B = (1, 0)$ in cluster 1, and $C = (10, 0)$, $D = (11, 0)$ in cluster 2, using Euclidean distance.
 - a. Compute the silhouette s_A for point A .
 - b. Compute the silhouette s_B for point B .
 - c. Without further computation, state s_C and s_D and explain why symmetry gives you these values for free.
6. Suppose we fit GMMs with $K = 1, 2, 3$ clusters to a dataset with $N = 100$ observations, obtaining maximized log-likelihoods $\log L_1^* = -150$, $\log L_2^* = -120$, $\log L_3^* = -115$, and suppose the corresponding number of free parameters are $\mathcal{P}_1 = 5$, $\mathcal{P}_2 = 11$, $\mathcal{P}_3 = 17$.
 - a. Compute BIC_1 , BIC_2 , and BIC_3 using $BIC_k = 2 \log L_k^* - \mathcal{P}_k \log N$.
 - b. Which K does BIC select? Explain why this need not be the K with the largest $\log L_k^*$.
7. For data in $\mathbb{R}^3$ ($d = 3$) with $K = 4$ clusters, count the number of free parameters (means, covariance parameters, and cluster probabilities combined) under
 - a. the EII model (equal volume, spherical shape, no orientation to speak of), i.e. $\Sigma_j = \lambda \mathbf{I}$ for all j .
 - b. the VVV model (fully unconstrained covariances).
8. Given cluster densities $f_1 = \mathcal{N}(0, 1)$ and $f_2 = \mathcal{N}(5, 1)$ (both univariate) with equal prior probabilities $p_1 = p_2 = 0.5$, compute the posterior probability $p(Z = 1 \mid x = 2)$ that a point at $x = 2$ belongs to cluster 1. Recall from Example 6.2 that this posterior is what allows GMMs to produce “fuzzy” cluster assignments.
9. Recall the kernel k -means distance $\|\varphi(\vec{x}_i) - \bar{\varphi}_\ell\|_{\mathcal{H}}^2 = k(\vec{x}_i, \vec{x}_i) - \frac{2}{|C_\ell|} \sum_{\vec{x}_j \in C_\ell} k(\vec{x}_i, \vec{x}_j) + \frac{1}{|C_\ell|^2} \sum_{\vec{x}_j, \vec{x}_{j'} \in C_\ell} k(\vec{x}_j, \vec{x}_{j'})$. Show that using the linear kernel $k(\vec{x}, \vec{y}) = \vec{x}^T \vec{y}$ recovers exactly the ordinary k -means distance $\|\vec{x}_i - \bar{\mu}_\ell\|^2$, so that kernel k -means with a linear kernel is just ordinary k -means.
10. Consider four points A, B, C, D with pairwise distances $d(A, B) = 2$, $d(A, C) = 8$, $d(A, D) = 9$, $d(B, C) = 7$, $d(B, D) = 8$, $d(C, D) = 3$.

- a. Build the dendrogram using **single** linkage. At what heights do the merges occur?
 - b. Build the dendrogram using **complete** linkage. At what heights do the merges occur?
 - c. The two dendrograms agree on which points merge first, but disagree on the height of the final merge. Explain this difference in terms of the definitions of single and complete linkage.
11. Using the single-linkage dendrogram from the previous exercise:
- a. Write down the cophenetic distance (Definition 6.1) between each of the 6 pairs of points.
 - b. Using R, compute the cophenetic correlation coefficient between the original distances and the cophenetic distances.
12. Let $C = \{\{1, 2, 3\}, \{4, 5\}\}$ be a clustering of 5 points and let $C' = \{\{1, 2\}, \{3, 4, 5\}\}$ be a reference partition (e.g. true labels).
- a. Build the 2×2 contingency table with entries n_{ij} as in Definition 6.4.
 - b. By directly examining all $\binom{5}{2} = 10$ pairs, count how many are concordant (both partitions agree the pair is together, or agree it is apart) and compute the Rand Index.
 - c. Using your contingency table from (a) and the formula $RI(C, C') = \left[2 \sum_{ij} \binom{n_{ij}}{2} + \binom{N}{2} - \sum_i \binom{n_{i\cdot}}{2} - \sum_j \binom{n_{\cdot j}}{2} \right] / \binom{N}{2}$, recompute the Rand Index and confirm it matches your answer to (b).
 - d. Compute the Adjusted Rand Index using Definition 6.4.
 - e. Using a calculator or R, compute the Normalized Mutual Information using Definition 6.5.
13. Two independent, uniformly random partitions of $N = 1000$ points, each into 50 clusters of 20 points, tend to have a Rand Index well above zero even though they share no real structure. Explain why, without computing anything – think about what fraction of pairs of points end up in *different* clusters under each partition, and how that fraction alone drives most of the agreement.
14. Let $\vec{x}_1, \dots, \vec{x}_4 = 0, 1, 5, 6 \in \mathbb{R}$, and build the (unweighted, binary) neighborhood graph for spectral clustering using $\epsilon = 2$, so that $\vec{x}_i, \vec{x}_j$ are connected exactly when $|\vec{x}_i - \vec{x}_j| \leq \epsilon$.
- a. Write down the adjacency matrix $\mathbf{W}$ and graph Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{W}$.
 - b. Find the eigenvalues of $\mathbf{L}$. How many are exactly zero, and how does this relate to the number of connected components of the graph?

- c. Give a pair of orthogonal eigenvectors spanning the null space of $\mathbf{L}$, and explain how using these as a 2-dimensional embedding followed by k -means with $k = 2$ recovers the two connected components exactly.
15. In the Kernel k -means section of this chapter, ordinary k -means fails on the nested rings example while kernel k -means with an rbf kernel succeeds. Using the equivalence between kernel k -means (with a graph-based kernel) and spectral clustering, explain conceptually why spectral clustering would also be expected to succeed on the same nested rings data.
16. The `iris` dataset contains known species labels for $N = 150$ flowers, giving us external ground truth to validate against.
- a. Standardize the four numeric iris features, then cluster the standardized data into $k = 3$ groups using k -means, hierarchical clustering (try both average and Ward linkage), and a Gaussian mixture model.
 - b. For each of the four clusterings, compute the silhouette score, the Adjusted Rand Index, and the Normalized Mutual Information against the true species labels.
 - c. Does the method with the best silhouette score also have the best ARI/NMI? If not, what does that tell you about relying on internal validation alone?
 - d. It is well known that two of the three iris species (versicolor and virginica) overlap substantially in feature space, while the third (setosa) is well separated from the other two. Does this show up in your results? Which pairs of species are most often confused by your clusterings?

Glossary

Adjusted Rand Index (ARI). A measure of agreement between two partitions of the same data (e.g. a clustering and a set of true labels) that corrects the ordinary Rand Index for the agreement expected by chance; see Definition 6.4.

Autoencoder. A neural network trained to reconstruct its own input through a lower-dimensional bottleneck layer, used as a nonlinear alternative to PCA for dimension reduction; see 5.10.

Bandwidth. A tuning parameter (often denoted σ or σ^2) controlling the scale of an RBF kernel or similar weighting function; larger bandwidths treat more distant points as similar.

Bayesian Information Criterion (BIC). A model selection criterion that balances a model's goodness of fit against its number of parameters, used to choose the number of clusters in a Gaussian mixture model.

Border point. In DBSCAN, a point that is not itself a core point but falls within the ϵ -neighborhood of one, and so is included on the edge of that core point's cluster; see 6.6.

Centering matrix. The matrix $\mathbf{H} = \mathbf{I} - \frac{1}{N} \mathbf{1}\mathbf{1}^T$, which subtracts the column means from a data matrix when applied on the left.

Chart. A local, continuous, invertible map from a neighborhood on a manifold to Euclidean space, used to assign local coordinates to nearby points; see 5.3.

Cluster. A subset of the data whose members are more similar to one another than to members of other subsets.

Clustering. The unsupervised learning task of partitioning data into groups of similar observations; see Chapter 6.

Cophenetic correlation. The correlation between the original pairwise distances in a dataset and the heights at which pairs of points are merged in a hierarchical clustering dendrogram, used to assess how faithfully a dendrogram represents the original distances; see Definition 6.1.

Core point. In DBSCAN, a point whose ϵ -neighborhood contains at least minPts points, marking it as sitting in the interior of a dense region; see 6.6.

Crowding problem. The geometric obstruction, addressed by t-SNE, in which a low-dimensional embedding has far less “room” at moderate distances than a high-dimensional space does, forcing methods that use the same notion of affinity in both spaces to crowd unrelated points together; see 5.8.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise). A clustering method that builds clusters from chains of overlapping, sufficiently dense neighborhoods, allowing it to recover arbitrarily shaped clusters and to explicitly label sparse points as noise rather than assigning them to a cluster; see 6.6.

Dendrogram. A tree diagram showing the sequence of merges (or splits) produced by hierarchical clustering.

Density-reachability. In DBSCAN, the relation by which a point can be reached from a core point through a chain of overlapping ϵ -neighborhoods of core points; the basis for how DBSCAN builds clusters of arbitrary shape; see Definition 6.2.

Dimension reduction. The unsupervised learning task of finding a lower-dimensional representation of data that preserves important relationships.

Elbow plot. A plot of a clustering loss (e.g. total within-cluster variation) against the number of clusters k , used informally to select k by looking for a bend, or “elbow,” in the curve.

Expectation-Maximization (EM) algorithm. An iterative algorithm for maximum likelihood estimation in models with latent variables, such as Gaussian mixture models, alternating between computing posterior cluster probabilities (E-step) and updating parameters (M-step).

Feature map. The (possibly implicit, possibly infinite-dimensional) function φ associated with a kernel k , such that $k(\vec{x}, \vec{y}) = \varphi(\vec{x})^T \varphi(\vec{y})$.

Gap statistic. A method for choosing the number of clusters k by comparing the within-cluster variation of the observed data to the within-cluster variation expected under a reference distribution with no clustering structure.

Gaussian Mixture Model (GMM). A probabilistic model for clustering in which the data are assumed to arise from a mixture of several Gaussian distributions, one per cluster; see 6.4.

Geodesic distance. The length of the shortest path between two points that stays on a manifold, as opposed to the (generally shorter) straight-line Euclidean distance between the same two points in the ambient space.

Graph Laplacian. The matrix $\mathbf{L} = \mathbf{D} - \mathbf{W}$, where $\mathbf{W}$ is a graph’s weighted adjacency matrix and $\mathbf{D}$ is its diagonal degree matrix; central to Laplacian eigenmaps and spectral clustering.

Hessian eigenmaps (HLE). A manifold learning method that recovers low-dimensional coordinates from the null space of an estimated Hessian operator on the manifold; see 5.7.

Intrinsic dimension. The true (typically much smaller) dimension of the manifold on which high-dimensional data are assumed to lie.

Isometry. A mapping between two spaces that exactly preserves distances; central to the assumptions behind ISOMAP; see Definition 5.6.

ISOMAP. A manifold learning method that applies classical multidimensional scaling to estimated geodesic distances rather than Euclidean distances; see 5.4.

k -center. A center-based clustering method that chooses cluster centers (from among the data points) to minimize the *maximum* distance from any point to its assigned center; see 6.1.

k -means. A center-based clustering method that partitions data into k clusters by minimizing the total squared Euclidean distance from each point to its cluster's mean; see 6.1.

k -medoids. A center-based clustering method, similar to k -means, that requires cluster centers to be data points and typically uses (non-squared) distance; see 6.1.

Kernel. A function $k(\vec{x}, \vec{y})$ that computes an inner product in some (possibly implicit) feature space, allowing linear methods to be applied to nonlinear structure without explicitly computing the feature map; see 5.1.

Kernel PCA. A nonlinear extension of PCA obtained by applying PCA to the (implicit) feature-space representation of the data via the kernel trick; see 5.2.

Kernel trick. The technique of replacing inner products $\vec{x}^T \vec{y}$ with kernel evaluations $k(\vec{x}, \vec{y})$ to obtain a nonlinear version of a linear method; see 5.1.

Laplacian eigenmap. A manifold learning method that recovers low-dimensional coordinates from the eigenvectors of the graph Laplacian, prioritizing the preservation of local neighborhoods; see 5.6.

Linkage. In hierarchical clustering, the rule (e.g. single, complete, average, or Ward) used to compute the dissimilarity between two clusters, each possibly containing more than one point; see 6.3.

Loadings. The eigenvectors of the sample covariance matrix in PCA, giving the directions along which the data vary the most.

Locally Linear Embedding (LLE). A manifold learning method that recovers low-dimensional coordinates by preserving the local, linear reconstruction weights of each point from its nearest neighbors; see 5.5.

Manifold. A space that locally resembles Euclidean space near every point, though it may be globally curved; see Definition 5.1.

Manifold hypothesis. The assumption that observed high-dimensional data lie on or near a manifold of much smaller intrinsic dimension; see 3.1.

Manifold learning. The branch of nonlinear dimension reduction concerned with recovering a low-dimensional manifold structure from high-dimensional data.

Mercer's condition. The condition a function k must satisfy to be a valid kernel, namely that every kernel matrix it produces is positive semidefinite; see Theorem 5.1.

Multidimensional Scaling (MDS). A family of dimension reduction methods that find a low-dimensional configuration of points whose pairwise distances best match a given set of original distances or dissimilarities; see 4.4.

Neighborhood graph. A graph connecting each data point to some set of nearby points (e.g. its k nearest neighbors, or all points within distance ϵ), used as the starting point for most manifold learning and spectral methods.

Noise point. In DBSCAN, any point that is neither a core point nor a border point, and is therefore excluded from every cluster entirely rather than forced into the nearest one; see 6.6.

Nonnegative Matrix Factorization (NMF). A dimension reduction method that factors a nonnegative data matrix into the product of two nonnegative matrices, often yielding an interpretable, parts-based representation; see 4.3.

Normalized Mutual Information (NMI). An information-theoretic measure of agreement between two partitions of the same data, normalized to lie between 0 (no shared information) and 1 (perfect agreement); see Definition 6.5.

Perplexity. In t-SNE, a user-specified tuning parameter, interpretable as a smooth stand-in for the effective number of neighbors of each point, used to locally calibrate the bandwidth of the high-dimensional affinities; see 5.8.

Principal Component Analysis (PCA). A linear dimension reduction method that finds the directions of maximum variance in the data; see 4.1.

Rand Index. The fraction of all pairs of samples on which two partitions of the data agree (both place the pair together, or both place it apart); see Definition 6.3.

Reconstruction error. The discrepancy between the original data and its approximation after dimension reduction and (if applicable) mapping back to the original space.

Scree plot. A plot of PCA variances (eigenvalues) against component number, used informally to choose the number of components to retain by looking for a bend, or “elbow,” in the curve.

Silhouette score. A measure of how well each point fits its assigned cluster relative to the next-closest cluster, averaged over all points to assess the quality

of a clustering; see 6.1.5.3.

Singular Value Decomposition (SVD). A matrix factorization $\mathbf{X} = \mathbf{USV}^T$ expressing any matrix as an orthogonal transformation, a scaling, and a second orthogonal transformation; closely related to PCA; see 4.2.

Spectral clustering. A clustering method that partitions data using the eigenvectors of a graph Laplacian built from the data; see 6.5.

Tangent space. The linear subspace that best approximates a manifold at a given point, spanned by the columns of the Jacobian of the manifold map at that point.

t-SNE (t-Distributed Stochastic Neighbor Embedding). A manifold learning method that matches Gaussian affinities in the original space to heavy-tailed (Student-t) affinities in the embedding, using the mismatch in tail shape to counteract the crowding problem; see 5.8.

UMAP (Uniform Manifold Approximation and Projection). A manifold learning method that constructs a fuzzy neighborhood graph and directly optimizes a low-dimensional embedding to match it, rather than solving an eigenvalue problem; see 5.9.

Unsupervised learning. The branch of machine learning concerned with finding structure in unlabeled data, as opposed to supervised learning's focus on predicting a labeled outcome; see Chapter 1.

Table of Notation

The table below collects the most frequently used notation in this book for quick reference. Some symbols are reused across chapters for closely related purposes (e.g. $\mathbf{W}$ denotes a graph adjacency matrix in the manifold learning and spectral clustering sections, but an NMF feature matrix in Chapter 4 and a neural network weight matrix in the autoencoder section); in these cases, the “Context” column indicates where each usage applies. A more detailed discussion of notation and conventions is given in Chapter 2.

Symbol	Meaning	Context
N	Number of samples/observations	General
d	Dimension of the original data	General
t	Reduced or intrinsic dimension	Manifold learning
k	Number of clusters	Clustering
$\vec{x}, \vec{y}, \vec{z}$	Vectors (lowercase, arrow)	General
$\vec{0}, \vec{1}$	Vector of all zeros / all ones	General
$\vec{x}^T$	Transpose of $\vec{x}$ (a row vector)	General
$\vec{x}^T \vec{y}$	Inner (dot) product	General
$\vec{x} \vec{y}^T$	Outer product (a matrix)	General
$\ \vec{x}\ $	Euclidean norm of $\vec{x}$	General
$\mathbf{A}, \mathbf{X}, \mathbf{\Sigma}$	Matrices (bold)	General
$\mathbf{A}_{ij}$	Entry in row i , column j	General
$\mathbf{A}^T$	Transpose of $\mathbf{A}$	General
$\mathbf{I}$	Identity matrix	General
$Tr(\mathbf{B})$	Trace of $\mathbf{B}$	General
$\det(\mathbf{B})$	Determinant, also written $ \mathbf{B} $	General
$\mathbf{H}$	Centering matrix $\mathbf{I} - \frac{1}{N} \mathbf{1} \mathbf{1}^T$	Ch. 2, 4
∇f	Gradient of f	Ch. 2
$\nabla^2 f$	Hessian matrix of f	Ch. 2, 5
$\hat{\mu}, \hat{\Sigma}$	Sample mean / sample covariance	Ch. 2, 4
$\lambda_1, \dots, \lambda_d$	Eigenvalues, decreasing order	Ch. 4
$\vec{w}_1, \dots, \vec{w}_d$	Eigenvectors, e.g. PCA loadings	Ch. 4
$k(\vec{x}, \vec{y})$	Kernel function	Ch. 5-6
$\mathbf{K}$	Kernel matrix, $\mathbf{K}_{ij} = k(\vec{x}_i, \vec{x}_j)$	Ch. 5-6
φ	Feature map for a kernel	Ch. 5
$\mathcal{H}$	Feature space for a kernel	Ch. 5
$C_1, \dots, C_k$	A partition of the data into clusters	Ch. 6
$\vec{\mu}_\ell, \vec{c}_\ell$	Center of cluster ℓ	Ch. 6
Ψ	A manifold map, $\vec{x}_i = \Psi(\vec{z}_i)$	Ch. 5
$\mathcal{M}$	A manifold	Ch. 5
ϕ	A chart on a manifold	Ch. 5
$\mathbf{W}$	Graph adjacency/weight matrix; also NMF and neural net weights	Ch. 5-6
$\mathbf{D}$	Diagonal degree matrix of a graph	Ch. 5-6
$\mathbf{L}$	Graph Laplacian, $\mathbf{L} = \mathbf{D} - \mathbf{W}$	Ch. 5-6
$\mathbb{I}(\cdot)$	Indicator function (1 if true, else 0)	General
$\mathcal{N}(\vec{\mu}, \mathbf{\Sigma})$	Multivariate normal, mean $\vec{\mu}$, cov. $\mathbf{\Sigma}$	General
Z_i	Latent cluster label for sample i	Ch. 6

Index

This index points to the section, definition, or theorem where each topic is introduced, rather than to a page number, so that it remains equally useful in the PDF and web editions of this book.

A

- Adjusted Rand Index, Definition 6.4
- Association rules (out of scope for this book), 1
- Autoencoders, 5.10

B

- Bayesian Information Criterion (BIC), 6.4
- Border point (DBSCAN), 6.6
- Bottleneck (autoencoder), 5.10

C

- Centering matrix, 1
- Charts and atlases, 5.3
- Classical (metric) scaling, 4.4
- Cluster validation, 6.7
- Clustering, overview, Chapter 6
 - Center-based, 6.1
 - DBSCAN, 6.6
 - Hierarchical, 6.3
 - Kernel k -means, 6.2
 - Model-based (GMM), 6.4
 - Spectral, 6.5
- Cophenetic correlation, Definition 6.1
- Core point (DBSCAN), 6.6
- Covariance constraints (GMM), 6.4
- Crowding problem, 5.8

D

- DBSCAN, 6.6
- Dendrograms, 6.3

- Density-reachability and density-connectivity, Definition 6.2
- Determinant, 1
- Dimension reduction, overview, 1

E

- Eckart-Young-Mirsky Theorem, Theorem 4.3
- Eigenmaps, see *Laplacian eigenmap*, *Hessian eigenmaps*
- Elbow plot, 6.1
- Expectation-Maximization (EM) algorithm, 6.4

F

- Feature map, 5.1

G

- Gap statistic, 6.1
- Gaussian Mixture Models, 6.4
- Geodesic distance, 5.4
- Graph Laplacian, 5.6

H

- Hessian eigenmaps (HLE), 5.7
- Hierarchical clustering, 6.3

I

- Intrinsic dimension, 5.3
- Isometry, Definition 5.6
- ISOMAP, 5.4

K

- k -center, 6.1
- k -means, 6.1
 - Kernel k -means, 6.2
 - Lloyd's algorithm, 6.1
- k -medoids, 6.1
- Kernel PCA, 5.2
- Kernel trick, 5.1

L

- Laplacian eigenmap, 5.6
- Linkage (single, complete, average, Ward), 6.3
- Loadings (PCA), 4.1
- Locally Linear Embeddings (LLE), 5.5

M

- Manifold, Definition 5.1
- Manifold distance, Definition 5.5

- Manifold hypothesis, 3.1
- Manifold learning, overview, Chapter 5
- Mercer's condition, Theorem 5.1
- MNIST dataset, 1
- Multidimensional Scaling (MDS), 4.4

N

- NCI60 tumor microarray dataset, 6.7
- Neighborhood graph, 5.4
- Noise point (DBSCAN), 6.6
- Nonnegative Matrix Factorization (NMF), 4.3
- Normalized Mutual Information, Definition 6.5

P

- Perplexity (t-SNE), 5.8
- Principal Component Analysis (PCA), 4.1
 - First PCA loading, Definition 4.1
 - Optimality of PCA, Theorem 4.2
 - PCA and eigendecomposition, Theorem 4.1

R

- Rand Index, Definition 6.3
- Recommendation systems, 4.2
- Reinforcement learning, 1

S

- Scree plot, 4.1
- Self-supervised learning, 1
- Semi-supervised learning, 1
- Silhouette score, 6.1.5.3
- Singular Value Decomposition (SVD), 4.2
- Spectral clustering, 6.5
- Supervised learning, 1

T

- Tangent space, 5.3
- t-SNE, 5.8

U

- UMAP, 5.9
- Unifying view (manifold learning as kernel PCA), 5.12
- Unsupervised learning, definition, 1

W

- Ward linkage, 6.3

Bibliography

- Locally linear embedding with additive noise. *Pattern Recognition Letters*, 123:47–52, 2019. ISSN 0167-8655. doi: <https://doi.org/10.1016/j.patrec.2019.02.030>. URL <https://www.sciencedirect.com/science/article/pii/S0167865519300820>.
- Md Ashad Alam and Kenji Fukumizu. Hyperparameter selection in kernel principal component analysis. *Journal of Computer Science*, 10:1139–1150, 01 2014. doi: 10.3844/jcssp.2014.1139.1150.
- Jeffrey D. Banfield and Adrian E. Raftery. Model-based gaussian and non-gaussian clustering. *Biometrics*, 49(3):803–821, 1993. ISSN 0006341X, 15410420. URL <http://www.jstor.org/stable/2532201>.
- Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps and spectral techniques for embedding and clustering. In T. Dietterich, S. Becker, and Z. Ghahramani, editors, *Advances in Neural Information Processing Systems*, volume 14. MIT Press, 2001. URL https://proceedings.neurips.cc/paper_files/paper/2001/file/f106b7f99d2cb30c3db1c3cc0fde9ccb-Paper.pdf.
- Mira Bernstein, Vin Silva, John Langford, and Joshua Tenenbaum. Graph approximations to geodesics on embedded manifolds. 02 2001.
- Jean-Philippe Brunet, Pablo Tamayo, Todd R. Golub, and Jill P. Mesirov. Metagenes and molecular pattern discovery using matrix factorization. *Proceedings of the National Academy of Sciences*, 101(12):4164–4169, 2004. doi: 10.1073/pnas.0308531101. URL <https://www.pnas.org/doi/abs/10.1073/pnas.0308531101>.
- P. Campadelli, E. Casiraghi, C. Ceruti, and A. Rozza. Intrinsic dimension estimation: Relevant techniques and a benchmark framework. *Mathematical Problems in Engineering*, 2015(1):759567, 2015. doi: <https://doi.org/10.1155/2015/759567>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1155/2015/759567>.
- Raymond B. Cattell. The scree test for the number of factors. *Multivariate Behavioral Research*, 1(2):245–276, 1966. doi: 10.1207/s15327906mbr0102_10. URL https://doi.org/10.1207/s15327906mbr0102_10. PMID: 26828106.
- Hong Chang and Dit-Yan Yeung. Robust locally linear embedding. *Pattern*

- Recognition*, 39(6):1053–1065, 2006. ISSN 0031-3203. doi: <https://doi.org/10.1016/j.patcog.2005.07.011>. URL <https://www.sciencedirect.com/science/article/pii/S0031320305003353>.
- Jing Chen and Yang Liu. Locally linear embedding: A survey. *Artif. Intell. Rev.*, 36:29–48, 06 2011. doi: 10.1007/s10462-010-9200-z.
- T.F. Cox and M.A.A. Cox. *Multidimensional Scaling, Second Edition*. Chapman & Hall/CRC Monographs on Statistics & Applied Probability. CRC Press, 2000. ISBN 9781420036121. URL <https://books.google.com/books?id=SKZzmEZqvqkC>.
- Adele Cutler and Leo Breiman. Archetypal analysis. *Technometrics*, 36(4): 338–347, 1994. ISSN 00401706. URL <http://www.jstor.org/stable/1269949>.
- Utkarsh J. Dang, Michael P.B. Gallagher, Ryan P. Browne, and Paul D. McNicholas. Model-Based Clustering and Classification Using Mixtures of Multivariate Skewed Power Exponential Distributions. *Journal of Classification*, 40(1):145–167, apr 2023. ISSN 14321343. doi: 10.1007/S00357-022-09427-7/TABLES/5. URL <https://link.springer.com/article/10.1007/s00357-022-09427-7>.
- A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum Likelihood from Incomplete Data Via the EM Algorithm. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 39(1):1–22, sep 1977. ISSN 1369-7412. doi: 10.1111/J.2517-6161.1977.TB01600.X. URL <https://dx.doi.org/10.1111/j.2517-6161.1977.tb01600.x>.
- Li Deng. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012.
- Inderjit S. Dhillon, Yuqiang Guan, and Brian Kulis. Kernel k-means: Spectral clustering and normalized cuts. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 551–556. ACM, 2004. doi: 10.1145/1014052.1014118.
- Alexander Dombowsky and David B. Dunson. Bayesian Clustering via Fusing of Localized Densities. *Journal of the American Statistical Association*, dec 2025. ISSN 1537274X. doi: 10.1080/01621459.2024.2427935;REQUESTEDJOURNAL:JOURNAL:UASA20;WGROU:STRING:PUBLICATION. URL <https://www.tandfonline.com/doi/abs/10.1080/01621459.2024.2427935>.
- David Donoho, Matan Gavish, and Elad Romanov. ScreenNOT: Exact MSE-optimal singular value thresholding in correlated noise. *The Annals of Statistics*, 51(1):122 – 148, 2023. doi: 10.1214/22-AOS2232. URL <https://doi.org/10.1214/22-AOS2232>.
- David L. Donoho and Carrie Grimes. Hessian eigenmaps: Locally linear embedding techniques for high-dimensional data. *Proceedings of the National Academy of Sciences*, 100(10):5591–5596, 2003. doi: 10.1073/pnas.1031596100. URL <https://www.pnas.org/doi/abs/10.1073/pnas.1031596100>.

- Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, KDD'96, pages 226–231, 1996.
- Brian S Everitt. *Cluster analysis*. Arnold ; Oxford University Press, London : New York, 4th ed. edition, 2001. ISBN 0340761199.
- Karl Pearson F.R.S. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572, 1901. doi: 10.1080/14786440109462720.
- Xiao Fu, Wing-Kin Ma, Kejun Huang, and N.D. Sidiropoulos. Blind separation of quasi-stationary sources: Exploiting convex geometry in covariance domain. *IEEE Transactions on Signal Processing*, 63:1–1, 05 2015. doi: 10.1109/TSP.2015.2404577.
- Cédric Févotte, Nancy Bertin, and Jean-Louis Durrieu. Nonnegative matrix factorization with the itakura-saito divergence: With application to music analysis. *Neural Computation*, 21(3):793–830, 2009. doi: 10.1162/neco.2008.04-08-771.
- Jihun Ham, Daniel D. Lee, Sebastian Mika, and Bernhard Schölkopf. A kernel view of the dimensionality reduction of manifolds. In *Proceedings of the Twenty-First International Conference on Machine Learning (ICML)*, page 47. ACM, 2004. doi: 10.1145/1015330.1015417.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. Springer New York Inc., New York, NY, USA, 2001.
- G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006. doi: 10.1126/science.1127647.
- Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of educational psychology*, 24(6):417, 1933.
- Alan Julian Izenman. *Modern Multivariate Statistical Techniques: Regression, Classification, and Manifold Learning*. Springer Publishing Company, Incorporated, 1 edition, 2008. ISBN 0387781889.
- Dimitris Karlis and Anais Santourian. Model-based clustering with non-elliptically contoured distributions. *Statistics and Computing*, 19(1):73–83, mar 2009. ISSN 09603174. doi: 10.1007/S11222-008-9072-0/METRICS. URL <https://link.springer.com/article/10.1007/s11222-008-9072-0>.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. *CoRR*,

- abs/1312.6114, 2013. URL <https://api.semanticscholar.org/CorpusID:216078090>.
- Daniel Lee and H. Sebastian Seung. Algorithms for non-negative matrix factorization. In T. Leen, T. Dietterich, and V. Tresp, editors, *Advances in Neural Information Processing Systems*, volume 13. MIT Press, 2000. URL https://proceedings.neurips.cc/paper_files/paper/2000/file/f9d1152547c0bd e01830b7e8bd60024c-Paper.pdf.
- J. M. Lee. *Introduction to Riemannian Manifolds (Second edition)*. Springer Nature, 2019. doi: <https://doi.org/10.1007/978-3-319-91755-9>.
- Chia-Hsiang Lin, Wing-Kin Ma, Wei-Chiang Li, Chong-Yung Chi, and Arulmurugan Ambikapathi. Identifiability of the simplex volume minimization criterion for blind hyperspectral unmixing: The no pure-pixel case. *IEEE Transactions on Geoscience and Remote Sensing*, 53, 06 2014. doi: 10.1109/TGRS.2015.2424719.
- Anna Little, Jason Lee, Yoon-Mo Jung, and Mauro Maggioni. Estimation of intrinsic dimensionality of samples from noisy low-dimensional manifolds in high dimensions with multiscale svd. pages 85 – 88, 10 2009. doi: 10.1109/SSP.2009.5278634.
- Weixiang Liu, Aifa Tang, Datian Ye, and Zhen Ji. Nonnegative singular value decomposition for microarray data analysis of spermatogenesis. In *2008 International Conference on Information Technology and Applications in Biomedicine*, pages 225–228, 2008. doi: 10.1109/ITAB.2008.4570528.
- Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- Sebastian Mika, Bernhard Schölkopf, Alex Smola, Klaus-Robert Müller, Matthias Scholz, and Gunnar Rätsch. Kernel pca and de-noising in feature spaces. In M. Kearns, S. Solla, and D. Cohn, editors, *Advances in Neural Information Processing Systems*, volume 11. MIT Press, 1998. URL https://proceedings.neurips.cc/paper_files/paper/1998/file/226d1f15ecd35f784d2a20c3ecf56d7f-Paper.pdf.
- R. Mojena. Hierarchical grouping methods and stopping rules: an evaluation*. *The Computer Journal*, 20(4):359–363, 01 1977. ISSN 0010-4620. doi: 10.1093/comjnl/20.4.359. URL <https://doi.org/10.1093/comjnl/20.4.359>.
- Adrian O’Hagan, Thomas Brendan Murphy, Isobel Claire Gormley, Paul D. McNicholas, and Dimitris Karlis. Clustering with the multivariate normal inverse Gaussian distribution. *Computational Statistics & Data Analysis*, 93: 18–30, jan 2016. ISSN 0167-9473. doi: 10.1016/J.CSDA.2014.09.006. URL <https://www.sciencedirect.com/science/article/abs/pii/S0167947314002667>.
- Maurice Roux. A comparative study of divisive and agglomerative hierarchical

- clustering algorithms. 35(2):345–366. ISSN 1432-1343. doi: 10.1007/s00357-018-9259-9. URL <https://doi.org/10.1007/s00357-018-9259-9>.
- Sam T. Roweis and Lawrence K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000. doi: 10.1126/science.290.5500.2323. URL <https://www.science.org/doi/abs/10.1126/science.290.5500.2323>.
- Lawrence K. Saul and Sam T. Roweis. Think globally, fit locally: Unsupervised learning of low dimensional manifolds. 4:119–155, 2003. doi: 10.1162/153244304322972667. URL <https://doi.org/10.1162/153244304322972667>.
- M. D. Spivak. *A comprehensive introduction to differential geometry*. Publish or Perish, Inc., 1979.
- Gilbert Strang. *Linear algebra and its applications*. Thomson, Brooks/Cole, Belmont, CA, 2006. ISBN 0030105676 9780030105678 0534422004 9780534422004. URL <http://www.amazon.com/Linear-Algebra-Its-Applications-Edition/dp/0030105676>.
- Gabor J Szekely, Maria L Rizzo, et al. Hierarchical clustering via joint between-within distances: Extending ward’s minimum variance method. *Journal of classification*, 22(2):151–184, 2005.
- Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000. doi: 10.1126/science.290.5500.2319.
- Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. SIAM, 1997. ISBN 0898713617.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.
- Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. ICML ’08, page 1096–1103. Association for Computing Machinery, 2008. ISBN 9781605582054. doi: 10.1145/1390156.1390294.